FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

# Multi-Agent Reinforcement Learning Control of Distributed Mixed Cooperative–Competitive Real-World Cyber-Physical Systems

## Case Studies in Smart Energy Management

**Tiago Fonseca**




**Programa Doutoral em Engenharia Informática**

Supervisor: Prof. Luís Miguel Moreira Lino Ferreira

Co-Supervisor: Prof. Armando Sousa

December 2026

"*For the things we have to learn before we can do them, we learn by doing them.*"

Aristotle

*Nicomachean Ethics*

© Tiago Fonseca, 2026

# Declaration of Honour

I hereby declare on my honour that this thesis (or dissertation) is my own original work and that all sources used have been acknowledged and properly cited. I am aware that the use of unauthorized third-party text, data, or figures constitutes plagiarism and is a serious breach of academic integrity.

Porto, _____ of _____ of 20_____

_____

*(Signature of the Student)*

# Declaration on the Use of Generative Artificial Intelligence

Generative Artificial Intelligence (AI) supported the work reported in this thesis, chiefly as a partner in the critical discussion of research, architecture, and implementation decisions. It was brought into the process while alternatives were still being considered, helping to question assumptions, compare possible designs, check consistency between requirements and technical specifications, and identify edge cases before changes reached the code. It also assisted with research organisation, drafting, language revision, documentation, software implementation, debugging, and testing.

Software-related assistance followed a specification-led process. Requirements and interfaces were established before implementation and served as the reference against which proposed designs and code were reviewed. For research and writing tasks, suggested interpretations, references, numerical values, and scientific claims were checked against primary sources, datasets, source code, tests, and experimental records. Generated material was rewritten or discarded whenever it did not meet these checks.

The author defined the research questions and methods, designed and verified the experiments, interpreted the results, and made all final decisions concerning the software and the manuscript. Responsibility for the originality, accuracy, arguments, conclusions, and final content of the thesis rests entirely with the author.

# Statement on Co-Authored Work

The research presented in this thesis was conducted within an integrated project whose overall architecture was conceived and defined by the author. This architecture provided a common conceptual and technical framework for the supporting components described throughout the thesis. Several of these components were developed collaboratively with supervisors, research colleagues, students, open-source contributors, and industrial partners. The corresponding lines of work were proposed and formulated by the author, who also contributed to their technical direction, development, supervision, validation, and integration.

As the nature and extent of the collaboration varied between components, the following table records the persons involved, the author's specific contribution, and the location of each component within the thesis. Publications arising from this work are reported separately in the list of research outputs and in the references.

| **Component or tool** | **Co-author(s) or contributor(s)** | **Author's contribution** | **Thesis location** |
|---|---|---|---|
| *[Title or name]* | *[Name(s)]* | *[Concise contribution]* | *[Ch./Sec.]* |
| *[Title or name]* | *[Name(s)]* | *[Concise contribution]* | *[Ch./Sec.]* |
| *[Title or name]* | *[Name(s)]* | *[Concise contribution]* | *[Ch./Sec.]* |
| *[Title or name]* | *[Name(s)]* | *[Concise contribution]* | *[Ch./Sec.]* |

The author retains responsibility for the selection, verification, integration, and interpretation of these components as presented in the thesis. Additional contributions are acknowledged in the relevant sections and do not alter the author's responsibility for the scientific arguments and conclusions reported herein.

# Acknowledgements

This work was supported by Fundação para a Ciência e a Tecnologia, I.P. (FCT), under the supervision and superintendence of the Ministry of Education, Science and Innovation, through doctoral grant 2024.00855.BD, financed by the Portuguese State Budget and co-financed by the European Social Fund Plus (FSE+) through the Programa Demografia, Qualificações e Inclusão (PDQI/PESSOAS 2030). Part of the research presented in this thesis was developed within the OPEVA project, which received funding within the Key Digital Technologies Joint Undertaking (KDT JU) from the European Union's Horizon Europe Programme and the National Authorities of France, Belgium, Czechia, Italy, Portugal, Turkey, and Switzerland, under grant agreement 101097267. This work was also supported by the DEMFLEX operation (COMPETE2030-FEDER-01657200 – 19215), supported by the Innovation and Digital Transition Programme (COMPETE 2030), under Portugal 2030, and co-financed by the European Union. The views and opinions expressed in this thesis are those of the author alone and do not necessarily reflect the views of FCT, the European Union, KDT JU, or the respective funding and granting authorities. Neither these institutions nor the granting authorities can be held responsible for them.

I would first like to express my sincere gratitude to my supervisor, Professor Luís Miguel Moreira Lino Ferreira, and to my co-supervisor, Professor Armando Sousa. Their guidance, availability, trust, and willingness to discuss the scientific and practical challenges encountered throughout this work were invaluable. They gave me not only knowledge and advice, but also the freedom to develop my own ideas, question assumptions, make mistakes, and learn from them. Their support throughout these years played an important role in my development as an independent researcher, engineer, and professional.

I would also like to acknowledge the industrial partners who contributed to the research developed during this thesis. I am particularly grateful to CleanWatts, especially Eng. Jorge Landeck, and to i-Charging, especially Eng. Paulo Rodrigues. Our collaboration brought an essential real-world perspective to the work, particularly through the research conducted within OPEVA. Their technical knowledge, operational experience, infrastructure, and understanding of the practical constraints of energy systems and electric-vehicle charging helped ensure that many of the problems investigated in this thesis were confronted with operational reality rather than considered solely as academic abstractions. I am equally grateful to the many other people from these organisations, project partners, and collaborating institutions with whom I had the opportunity to interact and who, directly or indirectly, contributed knowledge, discussions, data, infrastructure, feedback, or assistance to this work.

An especially important part of my doctoral experience was the opportunity to supervise, co-supervise, and work closely with students developing their own theses and research projects. I am grateful to [names to be added], and to all the students I had the opportunity to accompany during these years. Their projects often developed in parallel with my own research and created opportunities to investigate related problems from different perspectives. Supervising this work taught me a great deal about leading research, defining achievable research questions, organising

technical work, establishing development and validation pipelines, reviewing results critically, and helping a team move from an initial idea to a defensible outcome. It also taught me when to provide direction and when to allow someone else to find a different solution. Learning to guide and coordinate the work of others became one of the most valuable parts of my doctoral training.

I also want to thank the friends and colleagues at FEUP who shared their own doctoral journeys with me. A special thanks goes to João Vitorino and Tiago Dias for the conversations, discussions, frustrations, laughs, and countless moments shared throughout these years. Going through a PhD alongside people facing many of the same uncertainties and challenges made the difficult periods easier and the good ones considerably better. I am also grateful to the colleagues, researchers, project partners, and friends with whom I exchanged ideas, code, tools, reviews, experiments, and the many small forms of help that, accumulated over several years, became part of this work.

I have deliberately left my most personal thanks for last. I am deeply grateful to my parents, Vera and Abílio, and to my grandparents. My parents gave me the stability, education, encouragement, and opportunities that made this path possible long before the PhD began. They supported my choices even when those choices demanded time, patience, and sacrifices from them, and they taught me the value of work, responsibility, and perseverance more through their example than through words. I am equally grateful to my grandparents for their affection, presence, and for the values and sense of family that have accompanied me throughout my life. Reaching this point is also the result of the foundation they all helped build for me over many years.

My final and deepest thanks go to my girlfriend, Inês. She experienced this PhD more closely than anyone else, not only through its achievements and good moments, but through the pressure, uncertainty, frustration, exhaustion, and constant presence that such a long process gradually brings into everyday life. There were periods when the work occupied far more of me than it should have, and when I became more impatient, distracted, withdrawn, grumpy, or simply more difficult to be around than I would have liked. Too often, the problems of the PhD followed me home, and she had to live with a version of me shaped by difficulties that were not hers.

Through all of it, she stayed beside me. She listened when I needed to talk, gave me perspective when I had lost it, celebrated progress that sometimes seemed insignificant to me, and supported me through periods when everything appeared to be moving too slowly or not at all. She knew when to encourage me, when to challenge me, and when simply to be there. When I doubted myself or the value of continuing, her belief was often steadier than my own, and giving up was never allowed to become the easy answer. More importantly, she continued to love and support me even when the pressure of this work made that support harder to give than I would like to admit.

A finished thesis inevitably presents research as something ordered: problems lead to methods, methods lead to experiments, and experiments lead to conclusions. Living through it is very different. Much of this journey was uncertain, repetitive, frustrating, and far less elegant than the document that eventually emerged from it. Inês was beside me throughout that real version of the PhD. Her patience, strength, affection, and belief in me made an enormous difference not only to completing this thesis, but to the person I remained while doing it.

Nes, Pai, Mae, Aida, Vo Ek e Vo Fonso.

# Abstract

Real-world cyber-physical systems couple software decisions to physical processes. Industrial plants, autonomous robots, transport networks, smart buildings, and energy communities all depend on controllers whose decisions affect equipment, users, and shared infrastructure. These controllers are usually developed and tested in simulation, where participants, interfaces, data quality, constraints, and computing resources can be kept stable. Those assumptions weaken when a controller reaches a real installation. Assets appear or become unavailable, observations arrive late or incomplete, feasible actions change, and an error can affect service or equipment. Good simulated performance therefore provides only part of the evidence needed to establish real-world applicability.

This thesis studies that gap using Multi-Agent Reinforcement Learning (MARL) as the control approach for distributed decision-making. MARL is relevant when local agents act with limited information but their decisions interact through shared resources and collective objectives. Renewable Energy Communities and electric-vehicle charging provide the empirical setting because they combine heterogeneous participants, flexible assets, user deadlines, individual economic interests, and common electrical limits. The investigation is structured by four connected requirements, namely scalability, deployability, safety, and robustness.

EnergAIze connects system and algorithm design across problem representation, data, simulation, controller development, experimentation, and deployment. CityLearn v3 and annual 15-minute datasets make heterogeneity, constraints, faults, and reconfiguration reproducible. A shared protocol compares rule-based, optimisation, single-agent, and multi-agent methods and exposes their assumptions. TI-MARL (Typed-Interface Multi-Agent Reinforcement Learning) is the principal algorithmic contribution. It represents each agent's current interface as a versioned typed contract bound to identifiable entities. A deterministic compiler derives valid observations, action groups, dependencies, and bounds. A shared decentralised policy operates over this variable interface, a variable-population critic supports coordination during training, and an analytic projector enforces local feasibility.

The evaluation covers reconfiguration, faults, scalability, compatibility, heterogeneous runtimes, OPEVA, and DemFlex. The evidence indicates that real-world applicability depends on preserving the meaning of observations, actions, constraints, and policy artefacts across simulation and execution. TI-MARL supports reconfiguration among known semantic types, while lifecycle and assurance mechanisms preserve traceability, feasibility, and recovery. The conclusions remain bounded to the evaluated energy systems and do not claim unknown-type admission, universal formal safety, or autonomous deployment of coordinated policy sets.

**Keywords** Multi-Agent Reinforcement Learning; Cyber-Physical Systems; Renewable Energy Communities; TI-MARL; Scalability; Safety and Robustness; Policy Lifecycle.

# Resumo

Os sistemas ciberfísicos reais ligam decisões de software a processos físicos. Instalações industriais, robôs autónomos, redes de transportes, edifícios inteligentes e comunidades de energia dependem de controladores cujas decisões afetam equipamentos, utilizadores e infraestruturas partilhadas. Estes controladores são normalmente avaliados em simulação, sob condições estáveis de participantes, interfaces, dados, restrições e recursos computacionais. Esses pressupostos enfraquecem numa instalação real. Ativos surgem ou ficam indisponíveis, observações chegam tarde ou incompletas, ações exequíveis mudam e um erro pode afetar serviço ou equipamento. O desempenho simulado fornece apenas parte da evidência necessária para demonstrar aplicabilidade no mundo real.

A tese estuda essa distância recorrendo à Aprendizagem por Reforço Multiagente (MARL) para controlo distribuído. O MARL adequa-se a agentes locais com informação limitada cujas decisões interagem através de recursos e objetivos comuns. As Comunidades de Energia Renovável e o carregamento de veículos elétricos servem de domínio empírico ao reunirem heterogeneidade, flexibilidade, prazos de utilização, interesses individuais e limites elétricos. Quatro requisitos orientam a investigação, nomeadamente escalabilidade, capacidade de implantação, segurança e robustez.

O EnergAIze articula sistema e algoritmos ao ligar representação, dados, simulação, controladores, experimentação e implantação. O CityLearn v3 e os datasets anuais de 15 minutos tornam reprodutíveis heterogeneidade, restrições, falhas e reconfiguração. Um protocolo comum compara métodos baseados em regras, otimização, aprendizagem individual e aprendizagem multiagente, expondo os seus pressupostos. O TI-MARL (Typed-Interface Multi-Agent Reinforcement Learning) constitui a principal contribuição algorítmica. Representa a interface atual de cada agente como um contrato tipado e versionado ligado a entidades identificáveis. Um compilador determinístico deriva observações, grupos de ação, dependências e limites válidos. Uma política descentralizada e partilhada atua sobre esta interface variável, um crítico de população variável apoia a coordenação durante o treino e um projetor analítico assegura a viabilidade local.

A avaliação abrange reconfiguração, falhas, escalabilidade, compatibilidade, plataformas heterogéneas e os demonstradores OPEVA e DemFlex. A evidência indica que a aplicabilidade no mundo real depende de preservar o significado das observações, ações, restrições e artefactos de política entre a simulação e a execução. O TI-MARL suporta a reconfiguração entre tipos semânticos conhecidos, enquanto os mecanismos de ciclo de vida e garantia preservam rastreabilidade, viabilidade e recuperação. As conclusões limitam-se aos sistemas energéticos avaliados e não abrangem tipos desconhecidos, segurança formal universal ou implantação autónoma de conjuntos coordenados de políticas.

**Palavras-chave** Aprendizagem por Reforço Multiagente; Sistemas Ciberfísicos; Comunidades de Energia Renovável; TI-MARL; Escalabilidade; Segurança e Robustez; Ciclo de Vida de Políticas.

# Publications and Research Outputs Arising from the Thesis

The table maps the publications and research outputs associated with the thesis to their principal chapters. Publication status and the relationship between each output and the corresponding thesis contribution follow the available publication and project records.

| ID | Publication or output | Status | Chapter |
|---|---|---|---|
| P1 | *Flexible Loads Scheduling Algorithms for Renewable Energy Communities* | Published, 2022 | 2, 5 |
| P2 | *EnergAIze: Multi Agent Deep Deterministic Policy Gradient for Vehicle to Grid Energy Management* | Preprint, 2024 | 3, 5, 8 |
| P3 | *EVLearn: Extending the CityLearn Framework with Electric Vehicle Simulation* | Published, 2025 | 4 |
| P4 | *Control of Renewable Energy Communities using AI and Real-World Data* | Preprint, 2025 | 5, 8 |
| P5 | *TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems* | Manuscript in preparation | 6 |
| P6 | Progressive control, MATD3, hierarchy, data-efficient learning, and TI-MARL structural and reconfiguration studies | Experiment and manuscript family in preparation | 5–6 |
| P7 | EnergAIze experiment and MARL policy-set lifecycle orchestration | System contribution in preparation | 7 |
| P8 | *CityLearn v3 / SoftCPS REC Simulator*: public simulator, scenario, documentation, benchmark, and release record associated with the thesis | Public software/output record | 3–8 |

The publications document parts of the research, while the dissertation provides the complete contributions. It harmonises terminology and experimental assumptions across the outputs, incorporates extended and negative results where relevant, and provides the cross-cutting synthesis required to answer the research questions.

# Contents

**Declaration of Honour** i

**Declaration on the Use of Generative Artificial Intelligence** iii

**Statement on Co-Authored Work** v

**Acknowledgements** vii

**Abstract** ix

**Resumo** xi

**Publications and Research Outputs Arising from the Thesis** xiii

**List of Acronyms** xxix

**Glossary** xxxiii

**1 Introduction** 1
- 1.1 Context and Motivation . . . 1
  - 1.1.1 Distributed and Dynamically Reconfigurable Cyber-Physical Systems . . 1
  - 1.1.2 Renewable Energy Communities as Representative Cyber-Physical Systems 2
  - 1.1.3 MARL for Mixed Cooperative–Competitive Decentralised Control . . . . 3
- 1.2 Requirements for MARL in Real-World CPS . . . 5
- 1.3 Thesis Proposition, Research Questions, and Scope . . . 6
  - 1.3.1 Thesis Proposition . . . 6
  - 1.3.2 Research Questions . . . 7
  - 1.3.3 Research Objectives . . . 8
  - 1.3.4 Scope and Assumptions . . . 8
- 1.4 Research Contributions and Outcomes . . . 9
  - 1.4.1 Scientific Contributions . . . 9
  - 1.4.2 System, Software, and Open-Source Contributions . . . 11
  - 1.4.3 Societal and Environmental Relevance . . . 11
- 1.5 Research Path and Thesis Organisation . . . 11

**2 Real-World Multi-Agent Control in Cyber-Physical Systems: Foundations, State of the Art, and Research Gaps** 13
- 2.1 Literature Scope and Evidence Approach . . . 13
  - 2.1.1 Scope and Sources . . . 14

2.1.2 Evidence Synthesis and Limitations . . . 14
2.2 Distributed, Heterogeneous, and Dynamically Reconfigurable Cyber-Physical Systems . . . 15
2.2.1 Distributed Control and Operational Constraints . . . 15
2.2.2 Heterogeneous Agents, Assets, and Interfaces . . . 16
2.2.3 Dynamic Populations, Capabilities, and Reconfiguration . . . 18
2.3 Renewable Energy Communities as Representative CPSs . . . 20
2.3.1 Participants, Assets, and Heterogeneity . . . 21
2.3.2 Flexibility, Temporal Coupling, and Dynamic Availability . . . 23
2.3.3 Individual and Community Objectives and Physical Constraints . . . 25
2.4 Control Approaches for Distributed Energy Systems and Related CPS . . . 27
2.4.1 Explicit Control: Rules, Optimisation, and Predictive Control . . . 28
2.4.2 Reinforcement Learning and Multi-Agent Control . . . 30
2.4.3 Structured, Federated, Transfer, Hybrid, and Hierarchical Approaches . . . 31
2.4.4 When MARL Adds Value across CPS Domains . . . 32
2.5 Foundations of Reinforcement Learning and MARL Used in This Thesis . . . 34
2.5.1 Sequential Decision-Making and Partial Observability . . . 34
2.5.2 Reinforcement Learning and Actor–Critic Methods . . . 36
2.5.3 Markov Games, MARL, and CTDE . . . 37
2.5.4 MARL Challenges Relevant to This Thesis . . . 39
2.6 Simulation and Levels of Real-World Evidence . . . 41
2.7 Critical Analysis of the State of the Art . . . 42
2.7.1 RQ1: Dynamic and Heterogeneous Multi-Agent Control . . . 44
2.7.2 RQ2: Multi-Dimensional Scalability of Learning and Coordination . . . 46
2.7.3 RQ3: Safe and Robust Operation . . . 47
2.7.4 RQ4: Governed and Deployable MARL Lifecycle . . . 48
2.8 Adjacent Challenges and Scope Boundaries . . . 50
2.8.1 Supporting Requirements and Evaluation Conditions . . . 50
2.8.2 Important Challenges Beyond the Central Scope . . . 52
2.9 Synthesis of Residual Gaps and Thesis Requirements . . . 52
2.10 Chapter Summary . . . 55

**3 EnergAIze: An Integrated Framework for Intelligent Energy Management 57**
3.1 Architectural View I: A Supervised Energy Decision . . . 58
3.2 Architectural View II: A Distributed Renewable Energy Community . . . 59
3.3 Architectural View III: Functional Components and Interfaces . . . 60
3.3.1 Percepta and the Data Provider . . . 61
3.3.2 Prediction, PulseCharge, and User Flexibility . . . 62
3.3.3 Algorithms, Inference, and the Fail-Safe Supervisor . . . 63
3.4 Controller Development and Runtime Operation . . . 64
3.4.1 CityLearn, EVLearn, and Experimental Development . . . 64
3.4.2 Runtime Topologies and Deployment Boundary . . . 65
3.5 Architectural Rationale and Thesis Map . . . 65
3.6 Chapter Summary . . . 66

**4 CityLearn v3: A Simulation and Evaluation Environment for Renewable Energy Communities** **69**
4.1 The Simulator as an Experimental Research Contribution . . . 70
4.1.1 Research Instrument and Public Software . . . 70
4.1.2 CityLearn v3 Across the Controller Lifecycle . . . 71
4.2 Problem Definition, Requirements, and Positioning . . . 73
4.2.1 Limitations of the Inherited Environment and Required Experimental Contract 73
4.2.2 Lineage and position within existing environments . . . 74
4.3 CityLearn v3 Architecture and REC Modelling Capabilities . . . 76
4.3.1 System Architecture and the Scenario-to-Evidence Path . . . 76
4.3.2 Scenario, Time, Unit, and Data Contracts . . . 78
4.3.3 Stationary storage and electrical-service limits . . . 79
4.3.4 Electric vehicles and chargers . . . 81
4.3.5 Deferrable appliances . . . 81
4.3.6 Local energy sharing and settlement . . . 82
4.3.7 Changing community members and assets . . . 83
4.3.8 Controlled data and equipment failures . . . 84
4.3.9 Demand-response services . . . 85
4.3.10 Multiple communities . . . 86
4.3.11 Controller Interfaces and Applied-Action Traceability . . . 86
4.4 Datasets Developed to Evaluate REC Controllers . . . 87
4.4.1 Common temporal, provenance, and accounting contract . . . 87
4.4.2 Composition and the separation of infrastructure from mobility . . . 89
4.4.3 Experimental ladder and matched scenario variants . . . 90
4.4.4 Validation protocol and annual baseline execution . . . 93
4.5 Evaluation Protocol and Controller Scorecard . . . 93
4.5.1 Reference Controllers and Fair Comparisons . . . 93
4.5.2 Core Scorecard and Result Exports . . . 94
4.6 Chapter Synthesis . . . 95

**5 Control and Learning Methods for Renewable Energy Communities** **97**
5.1 Research Progression . . . 98
5.2 Common Control Problem and Experimental Protocol . . . 100
5.2.1 REC Control Formulation . . . 101
5.2.2 Information and Comparison Surfaces . . . 102
5.2.3 Dataset Ladder and Experimental Funnel . . . 103
5.2.4 Training Budgets, Repetition, and Statistical Reading . . . 104
5.3 Engineering the Control–Learning Interface . . . 105
5.3.1 From Simulator State to Policy Input . . . 105
5.3.2 Action Semantics and the Executed Transition . . . 106
5.3.3 Reward Lineage . . . 107
5.3.4 Replay, Demonstrations, and Rare Events . . . 108
5.3.5 Optimisation, Checkpoints, and Export Continuity . . . 109
5.4 Level I: Transparent and Constraint-Aware Controllers . . . 110
5.4.1 Business-as-Usual and the Rule-Based Ladder . . . 110
5.4.2 MILP Reference Problems . . . 111
5.4.3 Evidence Produced at Level I . . . 113
5.5 Level II: Adaptive Building Control . . . 113

5.5.1 Strict Local Learning Contract . . . 114
5.5.2 Proximal Policy Optimisation . . . 114
5.5.3 Twin Delayed Deep Deterministic Policy Gradient . . . 115
5.5.4 Residual Control, Service Protection, and Local Safety . . . 115
5.5.5 Entity-Structured PPO . . . 116
5.6 Level III: Decentralised Multi-Agent Control . . . 117
5.6.1 Centralised Training and Decentralised Execution . . . 117
5.6.2 MADDPG and the Original EnergAIze Formulation . . . 118
5.6.3 MATD3 . . . 118
5.6.4 Alternative MARL Comparators . . . 119
5.6.5 Entity-Structured MATD3 . . . 120
5.6.6 Evidence Produced at Level III . . . 121
5.7 Prior Knowledge and Data-Efficient Learning . . . 121
5.7.1 Rule-Guided Exploration and Residual Policies . . . 121
5.7.2 Behaviour Cloning and Demonstration Use . . . 122
5.7.3 Replay as a Reusable Knowledge Source . . . 123
5.7.4 Offline Pretraining and Policy Distillation . . . 123
5.8 Level IV: Hierarchical Community Coordination . . . 124
5.8.1 Decision Decomposition . . . 124
5.8.2 Global and Member-Specific Coordination . . . 125
5.8.3 Causal Signal Path and Neutral Parity . . . 126
5.8.4 Hierarchical Comparison . . . 126
5.9 Cross-Method Synthesis and Open Limitations . . . 127
5.9.1 Contribution to the Research Questions . . . 127
5.9.2 Open Limitations . . . 128
5.10 Chapter Summary . . . 130

**6 TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems 131**
6.1 Scientific Positioning and Problem Formulation . . . 132
6.1.1 Closest Approaches and the Residual Problem . . . 132
6.1.2 Typed-Interface Decision Processes . . . 134
6.2 Implemented TI-MARL Architecture . . . 136
6.3 Entity-Bound Runtime Typed Interfaces . . . 137
6.3.1 Schemas, Entities, and Dynamic Module Instances . . . 137
6.3.2 Typed Observation Parts and Community Context . . . 138
6.3.3 Dynamic Action Groups and Typed Parameterised Ports . . . 139
6.3.4 Health-Aware Causal Compilation . . . 140
6.4 Local Typed-Interface Policies . . . 141
6.4.1 Local Information Boundaries and Parameter Sharing . . . 141
6.4.2 Hierarchical Typed Encoding and Local Latent State . . . 142
6.4.3 Dynamic Grouped Action Decoding . . . 143
6.4.4 Constraint-Aware Local Bundle Construction . . . 144
6.5 Multi-Agent Learning and Community Coordination . . . 145
6.5.1 Joint Training and Decentralised Execution . . . 145
6.5.2 Local and Central Set Critics . . . 145
6.5.3 TI-PPO and TI-MAPPO . . . 146
6.5.4 Community Observations, Rewards, and Learned Coordination . . . 146

6.5.5 Training Stability and Time Resolution . . . 147
6.5.6 Reusing Prior Knowledge without Extending Runtime Authority . . . 147
6.6 Typed Constraints, Safety, and Runtime Robustness . . . 148
6.6.1 Typed Constraint Relations . . . 148
6.6.2 Boundary of Shared-Constraint Enforcement . . . 148
6.6.3 Runtime Assurance and Degraded Modes . . . 149
6.7 Compatibility and Typed Explainability . . . 149
6.7.1 Interface Snapshots and Admission . . . 149
6.7.2 Checkpoint, Export, and Actor-Only Handoff . . . 150
6.7.3 Execution-Level Decision Traces . . . 150
6.7.4 Formal and Operational Properties . . . 151
6.8 Experimental Methodology and Baselines . . . 151
6.8.1 Comparison Design and Reference Methods . . . 151
6.8.2 Reconfiguration Scenarios, Ablations, and Metrics . . . 152
6.8.3 Temporal Split, Seeds, and Selection Gates . . . 153
6.9 Results and Comparative Analysis . . . 154
6.9.1 Implemented and Verified Behaviour . . . 154
6.9.2 Multiple Assets, Reconfiguration, and Held-Out Compositions . . . 154
6.9.3 Health-Aware Robustness and Local Safety . . . 155
6.9.4 Multi-Agent Coordination and Shared Constraints . . . 156
6.9.5 Scalability and Runtime Cost . . . 156
6.9.6 Ablations, Explainability, and Boundary Conditions . . . 156
6.10 Discussion, Generality, and Contribution to the Research Questions . . . 157
6.11 Chapter Summary . . . 158

**7 EnergAIze Policy Lifecycle: Experiment Orchestration and Distributed Inference 159**
7.1 Deployment Problem and Lifecycle Contract . . . 160
7.2 End-to-End EnergAIze Lifecycle Architecture . . . 160
7.3 Experiment Orchestration on Heterogeneous Compute . . . 162
7.3.1 Job Orchestrator and Persistent Job State . . . 162
7.3.2 Common Worker Agent Contract . . . 162
7.3.3 Execution Target Profiles . . . 163
7.4 Experiment Evidence and Operator Supervision . . . 164
7.5 Policy Packaging and Distributed Inference . . . 165
7.5.1 Validated Bundle Contract . . . 165
7.5.2 energAIze Inference Runtime . . . 165
7.5.3 Inference Targets and Bundle Management . . . 166
7.6 Monitoring, Failure Handling, and Recovery . . . 166
7.7 Systems Evaluation, Availability, and Contribution to RQ4 . . . 167
7.8 Chapter Summary . . . 168

**8 EnergAIze in Operation: Real-World Deployment and Validation 171**
8.1 Evidence Levels and Comparative Reading . . . 171
8.2 OPEVA Demonstrator . . . 172
8.2.1 Control Problem and Physical Configuration . . . 172
8.2.2 EnergAIze Integration Path . . . 173
8.2.3 Scenarios and Validation Strategy . . . 174
8.2.4 Results and Demonstrated Boundary . . . 175

8.3 DemFlex Demonstrator . . . 176
8.4 Cross-Demonstrator Lessons and Simulation-to-Operation Differences . . . 176
8.5 Contribution to the Research Questions and Chapter Summary . . . 177

**9 General Discussion and Answers to the Research Questions 179**
9.1 What the Combined Evidence Establishes . . . 180
9.2 Answer to RQ1: Dynamic and Heterogeneous Multi-Agent Control . . . 181
9.3 Answer to RQ2: Multi-Dimensional Scalability of Learning and Coordination . . . 182
9.4 Answer to RQ3: Safe and Robust Operation . . . 183
9.5 Answer to RQ4: Governed and Deployable MARL Lifecycle . . . 184
9.6 Interactions among the Four Requirements . . . 185
9.7 Scientific Contributions, Generality, and Responsible Use . . . 186
9.8 Limitations and Threats to Validity . . . 187
9.9 Implications and Open Research Problems . . . 188

**10 Conclusions and Future Research 189**
10.1 What the Research Established . . . 189
10.2 Answers to the Research Questions . . . 191
10.3 Principal Contributions . . . 192
10.4 Assessment of the Central Proposition . . . 192
10.5 Limits of the Conclusions . . . 193
10.6 Future Research Directions . . . 194
10.7 Closing Remarks . . . 195

**References 197**

**A Reproducibility and Supplementary Material 207**
A.1 Claim and Evidence Inventory . . . 207
A.2 Scenario and Environment Manifests . . . 207
A.3 Canonical Annual Dataset Contract . . . 207
A.3.1 Calendar, price forecasts, and settlement . . . 208
A.3.2 Demand, PV, and stationary storage . . . 208
A.3.3 Electrical connections and phase allocation . . . 209
A.3.4 EVs, sessions, and deferrable services . . . 210
A.3.5 Topology and robustness calendars . . . 212
A.3.6 Matched annual baseline scorecard . . . 214
A.4 Simulator Audits and Demonstrative Results . . . 215
A.4.1 Software and contract verification . . . 216
A.4.2 Selected capability demonstrations . . . 216
A.4.3 Performance and scalability smoke benchmarks . . . 217
A.5 Simulator KPI Catalogue . . . 218
A.6 Controller, TI-RL, and TI-MARL Configurations . . . 219
A.6.1 Executable Controller Inventory . . . 219
A.6.2 Observation, Action, and Training Record . . . 221
A.6.3 TI-MARL Reproducibility Contract . . . 221
A.6.4 Reward Profile Ledger . . . 223
A.7 Hyperparameters, Seeds, and Statistical Procedures . . . 225
A.8 Supplementary and Negative Results . . . 225

A.9 Lifecycle and Deployment Records . . . . . 226
A.10 Software, Data, and Archival Record . . . . . 226

# List of Figures


1.1 Renewable Energy Community as the principal real-world cyber-physical-system instantiation used in the thesis. . . . . 4

2.1 Taxonomy of dynamicity and reconfiguration used throughout the thesis. . . . . 19

3.1 Architectural View I: the smallest complete EnergAIze decision loop. Heterogeneous information is converted into a coherent state, a controller proposes an action, supervision preserves physical and user constraints, and the external platform reports what was accepted and executed. . . . . 58

3.2 Architectural View II: heterogeneous installation instances participate in one community without being reduced to an identical asset model. The wider scope exposes population, interface, coordination, computing, deployment, shared-safety, and reconfiguration pressures. . . . . 60

3.3 Architectural View III: functional EnergAIze components and interfaces for one installation. Provider integration, predictive context, controller execution, safety supervision, experimental development, and the policy lifecycle remain distinct while contributing to one closed control loop. . . . . 61

4.1 CityLearn v3 representation of one or more Renewable Energy Communities. Each community retains its buildings, assets, controllers, local exchange, events, and KPIs; the enclosing environment synchronises communities and derives combined results. Adapted from the CityLearn v3 simulator manuscript developed alongside this thesis. . . . . 76

4.2 Peak visibility when the same physical trace is represented at one-hour, 15-minute, and 15-second resolution. Annual energy is preserved; the observable short-duration peak is not. . . . . 79

4.3 A deferrable service with an admissible start window, fixed cycle profile, and completion deadline. Cost or peak reduction is not credited as successful service when the required cycle is omitted or completed late. . . . . 82

4.4 Annual local-energy and settlement accounting for one unchanged controller trajectory. The comparison isolates the configured accounting rule and retains heterogeneous member outcomes. . . . . 83

4.5 Lifecycle of members and assets in one dynamic-topology scenario. The event log, active entity counts, structured observations, relations, and action rows change at the same simulation steps. . . . . 84

4.6 Failure channels in the controller loop. Measurement and forecast events alter controller-visible information; command dropout and equipment unavailability create traceable differences between requested and applied actions. . . . . 85

5.1 Progression of the controller studies. Each level introduces one additional control problem while retaining earlier methods as references, teachers, or fallbacks. . . 98
5.2 Hierarchical community coordination. The coordinator changes a compact signal; local controllers retain asset-level decisions and service protection. The dashed return path carries community aggregates and outcomes. . . . . 125

6.1 Implemented TI-MARL architecture. Each active member is compiled into an entity-bound snapshot and evaluated by the same typed actor. The raw hybrid bundle is projected against that member's service and electrical constraints before command encoding. TI-MAPPO uses the active set of member summaries only during training. . . . . 136

7.1 Lifecycle entities and provenance relations connecting an experiment specification to the policy bundle and its observed deployment. . . . . 161
7.2 Implemented EnergAIze lifecycle architecture. Experiment orchestration and heterogeneous execution produce traceable evidence and a policy bundle; deployment services catalogue and load the bundle into an observable inference target connected to the wider EnergAIze control path. . . . . 161
7.3 Implemented job states. Recovery after successful remote Union computation is represented separately from rerunning the experiment, while cancellation, cooperative stop, and failure remain explicit terminal outcomes. . . . . 162
7.4 Implemented bundle management and inference activation sequence. Bundle selection is explicit and observable for each configured target; coordinated fleet-wide activation remains a separate lifecycle requirement. . . . . 166

8.1 Validation evidence levels and the claims supported by each level. . . . . 172
8.2 OPEVA Demo 9 sites, assets, platforms, and EnergAIze integration path. . . . . 174
8.3 Timeline of one audited OPEVA control or integration period. . . . . 176
8.4 Observed differences between simulation assumptions and demonstrator conditions. 177
8.5 Verified OPEVA and DemFlex results by evidence level. . . . . 178

9.1 Research-question, contribution, finding, and evidence map. . . . . 185
9.2 Interactions among scalability, deployability, safety, and robustness. . . . . 186

10.1 Contribution stack and cumulative thesis argument. . . . . 193

# List of Tables


2.1 Examples of recurring properties across distributed CPS domains. . . . . 17
2.2 Three layers of heterogeneity in REC control. . . . . 22
2.3 Operational classification of dynamics in REC and EV control. . . . . 24
2.4 Mapping from renewable-energy-community characteristics to general CPS properties. . . . . 28
2.5 Evidence-backed comparison of controller families in REC and EV control. . . . 33
2.6 Recurring MARL challenges and their manifestation in energy communities. . . 41
2.7 Evidence levels used throughout the thesis for real-world-oriented control studies. 42
2.8 Overly broad claims, counterevidence, and defensible replacements. . . . . 43
2.9 Representative MARL mechanisms for varying team size, agent loss, or changing interaction graphs. . . . . 45
2.10 Lifecycle capabilities that should not be conflated in a MARL deployment. . . . . 49
2.11 Residual requirements derived from the state of the art and their connection to the research questions. . . . . 53

3.1 Principal EnergAIze components and their interfaces within the control loop. . . 64
3.2 Traceability between the four cross-cutting requirements, EnergAIze mechanisms, and the detailed thesis chapters. . . . . 66

4.1 Requirements used to develop and evaluate the extended simulation environment. 74
4.2 Role-based positioning against representative energy-control environments. . . . 75
4.3 Functional layers of CityLearn v3 and their role in an auditable controller experiment. 77
4.4 Installed composition of the canonical 2023 annual dataset suite. Counts are fixed in the full catalogue; the active population and available equipment can change in Dynamic and AllIn scenarios. . . . . 89
4.5 Matched Core-30 variants and the experimental question isolated by each schema. All variants reference the same annual physical time series and asset identities. . 91
4.6 Seventeen headline indicators in the core controller scorecard. . . . . 94

5.1 Controller and learner families implemented in the Algorithms repository and their role in this chapter. . . . . 100
5.2 Comparison surfaces used to preserve information and objective fairness. . . . . 102
5.3 Role of the annual dataset suite in the controller campaign. . . . . 104
5.4 Implemented entity-encoding profiles and their experimental use. . . . . 106
5.5 Main stages in the implemented reward lineage. Exact parameters and registered profile names are retained in Appendix A.6. . . . . 108
5.6 Transparent controller ladder used in the experiments. . . . . 110
5.7 MILP reference scopes and permitted interpretation. . . . . 112

5.8 MARL comparator families and the hypothesis tested by each one. . . . . 119
5.9 Scientific role and remaining boundary of each controller stage. . . . . 128

6.1 TI-MARL as a cumulative extension of the controller progression in Chapter 5. . 133
6.2 Main TI-MARL components and their interpretation in the running energy example. 137
6.3 Representative typed observation parts for a building with two chargers. Values are illustrative. . . . . 139
6.4 Representative action ports for two charger instances and one stationary battery. . 140
6.5 Structural baselines required to isolate the TI-MARL contribution. . . . . 152
6.6 Verified TI-MARL implementation evidence in the audited repository state. . . . 155

7.1 Heterogeneous EnergAIze worker targets and target-specific execution requirements. 163

8.1 OPEVA Demo 9 scenarios and the evidence boundary reported in deliverable D6.3. 175

9.1 Role of the principal evidence layers in the thesis argument. . . . . 181

A.1 Static-asset generation contract by member class. PV and BESS entries apply only to members that own the corresponding asset. . . . . 209
A.2 Routine archetypes used to construct heterogeneous demand and flexibility. . . . 209
A.3 Electrical-service and charger contract. Charger powers are compatible with the charging classes described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b). . . . . 210
A.4 Generated EV identities, annual session counts, and topology-observable departure obligations. A censored session is absent from the controller's service denominator because its member or physical charger does not exist at the relevant boundary. . 211
A.5 Core-30 dynamic topology calendar. Paired dates denote removal and restoration. 213
A.6 Premium-100 dynamic topology calendar. Member ranges denote simultaneous first-entry cohorts. . . . . 213
A.7 Annual matched cost and service scorecard. Paired entries are BAU/RBC Smart. . 214
A.8 Annual safety, demand-response, and robustness scorecard for the stress variants. Paired entries are BAU/RBC Smart. . . . . 215
A.9 Verification and validation evidence reported for the simulator manuscript checkout. 216
A.10 Selected application evidence and the bounded claim supported by each example. 217
A.11 Computational-cost evidence from matched simulator configurations. . . . . 217
A.12 Expanded CityLearn v3 KPI catalogue. . . . . 218
A.13 Executable Algorithms registry grouped by scientific role. . . . . 220
A.14 TI-MARL v1 configuration and evidence fields retained for reproduction. . . . . 222
A.15 Implemented Chapter 5 reward-profile lineage. . . . . 223

# List of Acronyms

**AC** Alternating Current

**AI** Artificial Intelligence

**API** Application Programming Interface

**BC** Behaviour Cloning

**BESS** Battery Energy Storage System

**BOPTEST** Building Optimization Testing Framework

**CPS** Cyber-Physical System

**CPU** Central Processing Unit

**CSV** Comma-Separated Values

**CTDE** Centralised Training with Decentralised Execution

**DC** Direct Current

**Dec-POMDP** Decentralised Partially Observable Markov Decision Process

**DER** Distributed Energy Resource

**DR** Demand Response

**EMS** Energy Management System

**EV** Electric Vehicle

**EVSE** Electric Vehicle Supply Equipment

**FMI** Functional Mock-up Interface

**G2V** Grid-to-Vehicle

**GDPR** General Data Protection Regulation

**GNN** Graph Neural Network

**GPU** Graphics Processing Unit

**HELICS** Hierarchical Engine for Large-scale Infrastructure Co-Simulation

**HIL** Hardware-in-the-Loop

**HPC** High-Performance Computing

**HRL** Hierarchical Reinforcement Learning

**HVAC** Heating, Ventilation, and Air Conditioning

**IoT** Internet of Things

**KPI** Key Performance Indicator

**LSTM** Long Short-Term Memory

**MADDPG** Multi-Agent Deep Deterministic Policy Gradient

**MADRL** Multi-Agent Deep Reinforcement Learning

**MAPPO** Multi-Agent Proximal Policy Optimization

**MARL** Multi-Agent Reinforcement Learning

**MAS** Multi-Agent System

**MATD3** Multi-Agent Twin Delayed Deep Deterministic Policy Gradient

**MDP** Markov Decision Process

**MILP** Mixed-Integer Linear Programming

**ML** Machine Learning

**MLOps** Machine Learning Operations

**MPC** Model Predictive Control

**OCPP** Open Charge Point Protocol

**ONNX** Open Neural Network Exchange

**OOD** Out of Distribution

**OPEVA** Optimisation of Electric Vehicle Autonomy

**PER** Prioritized Experience Replay

**POMDP** Partially Observable Markov Decision Process

**PPO** Proximal Policy Optimization

**PV** Photovoltaic

**RAL** Runtime Assurance Layer

**RBC** Rule-Based Controller

**REC** Renewable Energy Community

**REST** Representational State Transfer

**RL** Reinforcement Learning

**RLOps** Reinforcement Learning Operations

**RQ** Research Question

**SAC** Soft Actor-Critic

**SoC** State of Charge

**TCG** Typed Constraint Graph

**TD3** Twin Delayed Deep Deterministic Policy Gradient

**TI-MARL** Typed-Interface Multi-Agent Reinforcement Learning

**TI-MDP** Typed-Interface Markov Decision Process

**TI-RL** Typed-Interface Reinforcement Learning

**TIC** Typed Interface Compiler

**TIMG** Typed-Interface Markov Game

**TIP** Typed-Interface Policy

**ToU** Time of Use

**V2G** Vehicle-to-Grid

**VPP** Virtual Power Plant

# Glossary

**action group** A set of mutually exclusive action ports representing alternative operating modes for one controllable function or entity, while compatible action groups may be controlled simultaneously within the same local action bundle

**action port** A callable typed action exposed by the current agent interface, including its semantic name, parameters, bounds, validity conditions, and any projection or safety rule applied before execution

**action-group instance** A runtime controllable group bound to a specific module or entity instance, such as the operating mode of one particular EV charger; the same shared decoder can be applied to multiple instances of the same group type

**agent** A decision-making entity that receives observations, selects actions, and pursues one or more objectives within an environment. In this thesis, an agent may represent a participant, building, charging installation, or controllable resource, depending on the control setting

**behaviour cloning** Supervised policy learning from state–action demonstrations produced by an expert, rule-based controller, optimiser, or previously deployed policy

**capability schema** A versioned description of the potential observations, action groups, action ports, constraints, compatibility requirements, and fallback semantics that an agent or module type can expose

**community** A set of participants and energy assets coordinated under shared rules or objectives. In the principal case studies, this term refers to a renewable energy community

**compatibility signature** A versioned summary of interface and capability requirements used to determine whether a policy, compiler, schema, and runtime target can operate together safely

**controller** The software or physical component responsible for producing control actions. A controller may implement a rule-based strategy, an optimisation method, or a learned policy

**coordination-aware monitoring** Operational monitoring that detects failures in the relationships among agents and policies, such as incompatible versions, oscillating or conflicting actions, stale messages, unfair shared-resource allocation, or degradation of a community-level objective despite healthy processes

**deployability** The ability to package, validate, install, configure, execute, monitor, update, and recover a control solution or coordinated policy set within its intended operational environment

**dynamic reconfiguration** A change in the composition, availability, capabilities, or interaction structure of a system during or between operating periods, such as an electric vehicle arriving or leaving, an asset becoming unavailable, or a new participant joining a community

**EnergAIze** The integrated research platform and energy-domain reference implementation through which the methods, software components, lifecycle mechanisms, and demonstrators developed in this thesis are connected and evaluated

**energy asset** A physical or virtual energy resource that can be observed or controlled, such as an electric vehicle, charger, battery, photovoltaic system, heat pump, or flexible electrical load

**entity** A runtime object that gives observations or actions a concrete identity and operational meaning, such as a connected electric-vehicle session, charger, battery, or other controlled or observed object within an agent

**entity-bound typed interface** A runtime observation–action interface in which observation parts, action groups, and ports explicitly identify the module and controlled entity that give them meaning

**experiment manifest** An immutable or content-addressed record that links an experiment to its source revision, environment, scenario and data, reward, observations and actions, constraints, seeds, algorithm configuration, dependencies, resources, and resulting artefacts

**fail-safe mode** A controlled operating state in which a learned controller is constrained, bypassed, or replaced by a conservative controller because data, communication, policy compatibility, confidence, or safety conditions are not sufficient for normal operation

**health closure** The deterministic propagation of source or module health through declared dependencies to observation validity, action-port availability, parameter bounds, action groups, constraints, and operational mode

**interface snapshot** An immutable record of the observation parts, action ports, dependencies, validity states, bounds, health, and resource couplings exposed to an agent at one decision step

**local action bundle** The structured action produced by one local typed-interface policy, containing one selected port and its parameters for each active action-group instance after local hard-feasibility construction

**local feasibility** The construction or adjustment of an agent's local action bundle so that simultaneous actions remain within non-negotiable local physical, device, and service constraints while preserving policy intent as far as possible

**module instance** A concrete runtime instance of a capability-bearing component associated with an agent, such as one charger, battery, or controllable subsystem; module instances may appear, disappear, or change availability without redefining the complete policy architecture

**observation part** A typed block of information exposed to a policy, such as self state, scanner status, battery state of charge, deadline, entity relation, or measurement-quality indicator

**offline reinforcement learning** Reinforcement learning from a fixed dataset of previously collected transitions or trajectories without additional environment interaction during the offline training stage

**physics-informed reinforcement learning** A family of approaches that incorporates physical models, invariants, residuals, constraints, or model-based structure into policy learning, value estimation, action validation, or training objectives

**policy** A mapping from the information available to an agent to the action selected by that agent. A policy may be rule-based, optimisation-based, or learned

**policy registry** A lifecycle service that records policy identity, versions, provenance, evaluation evidence, compatibility metadata, promotion state, deployment associations, and rollback lineage, rather than storing only model bytes

**policy set** A governed collection of interdependent policies intended to operate together, including their agent-role mappings, observation and action schemas, communication assumptions, safety dependencies, compatibility rules, and shared deployment version

**real-world operation** Execution under physical, computational, communication, organisational, and lifecycle constraints that are not fully represented by a conventional isolated simulation experiment

**resilience** The ability of a system to recover after a fault, disruption, or period of degraded operation

**robustness** The ability to maintain acceptable behaviour and performance under uncertainty, disturbances, faults, imperfect data, dynamic reconfiguration, or differences between training and operation

**runtime health state** The current quality and availability state of observations, modules, communication channels, or controlled entities used by the Typed Interface Compiler to determine validity, bounds, degraded modes, and fallback behaviour

**safety** The prevention of unacceptable actions or system states through the satisfaction of physical, operational, and user-related constraints

**scalability** The ability of an algorithm, simulator, infrastructure, or deployed system to accommodate increasing numbers of agents, assets, interactions, experiments, or devices without unacceptable loss of performance or excessive resource use

**shared hard-constraint monitor** An exceptional runtime assurance mechanism that detects and, when necessary, minimally corrects or rejects joint actions that would violate non-negotiable constraints shared across agents; it is not the normal community optimisation mechanism

**transferability** The extent to which knowledge, representations, or policies learned in one setting can be reused or adapted in a different agent, configuration, scenario, or environment

**typed constraint graph** A runtime graph connecting agents, modules, entities, action ports, resources, topology, and local or shared constraints through typed relations; it supports local bundle feasibility, training-time graph critics, hard shared checks, compatibility, and decision traces

**typed decision trace** An execution record grounded in the actual interface snapshot, including active entities, observations and health, available and invalid ports, selected actions, local feasibility adjustments, shared safety interventions, and final commands

**typed interface** A runtime contract that describes an agent's available observation parts, action ports, parameter types and bounds, validity conditions, capabilities, and constraints rather than assuming one permanently fixed flat observation and action vector

# Chapter 1

# Introduction

This dissertation was developed within the Doctoral Programme in Informatics Engineering at the Faculty of Engineering of the University of Porto. It investigates the development and operation of MARL-based control for distributed, heterogeneous, and dynamically reconfigurable CPSs, using smart-energy systems as the principal empirical domain.

This chapter introduces the context that motivates the research, establishes the transition from simulated learning to operational control, states the central thesis proposition and research questions, defines the scope of the work, summarises its main contributions, and presents the research path reflected in the organisation of the dissertation.

## 1.1 Context and Motivation

The research begins from a class of control problems in which decision-making is distributed across physical assets, software components, and participants whose capabilities and availability may change over time. The following sections move from this broader cyber-physical setting to Renewable Energy Communities and, finally, to the role of multi-agent reinforcement learning in decentralised control.

### 1.1.1 Distributed and Dynamically Reconfigurable Cyber-Physical Systems

A CPS combines computation with physical processes. Sensors provide information about the physical environment, software interprets that information and decides what to do, and actuators change the physical process, creating a feedback loop between computation and the world. This connection is what distinguishes a CPS from a conventional software system: a software error is no longer confined to incorrect information on a screen; it can also lead to an incorrect physical action. Early work on CPSs already emphasised that physical dynamics, timing, safety, and reliability require abstractions that go beyond those used in general-purpose computing (Lee 2008).

Many contemporary CPSs are distributed. A factory includes machines, local controllers, sensors, and supervisory services. A fleet-management system coordinates vehicles that move in

and out of communication range. A group of robots may share a mission while retaining local sensing and control. An energy community connects buildings, meters, renewable generation, batteries, charging stations, and external platforms. In each case, information and decision authority are spread across several locations; no single controller has a complete view or exclusive ability to act.

Distribution changes the control problem in several ways. Each part of the system may observe only a local portion of the environment; communication introduces delays, bandwidth limits, and possible failures; available computation may range from cloud servers to embedded devices; and local decisions can have system-wide consequences. Local control can react quickly and preserve autonomy at the cost of a restricted view. Central control has access to broader context, with greater communication demands, scaling pressure, and exposure to a single point of failure.

The system may also be dynamically reconfigurable. In this thesis, dynamic reconfiguration refers to changes in participating agents, available assets, capabilities, or interaction structure. Consider an electric vehicle connected to a household charger. Before the vehicle arrives, the charger has no energy request to satisfy. During charging, the vehicle's state, required departure charge, and available power become relevant to control. Once the vehicle leaves, that particular flexibility disappears even though the physical charger remains. Similar changes occur when a robot joins a mission, a machine is taken offline, a device fails, or a communication link becomes unavailable.

Such changes are different from ordinary variation in the value of a fixed observation. A temperature changing from 20 °C to 21 °C changes the state of an otherwise stable interface. A device or participant joining or leaving can instead change which observations exist, which actions are meaningful, which constraints apply, and which objectives can still be achieved. Control methods built around a fixed and homogeneous population may perform well in a stable benchmark yet require substantial adaptation when the configuration changes during operation.

Centralised optimisation remains a strong option when the system is sufficiently well characterised and the corresponding optimisation can be completed within the available decision time. Decentralised or hierarchical decision-making becomes increasingly relevant as configurations change, local information cannot be shared continuously, and communication or computation constrain central control. The central question is how control can remain coherent when information, capabilities, and decision authority are distributed across both local and shared mechanisms.

### 1.1.2 Renewable Energy Communities as Representative Cyber-Physical Systems

The energy transition provides a concrete instance of these challenges. Electricity systems are incorporating increasing numbers of distributed and renewable resources, while electrification introduces flexible loads such as heat pumps and electric vehicles. Flexibility—the ability to shift production, consumption, storage, or exchange over time—is increasingly important for accommodating uncertainty and aligning demand with available renewable energy (Hussain et al. 2023).

European legislation defines a REC as a legal entity based on open and voluntary participation, effectively controlled by members located near the renewable-energy projects it owns or develops, and primarily intended to provide environmental, economic, or social community benefits rather than financial profit (European Parliament and Council of the European Union 2018). The control setting considered in this thesis is more concrete: a community may contain participants who consume and produce electricity and who expose different forms of flexibility through buildings, batteries, electric vehicles, chargers, heating systems, and other controllable loads.

A single participant may already combine assets with very different information and control interfaces. A battery can charge or discharge subject to power and SoC limits. A charger can exchange energy only while a compatible vehicle is connected and must account for the service expected at departure. A heat pump is coupled to thermal dynamics and comfort. These differences are not implementation details: they determine the observations, actions, constraints, and temporal dependencies of the control problem.

Time couples those decisions. Charging a battery now affects what it can do later. Delaying an electric-vehicle charge creates flexibility only until the vehicle must depart. Reducing heating demand in one interval may create a larger requirement later. Energy management is a sequential decision problem: the value of an action depends on its immediate effect and on the options it preserves for the future. This is one reason why RL has attracted sustained interest in demand response and building control (Vázquez-Canteli and Nagy 2019).

At community level, objectives are also mixed. A participant may seek lower cost or guaranteed service, while the community may seek lower collective peaks, increased use of local renewable production, or adherence to a shared infrastructure limit. These objectives can align, but they are not identical. Several participants may benefit from coordination while simultaneously competing for a limited resource. This produces a mixed cooperative–competitive setting.

The same community can reconfigure during operation. Vehicles connect and disconnect, devices become unavailable, participants can join or leave, and external signals can change the value of different actions. Renewable Energy Communities bring distributed control, heterogeneous capabilities, changing availability, mixed objectives, and shared physical constraints into one empirical setting. These properties also arise in other CPSs, including robotics, transport, and industrial systems, although the empirical claims of this thesis remain grounded in the energy systems that are actually evaluated.

### 1.1.3 MARL for Mixed Cooperative–Competitive Decentralised Control

Rule-based and optimisation-based controllers remain important in energy management. A rule-based controller can encode transparent operational behaviour, while an optimisation model can schedule several assets from forecasts, explicit constraints, and models of their dynamics. These approaches provide strong references and, in some cases, appropriate operational fallbacks. Learning-based control becomes attractive when the decision strategy is difficult to specify completely in

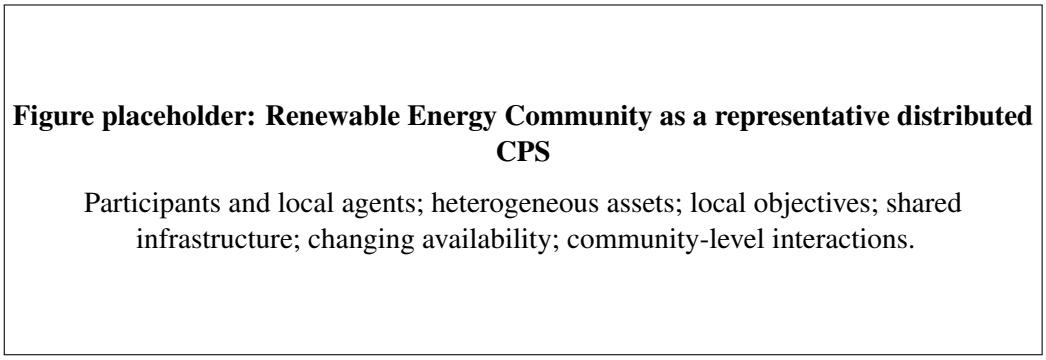



Figure 1.1: Renewable Energy Community as the principal real-world cyber-physical-system instantiation used in the thesis.

advance, when repeated interaction provides useful experience, or when decentralised decision makers must adapt to a changing environment.

RL formulates sequential control as a learning problem: an agent observes the environment, selects an action, receives feedback, and improves its policy through experience. The observations, actions, rewards, constraints, and training environment still have to be designed; learning does not remove the need for a sound control formulation. Its value lies in learning useful decision behaviour without encoding the complete strategy as a fixed rule set or solving a full optimisation problem at every decision step.

MARL extends this setting to several agents interacting in a shared environment. It includes cooperative, competitive, and mixed formulations as well as different assumptions about information, communication, and training (Albrecht et al. 2024; Gronauer and Diepold 2022). A common approach is CTDE, in which broader information can be used during training while each agent executes from its authorised local observations (Lowe et al. 2017). This is particularly relevant when operational control is distributed but learning can exploit richer information offline.

The mixed cooperative–competitive character is central to the energy setting. Two households with electric vehicles may share an interest in respecting a community capacity limit while each still prioritises the service required by its own vehicle. The agents must coordinate around shared outcomes without collapsing all participant objectives into one indistinguishable objective.

MARL introduces additional difficulties of its own. Other learning agents make the environment appear non-stationary from the perspective of an individual learner; joint decision spaces grow with the population; heterogeneous agents may expose different observations and actions; and communication or coordination can create additional computational and operational dependencies (Gronauer and Diepold 2022; Hernandez-Leal et al. 2019). These difficulties become more pronounced when the physical system itself also changes. The learning algorithm is one part of the wider control system considered in this thesis, alongside its interfaces, physical constraints, runtime infrastructure, and evidence.

## 1.2 Requirements for MARL in Real-World CPS

Simulation is indispensable for training and evaluating learning-based control. It enables repeated experiments, controlled comparisons, and the deliberate creation of disturbances that would be expensive, unsafe, or impractical to reproduce in a physical installation. At the same time, a simulator can provide cleaner observations, more stable configurations, and more computational resources than will exist during operation. Real-world use requires evidence that reaches beyond learning performance under those controlled conditions.

This thesis organises that transition around four cross-cutting requirements: scalability, deployability, safety, and robustness. These four requirements define the dimensions investigated explicitly across the architecture, algorithms, experimentation, lifecycle, and operational evidence developed in the thesis; they are not intended as a complete taxonomy of the challenges associated with real-world intelligent control.

**Scalability.** Scalability concerns more than increasing the number of agents. It includes the computational cost of learning and inference, growth in heterogeneous observations and actions, simulator throughput, communication and coordination overhead, the reproducible execution of many experiments, and the resources required by deployed components. A method that scales in one dimension can still fail in another; for example, parameter sharing can reduce model count while communication or critic cost continues to grow. Relevant evidence identifies the dimension that increased, the resources consumed, and the control quality maintained at that scale.

**Deployability.** A trained policy becomes an operational controller only when its experimental assumptions and runtime dependencies remain intact. It must stay associated with the simulator, scenario, interface, reward, and software versions under which it was produced; it must be validated for the intended target; and it must be packaged, installed, monitored, updated, and recoverable. In a distributed system, these concerns apply to compatible sets of runtime components, with no model file considered in isolation. Deployability in this thesis covers the controlled path from an experimental result to continued operation on heterogeneous edge and server-side infrastructure.

**Safety.** Physical control imposes limits that cannot be traded away for a higher reward. Batteries, chargers, buildings, and network connections have operating constraints, while users impose service requirements such as departure charge or comfort. Safety is used here in this operational sense: preventing unacceptable actions and states through the control formulation, validation, runtime constraints, and fallback mechanisms appropriate to each stage. Cybersecurity is important to real infrastructure but is not used as a synonym for safety in this thesis.

**Robustness.** Operational conditions do not remain identical to training conditions. Forecasts are imperfect, observations can be delayed or missing, devices can become unavailable, communication can fail, and the population or control interface can change. Robustness concerns whether useful

control behaviour is maintained under these deviations and how performance degrades when assumptions no longer hold. Recovery after a disruption is treated as a related resilience concern and is addressed particularly through lifecycle and runtime mechanisms.

These requirements interact. More communication can improve coordination while increasing dependence on connectivity; a larger policy can improve control while exceeding an edge latency budget; conservative safety mechanisms can reduce violations while limiting performance; and frequent policy updates can support adaptation while increasing validation and operational risk. The thesis evaluates the four requirements jointly where their interactions matter and separately where causal attribution is needed.

The detailed state-of-the-art analysis in Chapter 2 examines how these challenges have been addressed in prior work and identifies the residual gaps that motivate the contributions developed in the remainder of the thesis. Other concerns are also important to real-world CPSs, including cybersecurity and privacy, human factors and interpretability, regulatory and market constraints, maintainability, and long-term organisational adoption. They are considered where they constrain the proposed methods or their operation, but they are not investigated directly as primary dimensions of this thesis.

## 1.3 Thesis Proposition, Research Questions, and Scope

The research lies at the intersection of artificial intelligence, distributed systems, cyber-physical systems, edge computing, and intelligent energy control. Its central concern is computational: how distributed controllers can be represented, learned, evaluated, and operated when the physical and software interfaces through which they act are heterogeneous and can change. Energy is not treated as an interchangeable benchmark; asset dynamics, user requirements, shared infrastructure, and operational interfaces determine which decisions are meaningful and which evidence is required.

### 1.3.1 Thesis Proposition

The central proposition evaluated by this dissertation is:

> *Operationalising Multi-Agent Reinforcement Learning in distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems requires the integrated co-design of suitable system representations and simulation environments, scalable learning and coordination mechanisms, safe and robust control processes, and lifecycle-aware distributed deployment infrastructure.*

The proposition places the learning method within the system that makes its claims meaningful. A controller is trained against a representation of the physical system, evaluated under explicit assumptions, executed through concrete observation and action interfaces, and maintained through an operational lifecycle. The thesis investigates these elements together while evaluating the individual claims in the chapters where the corresponding evidence is produced.

### 1.3.2 Research Questions

Four research questions structure the investigation.

**RQ1 – Dynamic and Heterogeneous Multi-Agent Control.**

> *How can MARL-based control be formulated to accommodate heterogeneous and dynamically changing agent populations, capabilities, and availability, while balancing individual and collective objectives in distributed cyber-physical systems?*

RQ1 concerns the representation and learning problem when agents and the entities they control are not assumed to expose one permanently fixed interface. It also addresses the coexistence of local and collective objectives under decentralised control.

**RQ2 – Multi-Dimensional Scalability of Learning and Coordination.**

> *Under which conditions, and through which algorithmic, computational, and communication mechanisms, can MARL-based learning and coordination maintain acceptable control performance and resource use as agent population size, heterogeneity, environment complexity, and dynamic reconfiguration increase, while computational and communication resources remain constrained?*

RQ2 treats scale as a conditional, multi-dimensional property. It considers population size, heterogeneity, representation size, simulation and training cost, communication, inference, orchestration, and the resources available at deployment time.

**RQ3 – Safe and Robust Operation.**

> *How can physical and operational safety constraints, uncertainty, faults, dynamic reconfiguration, and shifts between training and operation be incorporated into and systematically assessed for MARL-based control?*

RQ3 addresses both the prevention of unacceptable control actions and the maintenance of useful behaviour when observations, forecasts, devices, communication, or system configuration deviate from nominal conditions.

**RQ4 – Governed and Deployable MARL Policy-Set Lifecycle.**

> *What lifecycle abstractions and orchestration mechanisms allow coordinated MARL policy sets to be reproduced, governed, and deployed across heterogeneous and dynamically reconfigurable edge cyber-physical systems, while supporting compatibility validation, monitoring, staged updates, rollback, and failure recovery?*

RQ4 follows the controller beyond training. It addresses experiment and environment lineage, compatibility between policies and runtime interfaces, validation and promotion, distributed deployment, monitoring, controlled updates, rollback, and recovery.

### 1.3.3 Research Objectives

The research questions are pursued through six objectives that connect the scientific argument to the experimental and systems work developed throughout the dissertation.

**O1 – Characterise and formalise the control setting.**
Establish the requirements of distributed, heterogeneous, and dynamically reconfigurable cyber-physical control, including mixed local and collective objectives and the four cross-cutting dimensions of scalability, deployability, safety, and robustness.

**O2 – Establish the simulation and experimental foundation.**
Extend and validate a reproducible environment capable of representing heterogeneous energy assets, changing participants and capabilities, physical constraints, disturbances, and the scales required for controlled learning experiments.

**O3 – Develop and compare progressive control and learning solutions.**
Design, implement, and evaluate rule-based, optimisation-based, reinforcement-learning, and multi-agent formulations under common scenarios and metrics, identifying what each approach solves and the assumptions under which it remains effective.

**O4 – Develop and evaluate TI-MARL.**
Design a multi-agent method that operates over entity-bound typed interfaces whose observations, action capabilities, bounds, and health can change at runtime, and assess its behaviour under heterogeneity, reconfiguration, local feasibility constraints, faults, and scale.

**O5 – Develop the governed policy lifecycle and distributed runtime.**
Define and evaluate the lineage, compatibility, validation, packaging, deployment, monitoring, update, rollback, and recovery mechanisms required to operate learned policies and policy sets across heterogeneous edge infrastructure.

**O6 – Integrate and validate the resulting approach in the energy domain.**
Connect the architectural, simulation, control, learning, and lifecycle contributions within EnergAIze and evaluate their interactions using Renewable Energy Communities, electric-vehicle charging, real data, external platforms, edge execution, and physical demonstrators at the evidence levels actually available.

### 1.3.4 Scope and Assumptions

The methods and system abstractions are developed for distributed and reconfigurable CPSs, while the empirical evidence is concentrated in smart-energy systems: Renewable Energy Communities, flexible buildings, electric vehicles, charging installations, and related energy services. Examples from robotics, transport, or industrial systems clarify the broader class of problems but do not

constitute equivalent empirical validation. Generality concerns the formulated mechanisms and abstractions; measured performance remains specific to the evaluated energy configurations.

The thesis focuses on sequential operational control; long-term infrastructure planning and market design remain outside its central scope. Rule-based, optimisation-based, single-agent, and multi-agent controllers are all retained when they provide useful baselines, references, fallbacks, or solutions to particular subproblems. Simulation is used for training, controlled comparison, scaling, and disturbance injection, while real data, external platforms, edge devices, charging infrastructure, testbeds, and physical demonstrators provide additional evidence where available. The evidence classification keeps real-data simulation distinct from direct physical control.

The work assumes that observations can be obtained through an operational interface and that admissible actions can be applied to the controlled system. Lower-level device protections remain active. The thesis evaluates selected physical and operational constraints, disturbances, failures, and reconfiguration events, but it does not claim universal formal safety certification. Security, privacy, regulation, and human factors are considered where they affect the architecture or operation, but they are not primary research questions unless supported by dedicated methods and evidence.

## 1.4 Research Contributions and Outcomes

The thesis contributes scientific methods and findings together with the software and experimental infrastructure required to realise and evaluate them. The publications and research outputs associated with these contributions are listed in the front matter; detailed provenance is recorded in the chapters where collaborative work contributes directly to the corresponding evidence.

### 1.4.1 Scientific Contributions

The scientific contributions form seven related families. TI-MARL is the principal algorithmic contribution, while the remaining contributions establish the empirical and systems foundations, solve complementary control problems, and provide the evidence needed to evaluate the complete thesis proposition.

**C1 – Evidence-based characterisation of operational MARL requirements.** The thesis develops a structured account of the conditions that separate conventional simulated MARL experiments from operational distributed CPSs. The literature analysis identifies existing solutions and counterexamples to simple absence claims, narrowing the residual challenge to the combined treatment of dynamic heterogeneity, multi-dimensional scalability, safe and robust operation, and a governed deployment lifecycle.

**C2 – The EnergAIze integrated research architecture.** The thesis defines an architecture connecting data and scenario preparation, simulation, controller development, reproducible experimentation, policy governance, edge deployment, runtime monitoring, fallback, and recovery. Its

contribution is the explicit organisation of responsibilities and contracts needed to study the path from a control problem to continued operation within one coherent research system.

**C3 – A physically audited simulation and experimentation environment.** The CityLearn-derived environment is extended with heterogeneous assets, electric vehicles and chargers, electrical and community mechanisms, dynamic topology, entity-structured observations and actions, operational events, data and unit contracts, diagnostics, reproducible scenarios, and computational performance evidence. The contribution lies in their verified combination, in the class of experiments made possible by that combination, and in the continuity of evidence between training and deployment. The same versioned environment, scenario, interface, constraint, and evaluation contracts support experiment reconstruction, compatibility validation, policy promotion, packaging, and incident replay throughout the deployment lifecycle.

**C4 – Progressive control and learning solutions.** The thesis develops and evaluates a progression of rule-based, optimisation-based, single-agent, and multi-agent control methods together with mechanisms for more stable, data-efficient, reusable, and hierarchical learning. Each stage addresses a concrete control problem and contributes empirical evidence about the assumptions under which it remains effective. These results provide both independent contributions and the design evidence from which the requirements of TI-MARL are derived.

**C5 – TI-MARL as the principal algorithmic contribution.** TI-MARL is an optimizer-agnostic multi-agent learning method for systems in which the active agents, controlled entities, and observation–action interfaces can change. Versioned capability information and runtime health are compiled into entity-bound typed interfaces containing the observation parts, action-group instances, parameterised action ports, dependencies, validity conditions, and bounds currently available to each agent. A decentralised typed policy operates over this variable interface and produces one coherent local action bundle, while multi-agent coordination is learned through authorised shared observations, coupled outcomes, the joint physical transition, and variable-population central critics during training. The method is evaluated through controlled reconfiguration, health and disturbance tests, structural baselines, ablations, and multi-dimensional scaling.

**C6 – A governed lifecycle for MARL policies and policy sets.** The thesis develops lifecycle abstractions linking experiments, environments, scenarios, interfaces, reward definitions, policies, compatible policy sets, validation states, packages, deployment targets, and runtime observations. The contribution focuses on lineage, compatibility-aware validation and promotion, distributed deployment, monitoring, staged updates, rollback, and recovery in multi-agent operational settings.

**C7 – Integrated real-world-oriented evidence in energy systems.** The preceding contributions are connected through real data, external platforms, edge devices, charging infrastructure, energy

communities, and fail-safe operation. The resulting evidence captures effects that do not appear in isolated learning experiments, including interface mismatches, runtime resource constraints, data-quality failures, deployment interventions, and the limits of the achieved operational evidence.

### 1.4.2 System, Software, and Open-Source Contributions

The scientific work is supported by a set of interconnected research artefacts whose energy-domain integration is realised through EnergAIze. These include data-collection and validation components; scenario and configuration tools; the extended simulation environment; rule-based, optimisation-based, single-agent, and multi-agent controllers; TI-MARL components; experiment workers and orchestration services; manifests and registries for experiments, policies, policy sets, packages, and deployment targets; validation and deployment tooling; edge-runtime components; monitoring, fallback, rollback, and recovery mechanisms; and reproducible configurations, documentation, and open-source releases where licensing permits.

These artefacts are not treated as substitutes for scientific evidence. They provide the implementation through which the corresponding claims can be tested, reproduced, integrated, and, where appropriate, transferred to operational infrastructure. Repository, release, licence, configuration, and experiment identifiers are retained for the submitted version, with access limitations stated where partner or data restrictions apply.

### 1.4.3 Societal and Environmental Relevance

Renewable Energy Communities aim to give participants a more active role in the production, use, and sharing of energy. Intelligent control can help use local flexibility without requiring users to coordinate every flexible asset manually, while community-level objectives can support local renewable-energy use and reduce simultaneous peaks. The technical results of this thesis inform wider questions of flexibility, electrification, and distributed energy participation.

The thesis nevertheless distinguishes measured technical outcomes from broader societal claims. Lower simulated cost does not establish long-term adoption, peak reduction in a testbed does not establish system-wide emissions impact, and edge execution does not by itself establish a complete privacy guarantee. Societal and environmental implications are discussed from the evidence actually produced and within the limits of the evaluated systems.

## 1.5 Research Path and Thesis Organisation

The research developed cumulatively. Early work established the data, simulation, and experimental foundations and examined progressively more capable control formulations, from rule-based and optimisation approaches to single-agent and multi-agent reinforcement learning. Each stage addressed a concrete problem under explicit assumptions and produced results that remained useful in their own right. As the systems and experiments became more heterogeneous and dynamic,

the remaining difficulty increasingly concerned how to preserve meaningful control when agents, assets, observations, actions, constraints, and operating conditions changed.

These findings motivated TI-MARL and, in parallel, the lifecycle and deployment mechanisms required to operate learned controllers beyond isolated experiments. The development was not a linear sequence: simulator limitations became visible during algorithm experiments, real operational data exposed assumptions in scenario design, and deployment targets introduced constraints that fed back into the control and software architecture. EnergAIze provides the common context in which these algorithmic and systems contributions are connected and evaluated.

The organisation of the dissertation follows this progression. Chapter 2 establishes the foundations and residual gaps; Chapter 3 presents the integrated architecture; and Chapter 4 develops the data, scenario, and simulation foundation. Chapter 5 presents the progression of control and learning solutions, leading to TI-MARL in Chapter 6. Chapter 7 addresses the governed lifecycle and distributed deployment of learned control, and Chapter 8 examines the integrated system in operational energy settings. Finally, Chapter 9 combines the evidence to answer the research questions, and Chapter 10 concludes the thesis and identifies the remaining research directions.

# Chapter 2

# Real-World Multi-Agent Control in Cyber-Physical Systems: Foundations, State of the Art, and Research Gaps

Chapter 1 established the thesis proposition and four research questions around dynamic and heterogeneous control, multi-dimensional scalability, safe and robust operation, and the governed deployment of MARL-based control. This chapter examines the knowledge that supports those questions and determines which parts of the problem are already addressed by prior work.

The class of systems under study provides the starting point for the argument. Distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems define the real-world conditions; Renewable Energy Communities make heterogeneity, temporal flexibility, mixed objectives, and physical constraints concrete. The chapter then compares established control families and identifies the conditions under which MARL offers a meaningful alternative. The formal RL and MARL foundations follow that comparison, once their role in the control problem is clear.

Simulation is an experimental instrument, not evidence of deployment. The chapter separates controlled simulation from evidence obtained through hardware, external platforms, testbeds, and real-world control. Chapter 4 later develops the detailed comparison of simulation and benchmarking platforms next to the simulator contribution itself. The final sections return to RQ1–RQ4, test broad gap claims against counterexamples, retain adjacent problems outside the thesis scope, and derive the requirements used by the contribution chapters.

## 2.1 Literature Scope and Evidence Approach

The state of the art is organised around the four research questions defined in Chapter 1. The objective is not to produce an exhaustive census of reinforcement learning, energy management,

simulation, or MLOps. It is to establish the strongest relevant prior capabilities, identify counterexamples to overly broad absence claims, and determine which limitations remain when the requirements of this thesis are considered together.

### 2.1.1 Scope and Sources

The literature base combines four bodies of evidence that are needed by the thesis: MARL in distributed cyber-physical systems; control and learning in Renewable Energy Communities and EV charging; simulation, benchmarking, and operational evidence; and lifecycle mechanisms for learned controllers and coordinated policy sets. These bodies are not presented as four independent reviews. They form one structured evidence base whose purpose is to support the argument developed across this chapter and the more detailed local state-of-the-art analyses in Chapters 4 and 7.

Peer-reviewed primary studies are preferred when a scientific claim depends on experimental evidence. Surveys and tutorials are used to establish terminology and to identify important lineages. Recent preprints are retained when they provide unusually relevant counterexamples or emerging capabilities, but their evidential status is stated explicitly. Official documentation, standards, and software repositories are used where a platform capability, interface, release history, or deployment mechanism cannot be established reliably from a research paper alone.

Searches combined targeted concept searches with backward and forward citation tracing. The concepts were selected from RQ1–RQ4 and from the operational conditions introduced in Chapter 1: heterogeneity and reconfiguration; scale and resource use; safety, robustness, and uncertainty; and deployment, monitoring, rollback, and recovery. The energy domain remained the principal empirical lens, while selected studies from robotics, transport, and industrial systems were retained when they tested whether a limitation was specific to energy or recurrent across distributed CPSs.

Detailed search expressions, screening records, evidence matrices, and source notes are retained with the research artefacts supporting the thesis. Keeping that operational record outside the chapter allows the dissertation to present the scientific synthesis without turning the state of the art into a procedural report of individual search passes.

### 2.1.2 Evidence Synthesis and Limitations

Three kinds of statement are kept distinct throughout the chapter. A *foundation* is an established concept used to formulate the work, such as an MDP, a Markov game, or CTDE. A *reported limitation* belongs to a particular study, platform, or line of work. A *research gap* is a broader conclusion that must remain defensible after strong counterexamples have been considered. A limitation reported by one study cannot establish that an entire field lacks the corresponding capability.

The synthesis deliberately gives counterexamples an active role. Candidate gaps were tested against work demonstrating dynamic teams, heterogeneous agents, large populations, explicit safety mechanisms, physical testbeds, operational deployment, model registries, edge orchestration, rollback, and related mechanisms. Evidence of an existing capability narrows the claim. The resulting gaps concern combinations, assumptions, and evidence boundaries; they do not rely on the blanket absence of individual techniques.

Evidence is also interpreted according to what was actually demonstrated. A documented software feature is not equivalent to an evaluated control mechanism; a physical testbed is not the same as sustained operational service; a policy evaluated on a larger population is not automatically scalable in communication, simulation throughput, or deployment cost. The relevant evidence level is stated when it changes the interpretation of a result.

This is a structured scoping review, with no claim of exhaustiveness. The retained literature covers the mechanisms, counterexamples, domains, and evidence levels needed to answer the thesis questions, including recent work whose status may change before submission. Absence from this corpus provides no proof that a capability has never been studied. Claims remain limited to the cited literature and archived evidence, and the detailed records allow the synthesis to be audited and updated.

## 2.2 Distributed, Heterogeneous, and Dynamically Reconfigurable Cyber-Physical Systems

The first step is to characterise the system class independently of any particular learning method. Chapter 1 introduced distribution, heterogeneity, and reconfiguration as recurring properties of the thesis problem. Here they are examined more closely because they determine what information is available, which actions are meaningful, how controllers can be coordinated, and what changes when the operational configuration evolves. These properties establish the setting for RQ1 and introduce communication, computation, and runtime constraints that later shape RQ2–RQ4.

### 2.2.1 Distributed Control and Operational Constraints

A CPS connects computation, communication, and physical processes through feedback. Sensors measure the physical world, software estimates or predicts its state, controllers select actions, and actuators affect the next physical state. Timing and physical dynamics are integral to that computation (Lee 2008).

Distribution changes this feedback loop. Information may be generated near an asset, processed at an edge device, aggregated by a platform, and used by a controller running elsewhere. Different actions may also be applied at different locations. A distributed architecture can reduce latency, preserve local data, and avoid a single control bottleneck, but it also creates coordination and

consistency problems. Reviews of distributed CPS control find these trade-offs across energy, transport, industrial, and robotic systems (Hamzah et al. 2023).

It is useful to distinguish three broad patterns.

**Centralised control.** A central controller receives the relevant information and computes the actions for the complete system. This can support globally coordinated decisions, but it depends on communication, a sufficiently accurate global model, and enough computation within the decision interval.

**Decentralised control.** Local controllers select actions using local information, possibly without a central decision maker. This can reduce latency and improve fault isolation, but global coordination is more difficult.

**Hierarchical control.** Local controllers retain selected responsibilities, while higher levels coordinate shared objectives, constraints, or resource allocation. Many operational systems use this intermediate pattern.

Decentralisation is a control structure, not an objective in itself. A centralised optimiser may remain the strongest baseline for a small, well-modelled system. The relevant question is whether the structure remains effective as the system grows, changes, or cannot share all data continuously. Some MARL studies describe decentralised agents while retaining a central simulator, global training information, or coordinated deployment outside the learning algorithm; those assumptions are reported explicitly.

**Examples across CPS application domains.** Table 2.1 shows how recurring system properties appear in energy, robotics or mobility, and industrial CPS. The examples establish generality at the level of the modelling and systems problem; they do not imply direct transfer of one controller between domains.

Evaluation remains domain-aware. Collision avoidance in robotics and battery or grid constraints in energy require different physical models and validation criteria, even when the systems share representations or lifecycle principles. The architectural argument is broader than energy, while the measured empirical claims remain grounded in the energy domain.

### 2.2.2 Heterogeneous Agents, Assets, and Interfaces

Heterogeneity can refer to several different properties. Agents may have different physical capabilities, sensors, action spaces, objectives, computing resources, communication links, or software interfaces. Treating all of these differences as one category can hide the reason a method fails.

The distinction between agent and energy asset is particularly important. An asset is the physical or virtual resource being observed or controlled. An agent is the decision-making entity. One agent may control several assets, several agents may coordinate around one shared resource, or

Table 2.1: Examples of recurring properties across distributed CPS domains.

| **Property** | **Energy community** | **Robotics or mobility** | **Industrial CPS** |
|---|---|---|---|
| Distribution | Buildings and chargers act at different sites | Robots or vehicles act at different locations | Machines and controllers are distributed across a plant |
| Heterogeneity | Batteries, EVs, PV, and heat pumps expose different interfaces | Aerial, ground, or specialised agents have different capabilities | Machines have different processes, sensors, and cycle times |
| Dynamic population | Vehicles arrive and depart; assets become unavailable | Agents join, leave, or move out of range | Machines enter maintenance or production cells are reconfigured |
| Mixed objectives | Individual cost and comfort coexist with community peaks and emissions | Local task progress coexists with mission-wide coordination | Local throughput coexists with plant-wide quality and energy targets |
| Physical constraints | Power, energy, comfort, and network limits | Collision, dynamics, battery, and motion constraints | Safety envelopes, capacity, timing, and maintenance constraints |
| Communication limits | Local meters and community platforms may be intermittently connected | Wireless links vary with position and interference | Field networks have timing, bandwidth, and reliability limits |
| Edge operation | Policies run in building or charger controllers | Policies run on robots or roadside units | Control runs on industrial PCs, PLC-connected devices, or embedded nodes |

a rule-based controller may operate an asset without being a learning agent. These alternatives lead to different observation spaces, action spaces, and deployment units.

Consider three energy assets. A battery accepts a continuous charge or discharge power constrained by its SoC. An EV charger may accept a current limit but only while a vehicle is connected. A heat pump affects a temperature state whose response is delayed by building thermal dynamics. Even if all three actions are normalised to the interval $[-1, 1]$, the meaning of the actions is not the same. A shared neural network cannot safely treat them as interchangeable without an explicit representation of asset type and capability.

The same issue appears in other CPS. A heterogeneous robot team may contain aerial and ground robots. A transport system contains vehicles, traffic signals, and charging infrastructure. A factory contains machines with different cycle times and maintenance states. The general challenge is not merely to train several policies; it is to connect each policy to an interface that correctly represents what its agent can observe and do.

Heterogeneity affects both learning and software architecture. During learning, it influences parameter sharing, representation, coordination, and generalisation. During deployment, it influences compatibility, packaging, hardware requirements, testing, and updates. A method that supports heterogeneous tensors inside a simulator may still require a separate real-world process for each physical interface.

The literature also reveals a useful distinction between *environmental heterogeneity* and *architectural heterogeneity*. Buildings can have different demand profiles, capacities, or locations while all agents still share one observation structure, action structure, and policy class. This is environmental heterogeneity. Architectural heterogeneity is stronger: agents differ in what they can observe, what they can control, how often they act, or which role they play. Residential-flexibility research has considered heterogeneous flexible resources under partial observability, and building-control work has explicitly compared homogeneous and heterogeneous MARL formulations (Charbonnier et al. 2022; Wu, Nweye, et al. 2023). These studies are important counterexamples to a claim that heterogeneity is absent. At the same time, most evidence remains tied to fixed agent sets and predeclared interfaces. Supporting a battery, an EV charger, and a heat pump in one experiment is not yet the same as supporting previously unseen interface types that join an operational system without controller redesign.

### 2.2.3 Dynamic Populations, Capabilities, and Reconfiguration

A fixed-agent problem assumes that the same agents remain present throughout training and operation. A dynamically reconfigurable system relaxes this assumption. Agents or assets may join, leave, fail, recover, or change capabilities. The relevant interaction topology may change with them.

Electric-vehicle charging provides the recurring example. The charger exists continuously, but the vehicle that creates the energy request is temporary. Its arrival time, departure time, initial SoC, desired SoC, battery capacity, and admissible charging power can vary. The control problem at 18:00 may contain a different set of active requests from the problem at 15:00.

There are several levels of change:

- **state variation**: values change while the agent and interface remain fixed;
- **availability variation**: an existing asset is temporarily unavailable;
- **population variation**: agents or controllable requests enter or leave;
- **capability variation**: the observation or action interface changes;
- **topology variation**: communication or coordination relationships change.

These cases should not be collapsed into one generic disturbance. A missing sensor value can be imputed. A departed vehicle should no longer receive a charging action. A new charger

type may require a policy compatible with a different action space. Representation and lifecycle mechanisms need the current data together with the identity and validity of the agents and interfaces that produced it.

Dynamic populations interact strongly with training. Padding a fixed-size input can handle a known maximum number of agents, but it does not by itself provide generalisation to unseen team sizes or types. Training separate policies for every configuration is often impractical. Type-conditioned policies, permutation-invariant representations, graph-based models, attention, role-based policies, or hierarchical coordination are possible responses, but their suitability depends on the domain and evidence.

Dynamic teams have already received targeted attention, and this evidence shapes the residual gap. In traffic, MARL has been applied to a varying number of connected vehicles under changing connectivity penetration (Su et al. 2020). In robotics, asynchronous policies have been designed to tolerate different action durations and robot loss (Yu, Yang, et al. 2023); event-triggered communication has been evaluated when the number of transport robots differs from training (Shibata et al. 2023); and physics-informed policies have handled time-varying communication graphs and robot teams larger than those observed during training, including physical Robotarium experiments (Sebastián et al. 2025). Variable agent count is already addressed in these settings. The remaining difficulty combines differences in capabilities, observations, actions, timing, and availability with agents that join, leave, fail, recover, or change role during continuing operation.

Figure 2.1 fixes the reconfiguration vocabulary used in the remaining chapters and prevents weaker forms of dynamicity from being mistaken for open heterogeneous agent populations.

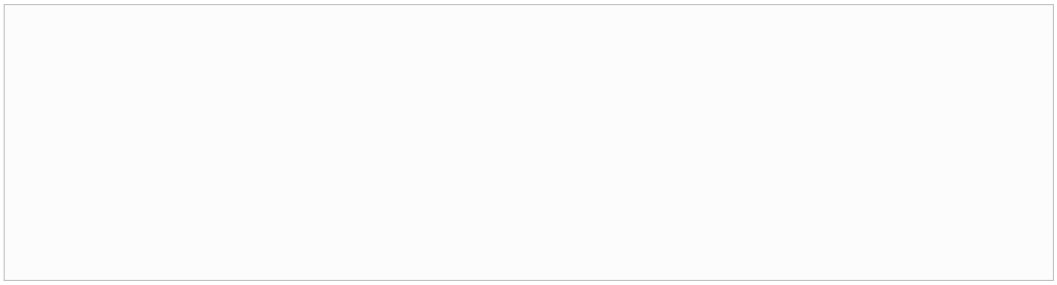


Figure 2.1: Taxonomy of dynamicity and reconfiguration used throughout the thesis.

**Physical, computational, and communication constraints.** A real CPS is constrained at several levels.

**Physical constraints.** Actions must respect power, energy, temperature, speed, mechanical, and other domain-specific limits. Physical systems also have inertia and delays: an action may not have an immediate or perfectly predictable effect.

**Operational constraints.** Users, operators, and organisations impose requirements beyond physical feasibility. A vehicle may need a minimum charge before departure. A building must preserve comfort. Maintenance windows, market rules, and contractual limits can restrict otherwise feasible actions.

**Computational constraints.** A policy may be trained on a cluster but executed on an embedded device. Memory, inference time, processor capability, and energy consumption constrain the deployed model. Edge computing can reduce latency and keep data local, but it also distributes hardware and maintenance responsibilities (Shi et al. 2016).

**Communication constraints.** Data can be delayed, lost, compressed, or unavailable. Bandwidth and privacy may limit what is shared. A local controller should know what to do when a community service is temporarily unreachable.

**Lifecycle constraints.** The controller must be installed, configured, monitored, updated, and, when necessary, rolled back. The operating system, dependencies, interface version, and policy version become part of the control solution. These concerns are not represented in a conventional Markov game, but they can determine whether the policy is usable.

The constraints interact. Sending more data can improve coordination while increasing communication cost and privacy exposure. A larger model can improve simulated performance while exceeding edge memory or latency limits. A conservative safety filter can reduce violations while also reducing nominal reward. Scalability, deployability, safety, and robustness are evaluated as connected properties whose interactions matter.

These properties define the real-world problem before any controller family is selected. For RQ1, reconfiguration changes the set and meaning of observations, actions, capabilities, and relationships, extending beyond numerical variation in a fixed state. For RQ2, it exposes communication, computation, and representation costs. The same changes alter the conditions under which safety evidence and deployed policy compatibility remain valid, connecting the system model directly to RQ3 and RQ4.

## 2.3 Renewable Energy Communities as Representative CPSs

Renewable Energy Communities make the preceding CPS properties concrete in the principal domain of this thesis. They combine physically distributed participants, heterogeneous assets, time-coupled flexibility, changing availability, local and collective objectives, and shared electrical constraints. Energy provides the domain in which these general system properties become measurable control requirements and can be evaluated directly.

This domain perspective gives RQ1 a concrete representation problem, gives RQ3 explicit physical and service constraints, and provides the operational scales and shared resources needed to examine RQ2. It also exposes the interfaces and organisational boundaries that later matter to RQ4.

### 2.3.1 Participants, Assets, and Heterogeneity

European legislation defines a REC through participation, control, geographical proximity, and community benefit (European Parliament and Council of the European Union 2018). This legal framing distinguishes a REC from an arbitrary collection of simulated buildings. It also means that technical control operates within an organisational setting containing members, operators, service providers, network actors, and possibly market participants. Reviews of energy-community business models show that governance, value distribution, and stakeholder roles are central to implementation (Reis et al. 2021).

For the control problem, participants can consume, produce, store, or shift energy. The term *participant* is preferable to inventing a different agent label for every possible role. A participant may be a consumer, a prosumer with local generation, or an entity providing flexibility through controllable assets. Whether that participant corresponds to one learning agent is a design choice addressed later by the EnergAIze architecture and controller formulations.

Typical assets include:

- photovoltaic generation;
- stationary batteries;
- electric vehicles and chargers;
- heat pumps and thermal storage;
- water heaters and other shiftable loads;
- smart meters, sensors, and controllable switches;
- external signals such as prices, emissions, weather, and grid limits.

The asset list is less important than the interfaces it creates. A photovoltaic system may be largely uncontrollable but provides uncertain production. A battery is controllable in both directions but constrained by energy and power limits. A flexible load may only be shifted within a comfort or deadline window. These differences explain why a community cannot be modelled simply as many copies of the same agent.

Stakeholders also define who is allowed to see data and apply actions. A community platform may know aggregate demand without receiving every raw household measurement. A charger operator may control the charging station but not the building's other loads. A network operator

may impose a limit without participating in the optimisation of individual costs. These boundaries later become part of deployment and information architecture.

**Three layers of heterogeneity.** The energy literature often describes a system as heterogeneous, but that word can refer to different things. Keeping the distinction explicit is important because each layer creates a different control problem.

**Asset heterogeneity.** Batteries, EVs, photovoltaic systems, heat pumps, and flexible loads have different physical dynamics, actions, limits, and time scales. This is well represented in optimisation, predictive-control, and learning studies (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Srithapon and Månsson 2023; Zhu, Yang, et al. 2022).

**Participant and objective heterogeneity.** Buildings and users differ in demand, routines, comfort, mobility requirements, tariffs, available data, and preferred objectives. MERLIN, for example, studies buildings with distinct occupant behaviour, while EnergAIze permits different local management objectives within one community (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Nweye, Sankaranarayanan, et al. 2023).

**Controller and interface heterogeneity.** Agents can expose different observation fields, valid actions, capabilities, operating timescales, communication interfaces, or policy architectures. This layer is harder. A system may contain different assets while still mapping all of them into a fixed, homogeneous controller schema. Dynamic participation becomes substantially more demanding when a joining controller introduces a capability or action meaning that was not present during training.

Table 2.2: Three layers of heterogeneity in REC control.

| Layer | Example | Control implication |
|---|---|---|
| Assets | One building has a battery and EV, another has a heat pump and photovoltaic production. | Models, constraints, and actions differ, but a common abstraction may still preserve one fixed controller interface. |
| Participants and objectives | One participant prioritises cost, another mobility readiness, while the community limits aggregate import. | Local and community outcomes must be represented and reported separately. |
| Controllers and interfaces | A new agent exposes different measurements, valid actions, timing, or hardware resources. | The architecture must discover or describe capabilities and bind a compatible controller without assuming an unchanged input–output contract. |

The first two layers are already well represented in REC and EV research. The third is the stronger residual problem for this thesis. It explains why a community with different load profiles is not automatically evidence of an architecture that can accept a new charger type, a new action space, or a participant with a different policy role during operation.

### 2.3.2 Flexibility, Temporal Coupling, and Dynamic Availability

Energy flexibility is the ability to change production, consumption, storage, or exchange relative to a baseline in response to an internal or external need. The literature uses several related definitions and modelling approaches, but a common element is the possibility of moving an energy action in time or changing its magnitude within constraints (Hussain et al. 2023).

Different assets provide flexibility in different forms:

- a battery moves energy across time;
- an EV charge can be delayed or modulated before departure;
- a heat pump uses thermal inertia to move electricity consumption while maintaining comfort;
- a flexible industrial process can be rescheduled within production requirements;
- controllable generation or curtailment changes the balance directly.

The value of flexibility depends on time. One kilowatt-hour of battery capacity is not equally useful in every interval. It may be valuable before a predicted community peak and less valuable after the peak has passed. Similarly, an EV with eight hours before departure provides more scheduling flexibility than an EV with twenty minutes remaining.

This temporal coupling is why one-step optimisation can be insufficient. A controller must consider the future opportunity cost of using flexibility now. Model predictive control addresses this through a receding horizon. RL addresses it through expected return. Rule-based control may encode selected temporal knowledge explicitly. The controller families differ, but the underlying sequential problem is shared.

Uncertainty further complicates the value of flexibility. Photovoltaic production, demand, arrival times, departure times, and prices may be forecast with error. A controller can reserve flexibility for a predicted event that does not occur or use it too early when the actual event is larger than expected. Robustness covers ordinary forecast error as well as sensor failures.

**Electric vehicles and dynamic asset availability.** Electric vehicles are both mobility assets and energy assets. Their primary purpose is transport, which means that energy control is subordinate to a mobility requirement. A charging policy that reduces cost but fails to provide the energy needed for departure is not acceptable.

An EV charging request is characterised by arrival time, expected or required departure time, initial SoC, target SoC, battery capacity, charger power, efficiency, and possibly vehicle-to-grid capability. These quantities differ between sessions and may be uncertain. The same charger can present a different control problem each time a vehicle connects.

This makes EV charging an intuitive example of a dynamic system, but the word *dynamic* must be used carefully. In most studies, the physical charger or controller identity remains fixed

while the session state changes. Puech et al. model uncertain starts, ends, energy requests, and early disconnections over 32 fixed charging slots using MPC and stochastic programming (Puech et al. 2024). Hermans et al. forecast occupancy and control 174 fixed chargers in an operational microgrid (Hermans et al. 2024). In both cases, users and sessions change, but the controller topology is known beforehand.

Table 2.3 defines the vocabulary used throughout the thesis. The categories are ordered by the extent to which the controller architecture itself changes.

Table 2.3: Operational classification of dynamics in REC and EV control.

| **Category** | **What changes** | **Representative evidence and limitation** |
|---|---|---|
| Session-level dynamics | An EV plugs in, disconnects, changes requested energy, or changes state while a charger/controller slot remains fixed. | Common in EV scheduling; Puech et al. and Hermans et al. explicitly represent changing sessions over fixed infrastructures (Hermans et al. 2024; Puech et al. 2024). |
| Asset availability | An asset becomes usable or unavailable inside a pre-existing building or agent. | Useful for masks and local fallbacks, but it does not create a new controller identity or interface. |
| Variable instance size | The same method is evaluated with a different number of chargers, vehicles, or buildings between runs. | Sadeghianpourhamami et al. use a size-independent aggregate state, while GNN-DT transfers one graph-compatible architecture across charger counts (Orfanoudakis, Panda, et al. 2025; Sadeghianpourhamami et al. 2020). |
| Dynamic participation | A participant or controller joins or leaves during the modelled process. | Wu et al. explicitly model buildings joining and departing a VPP and adapt a Multi-Agent Transformer within a predefined family of building representations (Wu, Qiu, et al. 2024). |
| Agent loss | A previously active controller fails or disappears during an episode. | Stronger evidence was found in robotics than in the energy literature; it tests continuity after failure rather than only planned session completion. |
| True open population | Controllers can be instantiated or removed online, potentially with new capabilities and interfaces, without redesigning or retraining for that exact composition. | No retained REC/EV study demonstrates this complete case. Recent graph methods approach variable scale and heterogeneous node types, but preserve a known feature and action grammar. |

The distinction changes how evidence should be interpreted. Sadeghianpourhamami et al. avoid a state representation that grows directly with the number of charging stations and demonstrate generalisation to larger station sets, but the policy still controls an aggregate problem with known semantics (Sadeghianpourhamami et al. 2020). GNN-DT uses graph embeddings so that a model trained at one charger count can be evaluated at other counts; this is size generalisation, not online creation of a new controller interface (Orfanoudakis, Panda, et al. 2025). HetGPS represents three EV types in a parameter-shared graph controller and evaluates 200–3218 EVs, but its formal agent index set remains fixed for each problem instance even though session arrival and departure are part of the exogenous process (Wang et al. 2026). As a recent preprint, it is a strong design counterexample but not yet peer-reviewed or physically deployed.

Wu et al. provide the clearest energy-domain counterexample to a claim that dynamic participation has not been studied. Their Multi-Agent Transformer with an adaptation module handles buildings that join or depart a virtual power plant and is evaluated across configurations with three to nine buildings of different types and regions (Wu, Qiu, et al. 2024). The result narrows the thesis gap. It does not remove the harder operational question of accepting agents with previously unseen observation fields, actions, roles, timing, or hardware requirements.

The vehicle can also participate at different architectural levels. It may be controlled as an individual agent, as one asset within a building agent, or as part of a central charging-site controller. Each choice changes the number of agents, the information available to the controller, and the meaning of coordination. Chapter 5 compares several such formulations rather than assuming that an EV must always correspond to one MARL agent.

Vehicle-to-grid operation adds another degree of freedom by allowing discharge to the building or grid. It can increase community flexibility, but it also introduces degradation, user acceptance, availability, market, and safety considerations. The presence of a technically possible action does not mean that it is contractually or operationally admissible.

### 2.3.3 Individual and Community Objectives and Physical Constraints

A community contains objectives at several levels.

At participant level, common objectives include electricity cost, comfort, mobility readiness, battery usage, self-consumption, and preservation of local autonomy. At community level, objectives can include aggregate cost, peak reduction, renewable self-consumption, emissions, energy sharing, and compliance with a shared import or export limit. Network and service operators may introduce additional objectives related to stability, congestion, or quality of service.

These objectives are not always contradictory. Charging a vehicle when local photovoltaic production is high can reduce both individual cost and community import. Conflict appears when several participants prefer the same limited period or when one participant must sacrifice local benefit to improve a community metric.

One aggregate score cannot capture this formulation. A community can achieve a low peak while one vehicle repeatedly misses its target. It can minimise total cost while allocating benefits unevenly. It can reduce emissions by shifting demand into a period that increases an individual bill. Reporting only the global objective can hide these effects.

There are several possible responses:

- combine local and community terms in each reward;
- use separate rewards and study the resulting general-sum interaction;
- impose participant requirements as constraints;
- coordinate local controllers through prices, incentives, budgets, or shared signals;

- use hierarchical control in which local and community decisions have different responsibilities.

A single reward cannot resolve the social and economic governance of a REC. The control formulations represent local and community objectives and report their measured effects, while broader questions of fairness and governance require evidence beyond aggregate control performance.

**Cooperative and competitive interactions.** The interaction between participants changes over time. During a period of abundant local renewable production, several participants can consume without conflict. During an evening peak or under a community import limit, the same participants may compete for available capacity. They still cooperate by accepting a shared coordination mechanism, but their preferred local actions are no longer simultaneously feasible.

A simple example illustrates the mixed motive. Suppose three EVs need energy before morning and the installation can safely supply only two at full power. All drivers benefit from reliable operation of the site, which is a cooperative objective. Each driver also prefers their own vehicle to charge first, which creates competition. A policy that ignores the shared limit is unsafe. A policy that always prioritises the same vehicle may be operationally unfair. A policy that divides power equally may be inefficient when departure times differ.

This example shows why "cooperative" and "competitive" should not be treated as fixed labels attached to the complete system. The relationship depends on the objective, resource, and time. Agents can share one reward component and retain different local reward components. They can cooperate during training through shared information while executing individual policies. They can also use a central coordinator for a constrained resource while retaining local control elsewhere.

The energy domain motivates general-sum formulations and hierarchical architectures. It also provides interpretable metrics for analysing the outcome: missed energy requirements, individual cost, community peak, renewable use, and distribution of benefits. These metrics make the mixed interaction more visible than a single game score.

**Physical constraints and mapping to general CPS properties.** Energy control is bounded by physical constraints. Batteries and EVs have minimum and maximum SoC, charge and discharge power, efficiency, and possibly degradation limits. Chargers have rated current and connection states. Buildings have thermal dynamics and comfort limits. Electrical installations have import, export, phase, and equipment limits.

Operational constraints add deadlines, preferences, contracts, and service requirements. An EV target may be expressed as a minimum departure energy. A participant may allow a heat pump to be interrupted only within a comfort band. A charger operator may limit how often a session can be modulated. A community may have an agreed peak or sharing rule.

Constraints can be handled at different layers:

- the environment can clip or reject invalid actions;

- the policy can be constructed to output only admissible actions;
- an optimisation or safety layer can project proposed actions into a feasible set;
- a fallback controller can replace a learned policy under selected conditions;
- validation gates can reject a policy before deployment if violations exceed a threshold;
- lower-level physical protections can remain independent of the learning system.

Safety mechanisms provide different levels of evidence. Action clipping protects the physical model but can hide frequent invalid proposals. A fallback improves operational safety while changing the controller that is effectively executed. Formal guarantees are stronger than empirical tests but may depend on restrictive models. Each later experiment identifies whether its evidence concerns policy behaviour, runtime enforcement, empirical stress testing, or a formal property.

**Mapping REC characteristics to general CPS properties.** Table 2.4 consolidates the role of energy communities in the thesis. Each REC characteristic is connected to a more general CPS property and to the resulting research concern.

The mapping establishes RECs as a representative real-world CPS while keeping the empirical scope explicit. It also prepares the comparison of control approaches: the relevant issue is how MARL compares with established alternatives and what evidence each family provides.

The domain provides more than an application example. It exposes a compact combination of the properties that motivate the thesis: heterogeneous capabilities, temporal coupling, dynamic availability, mixed objectives, partial information, and non-negotiable physical limits. The later experiments use these properties to test the research questions without extending energy-specific measurements to unrelated CPS domains.

## 2.4 Control Approaches for Distributed Energy Systems and Related CPS

The preceding sections establish the control problem but do not determine the controller that should solve it. Energy systems already use successful rule-based, optimisation-based, predictive, learning-based, and hybrid approaches. A credible comparison examines the assumptions, capabilities, and real-world costs of these families. Labels such as “traditional” and “intelligent” reveal little about which controller fits the problem.

The discussion introduces RL and MARL at the level needed to compare them with credible alternatives. Their formal foundations are developed in Section 2.5, after the motivation for using them has been established. This order keeps the state of the art centred on the control problem and avoids a general tutorial detached from the thesis questions.

Table 2.4: Mapping from renewable-energy-community characteristics to general CPS properties.

| REC characteristic | General CPS property | Research implication |
|---|---|---|
| Buildings and assets are geographically distributed | Distributed sensing, actuation, and control | Decide which information and responsibilities remain local or are coordinated |
| Batteries, EVs, PV, heat pumps, and loads expose different interfaces | Heterogeneous agents and assets | Represent capabilities explicitly and support compatible policies |
| EV sessions begin and end; assets become unavailable; selected participants may join or leave | Session dynamics, availability, and possible population reconfiguration | Distinguish state change from controller membership and test whether interfaces remain compatible |
| Individual cost and comfort coexist with community objectives | Mixed cooperative–competitive interaction | Evaluate both local and global outcomes |
| Decisions affect later flexibility | Sequential and temporally coupled control | Use models or learning that account for long-term consequences |
| Forecasts and measurements are imperfect | Partial observability and uncertainty | Assess robustness to error, delay, and missing data |
| Power, energy, comfort, and network limits are mandatory | Safety-critical physical constraints | Prevent or quantify invalid and unacceptable actions |
| Policies may run on local controllers | Edge and lifecycle constraints | Consider inference resources, packaging, updates, monitoring, and recovery |

The useful question is: which controller fits which assumptions, and what additional value justifies its complexity? A well-modelled charging site may be controlled effectively through MPC. A geographically distributed community with private observations and repeated local decisions may benefit from decentralised learning. A safety-critical action may still be projected through an optimisation or physics layer even when a learned policy proposes it.

### 2.4.1 Explicit Control: Rules, Optimisation, and Predictive Control

A RBC selects actions through predefined conditions. Examples include charging when a tariff falls below a threshold, reducing charger power when a site limit is approached, or activating a heat pump when temperature leaves a comfort band. Rules can be local or coordinated through a MAS, in which software agents exchange information and follow designed protocols.

Rule-based control is sometimes treated as a weak baseline because it does not learn. This is misleading. A small set of carefully designed rules can be reliable, inexpensive, easy to inspect, and suitable for constrained hardware. It can also provide a safe fallback when a learned controller is unavailable. Fonseca et al., for example, represent heterogeneous flexible loads through a common

FlexOffer abstraction and compare explicit scheduling mechanisms in a REC without requiring RL (Fonseca, Ferreira, Landeck, et al. 2022). The result is a useful reminder that a common interface can solve part of the heterogeneity problem before learning is introduced.

The limitation is that rules express the situations anticipated by their designers. As the number of assets, contexts, and interactions grows, the rule set can become difficult to maintain. Rules that are reasonable independently can conflict when combined, and a threshold selected for one participant may be inappropriate for another. RBCs remain important as transparent baselines, operational fallbacks, and lower-level protections. Their transparency does not provide long-horizon adaptation or coordination in every setting.

**Mathematical optimisation and multi-agent optimisation.** Mathematical optimisation expresses the task through decision variables, an objective, and constraints. In an energy community, decision variables can represent charger power, battery operation, flexible-load schedules, or energy exchanges. Constraints encode energy balances, device limits, deadlines, comfort, and network limits. When models and forecasts are adequate, the result is explicit and interpretable: feasibility can be checked and the effect of a new constraint can be inspected directly.

Optimisation is not inherently centralised. Decomposition and agent-based structures can distribute a problem while retaining mathematical solvers. Faia et al. organise an energy community as local agents and a coordinator, formulate the scheduling problem as mixed-integer linear optimisation, and report a large reduction in solution time relative to a centralised implementation for their high-EV-penetration case (Faia et al. 2023). This is a direct counterexample to the idea that a multi-agent architecture must be learning-based.

The practical burden lies in maintaining models, constraints, forecasts, and solver performance as the system changes. A new asset type may require new variables and equations. Integer decisions, network constraints, uncertainty, and participant-specific models can increase computation. These limitations depend on the problem and provide no universal reason to prefer RL. If a model can be maintained and solved within the decision interval, a learning-based alternative must justify itself through adaptation, local autonomy, privacy, transfer, or lower online optimisation cost.

**Model predictive and stochastic control.** MPC repeatedly solves a finite-horizon optimisation problem using the latest measurements and predictions. Only the first action is applied; the horizon then moves forward. This receding-horizon structure is well suited to energy systems because prices, photovoltaic production, occupancy, and charging requests change during the day.

Srithapon and Månsson combine home-level MPC with community coordination and AC optimal power flow for EVs, heat pumps, thermal storage, photovoltaic generation, and network constraints (Srithapon and Månsson 2023). Puech et al. use MPC and two-stage stochastic programming to represent uncertain session starts, ends, requested energy, and early disconnections over 32 charging slots (Puech et al. 2024). These studies demonstrate that heterogeneous assets and dynamic EV sessions can be treated explicitly without MARL.

The strongest operational counterexample in the energy literature is the implementation by Hermans et al. A quadratic-programming MPC controller was integrated into a corporate microgrid with photovoltaic production and 174 EV chargers. Over three weeks, it achieved a reported 59% average daily reduction in grid peak power (Hermans et al. 2024). The system uses charger-occupancy and photovoltaic forecasts while the charger infrastructure remains fixed. This result prevents two overstatements: large EV populations do not intrinsically require MARL, and operational intelligent charging is not absent from the literature.

MPC and stochastic control require models, forecasts, and repeated solver execution. Their value is strongest when constraints must be explicit and the optimisation remains tractable. Learning and MPC are also complementary. A learned model can support prediction; a policy can supply a warm start; an optimiser can project a learned action into a feasible set; and an MPC or rule-based controller can remain as fallback.

### 2.4.2 Reinforcement Learning and Multi-Agent Control

Single-agent RL can act locally or centrally. A local agent may control one battery or building. A central agent may observe an entire charging site and produce a joint or aggregate action. Both are single-agent formulations even when many physical assets are affected.

Sadeghianpourhamami et al. use fitted Q-iteration to coordinate EV charging through a state representation designed not to grow directly with the number of charging stations. Simulations based on real charging data test generalisation to larger station sets (Sadeghianpourhamami et al. 2020). This is important for the thesis because it shows that variable session availability and size-independent coordination can sometimes be achieved without modelling every EV as a separate MARL agent.

Aggregate representations reduce dimensional growth, but they can remove information that matters to individual participants or to network constraints. A central policy also concentrates computation and may become difficult to deploy when actions must be produced by several local systems. Conversely, a local single-agent policy can be easy to deploy but may ignore shared limits. Single-agent RL remains a valid controller family and a necessary comparison in its own right.

**Multi-agent, federated, and CTDE learning.** MARL represents decision makers separately and models their interaction during learning. Agents can correspond to buildings, participants, charging sites, EVs, or selected assets. The representation enables local observations and actions while coordination is learned through shared rewards, centralised critics, communication, markets, or community signals.

The retained studies demonstrate several forms of coordination. GridLearn combines building energy models with distribution-network power flow and uses MARL to pursue building and voltage objectives (Pigott et al. 2022). Zhu et al. use centralised training with decentralised execution, soft actor–critic, attention, a counterfactual baseline, and a Lagrangian reward for multi-energy industrial-park management (Zhu, Yang, et al. 2022). Shojaeighadikolaei et al. compare centralised

and decentralised critics for cooperative EV charging under CTDE (Shojaeighadikolaei et al. 2024). EnergAIze uses MADDPG for local prosumer objectives and community-level V2G and flexibility management (Fonseca, Ferreira, Cabral, Severino, and Praça 2024). Wilk et al. reduce data exchange by sharing strategic information during training and using local information during execution (Wilk et al. 2024).

Federated RL offers another decentralisation mechanism. Chu et al. coordinate a residential EV fleet while retaining local experiences and aggregating learned parameters (Chu, Wei, et al. 2022). This can reduce raw-data centralisation, but compatible model and interface structures are still required. Federated averaging does not by itself solve the case in which one agent observes temperatures, another controls charger current, and a third has a different action frequency.

These examples show why the label MARL is not enough. A method can be multi-agent while retaining a fixed population, homogeneous policy interfaces, ideal communication, or centralised deployment. The scientific comparison must state what is local, what is shared, how the agents differ, and what happens when composition or communication changes.

### 2.4.3 Structured, Federated, Transfer, Hybrid, and Hierarchical Approaches

Recent work introduces mechanisms that address selected limitations of fixed-vector and fixed-population controllers.

MERLIN trains independent offline policies in a digital twin of a real 17-building community and transfers policies between buildings with different occupant behaviour, reducing training effort while retaining comparable performance in the reported cases (Nweye, Sankaranarayanan, et al. 2023). Wu et al. use a Multi-Agent Transformer and adaptation modules for buildings that dynamically join or leave a VPP (Wu, Qiu, et al. 2024). GNN-DT combines a graph encoder with a Decision Transformer so that one architecture can process different charger counts and unseen action-space sizes (Orfanoudakis, Panda, et al. 2025).

Graph and set representations remove the need to concatenate every participant into one fixed vector, but they normally preserve a feature and action grammar. A new node can be added only if its attributes and actions can be expressed in the representation expected by the model. This is more flexible than fixed padding, but it is not arbitrary interface discovery.

HetGPS provides a particularly strong recent counterexample to simple scalability and safety claims. The August 2026 preprint combines parameter-shared heterogeneous-graph soft actor–critic with a learned risk model and a physics-anchored action projection. It evaluates 200–3218 EVs across nested distribution networks and keeps the learned parameter count constant across fleet sizes (Wang et al. 2026). It also demonstrates zero-shot topology transfer and runtime voltage protection. At the same time, graph computation and topology data still grow with system size, agent identities are fixed within each problem instance, and the work is not yet peer-reviewed or physically deployed.

The broader lesson is that modern energy control is increasingly hybrid. Learning can provide adaptive decisions, while explicit models, masks, optimisation, and runtime filters protect constraints or make the policy deployable. Evaluation must cover the complete control stack, including the neural network and the mechanisms around it.

**Centralised, decentralised, and hierarchical structures.** Controller family and control structure are related but different. Optimisation can be centralised or distributed. Rules can be local or coordinated. RL can use one central policy, independent local policies, federated learning, or CTDE. A useful comparison asks how the action is computed and where information, computation, constraints, and responsibility reside.

Centralised control can exploit global information and coordinate shared constraints directly. It is often appropriate when communication is reliable, privacy constraints are manageable, and the global problem remains tractable. Decentralised control places decisions near assets, potentially reducing communication and preserving local operation, but local actions can collectively violate a shared limit. Hierarchical control combines both: local controllers preserve device and user requirements, while a community level allocates shared capacity or objectives.

Controller families can also be combined. A community optimiser can send budgets to local policies, a learned controller can propose an action checked by a physics layer, and a rule-based fallback can take over when monitoring rejects a policy. MARL is one part of this hybrid control stack and does not imply complete decentralisation.

### 2.4.4 When MARL Adds Value across CPS Domains

The energy literature supports a more demanding standard for claims about the need for MARL. Beating uncontrolled charging or a simple threshold rule is not sufficient when a realistic optimisation or MPC baseline is feasible. The deployed 174-charger MPC study is especially important because it shows that a conventional model-based controller can operate at substantial physical scale (Hermans et al. 2024).

MARL is most strongly motivated when several conditions occur together: decisions are naturally distributed; participants possess local or private information; models are difficult to maintain; interactions repeat often enough to support learning; online solver budgets are restrictive; local and community objectives must coexist; and the controller must adapt across configurations. Even then, MARL must demonstrate value relative to the engineering and assurance cost it introduces.

A simpler method can remain preferable when the system is small and stable, a trusted model is available, constraints require transparent guarantees, operational data are limited, or the optimiser meets the decision deadline. The later algorithm chapter compares controller families without treating MARL as the default winner.

**Evidence from other CPS domains.** The same controller families appear in other CPS, although physical models and metrics differ. Robot teams combine trajectory optimisation, local rules,

Table 2.5: Evidence-backed comparison of controller families in REC and EV control.

| Family | Representative evidence | Where it is strong | What must be justified or maintained |
|---|---|---|---|
| Rules and explicit scheduling | FlexOffer-based scheduling of heterogeneous flexible loads (Fonseca, Ferreira, Landeck, et al. 2022). | Transparency, low runtime cost, fallback, and known operating logic. | Rule coverage, conflicts, maintenance, and long-horizon adaptation. |
| Optimisation and agent-based optimisation | Multi-agent MILP for high-EV REC scheduling (Faia et al. 2023). | Explicit objectives, feasibility, interpretable constraints, and strong baselines. | Model and formulation maintenance, solver time, and uncertain inputs. |
| MPC and stochastic control | Heterogeneous REC MPC/OPF, uncertain 32-slot charging, and operational control of 174 chargers (Hermans et al. 2024; Puech et al. 2024; Srithapon and Månsson 2023). | Feedback planning, explicit constraints, and uncertainty modelling. | Forecasts, models, repeated computation, and infeasibility handling. |
| Aggregate or single-agent RL | Size-independent EV coordination and central/community policies (Sadeghianpourhamami et al. 2020). | Model-free adaptation and avoidance of repeated online optimisation. | Information loss through aggregation, central action growth, and training data. |
| MARL, CTDE, and federated RL | GridLearn, industrial-park MARL, EV CTDE, federated charging, EnergAIze, and smart-community MARL (Chu, Wei, et al. 2022; Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Pigott et al. 2022; Wilk et al. 2024; Zhu, Yang, et al. 2022). | Local autonomy, coordination, privacy-aware execution, and mixed objectives. | Non-stationarity, compatible interfaces, communication, scale, and policy lifecycle. |
| Transfer, graph, and hybrid learning | MERLIN, dynamic-participation Transformer, GNN-DT, and HetGPS (Nweye, Sankaranarayanan, et al. 2023; Orfanoudakis, Panda, et al. 2025; Wang et al. 2026; Wu, Qiu, et al. 2024). | Reuse, variable-size representations, topology generalisation, and runtime protection. | Stable feature/action grammar, graph computation, evidence beyond simulation, and safety guarantees. |

learned policies, and MARL. Traffic systems use model-based scheduling, independent signal agents, coordinated learning, masks, and operational guards. Industrial systems combine production optimisation with local machine controllers and fault-handling logic.

These comparisons support the generality of the design questions, not direct policy transfer. A policy trained for EV charging cannot control a robot by changing a label. What can transfer are abstractions and questions: how capabilities are represented, how information is shared, how a changing population is processed, how constraints are enforced, and how policies are validated and deployed.

This comparison sets the standard used throughout the thesis. RQ1 cannot be answered simply by showing that MARL can coordinate several agents; the formulation must justify why separate

decision makers and changing interfaces matter. RQ2 requires resource and performance evidence across more dimensions than agent count. RQ3 requires explicit treatment of constraints and disturbances, including mechanisms outside the learned policy when appropriate. RQ4 requires the resulting controller to remain identifiable, compatible, and governable after training. These requirements motivate the formal concepts introduced next.

## 2.5 Foundations of Reinforcement Learning and MARL Used in This Thesis

The controller comparison above motivates the use of reinforcement learning and MARL in selected parts of the thesis. This section introduces only the formal concepts and notation needed to understand those formulations and the later TI-MARL method. It is not intended as a comprehensive treatment of reinforcement learning, game theory, or multi-agent systems; broader introductions and surveys are available in Sutton and Barto (2018), Oliehoek and Amato (2016), Hernandez-Leal et al. (2019), Gronauer and Diepold (2022), and Albrecht et al. (2024).

The common starting point is sequential decision-making. An action changes the present state and the decisions available later. Charging a battery now can reduce a local electricity bill while leaving less storage capacity for later photovoltaic production; delaying an EV charge can avoid a community peak only while sufficient time remains before departure. The concepts below provide the formal language used to represent those long-term effects, partial information, interacting agents, and centralised-training/decentralised-execution structures.

### 2.5.1 Sequential Decision-Making and Partial Observability

**Markov processes and Markov decision processes.** A Markov process describes a system that moves between states according to transition probabilities. Its defining idea is the Markov property: once the current state is known, the probability of the next state does not depend on the complete sequence of earlier states. This does not mean that the past is irrelevant. It means that the state representation is assumed to contain the information from the past that is needed for future prediction.

A simple battery example helps clarify this point. If the state contains only the current electricity price, it is not sufficient for deciding whether to charge. The decision also depends on the battery's current SoC, its power limits, and possibly the expected future demand. If these relevant quantities are included in the state, the controller no longer needs the entire history of earlier charging actions to understand the immediate consequences of the next action. The Markov assumption is only as sound as the state representation on which it relies.

An MDP adds decisions and rewards to this process. A discounted MDP is commonly represented by the tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle, \tag{2.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s,a)$ is the probability of reaching state $s'$ after action $a$ is selected in state $s$, $R(s,a,s')$ defines the immediate reward, and $\gamma \in [0,1]$ discounts rewards obtained further in the future (Sutton and Barto 2018).

At each decision step $t$, the agent observes a state $S_t$, selects an action $A_t$, receives a reward $R_{t+1}$, and the environment moves to $S_{t+1}$. A policy $\pi(a \mid s)$ specifies the probability of selecting action $a$ in state $s$. The objective is usually expressed through the expected return

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{2.2}$$

The state-value function $V^\pi(s)$ and action-value function $Q^\pi(s,a)$ describe the expected return obtained by following policy $\pi$ from a state or state–action pair. These functions connect local decisions to long-term consequences. A charging action with a small immediate cost can still be valuable if it prevents a larger cost later.

The MDP model is useful because it separates the elements of the decision problem. States describe what the controller knows, actions describe what it can change, transitions describe the environment dynamics, and rewards describe the objective. This separation is also useful when a control problem is implemented in software: observation and action interfaces can be checked independently from the learning algorithm.

The same separation exposes several modelling risks. A reward is not the complete physical objective of the system. It is a numerical signal designed to guide behaviour. If a community reward penalises electricity cost but ignores a charger's minimum departure requirement, an agent can obtain a good reward by leaving the vehicle insufficiently charged. The missing requirement should be represented as a constraint or included explicitly in the objective. Similarly, a state that omits a relevant variable can make the problem appear unpredictable even when the underlying physical system is not.

Finally, the standard MDP contains one decision-making agent. Other actors can be hidden inside the transition dynamics, but their behaviour is then treated as part of the environment. This becomes problematic when several actors are learning and adapting at the same time. Markov games, introduced in Section 2.5.3, extend the model to that setting.

**Partial observability and sequential decision-making.** The state of a physical system is rarely available to a controller in full. Sensors cover only selected variables, measurements contain noise, data may arrive late, and some information is intentionally kept local. A charger can observe the connected vehicle and the power available at its installation, but it may not know the exact flexibility of every other participant in the community. A local building controller can measure indoor temperature without knowing the complete thermal state of the building envelope.

A POMDP separates the underlying state from the observation available to the agent. It extends the MDP with an observation space $\mathcal{O}$ and an observation model $O(o \mid s)$, or a related formulation in which the observation also depends on the previous action. The agent selects actions from observations or from an internal representation of their history rather than from the true state directly (Kaelbling et al. 1998).

In the classical formulation, a belief state represents a probability distribution over possible underlying states. The belief is updated when a new action and observation are received. In practical deep reinforcement learning, recurrent networks, finite observation windows, learned latent states, or explicit forecasts may serve a similar purpose without maintaining an exact probability distribution.

Partial observability matters for two reasons. First, the same observation can correspond to different physical situations. A household consuming 3 kW may be close to a local limit or may still have considerable spare capacity, depending on other unobserved loads. Second, information sharing changes the decision problem. Providing every agent with the complete community state can improve coordination during an experiment, but it may be unrealistic in deployment because of communication, privacy, latency, or ownership constraints.

When several agents act under partial observability, the formal problem can be represented as a Dec-POMDP. In a Dec-POMDP, agents cooperate under a common objective but receive local observations and select actions using decentralised policies (Oliehoek and Amato 2016). Markov games are more general with respect to individual rewards, while Dec-POMDPs are particularly useful for cooperative planning under uncertainty. The two perspectives overlap in many practical MARL systems: training may use global information, while execution relies on local observations.

Execution-time information is an architectural and algorithmic choice. A policy that requires every global variable may perform well in simulation but be difficult to deploy; a policy based only on local observations is easier to place near the asset but may require stronger coordination mechanisms. This information boundary later affects scalability, deployability, safety, and robustness.

### 2.5.2 Reinforcement Learning and Actor–Critic Methods

RL addresses sequential decision-making through interaction. An agent acts without a labelled correct action for every state, observes the consequences, and adjusts its policy to increase expected return (Sutton and Barto 2018). This is attractive when a useful control strategy is difficult to write as a complete set of rules, when decisions have delayed consequences, or when the environment contains uncertainty that is difficult to model exactly.

The interaction loop is simple to describe:

1. the agent receives an observation;
2. the policy selects an action;
3. the environment changes;

4. a reward and a new observation are produced;

5. the agent updates its estimates or policy during training.

The difficulty is hidden inside each step. The observation may be incomplete, exploration may be unsafe, useful rewards may be delayed, and the data collected by the current policy may differ from the data that will be seen after deployment.

RL methods are often grouped according to what they learn. Value-based methods estimate the expected return of states or actions and derive a policy from those estimates. Policy-based methods optimise a parameterised policy directly. Actor–critic methods combine a policy, the actor, with a value estimate, the critic. Model-free methods learn without an explicit transition model, while model-based methods learn or use a model to plan. On-policy methods learn primarily from data generated by the current policy; off-policy methods can reuse data generated by other behaviour policies.

These distinctions do not by themselves determine whether a method is appropriate for a CPS. Off-policy learning improves data reuse, although stale experience becomes difficult to interpret when other agents are changing. Policy-gradient methods support continuous actions and still require mechanisms for constraint satisfaction. Model-based control can use physical knowledge, with accuracy bounded by the learned or specified model outside the training distribution.

Energy management illustrates both the attraction and the risk of RL. The controller can learn to shift consumption without requiring an exact closed-form model of every user and device. It can also account for future prices, renewable production, and vehicle departures. Reviews report extensive interest in RL for demand response, buildings, and power systems (Cao et al. 2020; Chen, Qu, et al. 2022; Vázquez-Canteli and Nagy 2019). At the same time, the cost of exploration, the need for large amounts of interaction data, and the consequences of invalid actions make direct learning on physical infrastructure difficult. Simulation becomes central under these conditions, while introducing its own gap between the environment used for learning and the system used for operation.

Real-world RL research identifies additional conditions that are often simplified in benchmarks, including delays, constraints, changing objectives, non-stationarity, limited data, explainability requirements, and the need to act safely while learning (Dulac-Arnold et al. 2021). These conditions become more difficult when several learning agents share the environment.

### 2.5.3 Markov Games, MARL, and CTDE

Stochastic games extend sequential decision-making to several players whose actions jointly influence the state and rewards. The model was introduced by Shapley (1953) and later adopted as a foundation for multi-agent reinforcement learning by Littman (1994). A Markov game can be represented by

$$\mathscr{G} = \langle \mathscr{N}, \mathscr{S}, \{\mathscr{A}_i\}_{i\in\mathscr{N}}, P, \{R_i\}_{i\in\mathscr{N}}, \gamma \rangle , \tag{2.3}$$

where $\mathcal{N} = \{1, \ldots, N\}$ is the set of agents, $\mathcal{S}$ is the state space, $\mathcal{A}_i$ is the action space of agent $i$, $P$ defines transitions from the current state and joint action, and $R_i$ defines the reward of each agent. The joint action is

$$\boldsymbol{a} = (a_1, a_2, \ldots, a_N) \in \mathcal{A}_1 \times \mathcal{A}_2 \times \cdots \times \mathcal{A}_N. \tag{2.4}$$

The transition depends on the combined decisions. A household may reduce charging, but the resulting community peak also depends on what every other participant does. Each agent affects the experience and learning signal of the others.

This creates the central difficulty of MARL: from the perspective of one learning agent, the environment is no longer stationary because the policies of the other agents change during training. In a single-agent MDP, the transition model is assumed to be stable. In MARL, an identical local action can produce different outcomes as other agents learn new behaviours. The agent is effectively trying to learn while part of the environment is learning back.

Early MARL work compared independent learners, which treat other agents as part of the environment, with cooperative agents that exchange information, experience, or policies (Tan 1993). Other foundational work distinguished agents that learn individual action values from agents that model joint actions, showing how exploration and game structure can lead to different equilibria even under a shared reward (Claus and Boutilier 1998). Modern deep MARL extends these ideas with neural policies, centralised critics, value factorisation, learned communication, role representations, and other mechanisms (Albrecht et al. 2024; Gronauer and Diepold 2022; Hernandez-Leal et al. 2019).

A standard Markov game normally assumes a fixed set of agents and defined action spaces. Real systems can violate both assumptions. If an electric vehicle leaves, an action that previously existed may disappear. If a new type of charger is added, its admissible actions and observations may differ from those of existing agents. These changes are not fully represented by simply changing the numerical state. They motivate explicit attention to dynamic populations, heterogeneous interfaces, and policy compatibility.

**Cooperative, competitive, and mixed-motive settings.** Multi-agent problems are often described as cooperative or competitive. In a fully cooperative setting, agents share a common reward or optimise the same team objective. In a two-player zero-sum setting, one agent's gain is the other's loss. Many real systems lie between these extremes and are better described as general-sum or mixed-motive games (Lowe et al. 2017; Wang et al. 2022).

An energy community provides a useful example. Participants may cooperate to increase collective self-consumption, reduce a peak, or avoid carbon-intensive electricity. At the same time, each participant may seek to minimise an individual bill, preserve comfort, and keep enough flexibility for personal needs. Two households can cooperate on a community objective while competing indirectly for a limited shared capacity. The word *competitive* does not imply malicious behaviour; it means that individual objectives are not always perfectly aligned.

The distinction affects problem formulation. If every agent receives only the same global reward, individual disadvantages can be hidden. A policy might reduce the community peak by repeatedly curtailing one participant. Conversely, if every agent optimises only its own cost, the community may experience synchronised behaviour, such as many vehicles charging at the beginning of a cheap tariff period. Mixed objectives require an explicit design choice regarding how local and collective terms are represented, weighted, constrained, or negotiated.

A higher total reward does not automatically imply a fair or acceptable outcome for all participants. Community-level and individual-level metrics are reported together. Where relevant, distributional measures and participation constraints expose outcomes hidden by an aggregate reward; a complete fairness analysis remains an adjacent research problem.

**Centralised training and decentralised execution.** CTDE is a common response to the tension between coordination and deployment. During training, a critic or learning process may access the global state, joint actions, or information from several agents. During execution, each agent selects actions using only its local policy and the observations available at its deployment location. This pattern was used in mixed cooperative–competitive actor–critic learning by Lowe et al. (2017) and in cooperative methods such as COMA and QMIX (Foerster et al. 2018; Rashid et al. 2018).

The appeal of CTDE is easy to see in an energy community. Historical data from all participants may be available in a secure training environment. A centralised critic can use this information to learn how local actions affect the community. The resulting actor policies can then be deployed to local controllers, where they operate without continuous access to every participant's raw data.

CTDE does not remove the deployment problem. The decentralised policies still need compatible observation and action interfaces, sufficiently small inference cost, appropriate packaging, and a process for coordinated updates. Training may also rely on agent identities, a fixed population, or global variables that are not stable over time. A method can satisfy decentralised execution in an algorithmic sense and still be difficult to install and maintain across heterogeneous devices.

The information available during training and execution must also be separated experimentally. A decentralised policy should not be evaluated with observations that are only available to the training critic. Otherwise, the experiment measures a more informed controller than the one that can be deployed. This information boundary is part of the system architecture and affects the validity of the reported controller.

### 2.5.4 MARL Challenges Relevant to This Thesis

Several challenges recur across MARL research (Albrecht et al. 2024; Gronauer and Diepold 2022; Hernandez-Leal et al. 2019). They are introduced here because later sections show how CPS conditions make them more severe.

**Non-stationarity.** Each agent changes its policy during learning, altering the effective environment experienced by the others. Experience collected earlier can become less representative of the current joint behaviour.

**Credit assignment.** When a community reward improves, it may be unclear which agent actions caused the improvement. Poor credit assignment can slow learning or encourage agents to rely on the efforts of others. Methods such as counterfactual baselines and value factorisation were developed partly to address this problem (Foerster et al. 2018; Rashid et al. 2018).

**Combinatorial growth.** The joint action space grows as the product of individual action spaces. Ten agents with five actions each already produce $5^{10}$ possible joint actions. Continuous actions and heterogeneous interfaces make the problem more complex still.

**Partial observability.** Local policies act with incomplete information. Communication can reduce uncertainty, but it introduces bandwidth, latency, reliability, and privacy considerations.

**Exploration and sample efficiency.** The number of possible joint behaviours is large, useful coordination may be rare, and physical systems cannot support arbitrary trial and error. Training often requires extensive simulated interaction.

**Heterogeneity.** Parameter sharing is effective when agents are sufficiently similar, but agents controlling batteries, chargers, and heat pumps do not necessarily share observations, actions, dynamics, or objectives. A shared policy may need explicit type or role information, while separate policies increase training and lifecycle complexity.

**Dynamic populations and generalisation.** Policies trained for a fixed group may fail when agents join, leave, or change capabilities. Generalisation across team size, topology, asset type, or operating distribution is part of robustness in this setting.

**Evaluation.** A high average return can hide unstable training, unequal individual outcomes, constraint violations, or excessive communication. Reliable evaluation requires repeated runs, appropriate baselines, uncertainty reporting, and metrics that reflect the physical task.

Table 2.6 connects these challenges to the recurring energy example and to later thesis concerns.

These challenges do not by themselves establish the thesis gaps. They are properties of the learning problem, and many have mature solution families. The critical question for RQ1–RQ4 is which assumptions remain when these mechanisms are placed back into the distributed, heterogeneous, physically constrained, and operational setting already established in Sections 2.2–2.4. The critical synthesis in Section 2.7 makes that distinction explicit.

Table 2.6: Recurring MARL challenges and their manifestation in energy communities.

| Challenge | Example in an energy community | Later concern |
|---|---|---|
| Non-stationarity | The value of delaying one vehicle depends on how other charging policies are changing | Learning stability and robustness |
| Credit assignment | A reduced peak results from several participants shifting load at different times | Individual and community objectives |
| Combinatorial growth | More buildings and assets increase joint observations and actions | Algorithmic and computational scalability |
| Partial observability | A local controller does not observe every community asset or user preference | Information architecture and communication |
| Sample efficiency | Unsafe or inconvenient actions cannot be explored freely on physical assets | Simulation, safety, and sim-to-real transfer |
| Heterogeneity | Batteries, chargers, and heat pumps expose different interfaces and dynamics | Policy representation and deployability |
| Dynamic population | Vehicles and participants can appear, disappear, or become unavailable | Reconfiguration and generalisation |
| Evaluation | Average reward can hide missed departures or local constraint violations | Multi-level evidence and validation |

## 2.6 Simulation and Levels of Real-World Evidence

Learning-based control requires experience, but physical systems cannot safely or economically provide unlimited trial and error. Simulation is a central experimental instrument for this thesis: it supports training, controlled comparison, scaling studies, deliberate disturbance injection, and experiments with configurations that would be unsafe, expensive, or unavailable in a physical demonstrator. The same strength can hide the assumptions that matter most in operation. Perfect observations, fixed populations, instant communication, deterministic devices, and unconstrained compute can make an algorithm appear more mature than the evidence supports.

Simulation quality must be interpreted relative to the claim being tested. A simplified environment can isolate a learning mechanism adequately, whereas a deployment claim requires stronger evidence about interfaces, timing, physical limits, hardware, communication, and recovery. Fidelity depends on whether the mechanisms that could change the conclusion are represented with sufficient accuracy. Reproducibility is equally important: a result should identify the scenario, environment version, data, randomisation, controller configuration, and evaluation procedure needed to reproduce it.

The detailed state of the art in energy, EV-charging, grid-aware, building-physics, and co-simulation environments is presented in Chapter 4, where it directly positions the extended CityLearn-derived environment developed in this thesis. That comparison includes platforms such as CityLearn, SustainGym, EV2Gym, PowerGridworld, CommonPower, BOPTEST, PowerGym, Chargax, and HELICS. Their capabilities are interpreted according to the claims they can support; the platforms are not treated as interchangeable benchmarks.

The empirical chapters use the evidence levels in Table 2.7 to keep this distinction explicit. The levels describe the evidence and do not form a maturity ranking. An experiment remains valid in simulation when its conclusions stay commensurate with that evidence.

Table 2.7: Evidence levels used throughout the thesis for real-world-oriented control studies.

| **Evidence level** | **Interpretation used in this thesis** |
|---|---|
| Domain simulation | The task represents a physical domain, but sensing, actuation, communication, and timing remain simulated. |
| High-fidelity simulation or digital replica | The environment is calibrated or physically detailed, but the controller does not close the loop with physical assets. |
| Hardware-in-the-loop or mixed reality | Some physical controllers, devices, or agents interact with simulated components. |
| Physical testbed | The controller acts on physical assets in an experimental setting, without a claim of sustained operational service. |
| Shadow operation | The controller receives operational data and produces decisions, but another controller remains responsible for actuation. |
| Operational closed-loop deployment | The controller participates in sustained operation under real interfaces and constraints. |

This evidence distinction supports all four research questions. RQ1 and RQ2 rely heavily on simulation because controlled heterogeneity, population change, and scale are difficult to reproduce physically. RQ3 requires disturbance and failure injection that would often be irresponsible to create on real assets. RQ4 requires progressively stronger evidence as policies move from reproducible experiments to packaging, edge execution, monitoring, updates, and recovery. Simulation supports that path while remaining a distinct evidence level.

## 2.7 Critical Analysis of the State of the Art

The preceding sections establish the system class, the principal energy-domain instantiation, the available controller families, the formal MARL concepts used by the thesis, and the distinction between simulated and operational evidence. The remaining task is to determine what the literature leaves unresolved after the strongest counterexamples are taken seriously. This synthesis is organised directly around RQ1–RQ4 so that the gaps retained here correspond to the questions stated in Chapter 1.

Strong examples of dynamic populations, large-scale control, safety mechanisms, sim-to-real transfer, and real-world deployment already exist in MARL and related control research. The residual problem is narrower: these capabilities are usually demonstrated under different assumptions, interfaces, scales, and evidence levels. Table 2.8 records several broad claims that are rejected and the scoped conclusions that replace them.

Table 2.8: Overly broad claims, counterevidence, and defensible replacements.

| **Tempting claim** | **Assessment** | **Counterevidence** | **Defensible replacement** |
|---|---|---|---|
| MARL has not been deployed in real-world CPS. | False | The I-24 programme used 67 operational VSL controllers (Zhang, Zhang, Ji, et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024). | Operational deployment is possible, but remains exceptional and concentrated in very few documented programmes. |
| Dynamic MARL populations have not been studied. | False | Varying vehicle populations, robot loss, changing team size, and time-varying graphs have been studied (Sebastián et al. 2025; Shibata et al. 2023; Su et al. 2020; Yu, Yang, et al. 2023). | Persistent operation with heterogeneous capabilities and online entry, exit, or role change remains weakly demonstrated. |
| MARL does not scale. | Too broad | Studies report 2–64 buildings, teams larger than training, and 67 field controllers (Almilaify et al. 2023; Sebastián et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024). | Scale is demonstrated along individual dimensions; multi-dimensional scalability remains less established. |
| Safety is ignored in MARL. | False | Constrained learning, physics shields, action masks, and runtime guards already exist (Chen, Liu, et al. 2023; Gu et al. 2023; Zhang, Dehghanpour, et al. 2021; Zhang, Zhang, Quiñones-Grueiro, et al. 2024). | Safety mechanisms exist, but are rarely assessed jointly with faults, reconfiguration, communication imperfections, and deployment shift. |
| Sim-to-real MARL has not been demonstrated. | False | Physical autonomous-driving, robot, and heating studies exist (Blad et al. 2023; Candela et al. 2022; Yu, Yang, et al. 2023). | Physical transfer exists, but testbed evidence must not be conflated with sustained operational deployment. |

The simulation literature requires the same restraint. Chapter 4 shows that current environments already provide EV session dynamics, V2G, grid constraints, large physical networks, accelerated simulation, safeguards, and dynamic co-simulation. The remaining limitation is the difficulty of evaluating dynamic composition, heterogeneous interfaces, physical validity, safety, scale, and reproducibility together under a common control contract.

### 2.7.1 RQ1: Dynamic and Heterogeneous Multi-Agent Control

Dynamic and heterogeneous control combines several changes that are often studied separately. An agent population can vary while all agents remain identical. Agents can be heterogeneous while the population remains fixed. A communication graph can change while observation and action interfaces remain unchanged. EV sessions can begin and end while every charger remains a permanent controller slot. Each relaxation is useful, but none alone represents the complete problem of an operational CPS.

The cross-domain literature contains clear counterexamples to a claim that dynamic teams are unexplored. Su et al. formulate traffic coordination with a varying number of connected vehicles and a random proportion of connected participants (Su et al. 2020). Yu et al. design an asynchronous, team-size-invariant exploration policy partly to accommodate unequal action duration and robot loss (Yu, Yang, et al. 2023). Shibata et al. consider cooperative transport with event-triggered communication and agent counts that differ from training (Shibata et al. 2023). Sebastián et al. use a policy representation for time-varying robot interaction graphs and demonstrate teams substantially larger than those seen during training (Sebastián et al. 2025).

The mechanisms used in these studies are not interchangeable. Table 2.9 separates four representative approaches. The first preserves a fixed neural-network shape by padding absent agents. The second learns a representation that can aggregate a variable set and explicitly tests agent loss during an episode. The third combines local consensus with event-triggered communication so that execution can use a different number of robots from training. The fourth embeds the interaction graph directly in a shared attention-based policy. Each approach solves a real part of the problem, but each also keeps part of the agent–system contract stable.

The energy review sharpens this distinction. Most EV studies represent *session-level dynamics*: users connect and disconnect while charger or agent slots remain fixed (Hermans et al. 2024; Puech et al. 2024). Sadeghianpourhamami et al. and GNN-DT address *variable instance size* through aggregate or graph-compatible representations, with changes made between evaluated configurations and no arbitrary controller creation during an ongoing episode (Orfanoudakis, Panda, et al. 2025; Sadeghianpourhamami et al. 2020). HetGPS combines heterogeneous EV types, graph coordination, and session availability at very large simulated scales, while retaining a fixed agent set within each instance (Wang et al. 2026).

Wu et al. are the strongest direct energy-domain counterexample. Their Multi-Agent Transformer with adaptation modules explicitly represents buildings that join or leave a VPP and is evaluated across dynamic configurations of three to nine buildings (Wu, Qiu, et al. 2024). This study means that the thesis cannot claim that dynamic energy participants are absent from MARL research. It also helps define the remaining boundary: the participating buildings still belong to a predefined family of representations, actions, and roles.

The environment literature, developed in detail in Chapter 4, reaches the same conclusion from a different direction. SustainGym contains realistic EV arrivals and departures while retaining a fixed EVSE-agent set (Yeh et al. 2023). EV2Gym and CityLearn/EVLearn can instantiate different

Table 2.9: Representative MARL mechanisms for varying team size, agent loss, or changing interaction graphs.

| Study | MARL basis | Mechanism used | What it demonstrates and retains fixed |
|---|---|---|---|
| Su et al. (2020) | Multi-agent actor–critic with decentralised actors and centralised critics | A fixed local observation uses the closest neighbouring vehicles; artificial entries pad missing observations and a fixed maximum number of actor–critic pairs pads the population. | Supports varying traffic population and connected-vehicle penetration in simulation. The maximum slots, feature meanings, vehicle actions, and controller architecture remain fixed. |
| Yu, Yang, et al. (2023) | ACE, extending MAPPO to asynchronous execution | Action-delay randomisation; team-size-invariant feature extraction; attention-based fusion of a variable set; low-dimensional feature exchange; explicit robot-loss experiments. | Demonstrates loss of agents during an episode and real-robot asynchronous execution. Robots remain largely homogeneous and expose the same observation and action semantics. |
| Shibata et al. (2023) | Event-triggered communication and consensus-based MARL | Each robot estimates resultant force and torque through local consensus and jointly decides control and when to communicate. | Demonstrates execution with robot counts different from training and real cooperative transport. The task, payload-control semantics, and minimum communication structure remain predefined. |
| Sebastián et al. (2025) | Soft actor–critic with a self-attention port-Hamiltonian policy | A shared physics-informed policy represents interactions through a time-varying graph; self-attention aggregates a variable neighbourhood without a fixed joint vector. | Demonstrates larger teams and changing interaction graphs. Robot roles, node features, and action meanings remain compatible with the learned policy. |

system sizes and represent connection states, but this normally occurs over predefined asset and interface schemas (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025; Orfanoudakis, Diaz-Londono, et al. 2025). HELICS demonstrates online entry and exit at co-simulation-federate level, yet leaves MARL observation, action, policy, and safety compatibility to the application (Hardy et al. 2024). These counterexamples place the policy–environment contract at the centre of the problem; component creation alone does not resolve it.

The residual gap concerns a joint and persistent case: agents differ in physical capabilities, observation and action interfaces, objectives, timing, or hardware; their availability changes during operation; and joining, leaving, failing, recovering, or changing role can be handled without rebuilding the complete controller. Targeted studies solve important pieces, but the cited literature provides little mature real-world evidence for this combined setting.

This residual gap is classified as **strong when stated jointly**. Targeted studies solve important pieces, but the cited literature does not provide a mature body of real-world evidence for open heterogeneous agent populations. The gap motivates RQ1 and constrains RQ2: population scaling must be tested together with composition and reconfiguration across heterogeneous agents.

### 2.7.2 RQ2: Multi-Dimensional Scalability of Learning and Coordination

Scalability spans several dimensions. A method can scale in agent count while requiring global communication. It can reduce communication while increasing training time. It can keep the number of learned parameters fixed while graph processing and environment simulation continue to grow. It can run a large experiment on a cluster but exceed the memory or latency budget of an edge device. A useful assessment identifies what grows, what remains fixed, and which outcome is preserved.

Meaningful counterexamples already exist across CPS domains. SCALEX explicitly compares building-control configurations from 2 to 64 buildings (Almilaify et al. 2023). Charbonnier et al. design distributed residential coordination to avoid the growth and information-sharing requirements of a fully centralised controller (Charbonnier et al. 2022). PowerNet exploits locality and learned neighbour communication for distributed grid control (Chen, Chen, et al. 2022). Large-scale traffic-signal control has been evaluated with local information structures (Chu, Wang, et al. 2020). Sebastián et al. test robot teams larger than those used during training and physical experiments with up to 16 robots (Sebastián et al. 2025). The I-24 deployment provides operational evidence from 67 field controllers (Zhang, Zhang, Quiñones-Grueiro, et al. 2024).

The energy literature adds several stronger population-scale counterexamples. Sadeghianpourhamami et al. design a state representation independent of charger count and evaluate generalisation to larger station sets (Sadeghianpourhamami et al. 2020). GNN-DT is evaluated at changing charger counts, including a 250-charger case, by combining graph embeddings with an offline Decision Transformer (Orfanoudakis, Panda, et al. 2025). The recent HetGPS preprint evaluates 200–3218 EVs, retains a constant learned parameter count through parameter sharing, and transfers a policy to larger distribution-network topologies (Wang et al. 2026). Hermans et al. provide a different form of scale through MPC control of 174 physical chargers (Hermans et al. 2024). These studies make population count alone an indefensible research gap.

None of the results proves general scalability, nor should it be expected to. Each demonstrates selected dimensions while holding others sufficiently controlled. In GNN-DT, the feature and action grammar remains graph-compatible. In HetGPS, learned parameter count remains fixed, but node-wise computation, topology data, and simulation burden still grow. In the deployed MPC study, the infrastructure is large but predeclared and centrally modelled. The residual problem is *multi-dimensional scalability*: whether acceptable control performance and resource use persist when agent count, interface heterogeneity, observation and action complexity, communication demand, computational limits, environment complexity, and reconfiguration increase together.

The evidence supports a **moderate residual gap**. RQ2 asks under which conditions algorithmic, computational, and communication mechanisms maintain acceptable performance and resource use as several dimensions grow together. The empirical evaluation separates simulator throughput, training and sample cost, communication, inference latency, orchestration, deployment resources, and reconfiguration behaviour before combining them into a scalability envelope.

The environment literature adds a further warning against collapsing these dimensions. Chargax reports very large simulator-throughput gains in its evaluated JAX configurations (Ponse et al. 2025). PowerGym includes an 8500-node feeder, demonstrating physical network scale (Fan et al. 2022). BOPTEST provides standard high-fidelity building emulators and has evolved toward vectorised experiment execution (Blum et al. 2021). EV2Gym exposes configurable EV, charger, and transformer topology (Orfanoudakis, Diaz-Londono, et al. 2025). These are all scalability results, but they measure different resources and preserve different assumptions. Chapters 4 and 7 report environment, learning, communication, orchestration, and deployment scale separately before examining their interaction.

### 2.7.3 RQ3: Safe and Robust Operation

Safety and Robustness answer different questions. Safety concerns unacceptable states or actions. Robustness concerns how behaviour changes when assumptions are violated. A controller may satisfy voltage limits under nominal communication and still fail when observations are delayed. Conversely, a controller may tolerate noise while occasionally violating a mobility or physical requirement.

Safety is already an explicit MARL research area. Safe policy learning for networked microgrids incorporates AC power-flow and operational constraints (Zhang, Dehghanpour, et al. 2021). Physics-shielded voltage control constrains learned actions using a model-based safety layer (Chen, Liu, et al. 2023). Constrained multi-agent policy optimisation has been evaluated for safe multi-robot control (Gu et al. 2023). Zhu et al. include storage constraints through a Lagrangian reward in multi-energy MARL (Zhu, Yang, et al. 2022). The I-24 operational stack combines the learned policy with invalid-action masking and runtime guards (Zhang, Zhang, Ji, et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024). These examples place Safety across the complete control stack, including mechanisms outside the learned policy.

The energy literature also contains robustness mechanisms outside MARL. Puech et al. model uncertain charging-session start and end times, requested energy, and early disconnection through MPC and stochastic programming (Puech et al. 2024). Hermans et al. integrate occupancy and photovoltaic forecasting in an operational MPC system (Hermans et al. 2024). HetGPS separates a learned task policy from a graph risk model and a physics-anchored action projection, reducing voltage violations across large simulated networks (Wang et al. 2026). As a recent preprint, this is important counterevidence but not a formal guarantee or deployment result.

Simulation frameworks also encode safety at different layers. CommonPower provides symbolic models, MPC baselines, and automatically synthesised safeguards around RL and MARL controllers (Eichelbeck et al. 2026). PowerGridworld and PowerGym expose grid variables and voltage-related physical constraints through power-flow models (Biagioni et al. 2022; Fan et al. 2022). These environments show that constraint-aware experimentation is already possible. They do not remove the need to test whether a safety mechanism remains valid under missing observations, communication faults, changing interfaces, or online reconfiguration.

Robustness has likewise been studied against selected disturbances in other CPS. Asynchronous robot exploration introduces action-delay randomisation and robot-loss handling (Yu, Yang, et al. 2023). Sim-to-real autonomous driving uses domain randomisation (Candela et al. 2022). Physics-informed multi-robot policies are evaluated under imperfect communication and changing team sizes (Sebastián et al. 2025). Real REC work reports a different set of problems: incomplete and noisy measurements, missing EV state of charge, heterogeneous subsystem integration, and synchronisation difficulties (Fonseca, Sousa, et al. 2025). These observations are operationally important even when they are not yet part of a complete controlled stress-test campaign.

Safe and robust methods already exist. The residual gap lies in the limited systematic evidence for safe operation under the *combination* of uncertainty, missing or delayed information, communication failures, agent or device faults, dynamic reconfiguration, unseen configurations, and train-to-operation shift. A paper may demonstrate voltage constraints but omit agent loss; address early disconnection without interface change; or report physical transfer without a defined fault campaign.

This residual gap is classified as **moderate**. RQ3 is defensible as an integration and evaluation question: how constraints are incorporated and how acceptable operation is assessed when several real-world disturbances occur, with the evidence level and protection layer stated explicitly.

### 2.7.4 RQ4: Governed and Deployable MARL Lifecycle

The literature does not support the claim that RL lacks lifecycle tooling or that MARL has never reached an operational CPS. Physical testbeds exist in autonomous driving, multi-robot systems, and building control (Blad et al. 2023; Candela et al. 2022; Shibata et al. 2023; Yu, Yang, et al. 2023). The I-24 programme provides the strongest counterexample: a simulation-trained MARL controller was integrated with real traffic-management infrastructure, deployed over 67 variable-speed-limit controllers, protected by invalid-action masking and independent runtime safety guards, and assessed over millions of decisions and several months of operation (Zhang, Zhang, Ji, et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024).

Operational intelligent energy control also exists, although the controller is not always MARL. Hermans et al. deploy MPC over 174 chargers (Hermans et al. 2024). Patrício et al. deploy a non-learning multi-agent community energy-management system with second-life batteries in a Portuguese REC (Patrício et al. 2026). Gonçalves et al. describe an RL-based MLaaS architecture with parallel training and an autoscalable inference service (Gonçalves et al. 2025). Fonseca et al. report preliminary evidence from an operational four-building REC and document practical data and integration problems (Fonseca, Sousa, et al. 2025). These examples reject an absolute claim that intelligent REC, EV, or MARL control has not reached real systems.

The lifecycle review further shows that generic MLOps is already well developed. An explicit RLOps lifecycle covers specification, development, validation, production serving, monitoring, reproducibility, and safety/security for RL (Li, Thomas, et al. 2022). MLflow provides a general precedent for experiment lineage, reproducibility, and deployment-oriented model management

(Zaharia et al. 2018). Distributed RL frameworks such as RLlib and Acme support scalable actor–learner execution, while MARLlib and MALib provide flexible agent–policy mappings and policy-population orchestration (Hoffman et al. 2020; Hu et al. 2023; Liang et al. 2018; Zhou et al. 2023). Pearl is explicitly designed as a production-oriented RL library that treats dynamic action spaces and safety as engineering concerns beyond benchmark design (Zhu, Braz, et al. 2024).

These systems are important counterexamples, but they govern different objects. A distributed training framework can execute several policies. A registry can version a model artefact. A container platform can deploy and roll back software. None of those facts, by itself, establishes that an interdependent policy set is represented, validated, promoted, deployed, observed, and recovered as one governed unit. The difference can be seen through five boundaries summarised in Table 2.10.

Table 2.10: Lifecycle capabilities that should not be conflated in a MARL deployment.

| **Existing capability** | **What it establishes** | **Additional requirement for coordinated MARL operation** |
|---|---|---|
| Experiment tracking | Parameters, metrics, artefacts, code revisions, and run history are recorded. | The environment, scenario, reward, constraints, observation/action schemas, seeds, agent–policy mapping, dependencies, and runtime must be recoverable as one experiment contract. |
| Model storage or registry | A model version has identity, lineage, metadata, and lifecycle state. | A policy-set registry must also represent member policies, roles, interface compatibility, communication and safety dependencies, allowed combinations, and rollback predecessors. |
| Serialisation or packaging | Weights, code, and dependencies can be moved to a target runtime. | The package must preserve preprocessing, schemas, communication adapters, safety layers, target requirements, configuration, and the identity of the complete policy set. |
| Multiple-model serving | Several independent models can be loaded and served. | The system must coordinate agent–policy assignment, mixed-version behaviour, atomic or staged policy-set updates, and compatibility checks across dependent policies. |
| Infrastructure monitoring | Processes, devices, resources, and communications appear operational. | Monitoring must also detect data-quality, control-performance, safety, and coordination failures, including incompatible versions, oscillation, unfair resource use, or loss of community-level performance. |

A simple energy example exposes the problem. Suppose a community deploys one policy for batteries, another for EV charging, and a third for community-level coordination. Each container may be healthy and each policy may work in isolation. Nevertheless, replacing only the EV policy can alter its action range, message schema, or response timing. The remaining policies may then operate with assumptions that are no longer valid. A generic rolling update has succeeded from a software perspective, while the coordinated controller has entered an unevaluated mixed-version state.

Dynamic composition narrows the gap further without closing it. HELICS supports federates that join after a co-simulation begins and explicitly documents EV federates joining and leaving a

running federation (Hardy et al. 2024). This proves that online component composition is feasible at the infrastructure layer. It does not decide which learned policy is compatible with the new component, whether its observation and action schemas match, which safety constraints apply, or how the other active policies should respond. Likewise, MALib's changing policy population is a training abstraction rather than evidence that a previously unknown physical controller can join an operational CPS safely (Zhou et al. 2023).

The residual gap concerns *governance and integration*. Existing research and production systems provide substantial lifecycle pieces independently: distributed training, experiment tracking, registries, packaging, scalable serving, progressive deployment, rollback, edge orchestration, runtime safety mechanisms, and sustained operational MARL. The available evidence provides much less support for a reusable lifecycle in which coordinated policy sets, agent–policy compatibility, environment–policy lineage, dynamic heterogeneous participation, validation gates, coordination-aware monitoring, staged edge deployment, rollback, and recovery are managed together.

This residual gap is classified as **strong**. RQ4 concerns the reproducibility, governance, validation, deployment, monitoring, updating, and recovery of changing collections of interdependent policies across heterogeneous edge targets. Chapter 7 develops the detailed platform comparison and evaluates the EnergAIze lifecycle. A trained model, a deployable package, and an operational multi-agent control service are treated as distinct research objects.

## 2.8 Adjacent Challenges and Scope Boundaries

The critical analysis in Section 2.7 isolates the four residual problems that motivate RQ1–RQ4. The same literature also identifies concerns that influence the design, evaluation, or eventual operation of a real-world control system without constituting independent contributions of this dissertation. Distinguishing these roles is necessary because an issue can be operationally important without being a research gap that the thesis claims to close.

The boundary adopted here is evidence-based. A concern is treated as a supporting requirement when it directly shapes the methods or evaluation developed later in the thesis. It remains outside the central scope when answering it would require a separate scientific formulation, dedicated methods, or an evidence base that is not provided by the present work. These boundaries are revisited in Chapter 9 when the final claims and limitations are assessed.

### 2.8.1 Supporting Requirements and Evaluation Conditions

Several concerns identified in the literature are incorporated into the thesis because they affect how the four research questions can be answered, even though they are not formulated as additional RQs.

**Traceability, reproducibility, and interoperability.** The lifecycle contribution depends on relating a result to the environment, scenario, reward, observation and action interfaces, source revision, dependencies, policies, and policy-set configuration that produced it. Existing platforms already provide strong experiment-tracking and lineage primitives; the requirement here is to keep those identities coupled when a learned controller is validated, packaged, deployed, updated, and reconstructed. This supports RQ4 within the selected control lifecycle and does not claim to solve reproducibility across machine learning. The same scope applies to interoperability: the thesis defines and tests explicit interfaces between selected components without proposing a universal standard for energy or CPS control software.

The experimental substrate introduces a related requirement. Reproducible comparison depends on explicit scenario semantics, randomisation, physical assumptions, and controller interfaces. Existing environments provide many of these mechanisms individually, as discussed further in Chapter 4. The thesis makes its own environment and scenario contracts explicit within this scope and does not claim a general cross-platform benchmark standard.

**Data quality, monitoring, and human oversight.** Missing, delayed, noisy, or inconsistent information affects both the behaviour of a learned controller and the meaning of the evidence collected from it. Data-quality disturbances form part of the robustness analysis under RQ3, while runtime monitoring of control performance, safety events, compatibility, and multi-agent coordination supports RQ4. Validation gates, fallback behaviour, and observable runtime state also preserve a meaningful role for human oversight. Human factors such as trust, workload, and decision support, and the development of a general-purpose data-governance platform, remain outside the evaluated scope.

**Privacy and data sovereignty.** Distributed and edge execution can reduce the movement of raw data and support local autonomy during communication loss, but these properties do not provide a complete privacy method. Access control, retention, inference attacks, privacy-preserving learning, and data-governance policy require separate treatment. Privacy constrains architectural choices and deployment assumptions without becoming an independent thesis claim.

**Mixed objectives and baseline discipline.** Personalised and community-level objectives are central characteristics of the control problem and already appear in prior work through local and collective objectives, hierarchical coordination, and market-based interaction (Faia et al. 2023; Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Srithapon and Månsson 2023). The thesis evaluates both participant and community outcomes without presenting mixed objectives as an unaddressed gap. This evidence also imposes an important evaluation condition: MARL must be compared with credible rule-based, optimisation, or MPC alternatives whenever the corresponding model and computational assumptions make those alternatives feasible. Otherwise, apparent gains can originate in a weak baseline instead of the multi-agent learning formulation.

### 2.8.2 Important Challenges Beyond the Central Scope

Other concerns remain outside the central scientific claims of the dissertation. They are acknowledged because they matter to operational CPSs, but resolving them would require methods and evidence beyond those developed here.

**Security, assurance, and regulation.** Adversarial security requires explicit threat models and evidence against malicious access, data manipulation, compromised communication, or attacks on software and devices. This is distinct from the operational meaning of safety and robustness adopted in the thesis. Formal verification and certification present a related but separate assurance problem: empirical constraint satisfaction and runtime safeguards do not constitute a formal safety certificate. Regulation and market design likewise shape which energy-community actions are legally and economically admissible, but the thesis treats these as operating conditions rather than proposing new regulation, tariff structures, or market mechanisms.

**Explainability, fairness, and long-term adoption.** Operators and participants may require understandable reasons for actions, but explaining a multi-agent policy and its interactions is a research problem in its own right. Similarly, reporting individual outcomes can reveal unequal effects without providing a normative definition of fair benefit allocation. Long-term acceptance introduces further questions of trust, incentives, usability, and behaviour that cannot be established from controller performance or short-duration demonstrators alone. These topics remain relevant to the deployment of intelligent energy systems, but they require dedicated human, social, economic, or longitudinal evaluation and are therefore retained as limitations and future research directions rather than central claims.

The resulting scope is deliberately narrower than the full set of challenges associated with real-world intelligent control. RQ1–RQ4 address the residual problems for which the thesis provides a connected methodological and empirical response; the concerns above either constrain that response or define important work that remains beyond it.

## 2.9 Synthesis of Residual Gaps and Thesis Requirements

The state of the art supports a narrower conclusion than the initial motivation alone might suggest. Dynamic populations, heterogeneous agents, large-scale MARL, constraint-aware control, robustness mechanisms, real-world deployment, experiment tracking, registries, rollback, and edge orchestration have all been demonstrated in some form. Limited evidence remains for their coherent combination when heterogeneous agents, observation and action interfaces, physical constraints, information quality, computational resources, and deployed policies change within the same distributed CPS.

The four research questions carry forward requirements conditioned by what prior work already establishes. Table 2.11 summarises the link between state-of-the-art evidence and the evidence required from the thesis.

Table 2.11: Residual requirements derived from the state of the art and their connection to the research questions.

| RQ | What prior work already establishes | Residual requirement carried into the thesis | Principal evidence required |
|---|---|---|---|
| RQ1 | Session dynamics, variable problem sizes, agent loss, changing graphs, dynamic participation, and reconfigurable co-simulation components have all been demonstrated (Hardy et al. 2024; Orfanoudakis, Panda, et al. 2025; Wu, Qiu, et al. 2024; Yeh et al. 2023; Yu, Yang, et al. 2023). | Control must remain meaningful when runtime membership changes coincide with heterogeneous capabilities, observation and action interfaces, local constraints, and compatible policy operation. | Typed and changing scenarios; controller comparisons across populations and capabilities; reconfiguration tests; integrated operational evidence. |
| RQ2 | Prior work demonstrates scaling in population size, physical-network size, topology, simulator throughput, parallel execution, and operational controller count (Almilaify et al. 2023; Blum et al. 2021; Fan et al. 2022; Orfanoudakis, Diaz-Londono, et al. 2025; Ponse et al. 2025). | Scalability must be evaluated across interacting dimensions rather than inferred from agent count alone, including heterogeneity, representation size, computation, communication, experimentation, reconfiguration, and edge execution. | Simulator and runtime benchmarks; repeated learning runs; compute, memory, communication, inference, orchestration, and control-quality measurements. |
| RQ3 | Constraint-aware learning, stochastic control, shielding, masking, runtime projection, safe-control environments, and selected disturbance tests are established techniques (Chen, Liu, et al. 2023; Eichelbeck et al. 2026; Puech et al. 2024; Yu, Yang, et al. 2023). | Safety and useful control behaviour must be assessed together across physical constraints, uncertain or degraded information, faults, dynamic reconfiguration, communication imperfections, and shifts between training and operation. | Constraint and service metrics; controlled disturbance campaigns; health and reconfiguration tests; fallback and recovery behaviour; evidence separated by simulation, testbed, and operational level. |
| RQ4 | Operational MARL, RLOps proposals, distributed RL/MARL frameworks, flexible policy mapping, registries, packaging, rollback, edge orchestration, and dynamic co-simulation show that the individual lifecycle mechanisms exist (Hardy et al. 2024; Hu et al. 2023; Li, Thomas, et al. 2022; Liang et al. 2018; Zaharia et al. 2018; Zhang, Zhang, Quiñones-Grueiro, et al. 2024; Zhou et al. 2023). | Interdependent policies must be governed as compatible policy sets with coupled environment–policy lineage, validation and promotion, staged distributed deployment, coordination-aware monitoring, rollback, and recovery. | Experiment reconstruction; compatibility tests; policy-set manifests; deployment and mixed-version tests; monitoring, rollback, disconnected-edge, recovery, and sustained-operation evidence. |

Two evaluation principles cut across all four questions. MARL is treated as one controller family among several. Rule-based control, optimisation, and MPC remain relevant comparisons or complementary mechanisms whenever their modelling and computational assumptions are credible. Evidence levels are also kept distinct: a simulator capability, an algorithmic result, successful execution on target hardware, shadow operation, a physical testbed, and sustained closed-loop operation support different claims.

The requirements are also interdependent. Dynamic heterogeneity changes representation and coordination cost, complicates safety assessment, and creates new deployment compatibility conditions. Safety mechanisms can alter performance and runtime cost. Monitoring and rollback support Deployability while also contributing to Robustness and recovery. Scalability must be demonstrated through control quality and resource use across the layers at which growth actually

occurs; one population-size result cannot establish it.

These requirements define the hand-off from the state of the art to the remainder of the dissertation without prescribing the solutions in advance. Chapter 3 assigns them to architectural responsibilities; Chapter 4 establishes the data, scenario, and simulation evidence needed to test them; Chapters 5 and 6 develop and evaluate the control methods; Chapter 7 addresses governance and distributed operation; and Chapter 8 examines their interaction in integrated energy settings. Chapter 9 then returns to RQ1–RQ4 and answers them against the complete body of evidence.

## 2.10 Chapter Summary

This chapter established the state-of-the-art basis for the thesis by moving from distributed and dynamically reconfigurable CPSs to their concrete expression in Renewable Energy Communities, the control approaches available for those systems, and the RL and MARL concepts needed by the remainder of the dissertation. It also distinguished the levels of experimental evidence used later, from controlled simulation to operational closed-loop control.

The literature shows that many capabilities relevant to real-world MARL already exist individually. The synthesis identifies the residual requirements that appear when heterogeneity, reconfiguration, scale, safety, robustness, and policy lifecycle constraints must be handled together. Section 2.9 carries those requirements forward through RQ1–RQ4, while Section 2.8 keeps adjacent challenges and the limits of the thesis explicit.

Chapter 3 now translates these requirements into the architectural responsibilities of EnergAIze, establishing the common structure for the simulation, control, learning, lifecycle, and operational contributions developed in the following chapters.

# Chapter 3

# EnergAIze: An Integrated Framework for Intelligent Energy Management

Control of a Renewable Energy Community extends well beyond the algorithm that selects an action. Before a controller can decide when an electric vehicle should charge, it must receive coherent measurements from different platforms, estimate information that is not directly observable, understand the user's flexibility, and translate its decision into a command accepted by the physical device. The same path must remain useful when measurements are delayed, a vehicle does not expose its state of charge, an asset becomes unavailable, or an external platform rejects an action. Data integration, prediction, optimisation, learning, safety, and deployment form one control system in a real community.

EnergAIze is the integrated framework developed in this thesis to connect those concerns. It turns heterogeneous energy and mobility data into supervised decisions for flexible assets while preserving a controlled route for simulation, learning, evaluation, packaging, and runtime execution. Percepta and the Data Provider establish the integration boundary; prediction and vehicle services complete the controller's view of each installation; the Algorithms component produces local and community-aware decisions; the Fail-Safe Supervisor preserves physical and user constraints; CityLearn and EVLearn provide the experimental environment; and the inference and lifecycle services carry selected controllers towards real execution (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Fonseca, Sousa, et al. 2025; Salvan and OPEVA Consortium 2026; Sousa et al. 2025).

The architecture is easiest to understand by widening the scope progressively. A single charging decision first exposes the information, safety, robustness, and execution requirements that exist even inside one building. Repeating that decision across heterogeneous participants then exposes the scale, coordination, and deployment problem. Only after these pressures are visible is the detailed component architecture introduced. This progression follows the way the research problem emerged in practice: scalability, deployability, safety, and robustness arise from the control setting and are visible before the software diagram is complete.

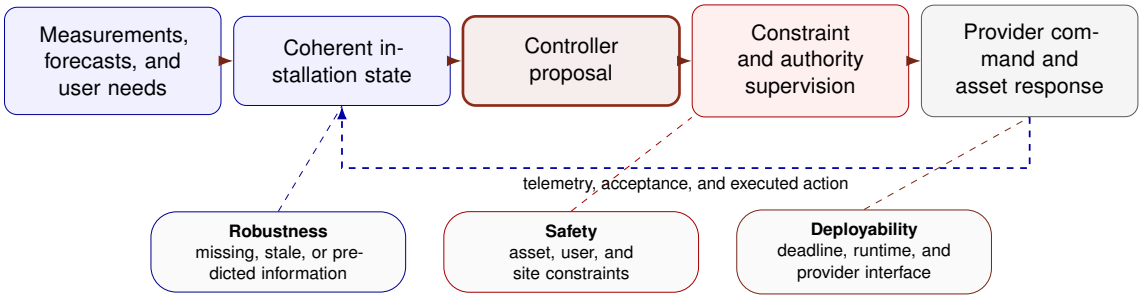



Figure 3.1: Architectural View I: the smallest complete EnergAIze decision loop. Heterogeneous information is converted into a coherent state, a controller proposes an action, supervision preserves physical and user constraints, and the external platform reports what was accepted and executed.

## 3.1 Architectural View I: A Supervised Energy Decision

Consider one decision for an EV connected to a prosumer building. The site meter and photovoltaic inverter may report through an energy-management platform, the charger through a charging-service platform, and the vehicle through a manufacturer-facing service. Their messages use different identifiers, units, sampling periods, and communication mechanisms. Consumption and photovoltaic production over the decision horizon are uncertain forecasts. The intended departure time and minimum departure energy can originate in a recurrent routine, a prediction, or an explicit user request. Before any optimisation is attempted, the controller must know what each value means, which physical entity it describes, when it was produced, and how much confidence can be placed in it.

The resulting installation state is not yet an action. A controller interprets that state together with prices, objectives, flexibility windows, and current constraints and proposes a charging, discharging, storage, or deferral decision. A supervision stage then checks whether the proposal remains admissible for the connected asset and the installation. Only the supervised result is translated into the protocol and command expected by the external platform. Telemetry and the provider response close the decision loop, allowing the system to distinguish what was proposed, what was requested, and what the device actually executed.

Figure 3.1 deliberately omits component names. It captures the smallest complete EnergAIze control loop and the first set of problems revealed by it.

Three cross-cutting requirements are already unavoidable in this single decision. *Robustness* is required because a number in an observation does not reveal whether it is measured, forecast, stale, substituted, or unavailable. The appropriate response can range from normal inference to a contracted action range or conservative fallback. *Safety* is required because state-of-charge limits, charger power, site capacity, connection state, and the user's departure requirement cannot be traded freely against cost or community reward. They influence state construction, the controller formulation, and the final supervision of the command.

*Deployability* appears as soon as the action must leave the model. The controller must execute within the decision interval, reconstruct the same preprocessing used during training, produce a

named and bounded action, and communicate it through an interface that the provider and device accept. An action that is optimal in a notebook but cannot be interpreted, executed, or traced at the installation is not a usable energy decision.

Scalability remains mostly latent at this first view. The loop concerns one installation and one decision deadline. Its importance becomes visible when the same semantic and execution contract must be preserved across participants whose assets, objectives, interfaces, connectivity, and computing resources are not identical.

## 3.2 Architectural View II: A Distributed Renewable Energy Community

Now consider the same decision loop across a Renewable Energy Community. One participant may contribute only demand and photovoltaic measurements. Another may expose a stationary battery and one domestic charger. A household can have two chargers whose sessions and actions vary independently. An office or service building can host several chargers under one site limit, while a larger installation can add local controllers, three-phase limits, and different actuation authority. Treating these participants as copies of one fixed vector would hide precisely the heterogeneity that determines what each local controller can observe and do.

EnergAIze uses a repeatable installation-level control boundary whose contents are configured by the assets and information available at that site. Local observations, user requirements, asset constraints, and physical actions remain associated with their installation. Community coordination contributes only the shared context required by the selected control formulation, such as a price, aggregate exchange, peak, ramping signal, demand-response request, or authorised message from other agents. This keeps local physical decisions identifiable within the community objective.

Figure 3.2 widens the first view. Each installation retains the same semantic loop, but its asset composition and execution environment differ. The community layer connects these loops while preserving their local differences.

At this scope, *scalability* is no longer equivalent to adding buildings. Population increases the number of decisions, but heterogeneity also increases the number and meaning of observations and action groups. Several chargers inside one building increase local action coupling without increasing the agent count. Community coordination introduces communication and aggregation costs. Training adds simulator throughput, replay, model, and experiment-management demands. Deployment adds memory, latency, image, network, and target-management requirements. The architecture must allow these dimensions to be measured separately because improving one does not guarantee improvement in the others.

Deployability also becomes a distributed-systems concern. One installation can execute centrally, another on a server-side container, and another near its assets on an edge device. Training can use a personal workstation, an institutional server, an HPC system, or a remote GPU platform whose runtime, storage, authentication, and image requirements differ. A common experiment

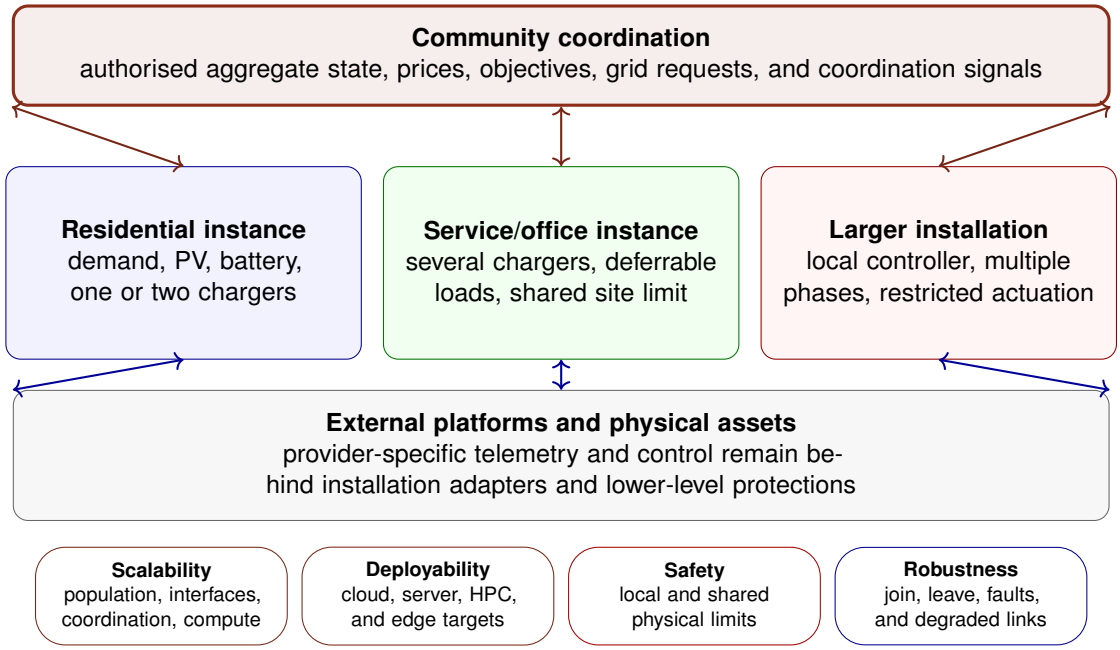



Figure 3.2: Architectural View II: heterogeneous installation instances participate in one community without being reduced to an identical asset model. The wider scope exposes population, interface, coordination, computing, deployment, shared-safety, and reconfiguration pressures.

and policy contract is needed so that changing the execution target does not silently change the scientific meaning of the run or the runtime meaning of the controller.

Safety and robustness become coupled across levels. A local charger action is constrained by its EV and circuit, while several chargers can compete for one site limit and several buildings can contribute to a shared grid condition. A participant can leave, an EV session can end, an asset can fail, or a community message can become stale. Local control should continue where its information and authority remain sufficient. Shared coordination must expose its degraded state and prevent obsolete context from producing an apparently valid action.

The two architectural views make algorithm selection only one part of the research problem. The control method operates inside a system that must preserve meaning across heterogeneous participants, increasing scale, physical constraints, degraded information, and different training and deployment environments.

## 3.3 Architectural View III: Functional Components and Interfaces

The complete component view assigns each of the preceding responsibilities to an explicit architectural boundary. It separates provider-specific communication from installation semantics, predictive context from measured state, policy inference from physical supervision, and experimental development from runtime authority. These separations allow individual components to evolve without requiring the complete control system to be rewritten.

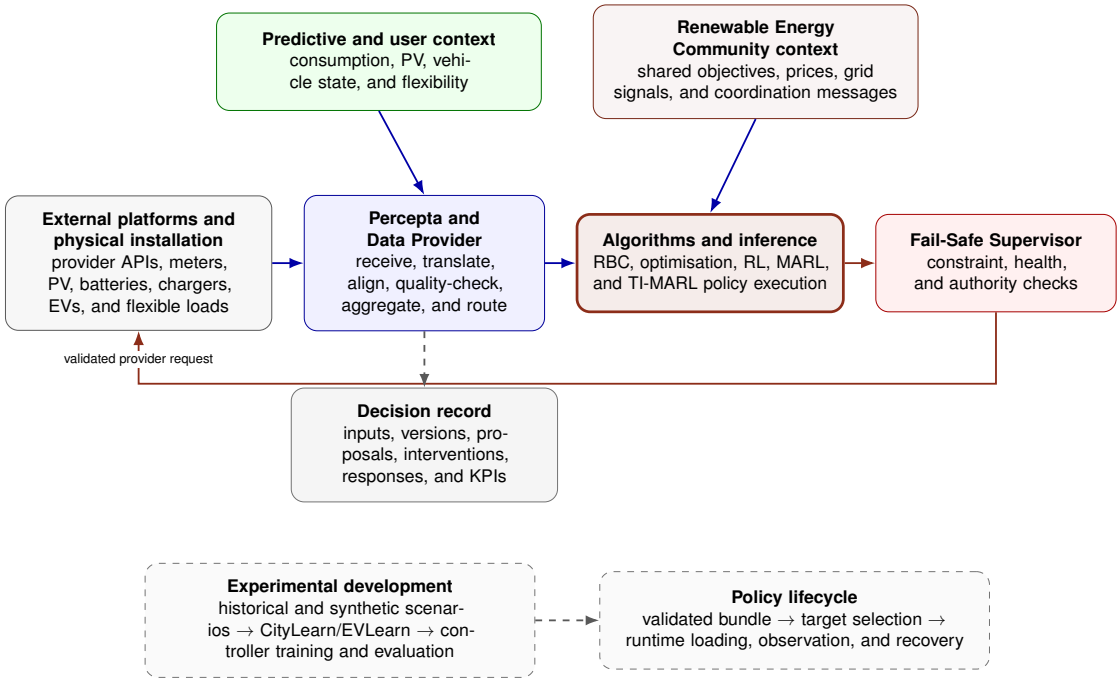



Figure 3.3: Architectural View III: functional EnergAIze components and interfaces for one installation. Provider integration, predictive context, controller execution, safety supervision, experimental development, and the policy lifecycle remain distinct while contributing to one closed control loop.

Figure 3.3 presents the functional architecture used throughout the remainder of the thesis. The figure shows one installation instance; the instance is repeated with different contents for the participants introduced in Architectural View II.

### 3.3.1 Percepta and the Data Provider

Percepta addresses the first obstacle to intelligent control: physical installations do not produce model-ready observations. Cleanwatts, my.i-charging, meters, local controllers, vehicle services, and other IoT providers expose different APIs, message formats, identifiers, units, and sampling rates. Coupling a controller directly to those details would make each provider or asset change a modification of the decision model. Percepta provides the stream-processing capability that separates transport and provider semantics from the installation-level representation (Sousa et al. 2025).

In the OPEVA instantiation, the Data Provider middleware realises this boundary for both historical-data preparation and runtime inference (Salvan and OPEVA Consortium 2026). A receiver contains the communication logic for a source, such as HTTP for a REST service or RabbitMQ for a message-based provider. A translator extracts the relevant fields and converts the provider message into the common EnergAIze representation. Installation queues and an accumulator decouple subsequent processing from the original source. A manager then aligns the

decision interval, checks missing values, reconstructs relationships among assets, and assembles the observation expected by the controller.

This organisation makes a new provider a bounded integration task: its receiver and translator can be added without modifying the other providers or the controller. It also keeps data treatment visible. Resampling a power measurement, interpolating a short gap, repeating a previous state, and substituting a forecast are not equivalent operations. The source, timestamp, quality, and transformation of a value are part of the information needed to determine whether normal control remains defensible.

The historical-data path retrieves and aligns installation records for simulation and training. The runtime path performs the same semantic work under a decision deadline, constructs an inference request, receives a supervised decision, and forwards the provider-specific request. Percepta and the Data Provider are bidirectional: they carry observations towards the controller and validated device requests towards the responsible external platform.

The OPEVA middleware calls the service that invokes an installation's AI model the *Predictor*. In this specific interface, the term denotes an inference gateway and does not refer to a forecasting model. Throughout the thesis, runtime inference executes a control policy, whereas prediction estimates future or unavailable quantities used by that policy.

### 3.3.2 Prediction, PulseCharge, and User Flexibility

A useful scheduling decision requires information that is not always measured directly. Near-term consumption and photovoltaic production influence whether flexible demand should be served immediately or deferred. A charger may report connection and delivered energy without exposing the EV state of charge. The user's departure time and required energy are behavioural service constraints, separate from electrical telemetry. EnergAIze represents each source explicitly and retains its meaning outside the numerical observation vector.

Consumption and photovoltaic-production services estimate the exogenous profiles used by the scheduling controller. Their outputs permit anticipation of a demand peak or local photovoltaic surplus and can provide a bounded substitute during a temporary sensor interruption. A forecast remains identifiable as a forecast, allowing the same controller to be evaluated with perfect, predicted, degraded, or unavailable information. Forecast models can evolve independently of provider adapters, safety supervision, and the control method.

PulseCharge supplies vehicle context that the charging infrastructure alone often cannot provide. It hides manufacturer-specific authentication and telemetry behind a common vehicle-data interface and retrieves state of charge, connection, and charging state where available. Request budgets and periodic-task supervision reflect the rate limits and temporary failures of manufacturer-facing services.

The associated flexibility model orders information by authority and confidence. A non-integrated or uncontrollable vehicle exposes no actionable flexibility. A recurrent schedule provides a low-interaction baseline. Flexibility Prediction estimates likely connection windows and energy

needs from historical behaviour. Explicit user input takes precedence when the current journey differs from the routine. This hierarchy prevents a statistically convenient prediction from silently overriding a declared departure requirement (Salvan and OPEVA Consortium 2026).

The Route Planner extends the same information model to an EV in transit by combining route, range, charger, price, and community information. It is an optional application component; installation-level control can operate without it. The evidence for its integration is bounded separately in Chapter 8.

### 3.3.3 Algorithms, Inference, and the Fail-Safe Supervisor

The Algorithms component is the decision-making core of EnergAIze. In the OPEVA demonstrator it is also described as the Flexibility Scheduling Algorithm. It receives the installation state assembled by the Data Provider, including measurements, prices, forecasts, asset availability, vehicle requirements, and the community information permitted by the selected formulation. It returns named actions such as a battery charge or discharge request, an EV charging limit, or the schedule of a deferrable load (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Salvan and OPEVA Consortium 2026).

Each installation controller represents local objectives and available assets. Community coordination introduces effects that cannot be inferred from one local meter alone, including aggregate peak, renewable sharing, ramping, and a common demand-response request. EnergAIze is not tied to one algorithm family: rule-based control provides an interpretable baseline and fallback, optimisation exposes explicit objectives and constraints, reinforcement learning supports adaptive local control, and MARL or hierarchical methods coordinate several participants. TI-MARL later addresses the fixed-interface assumptions that remain when assets, agents, and capabilities change. Their formulations and experimental evidence belong to Chapters 5 and 6; this chapter treats every method through its declared observation and action contract.

The energAIze inference runtime preserves that contract outside the training process. A deployable bundle associates policy artefacts with observation names, encoders, action names, action bounds, topology, and deployment metadata. The runtime reconstructs preprocessing and executes either a learned model or a deterministic policy behind the same request interface. Explicit aliases keep transport-specific field names outside the policy. Chapter 7 describes the bundle, runtime, and deployment mechanisms in detail.

The result of inference is a proposed action, not unconditional physical authority. The Fail-Safe Supervisor checks it against rated power, state-of-charge bounds, connection state, vehicle departure requirements, site and phase limits, unavailable observations, and the authority granted to the current controller. According to the installation and failure condition, it can bound or reject the action, retain a previously safe value, or invoke a conservative rule-based alternative. External platforms, local controllers, and device firmware retain their lower-level protection and can still reject the resulting command.

EnergAIze records the observation context, controller and model version, proposed action, supervised request, provider response, executed value where available, and relevant KPIs. These records separate policy behaviour from the protection applied around it. A stable physical trajectory accompanied by frequent supervisor corrections demonstrates an effective protection layer, but it does not by itself demonstrate a reliable learned policy.

Table 3.1 condenses the functional responsibilities without turning them into independent contributions.

Table 3.1: Principal EnergAIze components and their interfaces within the control loop.

| **Component** | **Principal input** | **Responsibility and output** |
|---|---|---|
| Percepta and Data Provider | Provider messages, identifiers, units, timestamps, and commands | Normalised installation state, data-quality context, and provider-specific forwarding |
| Prediction services | Historical and recent demand, PV, weather, and contextual variables | Forecast profiles with explicit provenance and confidence |
| PulseCharge and flexibility services | Vehicle telemetry, schedules, behaviour, and user declarations | Vehicle state, connection window, energy need, and flexibility authority |
| Algorithms and inference | Declared observations, objectives, constraints, topology, and policy bundle | Named local and community-aware proposed actions |
| Fail-Safe Supervisor | Proposed actions, health, authority, and physical/user limits | Bounded, rejected, retained, or fallback device requests |
| Decision record | Inputs, versions, proposals, interventions, provider responses, and telemetry | Traceable relation between context, proposed control, requested control, and observed execution |

## 3.4 Controller Development and Runtime Operation

### 3.4.1 CityLearn, EVLearn, and Experimental Development

Trial-and-error learning cannot begin on occupied buildings or connected vehicles. An exploratory charging action can violate a site limit, increase cost, or leave a user without the required departure energy. EnergAIze develops controllers against a simulated representation of the intended installations before considering real-world inference or actuation.

CityLearn provides the building, demand, photovoltaic, storage, weather, tariff, and community environment. EVLearn extends it with vehicles, chargers, connection sessions, battery dynamics, and V1G/V2G flexibility (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025). Historical data processed through the integration path can populate this environment, while synthetic and hybrid scenarios expose controllers to reconfiguration, faults, tight limits, and rare events that are impractical or unsafe to create physically. Rule-based, optimisation, RL, and MARL controllers can then be compared under the same assets, information, time period, and KPI definitions.

The simulator is an experimental representation of selected physical mechanisms. It does not copy the complete physical system. Provider latency, rejected commands, unavailable vehicle telemetry, human overrides, and electrical protections require explicit disturbance models or

separate real-world evidence. Chapter 4 establishes the scenario and simulation contracts; Chapter 8 identifies which assumptions survived interaction with external platforms and demonstrators.

Training and evaluation bind the controller configuration to a scenario, dataset, simulator version, seed, reward, and compute target. Logs, progress, KPIs, checkpoints, and the final policy artefacts remain associated with that job. This record allows a result to be reconstructed and prevents a model file from becoming detached from the world in which its behaviour was evaluated.

### 3.4.2 Runtime Topologies and Deployment Boundary

An EnergAIze instance can execute centrally for several installations or locally beside one installation. Central execution suits a community platform that already aggregates participant data. Edge execution suits an installation requiring lower latency, reduced data movement, or continued local service during a cloud interruption. These placements implement the same logical control loop, but they have different connectivity, hardware, packaging, observation, and recovery requirements.

The model runtime and integration middleware remain separable. A policy bundle can be updated without replacing every provider adapter, while a new provider receiver does not require the controller to be retrained solely because the transport protocol changed. In OPEVA, OCPP links chargers and the charging platform; Modbus TCP/IP links selected chargers, meters, and local controllers; OCPI carries charging information and profiles between platforms; and REST or RabbitMQ connects data and algorithm services. The integration boundary absorbs these protocols and keeps them outside the policy.

The lifecycle infrastructure connects personal and institutional compute, remote HPC and GPU platforms, shared experiment storage, policy export, and one or more inference services. Its purpose is not merely to make training faster. It ensures that the configuration executed on heterogeneous workers produces a traceable artefact that can be validated and loaded by the runtime without a manual reimplementation of preprocessing or action semantics. Chapter 7 develops this path and its failure semantics.

The presence of a component in the architecture does not assign the same evidence level to every site. An installation can contribute historical data, participate only in simulation, execute inference without actuation, or accept bounded physical commands. Chapter 8 reports OPEVA and DemFlex by evidence level and keeps their deployment conditions separate.

## 3.5 Architectural Rationale and Thesis Map

The progressive views establish the architectural argument. Architectural View I showed that a useful local decision already depends on robust information treatment, explicit safety boundaries, and an executable runtime contract. Architectural View II showed that a community multiplies the number of decisions, interface diversity, coordination, shared constraints, computing demands, and deployment targets. Architectural View III assigned those pressures to explicit functional boundaries. EnergAIze provides the common system context in which the later contributions can

be stated and evaluated. Evidence that the four requirements have been satisfied must still come from the experiments and demonstrators.

Table 3.2 records where the four requirements enter the architecture and where the thesis develops the corresponding methods and evidence.

Table 3.2: Traceability between the four cross-cutting requirements, EnergAIze mechanisms, and the detailed thesis chapters.

| **Requirement** | **Architectural manifestation** | **Detailed treatment** |
|---|---|---|
| Scalability | Repeatable heterogeneous installation instances; bounded local interfaces; explicit community information; separable simulation, training, orchestration, and inference resources | Simulator performance and scenario scale in Chapter 4; control and learning scale in Chapters 5 and 6; worker and runtime scale in Chapter 7 |
| Deployability | Stable data and action boundaries; self-contained controller bundles; central and edge topologies; target-specific execution behind common experiment and inference contracts | Policy lifecycle, heterogeneous workers, packaging, inference, monitoring, and recovery in Chapter 7; demonstrated integrations in Chapter 8 |
| Safety | Entity, user, site, and shared constraints; distinction between proposal and executed action; independent supervision and lower-level protection | Constraint representation in Chapter 4; controller mechanisms in Chapters 5 and 6; runtime evidence in Chapter 8 |
| Robustness | Provenance and quality of observations; explicit health and availability; local degraded operation; traceable failures, reconfiguration, fallback, and recovery | Disturbance scenarios in Chapter 4; robust and dynamic control in Chapter 6; lifecycle recovery in Chapter 7; real-world boundaries in Chapter 8 |

The heterogeneous installation model also establishes the architectural setting for RQ1: participants, assets, capabilities, and interfaces are allowed to differ and change while local and community objectives remain connected. RQ2 is investigated through the several dimensions of scalability made visible in the second view. Safety and robustness provide the two complementary parts of RQ3. The experiment, bundle, deployment, monitoring, and recovery path provides the system setting for RQ4.

This map determines the remaining thesis progression. Chapter 4 constructs the controlled world in which the relevant conditions can be represented and repeated. Chapter 5 develops progressively more capable controllers, while Chapter 6 addresses dynamic typed interfaces as the principal algorithmic contribution. Chapter 7 governs experiments and policy execution across heterogeneous infrastructure. Chapter 8 then returns to the complete EnergAIze architecture and examines which components, interfaces, and control paths were exercised in OPEVA and DemFlex.

## 3.6 Chapter Summary

EnergAIze connects heterogeneous energy and mobility information to a supervised physical decision. Its architecture becomes necessary progressively. A single decision exposes robust information treatment, physical and user safety, and an executable deployment contract. A distributed community adds multidimensional scale, heterogeneous interfaces, shared constraints, reconfiguration, and heterogeneous computing targets. Percepta, prediction, PulseCharge, the Algorithms and

inference services, the Fail-Safe Supervisor, CityLearn and EVLearn, and the experiment lifecycle give these responsibilities explicit boundaries.

The framework integrates the research context while the methods and evidence remain the responsibility of the later chapters. The next chapter begins that evidence chain by establishing the data, scenario, and simulation foundation used to train and compare controllers under the conditions revealed by the architectural views.

# Chapter 4

# CityLearn v3: A Simulation and Evaluation Environment for Renewable Energy Communities

A controller can appear successful for the wrong reasons. It can reduce electricity cost by leaving an electric vehicle below its required departure state of charge, postpone a flexible load beyond its deadline, request more charging power than the building connection can supply, or exploit measurements that would be incomplete in operation. These failures are difficult to recognise when an experiment represents a Renewable Energy Community (REC) as a fixed set of buildings, assumes one uniform and perfectly observed clock, and reports only aggregate energy or cost.

The simulator developed in this thesis addresses that methodological problem. Starting from the CityLearn lineage, it extends the environment from a predominantly building-centred benchmark into CityLearn v3: an auditable, dataset-driven simulation and evaluation environment for realistic REC control studies. The extension combines heterogeneous buildings and flexibility assets, explicit temporal and unit semantics, local energy sharing, demand-response services, changing participants and equipment, controlled data and equipment failures, structured controller interfaces, multiple communities, service-aware reference controllers, and traceable evaluation outputs.

The purpose is not to reproduce every physical process at maximum fidelity. It is to represent, with sufficient fidelity and explicitness, the mechanisms that can change the conclusion of a controller experiment. The resulting environment is the experimental substrate for the controller progression in Chapter 5 and the typed-interface methods in Chapter 6. It also provides the controlled counterpart to the OPEVA and DemFlex evidence examined in Chapter 8.

The central argument is methodological. A REC controller should be evaluated against a declared scenario, under the same service and physical rules as its reference controller, with a record that connects what the controller observed, what it requested, what the simulator applied, and what outcome followed.

## 4.1 The Simulator as an Experimental Research Contribution

At the beginning of this research, the simulator could reasonably have been treated as supporting infrastructure: a means of training a controller and obtaining the results needed elsewhere in the thesis. That interpretation became inadequate as the research moved from conventional building-level demand response towards heterogeneous RECs, EV charging, flexible services, changing system composition, and field-informed operating conditions. Under these conditions, the environment determines which physical and service obligations exist, what information reaches the controller, which actions are feasible, and how success is measured. The simulator became part of the experimental method itself.

A missing simulator capability can change the conclusion of an experiment. If mobility requirements are omitted, a controller can reduce cost by withholding energy from an EV. If a short connection peak is averaged into an hourly value, an electrically infeasible action can appear acceptable. If a changing asset population is converted into a fixed vector before it reaches the policy, an experiment no longer evaluates adaptation to online composition change. If requested and applied actions are not separated, a failed command can be misinterpreted as a poor control decision. Each omission weakens the construct validity of the reported comparison.

The substantial effort devoted to CityLearn v3 followed directly from the research questions. The controllers developed in this thesis could only be assessed under realistic heterogeneity and change if the environment could express those conditions, apply them consistently, and retain enough evidence to explain their effects. Simulator development, controller development, and demonstrator interpretation progressed together. New operating requirements exposed missing environmental semantics; implementing those semantics changed the controller contract; and the revised contract required new baselines, indicators, tests, and documentation.

### 4.1.1 Research Instrument and Public Software

CityLearn v3 emerged through repeated movement between three contexts. Algorithm development identified what controllers needed to observe and control; OPEVA and DemFlex exposed the timing, integration, asset, and data conditions that occur outside a benchmark; and simulator experiments made those conditions repeatable before a controller was taken to a physical site. These contexts formed a feedback loop. A field constraint was formalised as a scenario or service contract, introduced under controlled conditions, evaluated with a reference controller, and then reflected in the interfaces used by the learning and deployment pipeline.

The importance of this loop is visible in several design decisions developed later in the chapter. Absolute measured PV data prevent a normalisation convention from altering the energy balance. Requested and applied actions distinguish policy behaviour from physical enforcement or command failure. Dynamic topology events retain the identity and active period of a changing entity instead of hiding the change through padding alone. Mobility, cycle, and demand-response indicators state whether the requested service was delivered, while participant-level settlement prevents a

favourable community total from concealing an unfavourable allocation. These are simulator features, but each was introduced to protect an experimental inference.

The resulting scope is intentionally broader than the minimum needed to run any one controller reported in this thesis. That breadth allowed the same environment to support conventional RBC and optimisation baselines, single-agent and multi-agent learning, entity-structured TI-MARL experiments, robustness studies, and multi-community execution. It also reduced the need to create a separate, weakly documented simulation script for every study. Scientific value comes from the inspectable common environment and the attribution it permits across controller experiments; reuse is a useful consequence.

**Public release and external reuse.** The simulator subsequently became a research output in its own right. Throughout this thesis, *CityLearn v3* denotes the generation developed in this work and distributed by the SoftCPS Research Group as `softcpsrecsimulator`, while retaining the `citylearn` Python import path for migration. The source identifies it as a research fork that extends, but does not replace, the separate upstream CityLearn codebase (Intelligent Environments Laboratory 2026; SoftCPS Research Group 2026a). At the thesis snapshot of 20 August 2026, version 1.7.0 was publicly available through the Python Package Index under the MIT licence, with source and wheel distributions, scenarios, examples, tests, a user interface, migration records, and bilingual documentation (SoftCPS Research Group 2026b).

Public release made the environment reusable beyond the experiments reported here and led to technical exchanges with researchers from several universities. These contacts indicate external relevance and initiated reuse and collaboration, but are not treated as a measured adoption rate or independent validation. CityLearn v3 enabled the thesis experiments, preserved the contracts required by the controller-to-deployment pipeline, strengthened the method used to evaluate REC controllers, and created a public basis for subsequent research.

### 4.1.2 CityLearn v3 Across the Controller Lifecycle

CityLearn v3 connects controller development to deployment by preserving the assumptions under which a policy was produced and challenged. Its controlled environment, compatibility information, and replay evidence support decisions made before and after deployment. Edge execution and field certification remain outside the simulator's responsibility, which keeps simulation and operation as distinct evidence levels inside the EnergAIze lifecycle.

During *controller development*, the simulator binds the policy to a declared scenario: data and event versions, timestep and units, active entities, observations, actions, rewards, constraints, reference controllers, seeds, and evaluation outputs. RBC, optimisation, RL, MARL, and TI-MARL methods can be compared under the same physical and service obligations. The resulting checkpoint remains accompanied by the experimental conditions that give its reported performance meaning.

During *pre-deployment qualification*, the same contract becomes a source of challenge cases and compatibility checks. Topology changes, missing measurements, forecast deviations, command

loss, equipment unavailability, phase limits, mobility deadlines, and demand-response requests are introduced reproducibly. The resulting evidence shows whether a candidate depends on unavailable inputs, assumes a different timestep or asset role, repeatedly requests infeasible actions, or loses required service under the declared disturbances. Chapter 7 uses these artefacts as inputs to promotion, packaging, and target-compatibility decisions.

During *operation and recovery*, selected field traces and incidents can be translated back into regression or replay scenarios. A problematic action sequence can then be reconstructed against the relevant software, interface, event, and constraint versions before a policy is updated, rolled back, or returned to service. This feedback path is deliberately asymmetric: field evidence can strengthen the simulator's challenge set, whereas a successful simulation remains only one part of the evidence required for safe operation.

The handoff across these stages uses a versioned evidence package. It links scenario declaration, controller-visible information, requested and applied actions, service and physical outcomes, software identity, and evaluation records. The package provides a verifiable connection between experimentation and deployment while the runtime adapters, monitoring, governance, and physical demonstrators retain their distinct responsibilities.

This lifecycle role connects the simulator to all four research questions without assigning one requirement to one chapter. For RQ1, it represents stable identities, heterogeneous capabilities, and online changes to entity, observation, and action sets. For RQ2, it varies population, community count, temporal resolution, interface payload, and computational workload. For RQ3, it makes physical and service constraints explicit, separates requested from applied actions, and introduces repeatable data, communication, equipment, and topology disturbances. For RQ4, it preserves the scenario, interface, constraint, baseline, software, and result lineage needed for reconstruction, compatibility checks, incident replay, and revalidation.

These mechanisms make scalability measurable, deployability traceable, safety constraints observable, and robustness deviations reproducible. The simulator supplies the controlled conditions and evidence contract; the controller chapters determine algorithmic behaviour, Chapter 7 governs transfer and continued operation, and Chapter 8 establishes the available real-world evidence.

## 4.2 Problem Definition, Requirements, and Positioning

Consider one plausible operating day. A household joins the community and its photovoltaic (PV) system enters the local balance. An EV arrives at a charger and must receive a specified amount of energy before departure. A dishwasher or industrial process can be shifted, but its full cycle must still finish before a deadline. During the evening peak, a system operator asks the community to reduce import. At the same time, a meter stops reporting and one battery command is lost. None of these conditions is exceptional in isolation; the scientific difficulty lies in representing them together without making the controller interface or the reported result ambiguous.

Three roles follow from this problem. First, the simulator is a *comparative benchmark*: rule-based control (RBC), mathematical optimisation, single-agent reinforcement learning (RL), multi-agent reinforcement learning (MARL), hierarchical control, and TI-MARL can be evaluated under common data, assets, constraints, services, and indicators. Second, it is a platform for *controlled intervention*: topology changes, missing observations, forecast errors, action failures, and demand-response requests can be introduced reproducibly without exposing a physical pilot to experimental risk. Third, it is an *integration rehearsal*: the scenario, entity, observation, and action contracts used during simulation can also inform training services, inference packages, and operational adapters. This third role narrows the simulation-to-operation gap, but does not turn the environment into a fully synchronised digital twin.

### 4.2.1 Limitations of the Inherited Environment and Required Experimental Contract

CityLearn v2 and EVLearn provided a strong starting point: a common simulation loop, coordinated multi-building control, established energy and cost indicators, storage and thermal assets, and an initial representation of EVs, chargers, connection schedules, and bidirectional charging (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025). The research developed here, however, required these mechanisms to remain coherent when participants and assets changed during an episode, temporal resolution exposed short power peaks, several devices competed for a building or phase connection, service deadlines became part of the objective, and data, commands, or equipment could become unavailable.

This gap cannot be closed by adding isolated asset classes. Introducing a charger during an episode, for example, also requires persistent identity, an event model, updated observations and actions, electrical headroom enforcement, mobility accounting, stable exports, and compatibility tests. The same dependency extends across scenario definition, physical progression, controller visibility, action feasibility, evaluation, and reproducibility. A plausible trajectory is not sufficient if a baseline receives different information, an infeasible request is silently clipped, or the export omits the event that explains a KPI.

Table 4.1 provides the formal summary of this transition. The first eight requirements define what the simulated REC must represent; the remaining five define the evidence needed to interpret

and reproduce a controller comparison. Together they turn the inherited simulation basis and its identified research pressures into an experimental contract.

Table 4.1: Requirements used to develop and evaluate the extended simulation environment.

| ID | Requirement | Experimental consequence |
|---|---|---|
| D1 | Multiple communities | Preserve local physics, controllers, and results while also reporting portfolio outcomes. |
| D2 | Demand-response requests | Declare the issuer, direction, target, window, baseline, delivered response, shortfall, and settlement. |
| D3 | Local sharing and settlement | Retain participant-level local import/export, residual grid exchange, and economic allocation. |
| D4 | Changing members and assets | Apply time-stamped additions and removals consistently to state, control interfaces, settlement, and evaluation. |
| D5 | Flexible-load services | Represent cycle profiles, start windows, completion deadlines, and missed or completed service. |
| D6 | Data and equipment failures | Introduce repeatable changes to observations, forecasts, actions, and logical equipment availability. |
| D7 | Time and unit semantics | Declare the timestep and distinguish power, energy per step, price, emissions, and normalised versus measured data. |
| D8 | Electrical-service limits | Enforce total and per-phase import/export limits while preserving requested and applied power. |
| S1 | Controller traceability | Record controller-visible inputs, availability, feasible actions, requested actions, and applied actions. |
| S2 | Multi-criteria evaluation | Report energy, cost, emissions, service, settlement, robustness, and constraint indicators with inspectable trajectories. |
| S3 | Fair reference operation | Compare controllers against references that face the same services, limits, disturbances, and accounting rules. |
| S4 | Software and physical checks | Protect temporal, physical, interface, accounting, and export behaviour through repeatable tests and audits. |
| S5 | Computational practicality | Measure loading, step, memory, export, interface, and multi-community costs under declared workloads. |

These requirements originated in the attempt to translate real REC and charging problems, including those encountered in OPEVA and DemFlex, into repeatable controller studies. They do not imply field validation of every simulator capability. Instead, they identify the operational assumptions that must be made explicit before a simulation result can support a stronger claim.

### 4.2.2 Lineage and position within existing environments

CityLearn introduced a common Gym-compatible environment for demand response and coordinated control across multiple buildings (Vázquez-Canteli, Kämpf, et al. 2019). The CityLearn Challenge added shared tasks, data, held-out evaluation, and benchmark governance (Nagy et al. 2021). CityLearn v2 broadened the represented assets, occupant and resilience mechanisms, and reference control methods, while EVLearn extended the lineage with EVs, chargers, connection schedules, charging requirements, and grid-to-vehicle/vehicle-to-grid operation (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025). This lineage provides familiar centralised and decentralised interfaces and established building- and district-level indicators.

It also exposes the limits of a static, building-centred formulation. Fixed flat observation vectors become difficult to interpret when participants or assets change. Hourly data can conceal power–energy mistakes and short peaks. A requested action may differ from physical operation because a vehicle has disconnected, a phase limit is binding, or a command has failed. Finally, aggregate cost and import do not show whether mobility, flexible-load, demand-response, or participant services were delivered.

CityLearn v3 retains useful compatibility with its lineage and adds explicit services around the simulation kernel. Representative environments already provide EVs, high-fidelity buildings, power-flow simulation, accelerated execution, and dynamic co-simulation. The contribution developed here is their integration into the experimental contract required for auditable REC controller studies, together with the evidence retained across scenario declaration, control, and evaluation.

Table 4.2: Role-based positioning against representative energy-control environments.

| **Environment family** | **Strongest relevant capability** | **Boundary relevant to this thesis** |
|---|---|---|
| CityLearn, its Challenge, and EVLearn | Shared community scenarios, multi-building control, established KPIs, EV charging, and V2G (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nagy et al. 2021; Nweye, Kaspar, et al. 2025) | Direct software lineage; online heterogeneous interface change and its audit trail are not established by asset presence alone. |
| ACN-Sim, SustainGym, and EV2Gym | Real charging sessions, charging-network constraints, configurable EV/V2G scenarios, and single- or multi-agent interfaces (Lee et al. 2021; Orfanoudakis, Diaz-Londono, et al. 2025; Yeh et al. 2023) | EV arrival and departure can occur while the charger-agent roster remains fixed; variable scenario size is not necessarily online agent entry. |
| Sinergym and BOPTEST | Reproducible, high-fidelity EnergyPlus- or Modelica-based building emulation (Blum et al. 2021; Campoy-Nieves et al. 2025) | Stronger references for detailed building physics; their purpose differs from high-throughput heterogeneous REC/MARL studies. |
| PowerGridworld, CommonPower, and PowerGym | Power-flow-aware objectives, heterogeneous grid devices, safeguards, and large feeder cases (Biagioni et al. 2022; Eichelbeck et al. 2026; Fan et al. 2022) | Stronger references for network physics and formal safety; site and phase limits in CityLearn v3 do not replace a feeder solver. |
| HELICS and mosaik | Distributed co-simulation, communication-aware studies, hardware-in-the-loop, and runtime federation changes (Gryga and Rossi 2021; Hardy et al. 2024) | Dynamic simulator composition does not itself define policy compatibility, typed observations/actions, or safe MARL adaptation. |

This comparison also clarifies the meaning of *dynamic*. An EV session can begin or end while a known charger remains present; different runs can instantiate different numbers of assets; a variable-set policy can consume a changing collection; and an agent or an entirely new interface can appear online. These are distinct levels of change, as established in Chapter 2. The simulator exposes several of them explicitly and makes the controller's response the object of evaluation.

## 4.3 CityLearn v3 Architecture and REC Modelling Capabilities

### 4.3.1 System Architecture and the Scenario-to-Evidence Path

CityLearn v3 is organised around one responsibility: converting a declared REC scenario and a controller request into an auditable physical and service outcome. Figure 4.1 follows this path from the scenario boundary to the evidence boundary. Within each REC, buildings retain their demand, generation, assets, electrical-service limits, services, settlement, and controller-facing identities. An enclosing orchestration layer can advance several independent RECs on a common clock without erasing their local state, policies, or results. Grid exchange, demand-response requests, topology changes, and data or equipment disturbances enter through explicit contracts and require no controller-specific modification.

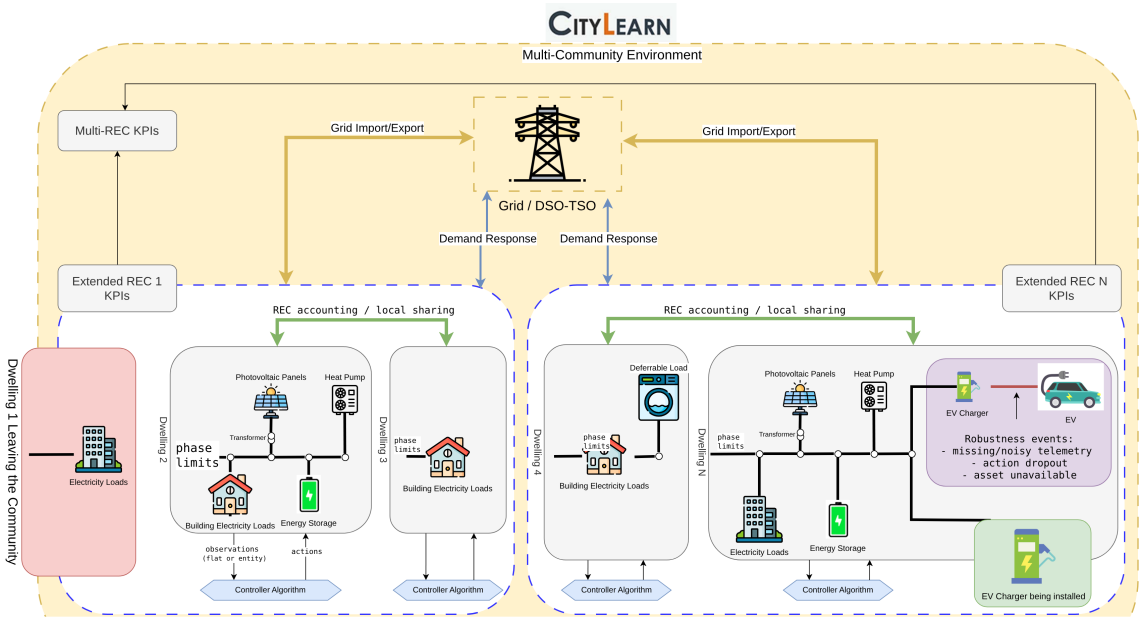



Figure 4.1: CityLearn v3 representation of one or more Renewable Energy Communities. Each community retains its buildings, assets, controllers, local exchange, events, and KPIs; the enclosing environment synchronises communities and derives combined results. Adapted from the CityLearn v3 simulator manuscript developed alongside this thesis.

The architecture separates four functional layers whose outputs form one continuous execution path. Table 4.3 states the responsibility and observable evidence of each layer. This structure has a direct experimental role: a disturbance can be varied without rewriting a policy, an interface can change without altering physical equations, and a KPI can be added without changing the state transition. A matched experiment can then identify which assumption changed and where its effect entered the result.

Table 4.3: Functional layers of CityLearn v3 and their role in an auditable controller experiment.

| Layer | Responsibility | Evidence retained |
|---|---|---|
| Scenario, data, and events | Resolves the clock, units, measured and forecast data, entities, services, topology changes, demand-response requests, and controlled disturbances. | Schema and software versions, source provenance, checksums, seeds, event calendars, active periods, and declared interfaces. |
| Simulation kernel | Advances building and asset states, energy balances, service progress, electrical limits, local sharing, and settlement. | Physical trajectories, state bounds, balances, feasible headroom, service state, and participant/community exchanges. |
| Control and evaluation | Constructs flat or entity-structured inputs, receives requested actions, applies feasibility and availability rules, calculates rewards and KPIs, and exports results. | Controller-visible values, masks, requested/applied/rejected actions, reasons, KPI rows, and time-series outputs. |
| Assurance | Checks schema integrity, physical and accounting invariants, interface stability, regression behaviour, and computational cost. | Test reports, golden cases, audit manifests, performance measurements, and release-linked validation records. |

The assurance layer distinguishes four levels of evidence. Software verification covers unit, integration, regression, and export behaviour; physical and accounting checks cover time conversion, energy balance, state bounds, action feasibility, settlement, and KPI identities; behavioural demonstrations confirm that each declared mechanism produces an observable trace; and external validity asks whether the selected abstraction is adequate for the real process behind a claim. The first three levels are reproducible within the simulator and their detailed audits are retained in Appendix A.4. The fourth relies on field or specialised-simulator evidence and is addressed through the demonstrators and explicit scope statements. Inherited thermal and HVAC models are preserved for compatibility; no new CityLearn v3 physics contribution is claimed for them.

At each timestep, events scheduled for that instant are first resolved; the controller-visible state and available actions are then constructed; the policy returns a requested action; feasibility, availability, command, building, and phase rules determine the applied action; and the simulation kernel advances the physical and service state. In compact form,

$$\begin{aligned} \tilde{o}_t &= \mathscr{O}(x_t, w_t, e_t), & a_t^{\mathrm{req}} &= \pi(\tilde{o}_t), \\ a_t^{\mathrm{app}} &= \Gamma(a_t^{\mathrm{req}}, x_t, e_t), & (x_{t+1}, y_t) &= F(x_t, a_t^{\mathrm{app}}, w_t, e_t), \end{aligned} \tag{4.1}$$

where $x_t$ is the physical state, $w_t$ contains exogenous data, $e_t$ contains active topology, service, or failure events, $\tilde{o}_t$ is the information actually exposed to the controller, and $y_t$ contains the step-level physical, service, and accounting outcomes. The feasibility operator $\Gamma$ deliberately separates controller intent from execution: $a_t^{\mathrm{req}}$ remains visible even when availability, equipment, building, phase, or command constraints reduce or reject it.

The complete trace associated with one decision is represented by the following evidence path:

$$\boxed{\mathscr{S}_t \longrightarrow \tilde{o}_t \xrightarrow{\pi} a_t^{\mathrm{req}} \xrightarrow{\Gamma} a_t^{\mathrm{app}} \xrightarrow{F} (x_{t+1}, y_t) \longrightarrow \mathscr{R}_t} \tag{4.2}$$

Here, $\mathscr{S}_t$ is the resolved scenario state at timestep $t$ and $\mathscr{R}_t$ is the archived evidence record from

which rewards, KPIs, service summaries, event logs, and exported trajectories are derived. This formulation makes the causal chain explicit: the result can be traced back to what existed, what was known, what was requested, what was physically applied, and which outcome followed. The remaining subsections explain the capabilities that implement each part of this path.

### 4.3.2 Scenario, Time, Unit, and Data Contracts

A dataset alone does not define a reproducible scenario. A time series does not state whether PV is an absolute measurement or a normalised profile, whether a price applies per kWh, which charger is connected to which building and phase, which services are mandatory, or which disturbances are active. CityLearn v3 defines the scenario through a schema, referenced data and event files, and versioned assumptions. The schema declares the simulation and episode windows, temporal resolution, units, buildings, assets, control mode, observations, actions, services, events, KPI scope, and export settings.

The same base data can support controlled interventions without changes to the simulator code. A nominal scenario can disable disturbances; a robustness scenario can add measurement and command failures; a topology scenario can activate time-stamped additions and removals; and a demand-response scenario can add externally issued service requests. These are different experimental scenarios even when their load and PV files are identical.

The physical contract uses kW for power and instantaneous limits, kWh per simulation step for energy-series columns, currency/kWh for prices, and $kgCO_2$/kWh for carbon intensity. If $P_t$ is an average power measurement in kW and $\Delta t$ is the step duration in seconds, the corresponding energy is

$$E_t = P_t \frac{\Delta t}{3600}. \tag{4.3}$$

The conversion is elementary but consequential. A 7.2 kW charger delivers 7.2 kWh in a one-hour step, 1.8 kWh in a 15-minute step, and 0.03 kWh in a 15-second step. Interpreting each value as 7.2 kWh would overstate the 15-second energy by a factor of 240.

The schema also distinguishes a normalised PV profile from measured production. The former is multiplied by installed capacity; the latter already represents the physical output and is not scaled again. Timestamp spacing, the declared simulation clock, and episode bounds are checked during loading so that a 15-second trace is not silently treated as hourly data. Fixtures exercise 60, 300, 900, and 3600-second operation, while dedicated datasets extend the checks to 15-second power traces.

Temporal resolution changes more than storage requirements. Figure 4.2 replays the same synthetic annual physical trace at three resolutions. Annual load, PV, and EV energy remain unchanged, but the observed peak changes from 96.8 kW at one hour to 188.2 kW at 15 minutes and 281.5 kW at 15 seconds. On the selected day, the hourly view reduces a 231.0 kW short event

to 63.9 kW. This deterministic trace-replay test demonstrates why the timestep belongs to the scientific claim. It provides no field-validation evidence.

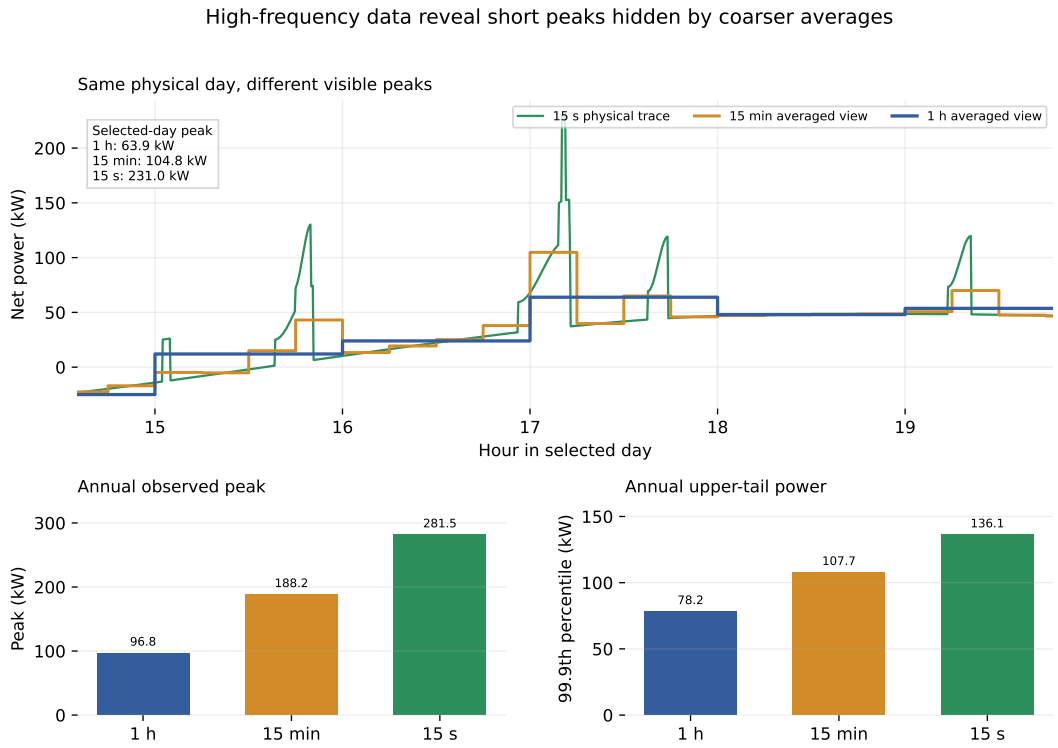



Figure 4.2: Peak visibility when the same physical trace is represented at one-hour, 15-minute, and 15-second resolution. Annual energy is preserved; the observable short-duration peak is not.

High-frequency access is implemented through windowed loading and columnar formats. In the evaluated evidence bundle, two 15-second CSV datasets of 2.446 and 2.619 GB were represented as 12.405 and 12.740 MB Parquet files. Initialisation time decreased from 25.37 to 15.61 s and from 28.59 to 18.92 s, respectively, while post-load step time remained similar. These figures characterise those datasets and encodings; they are not a universal compression ratio.

The simulator treats controllable demand as a collection of services embedded in buildings. Buildings retain their non-controllable demand, weather-dependent or inherited thermal components, PV generation, and grid exchange. Stationary batteries, EVs, chargers, and deferrable appliances introduce different state, availability, and service semantics. These distinctions are necessary because the same energy shift can have different consequences for mobility, comfort, appliance completion, and electrical feasibility.

### 4.3.3 Stationary storage and electrical-service limits

Stationary storage is represented through energy capacity, charge/discharge power, efficiency, and state-of-charge bounds. The simulator calculates the feasible action from the current state and retains feedback whenever the requested action is clipped or cannot be applied. This allows storage

to support self-consumption, peak reduction, price response, and demand-response headroom while keeping infeasible requests visible in the record.

Each building can additionally declare total import/export limits and, for three-phase connections, a separate limit for each phase. In the canonical suite, household and small-service import ceilings use Portuguese normalised contracted-power levels. Single-phase residential connections do not exceed 10.35 kVA; three-phase BTN levels extend to 41.4 kVA; and the larger-consumption scenario class uses representative BTE-level connections of 69, 103.5, or 138 kVA (E-REDES 2018; Entidade Reguladora dos Serviços Energéticos 2026a). Residential chargers use 3.7, 7.4, or 11 kW classes and service chargers use 11 or 22 kW, within the normal and fast charging powers described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b). Single-phase connections and chargers are rotated across the community's L1, L2, and L3 labels; balanced three-phase uncontrollable demand and three-phase assets are divided across all three phases. Explicit phase-allocation metadata records these choices: a charger may be connected only to L1, while a three-phase battery can distribute its power across L1–L3.

CityLearn currently advances active energy and does not calculate reactive power. The suite maps the numerical kVA connection level to a kW ceiling under an explicit unity-power-factor assumption; a balanced three-phase ceiling is one third of the total on each phase, whereas a single-phase member receives the complete ceiling on its assigned phase. The export ceiling is the smaller of the import ceiling and the installed PV-plus-battery power. These rules produce Portuguese-comparable connection-headroom cases but do not constitute the engineering design or certification of a particular installation. Voltage drop, feeder impedance, protection coordination, and phase-specific measured household baselines remain outside this model. With positive power denoting import, an applied state is feasible when

$$-X_b^{\max} \leq P_{b,t} \leq I_b^{\max}, \qquad -X_{b,\phi}^{\max} \leq P_{b,\phi,t} \leq I_{b,\phi}^{\max} \quad \forall \phi \in \Phi_b, \tag{4.4}$$

where $I$ and $X$ are import and export limits. A building can be below its total contracted limit and still exceed the phase to which a charger is attached. For example, if a house imports 4 kW on L1 and L1 is limited to 7 kW, a 7.2 kW single-phase charging request has only 3 kW of feasible headroom even if the total building limit would allow more. CityLearn v3 applies the feasible portion and records both the original request and the rejected power. Two KPI families preserve the causal distinction. *Requested pressure* measures the excess implied by the controller request before projection and exposes how strongly a policy depends on the safety layer. *Residual violation* is calculated from the applied total and phase trajectories after projection and is the quantity used for the electrical hard gate. If non-controllable demand already violates a limit, the constraint layer leaves that residual visible. The controller-visible constraint signal retains requested-pressure semantics so that an unsafe intention can be penalised even when the applied trajectory is safe.

### 4.3.4 Electric vehicles and chargers

EV control is both time-coupled and service-constrained. The relevant state includes connection, vehicle and charger identities, arrival state of charge, target or minimum departure state, charging/discharging bounds, expected departure, and vehicle-to-grid capability. From these quantities the environment derives remaining energy, time to departure, average required power, slack, and urgency.

The implementation distinguishes the vehicle session from the charging asset. A charger may remain installed while successive vehicles connect and disconnect; this is session dynamicity and does not necessarily change agent identity. A topology event can separately install, remove, activate, or deactivate the charger or its controller-facing entity. The experiments in Chapters 5 and 6 can then state precisely which form of reconfiguration they exercise.

Several edge cases are explicitly tested: reproducible initial state of charge, arrival and departure transitions, state progression when the charger action is zero, disconnected behaviour, charging and V2G bounds, terminal-step handling, and required departure service. The KPI layer reports departure counts, target/minimum/tolerance success, energy deficit, charged energy, and V2G energy. It also applies an independent energy-accounting invariant: the positive connected-session increase in stored EV energy cannot exceed metered charger input, apart from numerical tolerance. A positive unexplained shortfall is a hard audit failure. Grid improvement cannot be credited when mobility service is silently missed, and a boundary-state reference cannot introduce unmetered energy.

Episode and topology boundaries receive explicit state semantics. If an experiment starts, a member enters, or a charger is installed while its declared session is already occupied, the runtime initializes the EV from the session's current-SoC boundary reference and does not substitute an unrelated annual-catalogue initial value. The reference is used once at runtime association; subsequent SoC is determined by the applied actions and battery model. Likewise, a deferrable request whose admissible start window closed before a member or appliance became active is excluded from controller-failure counts. These rules make partial-window training and dynamic full-year evaluation comparable without inventing decisions in history that the controller never observed.

### 4.3.5 Deferrable appliances

The earlier appliance-specific representation was generalised into a reusable deferrable-service model. A cycle catalogue defines the energy profile $q_{c,\tau}$ and duration $d_c$; a schedule defines the earliest and latest admissible start, completion deadline, priority, and whether the service is mandatory. The controller selects a start time but does not reshape or truncate the cycle.

For requested cycle $c$, start $s_c$ is feasible when

$$t_c^{\text{earliest}} \leq s_c \leq t_c^{\text{latest}}, \qquad s_c + d_c - 1 \leq t_c^{\text{deadline}}. \tag{4.5}$$

Once started, the stored profile is applied until completion and remains subject to the building's electrical limits. Figure 4.3 shows the modelling boundary. The simulator does not reproduce the internal physics of every appliance or industrial process; it represents the complete time-shiftable service so that omitted or late demand is visible through completed cycles, missed cycles, start delay, served energy, and unserved energy.

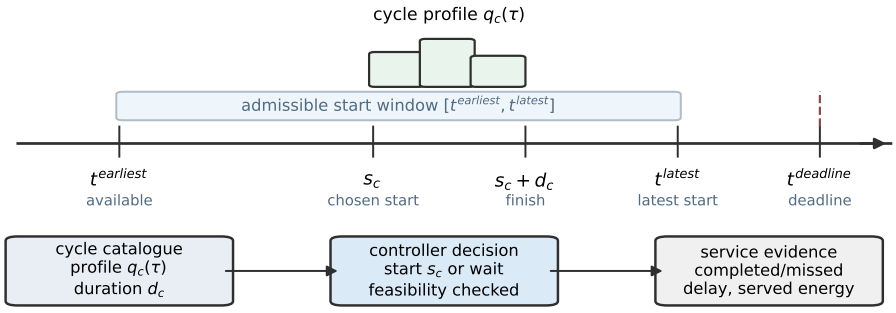



Figure 4.3: A deferrable service with an admissible start window, fixed cycle profile, and completion deadline. Cost or peak reduction is not credited as successful service when the required cycle is omitted or completed late.

### 4.3.6 Local energy sharing and settlement

REC operation cannot be inferred from district net load alone. If one member exports PV while another imports, aggregation may show a near-zero balance without revealing whether local exchange was permitted, which participant supplied or consumed it, or how value was allocated. CityLearn v3 records member net demand, local import/export, residual grid exchange, and settlement separately.

For active members $\mathscr{C}_t$, let $E^{\text{net}}_{b,t}$ be positive for demand and negative for surplus. Community demand, surplus, and locally shared energy are

$$D_{\mathscr{C},t} = \sum_{b \in \mathscr{C}_t} \max(E^{\text{net}}_{b,t}, 0), \tag{4.6}$$

$$S_{\mathscr{C},t} = \sum_{b \in \mathscr{C}_t} \max(-E^{\text{net}}_{b,t}, 0), \tag{4.7}$$

$$L_{\mathscr{C},t} = \min(D_{\mathscr{C},t}, S_{\mathscr{C},t}). \tag{4.8}$$

Participant weights allocate scarce surplus without exceeding demand, and residual demand or surplus is exchanged with the grid. The current settlement configuration prices local energy as a declared fraction of the retail import price and retains counterfactual cost, savings, and member-level outcomes. It is an experimental accounting model, not a universal legal or market rule.

The historical annual diagnostic in Figure 4.4 uses the same community RBC trajectory in both accounting cases. Under the configured rule, 17.0 MWh is shared locally, annual cost changes from 24.8 to 22.0 kEUR, and all 17 members reduce cost, although savings range from 68.8 to 525.7 EUR. This result predates the canonical suite and is retained for exact reproduction. It demonstrates that participant-level consequences are measurable; it does not establish that the selected rule is equitable or applicable to every REC.

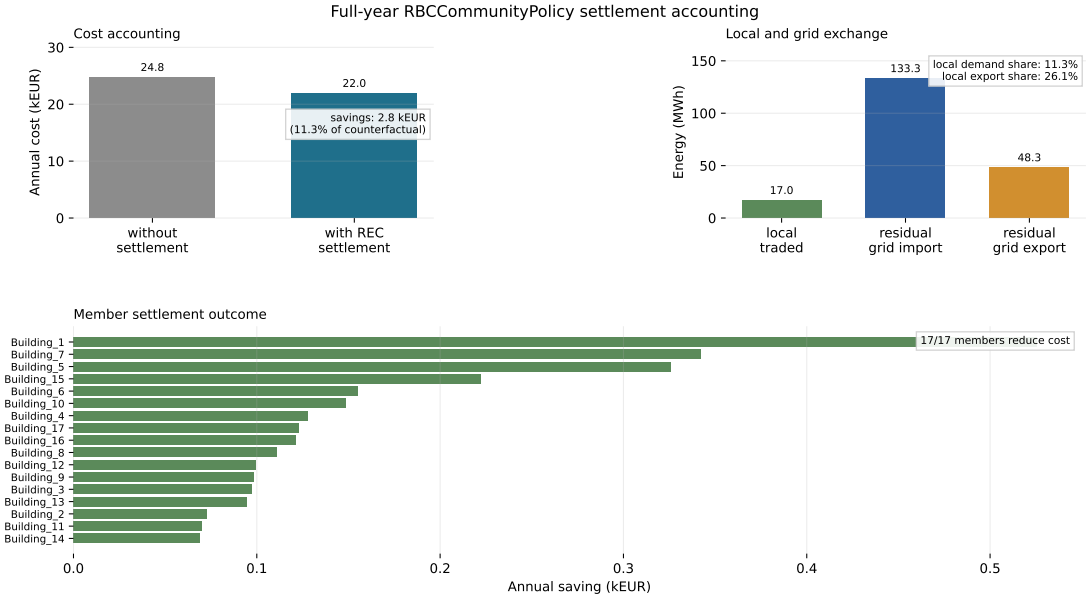



Figure 4.4: Annual local-energy and settlement accounting for one unchanged controller trajectory. The comparison isolates the configured accounting rule and retains heterogeneous member outcomes.

Asset models describe what can operate; event layers describe how the conditions of operation change. CityLearn v3 represents these changes as scenario data, keeping them outside controller-specific code. The same policy can be run against a nominal scenario and against a matched scenario containing topology, demand-response, or robustness events.

### 4.3.7 Changing community members and assets

A member may join or leave, a charger may be installed, or PV and storage may become part of the community only during a defined active period. Each topology event contains a timestep, operation, affected identity, and, where required, replacement data or parameters. Events scheduled at $t$ are applied before the controller acts at $t$, so the physical state, available observations and actions, settlement, and KPI denominators use the same active topology.

Removal does not erase history. A departed member retains its earlier rows but no longer contributes demand, generation, settlement, or actions. Similarly, an installed charger has no action or service denominator before its activation. A reinstalled stateful asset is represented by a new runtime instance initialized at its declared boundary state; it does not silently inherit SoC or pending

service from the removed instance, whose earlier trajectory remains available to KPI aggregation. Energy carried at re-entry is an exogenous boundary condition, with no grid energy created inside a control step. The exported topology log identifies which entity changed and why the shape of the controller interface or result set changed.

Figure 4.5 shows one executable lifecycle scenario. Building 18 is added at step 1000, a charger is added at 1300, a charger is removed at 1500, and PV is removed at 1600. At the same points, identified controller-input rows change from 67 to 74 and finish at 71; relation rows change from 74 to 84 and finish at 80; and action rows change from 26 to 30 and finish at 29. These figures are interface-propagation evidence, not a claim about controller performance under reconfiguration.

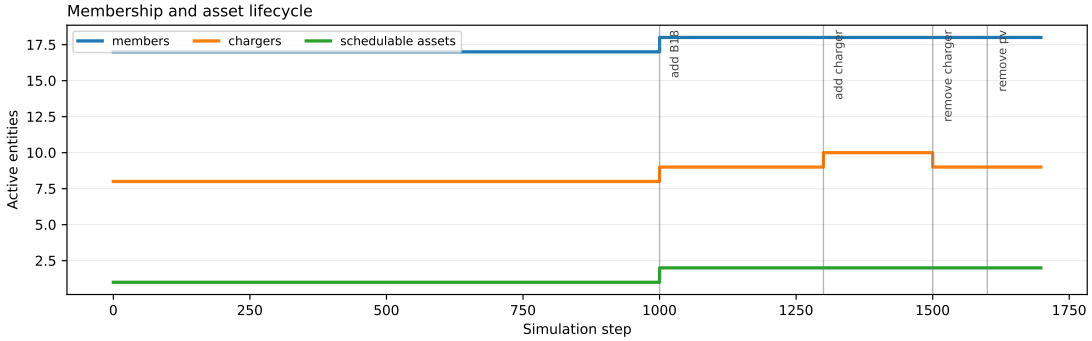



Figure 4.5: Lifecycle of members and assets in one dynamic-topology scenario. The event log, active entity counts, structured observations, relations, and action rows change at the same simulation steps.

### 4.3.8 Controlled data and equipment failures

Operational failures enter different parts of the control loop and must not be conflated. A missing measurement or biased forecast changes what the controller knows; an action dropout changes what is actuated; equipment unavailability changes which action is physically possible. CityLearn v3 stores these conditions as time-stamped event records with channel, target, feature, interval, failure mode, and parameters.

Let $\mathscr{E}_t^o$, $\mathscr{E}_t^f$, $\mathscr{E}_t^a$, and $\mathscr{E}_t^u$ denote active observation, forecast, action, and availability failures. Their location in the control loop is

$$\tilde{o}_t = \rho_o(o_t, \mathscr{E}_t^o, \mathscr{E}_t^u), \qquad \tilde{f}_t = \rho_f(f_t, \mathscr{E}_t^f), \tag{4.9}$$

$$a_t^{\text{app}} = \rho_a(a_t^{\text{req}}, \mathscr{E}_t^a, \mathscr{E}_t^u), \qquad x_{t+1} = F(x_t, a_t^{\text{app}}, w_t). \tag{4.10}$$

Corrupting observations or forecasts affects controller-facing data without changing the underlying physical state. Action and availability failures preserve the request but alter the applied action. Supported modes include missing, noise, bias, stuck, and clipping for data; dropout, delay,

noise, bias, stuck, and clipping for actions; and temporary unavailability for assets. A seed makes stochastic events repeatable, and a missing-value marker remains distinct from physical zero.

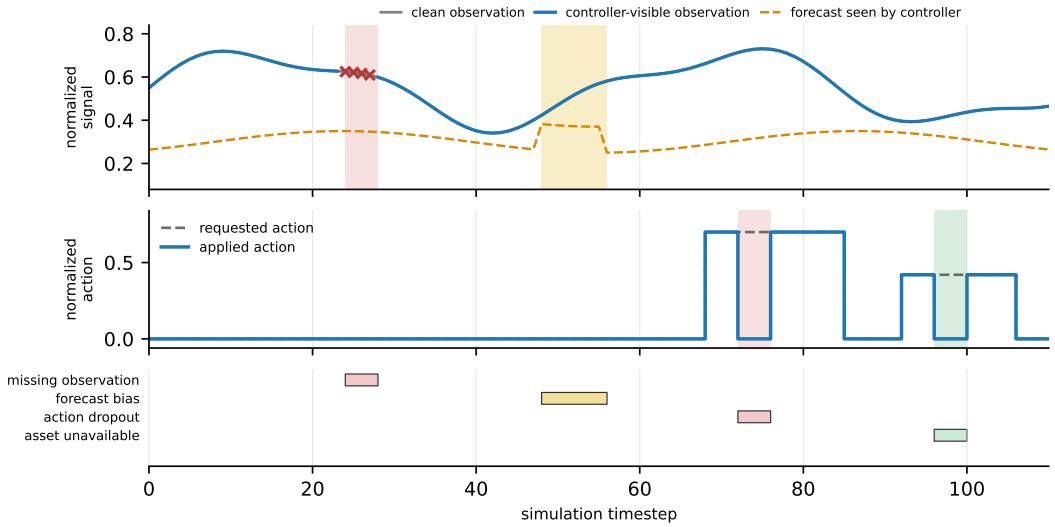



Figure 4.6: Failure channels in the controller loop. Measurement and forecast events alter controller-visible information; command dropout and equipment unavailability create traceable differences between requested and applied actions.

In the diagnostic scenario underlying Figure 4.6, a missing load measurement, biased price forecast, storage-command dropout, and temporary storage unavailability are activated in separate windows. The resulting export records changed measurements and forecasts, lost commands, unavailable-equipment timesteps, and requested/applied-action differences. The supported claim is that the failure is reproducible and measurable, not that the diagnostic policy is robust to every failure class.

### 4.3.9 Demand-response services

A demand-response request is represented as an explicit service with its own issuer, direction, start and end, target power, tolerance, activation price, shortfall penalty, and baseline method. The request is kept visible outside the price and reward. With positive $P_t$ denoting community import and $B_{r,t}$ the baseline for request $r$, delivered power is

$$d_{r,t} = \begin{cases} P_t - B_{r,t}, & \text{up request,} \\ B_{r,t} - P_t, & \text{down request.} \end{cases} \tag{4.11}$$

Positive delivery always means movement in the requested direction. Credited response is non-negative and capped by the target, whereas full delivery remains in the record. Shortfall is calculated after tolerance, and both quantities are converted from power to per-step energy using Equation (4.3). An invalid or unavailable baseline receives an explicit status and is excluded from scoring.

The evaluated 180-step scenario contains three non-overlapping requests totalling 190.0 kWh. Relative to reference operation, a diagnostic event-aware dispatch reduced shortfall from 101.1 to 51.6 kWh, increased compliance from 0.43 to 0.70, and reduced cost after settlement from 863.6 to 790.6 EUR. Net demand-response revenue remained negative after penalties. The diagnostic establishes that request, delivery, credit, shortfall, payment, penalty, and final cost can be interpreted together. It leaves the profitability and optimality of individual requests open.

### 4.3.10 Multiple communities

`MultiCommunityEnv` advances several independent CityLearn environments in lockstep. Each community retains its scenario, members, assets, events, controller, reward, KPIs, and identifier. Actions and observations are organised by community, enabling local, portfolio, or hierarchical controllers without merging all buildings into one artificial district.

Energy, cost, emissions, payments, and event counts are additive. Ratios are aggregated through declared reporting weights that affect only the combined result, not the simulated physics or local control. Communities in one synchronised run must share timestep, episode length, controller-interface mode, and control setting, whether centralised or decentralised.

In the four-community evidence case, the environments were evaluated over the same 600-step window. Individual import/export trajectories remained visible alongside the portfolio result, and 2.45 MWh of simultaneous export and demand was identified as a potential inter-community match. This was an accounting diagnostic under independent control, not physical power transfer, centralised market clearing, or optimised inter-community exchange.

### 4.3.11 Controller Interfaces and Applied-Action Traceability

The simulator preserves the original CityLearn/Gymnasium interaction pattern: reset the environment, receive an observation, submit an action through `step`, and receive reward and diagnostic information. Two complementary controller views are available.

The *flat interface* preserves compatibility with existing RBC, optimisation, and RL implementations that expect fixed arrays. The *entity interface* provides tables organised by entity type, stable identifiers, relations, masks, feasible ranges, action ports, and metadata. A newly installed charger appears as an identified row linked to its building and current EV; its addition is not reduced to a change in vector position. Entity bundles can expose basic or more detailed feature sets for graph-, set-, or transformer-based policies and for offline datasets.

The structured interface does not itself make a policy capable of handling a new entity or interface. It makes the change explicit and testable. A fixed-head controller can use a compatible fixed view or padding and masks; a variable-set controller can consume active rows; and TI-MARL can be tested against the stronger typed-interface conditions developed in Chapter 6. The environment remains neutral about whether the controller succeeds.

Action tables carry identity, bounds, masks, and feasible capacity. At every step, requested, clipped, applied, or rejected values can be exported with a reason. This prevents a recurrent interpretive error: a policy that repeatedly proposes infeasible power may produce an apparently safe physical trajectory only because the environment intervenes. Learning diagnostics and evaluation retain controller intent and executed operation as separate records.

## 4.4 Datasets Developed to Evaluate REC Controllers

The simulator capabilities described above require an experimental progression that makes their effects separable. A collection of unrelated scenarios would permit each controller to be tested, but would make it difficult to determine whether a result changed because of the algorithm, the participant set, the data resolution, the service obligations, or an unnoticed change in the physical inputs. CityLearn v3 includes a canonical annual suite constructed as a ladder: a small diagnostic case, a reduced development case, a 30-member reference family with isolated interventions, and a 100-member integrated stress case. Each level preserves a common contract while increasing the dimensions that matter to RQ1–RQ4. No individual dataset is intended to represent every REC.

The earlier hourly case with 17 members is retained for historical reproduction of experiments developed before this suite. It remains useful when an exact earlier result must be reconstructed, including the annual settlement diagnostic reported later in this chapter. It is not treated as a member of the new canonical family because its timestep and asset contract differ.

### 4.4.1 Common temporal, provenance, and accounting contract

Every canonical dataset spans the civil year 2023 with 35,040 steps of 15 min and `seconds_per_time_step=900`. The progression uses one continuous UTC timeline from 1 January at 00:00 to 31 December at 23:45. A separate calendar table preserves the corresponding `Europe/Lisbon` local time, UTC offset, weekday, hour, minute, and daylight-saving status. This choice avoids an artificial duplicate or missing simulator index at the daylight-saving transitions while retaining the local attributes needed by prediction and control.

The canonical files contain exactly 35,040 physical interval rows. Since the environment needs the state at both ends of an action interval, a complete annual rollout uses 35,041 state points to execute 35,040 control transitions. The extra point is an observation-only boundary at 1 January 2024 00:00; it carries no additional demand, generation, price, settlement, or service-energy interval. The runtime contract makes this boundary explicit and preserves the final 15-minute action, including a departure at midnight, in the evaluation.

The price series is the historical component held common across the suite. It uses the official 2023 Portuguese day-ahead prices distributed by OMIE (OMIE 2023). The daily source files contain the physical market periods, including the 23- and 25-period daylight-saving days. Concatenating those periods produces 8,760 physical hours. Since the day-ahead market only moved to a 15-minute market time unit in October 2025 (OMIE 2025), each official 2023 hourly price is repeated

in the four corresponding simulation steps, without interpolation, and converted from EUR/MWh to EUR/kWh. The generated provenance table retains the daily OMIE filename, published revision, URL, number of periods, and SHA-256 digest, making the transformation inspectable down to the source file used for each day.

The remaining exogenous series are reproducible scenario profiles, but they are not copies of one daily template with independent noise. Each complete member class is stratified over five explicit routine archetypes. Residential members include early, standard, and late commuters, home workers, and predominantly home-day routines. Small services include office, six-day retail, split-day clinic, extended hospitality, and early-workshop schedules; larger consumers include daytime, extended, continuous, morning-intensive, and evening-intensive processes. The archetype changes the operating window, the relative morning, daytime, and evening demand, occupancy, eligible EV visits, and deferrable-service windows. Member-specific peak locations, correlated short-term variation, day-level temporal shifts, reduced-activity days, weekends, holidays, and seasonality then prevent members within one archetype from becoming exact replicas.

Each annual demand trace is scaled to a target drawn from a bounded log-normal scenario distribution, without accumulating observations at clipping limits. Its class centre is anchored to 2023 Portuguese consumption per consumer: approximately 2.49 MWh for residential, 16.58 MWh for non-domestic, and 175.89 MWh for industrial consumption (Direção-Geral de Energia e Geologia 2023; Pordata 2023). These national classes provide an order-of-magnitude anchor; neither the ranges nor the frequency of each routine are presented as a fitted distribution of REC participants. PV capacity is coupled to the host's annual demand and separately varies east/south/west orientation, derating, and reproducible shading. Stationary batteries use several product-like energy and power classes selected from a stratified daily-load-autonomy rule with a host-PV floor; efficiency, initial state of charge, usable depth, standing loss, and degradation coefficient vary by member. Weather and carbon remain constructed scenario series, while weather forecasts include temporally correlated errors that increase with forecast horizon. Load and PV forecast bundles use causal daily persistence: a value at horizon $h$ is based on the observation one day before $t+h$, with the current step used only for cold start. They contain no realised future load or generation. The OMIE price bundles are publication-aware: an official realised price is visible only after the declared 13:00 local day-ahead cut-off; an as-yet unpublished horizon uses causal daily persistence instead. Stored price forecasts cover 1, 6, and 24 hours. The entity interface obtains its additional 15-minute and 3-hour features from an earlier issue of the shortest stored horizon that targets the same delivery step and never reads the realised future-price series. The cut-off and alignment rule are conservative benchmark assumptions recorded in the schema and interface metadata.

The datasets are deliberately described as *hybrid*. The price is historical, annual demand has a Portuguese statistical anchor, and PV, weather, carbon, mobility, and deferrable behaviour create controlled heterogeneous operating conditions. They are not measurements of one physical community. This boundary keeps the controlled evidence in Chapters 5 and 6 separate from the field-

derived data and demonstrator evidence in Chapter 8. The exact generation ranges, asset parameters, topology calendars, and claim boundaries are fixed in Appendix A.3 and in the machine-readable family manifests.

All families use the same local settlement rule. Surplus and demand are matched within the same 15-minute step; importers receive equal allocation weights; local energy is priced at 80% of the OMIE value; and any residual export receives no remuneration. A grid-only calculation, with no local matching, is retained as the counterfactual for participant and community results. Retail margins, network tariffs, taxes, and regulated compensation are intentionally excluded. The settlement provides an internally consistent experimental accounting rule and does not reconstruct a Portuguese electricity bill.

### 4.4.2 Composition and the separation of infrastructure from mobility

Table 4.4 fixes the installed catalogue for each level. The counts refer to distinct members and physical assets, not to observation rows or EV visits. Core-15-Stripped is a byte-identical physical subset of the first 15 members of Core-30 for the files and assets it retains. This makes it possible to move from debugging to the principal comparison without changing the behaviour of the shared members.

Table 4.4: Installed composition of the canonical 2023 annual dataset suite. Counts are fixed in the full catalogue; the active population and available equipment can change in Dynamic and AllIn scenarios.

| Dataset | Members | PV | BESS | Charging members | Chargers | Deferrables |
|---|---|---|---|---|---|---|
| Micro-4-Q | 4 | 2 | 1 | 2 | 2 | 1 |
| Core-15-Stripped | 15 | 7 | 3 | 6 | 7 | 4 |
| Core-30 | 30 | 15 | 6 | 12 | 15 | 8 |
| Premium-100 | 100 | 55 | 22 | 45 | 80 | 30 |

Four identities are kept distinct throughout the suite. A *member* is the participant or building that owns demand, production, and assets. A *charger* is a physical controllable connection attached to that member and to an electrical phase. An *EV* is the mobile energy-storage object whose state of charge and mobility requirement persist beyond one connection interval. A *charging session* associates one EV with one charger over a declared arrival–departure window. Counting visits as chargers would inflate the control interface, while assigning one permanent EV to every service charger would suppress the shared-infrastructure problem that those buildings are meant to represent. Contract v1.9 propagates all four identities into the executable schedule. A new session remains observable even when the same EV reconnects at the exact boundary at which its previous session ends and charger occupancy never changes. Runtime arrival SoC and departure KPIs use the session boundary directly, without inferring it only from charger or vehicle identity.

The smaller datasets already include more than one charger at one household. Core-15-Stripped has five homes with one charger and one with two. Core-30 has nine homes with one and three

homes with two. The 80-charger Premium catalogue expands both multiplicity and building type. Thirty equipped homes contain 36 chargers: 24 have one and six have two. Ten small service or commercial members contain 20 chargers, two per member. Five larger office, public-service, or light-process members contain the remaining 24: three have four chargers and two have six. Chargers at one member remain separate action entities and phase-connected loads, so simultaneous requests must satisfy both individual limits and the shared building and phase headroom.

Residential charger–vehicle relations are normally stable. Shared chargers in service and office members receive successive EVs during the day. The Premium catalogue contains 212 EV identities and 41,173 annual session identities for 80 physical chargers. Arrival and departure times vary reproducibly by member, charger, day, weekday, building class, and routine archetype; missed residential visits and several daily service-charging windows prevent an artificial all-at-once connection pattern. At the 15-minute clock, Premium uses all 96 possible arrival times and has no more than 10 simultaneous arrivals in the generated year.

The energy requested in each session is sampled by building class and translated into arrival and target state of charge. The generator reserves a 15% charger-window margin for residential sessions and a 28% margin for shared service sessions, and caps every request at 72% of EV capacity. Every ordinary session is individually deliverable at the charger's declared power and 95% efficiency before community constraints are applied. The audit found zero individually infeasible sessions across all four families. Joint attainability still depends on simultaneous chargers, non-controllable demand, phase limits, demand-response obligations, outages, and command failures, which can create deliberate competition for scarce headroom. A failure under Safety or AllIn can be attributed to this tested interaction without an impossible mobility request already embedded in the clean data. The session distributions are scenario assumptions and do not constitute a national mobility survey.

Deferrable services follow the same principle. Each asset has a catalogue of complete cycle profiles with duration and step energy, and a sparse annual request schedule with earliest and latest start, completion deadline, priority, and `must_run`. Residential archetypes produce distinct washing-machine, dishwasher, washer–dryer, and domestic-hot-water services. Service and larger-consumption routines introduce cleaning, cold storage, sterilisation, hospitality hot water, compressors, pumping, and batch-process loads. Profile energy is scaled per asset; request occurrence, eligible days, clock window, completion margin, and mandatory-service probability vary by routine and day. Requests for one physical appliance are explicitly non-overlapping. Completion, delay, missed deadline, and unserved-energy indicators allow the later algorithm comparisons to distinguish legitimate load shifting from omission of the requested service.

### 4.4.3 Experimental ladder and matched scenario variants

Micro-4-Q is the smallest end-to-end contract. It is used for loader, interface, accounting, and controller smoke tests where a four-member trajectory is easier to inspect manually. Core-15-Stripped retains enough heterogeneity for initial baselines and algorithm development but reduces

training and debugging cost. Its clean data and normally non-binding connection limits make an error easier to attribute before the same method is moved to the full reference family.

Core-30 is the principal controlled family. One annual physical base produces the five variants in Table 4.5. Safety, Health, and Dynamic first introduce one experimental dimension at a time. Combined then asks whether a method that appeared adequate under isolated changes remains adequate when tight limits, faults, topology changes, and demand-response commitments interact. Its dynamic trajectory begins with 26 active members: four previously inactive members join and Members 019 and 008 later leave permanently, producing 26–30 active participants and 28 at year end. A charger, stationary battery, deferrable appliance, and PV system are each removed and later restored independently of their member. A member event separately activates or deactivates all of that participant's assets. Equipment unavailability is recorded as a health event, preserving the identity of an entity that still exists but cannot currently be observed or controlled. A topology event records that the entity itself is absent.

The corresponding state semantics are explicit. Re-adding an asset from its catalogue template represents replacement or recommissioning and starts a new runtime instance from the declared initial state. No continuity is assumed for an unobserved SoC, degradation state, or unfinished service across removal. A temporary outage of the same physical instance is represented by a robustness event, which retains its identity and accumulated state while blocking the affected observation or action channel. In the scenario calendar, "restored" denotes restoration of the declared capability without reconstructing hidden physical history.

Table 4.5: Matched Core-30 variants and the experimental question isolated by each schema. All variants reference the same annual physical time series and asset identities.

| **Variant** | **Added condition** | **Purpose** |
|---|---|---|
| Core-30-Nominal | Static membership, available equipment, and no injected faults | Reference comparison under the complete but undisturbed annual contract. |
| Core-30-Safety | Total and per-phase import/export limits | Tests whether requested actions remain feasible when several assets at one member act simultaneously. |
| Core-30-Health | Missing observations, forecast bias, command delay/dropout, and asset unavailability | Isolates degradation caused by data, communication, and equipment health without changing membership. |
| Core-30-Dynamic | Member entry/exit and independent charger, PV, storage, and deferrable removal/reinstallation | Isolates changes to the active entity, relation, observation, and action sets. |
| Core-30-Combined | Safety, Health, Dynamic, and demand-response requests | Evaluates interacting obligations and disturbances after each mechanism has been examined separately. |

Premium-100 is the integrated scale and heterogeneity case. Its complete catalogue contains 70 residential members, 20 small service or commercial members, and 10 larger-consumption members. Each class contains all five of its routine archetypes; the frequencies are stratified scenario choices and are not inferred population proportions. The scenario begins with 70 active

participants; 30 first-entry events activate the remaining catalogue and six active members later leave permanently. The active population spans 70–100 and ends at 94. Two chargers and one PV system, stationary battery, and deferrable appliance each undergo independent removal and restoration. PV, stationary batteries, chargers, deferrables, building types, and service patterns are deliberately unevenly distributed, so a policy must handle different capability sets across the 100 members.

Two Premium schemas share the same physical files, topology timeline, electrical limits, settlement, and demand-response services. Premium-100-Clean contains that already demanding structure without injected health faults. Premium-100-AllIn adds missing and degraded telemetry, forecast error, delayed or dropped actions, charger and storage unavailability, and selected grid-outage intervals, with some events coinciding with safety pressure, mobility deadlines, or demand-response requests. AllIn is reserved for final confirmation. Repeated tuning on the integrated stress case would turn it into another development set and weaken its value as a held-back challenge.

Service denominators follow the active topology. The complete catalogue remains available for auditing but does not define the obligations visible to the controller. An EV departure is observable only when its member and physical charger exist during the final control interval preceding departure. The static Micro, Core-15, and Core-30 variants expose 554, 1,757, and 3,774 departures, respectively. Core-30-Dynamic and Combined expose 3,690; 56 catalogue sessions occur while their member is inactive and 28 while the charger is absent. Each Premium variant exposes 28,108; 13,048 sessions belong to an inactive member at the relevant boundary and 17 to an absent charger. The same audit distinguishes deferrable requests that are fully observable, never acquire a valid start opportunity, or are interrupted by a topology change before their deadline. The dynamic Core case contains 1,657 fully observable, 234 censored, and one potentially interrupted cycle; Premium contains 5,950, 1,409, and one, respectively. Excluded and interrupted services receive separate counts instead of being assigned as controller failures. The annual audit independently reconstructs the expected denominators from the schema and requires agreement with the runtime KPI counts.

This ladder assigns a clear role to each evidence level. Core-15 supports fast implementation checks and early controller progression. Matched Core-30 variants support the principal attribution of algorithm effects. Premium-100 tests whether the resulting method and interface remain usable under scale, mixed building roles, several action groups per member, dynamic composition, safety constraints, and health degradation. In this way the datasets provide a common bridge from the RBC and learning progression in Chapter 5 to the typed, variable-set TI-MARL evaluation in Chapter 6. They do not guarantee that an algorithm is representative merely because it performs well on all levels; they make the conditions under which that claim is tested explicit.

The suite connects directly to the thesis-wide research questions without repeating their full mapping. Dynamic members, installed assets, EVs, and sessions expose the representation problem in RQ1; population and asset multiplicity provide controlled RQ2 scale; matched Safety and Health variants provide repeatable RQ3 conditions; and versioned schemas, source provenance,

deterministic generation, checksums, and validation reports provide the RQ4 lineage that accompanies a controller into the deployment workflow. The validation protocol and its numerical results are reported in Appendix A.3. Passing those checks establishes consistency within the declared dataset and simulator contract; it does not establish population-level representativeness, feeder-level electrical validity, or controller effectiveness.

### 4.4.4 Validation protocol and annual baseline execution

Structural validation cannot by itself show that a controller can traverse every session boundary, topology event, fault, service window, and settlement step without corrupting the experiment contract. The annual execution audit pairs BAU and RBC Smart on every schema, binds both runs to the same dataset and simulator attestations, and independently re-reads the exported KPIs. It checks the terminal-state convention, topology-derived service denominators, causal price interface, finite outputs, event counts, EV energy conservation, and total and per-phase residuals.

BAU uses no stationary-battery action, charges EVs immediately until they enter the declared departure tolerance, and starts each deferrable cycle at its earliest feasible time. RBC Smart uses current and causal forecast information to shift storage and flexible services. The campaign is an execution and interpretability audit, not an algorithm-ranking result: it verifies that the datasets are traversable, the references are matched, required services remain visible, and competing cost, peak, ramping, demand-response, and feasibility outcomes are retained. The complete structural audit, annual scorecards, service diagnostics, and event evidence are reported in Appendix A.3.

## 4.5 Evaluation Protocol and Controller Scorecard

A controller comparison is only meaningful when all policies face the same scenario contract and when performance is interpreted together with service and feasibility. CityLearn v3 records the optimisation reward, core thesis scorecard, extended KPI catalogue, and exported trajectories separately. Reward guides a controller during training but cannot establish controller quality on its own.

### 4.5.1 Reference Controllers and Fair Comparisons

The principal reference is business-as-usual (BAU): normal EV charging, earliest feasible execution of mandatory deferrable services, and no deliberate stationary-battery optimisation. A no-battery variant isolates native service demand, while progressively informed RBCs add stationary storage, current price, PV, peak, or community context. These policies provide transparent reference behaviours and carry no claim of optimality.

Every comparison binds the evaluated controller and its reference to the same data, topology, services, settlement, limits, disturbances, timestep, information boundary, and KPI definitions. Raw indicators are always retained. Ratios or differences to BAU are reported only when the

reference value is valid and the direction of improvement is unambiguous. Missing service, invalid demand-response baselines, and constraint violations remain explicit counts, rates, or gates outside the normalised aggregate.

### 4.5.2 Core Scorecard and Result Exports

The thesis uses a compact scorecard with several interpretable indicators. Run integrity and scenario-specific obligations, including mandatory deferrable completion and valid demand-response baselines, are checked before the headline indicators are interpreted. The reference is the exported BAU when available and otherwise the simulator baseline declared in the result. Cost saving is $C_{\text{ref}} - C_{\pi}$, so a positive value means that policy $\pi$ is cheaper. Member improvement is reported as both $x/N$ and a percentage, where $x$ is the number of eligible members whose cost is lower than under the matched reference.

Table 4.6: Seventeen headline indicators in the core controller scorecard.

| Dimension | Headline indicators | Preferred reading |
|---|---|---|
| Economic | **Cost saving vs reference** [EUR and %] | Positive; larger is better. |
| Service and safety | **EV minimum feasible service** [%]; **EV within tolerance** [%, declared ±5% SoC band]; **Electrical violations** [kWh] | Service ↑; violations = 0. |
| Flexibility and solar | **Battery throughput** [kWh]; **V2G export** [kWh]; **Solar self-consumption** [%] | Throughput ↓ at equal service; V2G contextual; self-consumption ↑. |
| Community exchange | **Community import** [kWh]; **Community export** [kWh]; **Community net exchange** [kWh] | Import ↓; export ↓ under the declared zero-remuneration rule; net exchange towards zero. |
| Grid shape | **Daily peak ratio**; **All-time peak ratio**; **Load factor penalty**; **Ramping** [raw value or reference ratio] | Ratios < 1; ramping ↓. |
| Participant outcomes | **Members saving vs reference** [$x/N$ and %] | Larger coverage is better; retain the savings range. |
| Computation | **Inference latency** [ms/action]; **Training time** [h] | Lower under matched hardware, software, and training budget. |

Electrical violations use the residual applied total and per-phase trajectory and remain a hard feasibility gate. EV service uses topology-feasible departures as its denominator; the minimum threshold and tolerance band remain separately visible. Battery throughput and V2G export are diagnostic measures with no universal direction of improvement: additional cycling or export is beneficial only when its economic, grid, service, and degradation consequences justify it. The same principle applies to community export whenever the settlement rule changes.

CityLearn v3 retains a wider KPI catalogue covering emissions, local settlement, deferrable and demand-response service, requested electrical pressure, rejected actions, topology, controlled failures, inherited comfort quantities, and multiple communities. Definitions, units, and interpretation boundaries are provided in Appendix A.5. Exports include these KPI rows together with trajectories,

service summaries, event logs, entity relations, masks, and requested, applied, reduced, or rejected actions. The user interface supports comparative inspection, while the archived machine-readable result remains the authoritative record.

Computational evidence forms part of the controller comparison. Training time and inference latency are reported with hardware, software, controller, population, timestep, interface, executed steps, training budget, and aggregation convention. Matched smoke benchmarks additionally record initialisation, reset, tail step time, memory, export time, data format, and community count. Their detailed current-checkout results are retained in Appendix A.4. These measurements bound the evaluated workloads and do not establish linear scaling to arbitrary systems.

## 4.6 Chapter Synthesis

CityLearn v3 contributes a coherent simulation and evaluation substrate. Its scenario contract binds data, units, services, topology, disturbances, controller interfaces, and evaluation; the annual dataset ladder turns that contract into matched development, attribution, scale, safety, and robustness cases; and the evidence path connects entity identity, controller-visible information, requested action, applied action, and outcome. Together, these mechanisms make heterogeneous and changing REC-controller experiments comparable, traceable, and reusable across the algorithm and deployment lifecycle developed in the following chapters.

The supported claim remains deliberately scoped. The chapter establishes internal consistency, executable mechanisms, and an auditable comparison contract; it does not establish population-level representativeness, detailed feeder physics, controller superiority, universal settlement validity, or field certification. The corresponding technical limits are stated beside the electrical, settlement, robustness, multi-community, and interface mechanisms to which they apply. Real-world evidence remains the role of the demonstrators in Chapter 8.

> *CityLearn v3 provides an auditable and physically checked experimental substrate for REC controller studies in which heterogeneous assets, service obligations, high-frequency data, dynamic topology, external requests, controlled operational failures, and multiple communities remain traceable from scenario declaration to controller-visible input, requested action, applied action, reported outcome, and the versioned validation contract used by the deployment lifecycle.*

The next chapter uses this substrate to compare the progression from transparent rules and optimisation to single- and multi-agent learning. The question is no longer only whether a method improves an aggregate metric, but whether it does so under common information, service, physical, and evaluation conditions.

# Chapter 5

# Control and Learning Methods for Renewable Energy Communities

A dwelling with a stationary battery, two EV chargers, and a deferrable appliance already presents a difficult scheduling problem. The controller has to use photovoltaic production and low-price periods without missing an EV departure, starting an appliance too late, or exceeding the electrical service allocated to the building. Placing that dwelling in a renewable energy community adds another layer: an action that is sensible locally can coincide with the actions of other members and create a community peak, increase ramping, or consume scarce phase headroom. The controller must then balance a participant's objectives with outcomes that are only visible at community level.

No single method was assumed to solve this complete problem at the start of the research. The work developed through a sequence of increasingly demanding control settings. Business-as-usual operation and rule-based controllers made feasible behaviour inspectable. Mixed-integer optimisation made objectives and constraints explicit and supplied reference schedules. Building-level reinforcement learning tested whether a controller could adapt without solving a full model at every step. Multi-agent methods introduced decentralised execution while retaining community information during training. The learning system itself also changed: observations became causal and asset-aware, action heads acquired semantic structure, rewards were corrected when they rewarded the wrong behaviour, and replay was adapted to retain rare service events. Demonstrations and residual policies reused knowledge that had already been acquired. Hierarchical coordination separated the slower community decision from the faster decisions taken inside each building.

This progression is both historical and methodological. The first EnergAIze experiments used an hourly, 17-dwelling case and a MADDPG controller with rule-guided exploration (Fonseca, Ferreira, Cabral, Severino, and Praça 2024). Later work exposed stronger baselines, stricter information boundaries, service failures hidden by aggregate reward, and the need to distinguish local adaptation from community coordination. The annual 15-minute suite introduced in Section 4.4 provides the common substrate on which those lessons are consolidated. Historical experiments remain part of the provenance of the research; the final comparisons use the current simulator, datasets, scorecard,

and controller contracts.

The chapter first formulates the shared control problem and fixes the comparison protocol. It then describes the control–learning interface shared by the methods, including the input representation, action path, reward lineage, replay design, and model continuity. Four levels of control complexity organise the controller progression: transparent and model-based control, adaptive building control, decentralised multi-agent control, and hierarchical community coordination. Knowledge reuse is treated separately because it cuts across several algorithm families. The closing synthesis records what each level is designed to establish and the limitations that remain after the progression. Numerical findings are inserted only from frozen, replayable experiments; the present text fixes their scientific position without anticipating their values.

## 5.1 Research Progression

The control problem grew along four dimensions. The decision boundary moved from one asset to a building and then to the community. The available information expanded from current local measurements to forecasts and authorised community aggregates. The method changed from explicit rules and models to policies learned through interaction. The evidence requirements also became stricter: cost and energy outcomes had to be read alongside service, physical feasibility, learning stability, and computational cost.

The four levels in Figure 5.1 reflect this growth in complexity. They are not a ranking in which every later method supersedes the earlier ones. A rule-based controller remains useful when its assumptions fit the task, an optimisation model remains the clearest reference for a bounded scheduling problem, and a local policy remains preferable when community coordination adds no measurable value. Each level answers a different question.

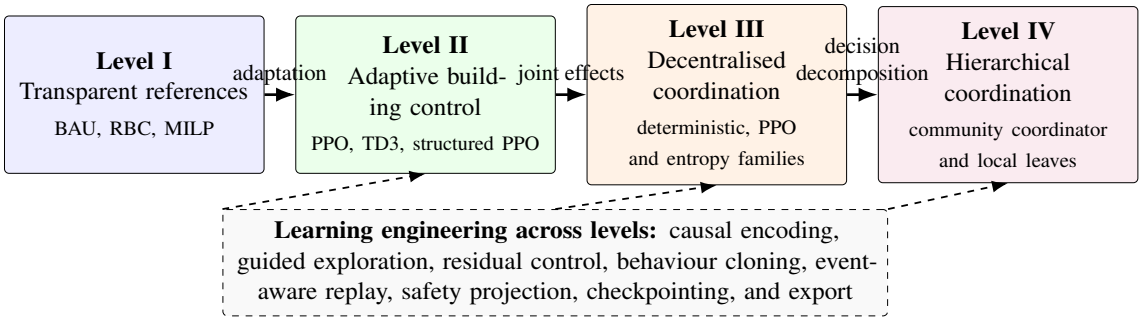



Figure 5.1: Progression of the controller studies. Each level introduces one additional control problem while retaining earlier methods as references, teachers, or fallbacks.

**Level I: transparent references.** The first question is whether flexibility can be scheduled plausibly and whether the relevant objective can be stated explicitly. BAU reveals the cost and service consequences of ordinary operation. Progressively informed RBCs show the value of price, forecast, flexibility, and community information without a learning process. MILP supplies a

model-based schedule under declared assumptions. This level establishes feasibility, interpretability, and an opportunity bound before neural control is considered.

**Level II: adaptive building control.** The second question is whether a building can improve its operation from local experience. Each dwelling is treated as a separate decision problem, with its own observations, actions, reward, and service obligations. PPO and TD3 represent on-policy and off-policy approaches to continuous control. Strictly local experiments isolate adaptation: they do not credit an algorithm with a gain that came from hidden access to community measurements or settlement outcomes.

**Level III: decentralised multi-agent control.** The third question concerns interacting decisions. Each building still executes a local policy, while training can use joint observations and actions to account for community effects. MADDPG supplied the original EnergAIze formulation. MATD3 retained the same execution structure while addressing value overestimation and training instability through twin critics, target smoothing, and delayed actor updates. PPO- and entropy-based multi-agent alternatives test whether an apparent gain is specific to one actor–critic family.

**Level IV: hierarchical community coordination.** The fourth question is whether local and community decisions can be separated without collapsing all assets into one central action vector. A community coordinator acts at a slower timescale and issues a compact signal. Local controllers continue to manage EV deadlines, storage, and deferrable services. This level examines the authority of the signal, the information exposed to the coordinator, the distribution of benefits, and the stability of composing learned controllers.

The repository contains more methods than the four names used in the first EnergAIze experiments. Table 5.1 records the current implementation surface. This table prevents three common ambiguities: treating a multi-agent implementation as a single-agent method, presenting an experimental comparator as a selected controller, and attributing a structural interface change to the optimisation algorithm alone.

A standalone SAC controller is not registered in the current repository. The soft actor–critic branch is implemented as MASAC, with decentralised stochastic actors and central twin critics. TD3 is the explicit one-member reduction of the MATD3 path. These distinctions are retained in the method names and result ledger.

These levels contribute differently to the research questions. Local and community reward formulations, decentralised execution, and hierarchical coordination address the balance between individual and collective objectives in RQ1. Population size, critic growth, simulator interactions, training time, memory, and inference latency provide RQ2 evidence. Constraint-aware references, service-preserving baselines, action projection, faults, and forecast errors support RQ3. Exportable policies, deterministic inference, declared preprocessing, and controller dependencies create algorithm-side evidence for RQ4; the governed lifecycle itself is developed later in the thesis.

Table 5.1: Controller and learner families implemented in the Algorithms repository and their role in this chapter.

| Family | Implemented methods | Scientific role | Interface boundary |
|---|---|---|---|
| Passive and rule-based | Normal, Normal No Battery, Random, RBC Basic, RBC Smart Local, RBC Community, Rule Based, and signal-aware variants | Ordinary-operation denominator, informed baselines, teachers, causal probes, and deterministic fallbacks | Reconstruct actions from known asset semantics; no learned representation |
| Optimisation and replay | Fixed-service, total-home, total-energy, individual and community MILP oracles, with replay policies | Explicit constrained schedules, opportunity estimates, and demonstrations after simulator replay | Variables and horizon are declared before solving; the current reference family uses perfect information |
| Strict local RL | PPO and TD3, instantiated once per controlled member | On-policy and off-policy tests of adaptive local control | Fixed local observation and action dimensions |
| Independent MARL | IPPO | Independent PPO learners inside one multi-member run | No central value or action-value function |
| Centralised-training MARL | MADDPG, MATD3, MASAC, MAPPO, and HAPPO | Deterministic, entropy-regularised, simultaneous PPO, and sequential PPO comparisons under decentralised execution | Flat actor layouts; central value or critic inputs grow with the registered population |
| Entity-structured learners | AgentTransformerPPO and AgentTransformerMATD3 | Tests whether shared asset-type projections and variable token counts improve representation and accommodate changes in known asset instances | Known entity types and feature widths; building-count changes remain a rebuild boundary |
| Hierarchical control | Building Agent, Community Coordinator, CC Level 1, CC Level 2, fixed-price and causal-price signal agents | Separation of asset service from slower community coordination | Signal semantics and recipient layout must be fixed and tested |
| Execution compositions | Pipeline and Ensemble | Frozen leaves, teacher–student stages, independent local learners, and composite export | Composition is an execution structure, not a new learning algorithm |

The same progression also exposes the four cross-cutting requirements introduced in Section 1.2. Scalability appears when a central action vector or joint critic grows with the community. Deployability appears when a policy depends on a teacher, an observation encoder, or a coordination signal that is absent from its exported artefact. Safety appears when a low-cost action conflicts with electrical headroom or a service deadline. Robustness appears when forecasts degrade, commands are delayed, equipment becomes unavailable, or the active control interface changes. These properties are observed throughout the chapter instead of being attached to one final algorithm.

## 5.2 Common Control Problem and Experimental Protocol

A fair controller comparison begins before training. The physical problem, information boundary, action semantics, accounting rule, and evidence threshold must be fixed first. Otherwise, a controller can appear stronger because it sees additional forecasts, uses community information unavailable to the baseline, receives a different settlement rule, or is evaluated on an easier service schedule.

### 5.2.1 REC Control Formulation

Consider a community with active members $\mathcal{N}_t$. Most experiments in this chapter use a fixed population $\mathcal{N}$; scenarios in which $\mathcal{N}_t$ or the assets of a member change are used to test the boundary of those methods. Member $i$ has inflexible demand $P_{i,t}^{\text{load}}$, photovoltaic generation $P_{i,t}^{\text{PV}}$, and a set of controllable assets $\mathcal{A}_{i,t}$. The local net exchange follows the sign convention

$$P_{i,t}^{\text{grid}} = P_{i,t}^{\text{load}} - P_{i,t}^{\text{PV}} + \sum_{b \in \mathcal{B}_{i,t}} P_{i,t}^{b} + \sum_{c \in \mathcal{C}_{i,t}} P_{i,c,t}^{\text{EV}} + \sum_{d \in \mathcal{D}_{i,t}} P_{i,d,t}^{\text{def}}, \tag{5.1}$$

where positive power denotes import. $\mathcal{B}_{i,t}$, $\mathcal{C}_{i,t}$, and $\mathcal{D}_{i,t}$ contain stationary batteries, chargers with active EV sessions, and deferrable services. Community exchange is

$$P_t^{\text{com}} = \sum_{i \in \mathcal{N}_t} P_{i,t}^{\text{grid}}. \tag{5.2}$$

These two balances are related but do not define the same optimisation problem. A local controller can reduce $P_{i,t}^{\text{grid}}$ while increasing $P_t^{\text{com}}$ at an already congested time. Community settlement can also change a participant's cost without changing the physical exchange of that participant. Local retail cost, community settled cost, and grid-only counterfactual cost are kept as distinct quantities.

The principal control actions are continuous battery and charger set-points together with discrete or bounded start decisions for deferrable services. A compact local action is

$$a_{i,t} = \left[ \{P_{i,t}^{b}\}_{b \in \mathcal{B}_{i,t}}, \{P_{i,c,t}^{\text{EV}}\}_{c \in \mathcal{C}_{i,t}}, \{u_{i,d,t}^{\text{def}}\}_{d \in \mathcal{D}_{i,t}} \right]. \tag{5.3}$$

Several physical and service constraints restrict this vector. Storage energy evolves according to

$$E_{t+1} = E_t + \eta^{\text{ch}} P_t^{\text{ch}} \Delta t - \frac{P_t^{\text{dis}} \Delta t}{\eta^{\text{dis}}}, \qquad E^{\min} \le E_t \le E^{\max}, \tag{5.4}$$

with charging and discharging bounded by the asset's power limits. An EV action is valid only while the associated session occupies its charger, and the energy at departure must satisfy its declared minimum service requirement. A deferrable cycle can start only inside its admissible window and must then execute its complete profile. Total and per-phase imports and exports are bounded at building and service level. These restrictions are evaluated on the applied trajectory, while requested actions and simulator interventions remain visible separately.

For learning, the community is represented as a partially observed stochastic game

$$\mathcal{G} = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{O}_i\}_{i \in \mathcal{N}}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, P, \{R_i\}_{i \in \mathcal{N}}, \gamma \rangle, \tag{5.5}$$

where $s_t \in \mathcal{S}$ is the simulator state, $o_{i,t} \in \mathcal{O}_i$ is the information available to agent $i$, $a_{i,t} \in \mathcal{A}_i$ is its action, and $P$ is the transition induced by the physical models, exogenous data, user services, and joint action. The objective of agent $i$ is

$$J_i(\pi) = \mathbb{E}_{\pi,P}\left[\sum_{t=0}^{T-1} \gamma^t r_{i,t}\right]. \tag{5.6}$$

The reward is a training signal, not the final definition of controller quality. A general reward family used across the experiments has the form

$$r_{i,t} = -w_C\tilde{C}_{i,t} - w_P\tilde{\Psi}^{\text{peak}}_{i,t} - w_R\tilde{\Psi}^{\text{ramp}}_{i,t} - w_S\tilde{L}^{\text{service}}_{i,t} - w_V\tilde{V}_{i,t} - w_D\tilde{L}^{\text{degradation}}_{i,t}, \tag{5.7}$$

where the tildes denote scale-normalised components. $C_{i,t}$ can be local retail cost, an allocated community settlement cost, or a community objective shared by several agents. $V_{i,t}$ records physical violations or unsafe requests; service loss covers EV departures and incomplete deferrables. Peak, ramping, and degradation terms are included only when they belong to the registered experimental question. Every run records the selected components, weights, normalisers, and aggregation rule. A change to any of them defines a different method variant.

### 5.2.2 Information and Comparison Surfaces

The Algorithms repository supports several valid controller views. Mixing them in one ranking would confound the source of an observed improvement. The final protocol separates three comparison surfaces, shown in Table 5.2.

Table 5.2: Comparison surfaces used to preserve information and objective fairness.

| Surface | Controller information | Primary references | Principal question |
|---|---|---|---|
| Strict building-local | Local asset, demand, generation, service, current/causal forecast, and public exogenous data; no private observations from other members | Normal, RBC Smart Local, and individual MILP | Does adaptation improve a building's own outcome under the same local information? |
| Decentralised community | Local actor observations; declared community features where authorised; joint information available only to central critics during training | RBC Community and community MILP; MADDPG as the historical learning reference | Does explicit multi-agent training improve local and community outcomes without centralising execution? |
| Hierarchical community | Frozen or separately trained local leaves; coordinator receives declared aggregates or compact member summaries and emits a coordination signal | Neutral signal, fixed-signal probes, and the same local controller without a coordinator | Does the additional coordination layer create causal value beyond the local controller? |

The strict building-local surface is deliberately demanding. A policy composed as $N$ independent learners does not become a local method merely because its actors are separate. Community observations, community settlement in the reward, or a teacher with community access would still cross the boundary. The local reference is therefore `RBCSmartLocalPolicy`, and each building has its own cost, service gates, checkpoint, and exported actor.

The decentralised community surface permits coupled training. In MADDPG and MATD3, actor $i$ still observes $o_{i,t}$ at execution, while its critic can use the joint state and action during training. Community settlement, aggregate demand, or other shared observations are available only when the experiment declares them. The comparator is `RBCCommunityPolicy` or the community MILP, not a strictly local RBC whose information is weaker.

The hierarchical surface fixes the leaf before attributing a result to the coordinator. A multiplier of one is the neutral intervention and must reproduce the standalone leaf exactly. Fixed signals are run before a learned coordinator to show that the signal reaches a controllable mechanism. This causal check prevents a neural coordinator from being credited for variation caused by a changed leaf, a different reward, or an out-of-distribution observation transformation.

Perfect-information schedules form a fourth category, but not a competitive surface. They are labelled as oracle references. A controller that uses realised future prices, demand, production, sessions, or faults cannot be ranked as an operational alternative to a causal policy. The repository also defines an operational rolling-horizon protocol in which the model receives only information available at decision time. That variant becomes a controller comparator only after its complete replay, forecast, solve-time, fallback, and infeasibility audit; until then, it remains a planned entry in the evidence ledger.

### 5.2.3 Dataset Ladder and Experimental Funnel

The annual datasets in Section 4.4 are used as a funnel rather than a Cartesian product of every controller, scenario, and hyperparameter. This keeps the campaign computationally defensible and reduces selection on the final stress cases.

An algorithm is admitted only to scenarios compatible with its declared interface. The fixed-vector PPO, TD3, MADDPG, MATD3, and fixed-output coordinators can be evaluated on static Core-30 variants. Their inability to absorb an added charger or a changed population is recorded as an applicability boundary, not hidden by silently padding a new vector or retraining on the test event. RBCs that consume the entity interface can traverse dynamic cases because their decisions are reconstructed from the active assets. The larger dynamic datasets are therefore useful even before a learning method can be ranked on them: they reveal which assumptions are structural.

Temporal evaluation also follows a funnel. Short windows are used to diagnose wiring, reward behaviour, and service failures. Algorithm selection then uses seasonally distributed training and validation blocks with causal forecasts. Final claims of temporal generalisation require a disjoint evaluation interval and a frozen checkpoint. A deterministic replay over the complete year measures annual operation and rare-event coverage; when the same annual exogenous trajectory contributed to training, that replay is identified as an annual in-sample or robustness result rather than an independent generalisation test.

Expert schedules obey the same split. A perfect-information MILP can describe the opportunity available in an evaluation interval but cannot supply demonstrations for a policy that is then claimed to generalise on that interval. Training demonstrations are generated only from the training portion

Table 5.3: Role of the annual dataset suite in the controller campaign.

| Dataset or family | Use in Chapter 5 | Evidence admitted | Selection status |
|---|---|---|---|
| Micro-4-Q | Loader, action, reward, export, and short end-to-end checks | Functional smoke evidence only | Reusable during debugging |
| Core-15-Stripped | Algorithm development, ablations, learning diagnostics, and early resource screening | Development evidence; no final superiority claim | Open during development |
| Core-30-Nominal | Principal matched comparison under a fixed population and complete asset mix | Seed-level learning and held-out control evidence | Frozen before final runs |
| Core-30-Safety | Total and per-phase headroom under otherwise matched conditions | Safety and service attribution | No safety-specific retuning after evaluation |
| Core-30-Health | Missing or biased observations, delayed/dropped commands, and equipment unavailability | Robustness degradation and recovery evidence | No health-specific retuning after evaluation |
| Core-30-Dynamic and Combined | Membership and asset changes, followed by interacting disturbances | Applicability and failure-boundary evidence for fixed-interface methods; performance evidence only for compatible controllers | Held back until nominal selection |
| Premium-100-Clean | Scale, heterogeneity, multiple chargers, and variable participation | Final scale and interface evidence for eligible methods | Held back |
| Premium-100-AllIn | Coincident scale, safety, health, and reconfiguration pressure | Confirmatory stress evidence | Final confirmation only |

or from a separately declared historical dataset. The manifest records the source interval, oracle scope, solver status, model gap, and simulator replay status.

### 5.2.4 Training Budgets, Repetition, and Statistical Reading

Learning algorithms are compared under matched environment interactions within a declared surface. Wall-clock parity alone is unsuitable because an on-policy method, an off-policy method, and an optimisation solver use data differently. Every result therefore reports environment transitions, update count, episodes, training time, hardware, peak memory, checkpoint-selection rule, and inference latency. Hyperparameter search is part of the computational cost and is reported separately from the final training run.

Stochastic methods use at least three independent training seeds in the principal comparison. The checkpoint rule is fixed before the confirmatory evaluation and cannot select directly on the final test score. Learning curves report the centre and dispersion across seeds. Controller KPIs are paired with the same scenario and reference trajectory. A single favourable seed cannot establish a gain, while a reduction in seed variance can be valuable even when the mean KPI changes little.

The 35,040 time steps of one annual trajectory are temporally dependent and are not treated as 35,040 independent samples. Confidence intervals for annual energy and cost differences use paired temporal blocks, such as days or weeks, chosen to preserve autocorrelation. Member-level

comparisons retain the complete distribution, the number $x/N$ of eligible members that improve, the worst member, and a fairness measure. The members form a designed heterogeneous scenario rather than a random sample of the Portuguese population, so inference across members is interpreted within the scenario.

The final decision is lexicographic. Run integrity is checked first. Electrical, EV, deferrable, SoC, and outage gates follow. Cost, peak, ramping, solar use, import/export, and participant outcomes are interpreted only after those gates pass. Computational and deployment properties then distinguish candidates with comparable control quality. A weighted reward or aggregate score cannot compensate for a physical violation or an unserved mandatory service.

The core scorecard is the 17-indicator contract in Table 4.6. Extended KPIs and action traces diagnose why a result changed. In particular, requested and applied violations are kept separate: a safe physical trajectory produced only because the simulator clipped unsafe requests is not evidence that the policy learned safe behaviour.

## 5.3 Engineering the Control–Learning Interface

Changing MADDPG to MATD3, or PPO to MAPPO, addresses only part of the learning problem. The controller also needs a stable interpretation of simulator state, an action vector whose entries retain their physical meaning, a reward that does not favour an unwanted shortcut, and a training record that distinguishes what the policy proposed from what was executed. Several apparently small implementation changes were introduced after a concrete failure in one of those contracts. They form part of the method because removing them changes the decision problem seen by the learner.

### 5.3.1 From Simulator State to Policy Input

The simulator's entity interface exposes tables, relations, and metadata. The Algorithms wrapper resolves those records into one observation and one action contract per building. District context, local demand and generation, batteries, physical chargers, connected and incoming EV sessions, deferrable services, electrical limits, action feedback, and topology metadata retain their identities during this conversion. A policy never infers that identity from column position alone; the ordered feature and action names are attached to the controller and written to its artefact manifest.

For a fixed-vector learner, the input at time $t$ is

$$x_{i,t}^{(p)} = \mathscr{E}_p\left(\mathscr{F}_{i,p}(O_t)\right), \tag{5.8}$$

where $O_t$ is the entity payload, $\mathscr{F}_{i,p}$ applies the member and information filters of profile $p$, and $\mathscr{E}_p$ encodes the retained values. Filtering precedes learning. A central feature removed only after it influenced a derived variable would still leak information into a strict-local experiment.

Table 5.4: Implemented entity-encoding profiles and their experimental use.

| **Profile** | **Representation** | **Use** |
|---|---|---|
| Min–max space | Complete surface, normalised from the simulator spaces without changing feature order or width | RBC inspection, debugging, schema audits, and the structured Transformer controllers |
| MADDPG v1 | Complete surface with cyclic time, normalised SoC, relative deadlines, physical scaling, and explicit missing-value treatment | Compatibility and detailed audits |
| MADDPG v2 compact | Earlier compact vector | Reproduction of experiments created before the operational feature set stabilised |
| MADDPG v3 operational | Compact state plus feasible EV actions, departure margin, charge/discharge availability, applied-action feedback, deferrable must-start state, and compact forecasts | Principal fixed-vector training profile |
| MADDPG v3 real-time | Operational profile without forecasts derived from realised simulator futures | Causal, real-world-safe comparison and deployment preparation |

Calendar variables use cyclic or categorical encodings; connection, availability, and operating states retain explicit indicators; and missing continuous values are represented through a declared missing-value convention. Departure time is expressed relative to the current step, while feasible charging margin is expressed in physical or normalised action units. The absolute episode index remains metadata. Feeding it to the actor would make memorising one annual trajectory easier than learning the operational state that recurs in another year.

The first vectorisation path repeatedly searched and transformed the complete observation list. It was replaced by a compiled source plan grouped by table, row, and column, followed by vectorised encoding. A direct entity-to-model path avoids constructing a second raw vector when the controller does not need it. This change is computational rather than conceptual, but it matters to RQ2 and RQ4: preprocessing latency is part of inference latency, and the same compiled feature order must be reproduced outside the training process.

### 5.3.2 Action Semantics and the Executed Transition

Every controllable battery, charger, and deferrable service has its own action entry. Two chargers in one building therefore remain two independently addressable controls. Shared-trunk actors initially ended in one dense output layer. Multi-head actors separated individual commands, and semantic multi-head actors later grouped EV, stationary-storage, deferrable, and other actions while scattering their outputs back to the simulator's original order. This reduced direct competition between unrelated outputs without changing the external action contract.

The complete action path is written as

$$
\begin{aligned}
z_{i,t} &= \pi_{\theta_i}(x_{i,t}),\\
a^{\mathrm{pol}}_{i,t} &= \ell_i + \frac{\tanh(z_{i,t})+1}{2}(u_i-\ell_i),\\
a^{\mathrm{cmp}}_{i,t} &= \mathrm{clip}_{[\ell_i,u_i]}\left(a^{\mathrm{base}}_{i,t} + g_{i,t}\odot a^{\mathrm{pol}}_{i,t}\right),\\
a^{\mathrm{req}}_{i,t} &= \Pi^{\mathrm{loc}}_{i,t}\left(a^{\mathrm{cmp}}_{i,t}; O_t\right), \qquad a^{\mathrm{app}}_{i,t} = \mathscr{S}\left(a^{\mathrm{req}}_{i,t}, s_t\right),
\end{aligned}
\tag{5.9}
$$

where $[\ell_i, u_i]$ are the active bounds, $a^{\mathrm{base}}$ is zero for an autonomous policy or comes from a declared teacher, $g_{i,t}$ is the residual authority, $\Pi^{\mathrm{loc}}$ is the optional controller-side projection, and $\mathscr{S}$ is the simulator transition. The recorded trace retains the policy, base, composed, requested, projected, and applied interpretations whenever the selected controller uses them. This separation prevents an imitation target from being confused with an executed action and prevents simulator clipping from being presented as learned safety.

Initial exploration also became asset-aware. Uniform full-range actions are useful for checking bounds, but they fill early replay with saturated storage, negative EV commands, and arbitrary deferrable starts. The implemented alternatives include no-op-centred sampling, separate storage and negative-EV noise scales, controlled deferrable trigger probabilities, and deterministic rule-guided warm starts. Warm starts leave through a registered probability or blending schedule. No-op actor initialisation and residual zero-initialisation make the initial neural contribution small enough to audit. These choices are ablated because an exploration strategy changes both safety exposure and the distribution from which an off-policy critic learns.

### 5.3.3 Reward Lineage

The general decomposition in Equation (5.7) was not written once and left unchanged. Early cost-focused rewards exposed shortcuts: V2G could lower cost while an EV remained below its target; dense urgency could make immediate charging look better than waiting for a cheaper feasible interval; infeasible departure deficits could dominate every critic target; and a high storage-throughput penalty could remove the very flexibility being studied. Each correction was kept as a named profile so that an old run could still be reproduced.

The common reward implementation emits both per-agent and community components. Its economic term can use local retail import, exact member settlement, or a declared community aggregation. EV penalties distinguish connected deficit, recoverable schedule deficit, departure failure, V2G below service, and over-service. The feasible schedule calculation uses the charger power available after building and phase headroom when those observations exist. Stationary-storage terms use observed SoC limits where possible, and state penalties can scale with the 900-second timestep. Deferrable urgency and missed deadlines remain separate components. Community terms can include import, peak, ramping, export, emissions, and electrical violations.

Reward and scorecard retain different jobs. Reward supplies gradients and may deliberately smooth a sparse consequence. The scorecard measures settled cost, feasible EV service, target

Table 5.5: Main stages in the implemented reward lineage. Exact parameters and registered profile names are retained in Appendix A.6.

| Stage | Profiles | Problem corrected |
|---|---|---|
| Cost with hard-service terms | Cost Hard Constraint, Service Guard V2, and Cost Balanced V3 | Makes electrical violations, outages, EV departure loss, V2G below target, storage bounds, and deferrable deadlines visible beside cost |
| Strict local objectives | Local V1, Scorecard Guard V2, Economic Safety V3, and Individual Scorecard Aligned V3 | Separates local information from settlement-aware reward, calibrates service gates, and removes dense urgency when a feasibility projector already protects recoverability |
| Community settlement and service bands | Community V4, V41, and V42 | Aligns the economic term with member settlement, introduces over-service precision, and tests storage versus service pressure |
| Physical feasibility and timestep scaling | Community V43–V47 | Uses observed storage bounds, scales state penalties by timestep, smooths recoverable deficits, accounts for phase-limited effective EV power, and caps only the training contribution of infeasible gaps |
| Flexibility value | Community V49–V52 | Rebalances settlement, peak, export, storage throughput, deadline service, and target-band precision so that useful cycling is neither free nor suppressed |
| Residual learning | Community V53–V55 | Makes small improvements over an RBC visible, densifies sparse EV consequences, and removes service terms from a storage-only critic only when service actions are copied unchanged from an audited teacher |
| Coordinator rewards | CC, CC Level 1, CC Level 2, and CC Phase 1 | Moves from price-arbitrage signals to normalised community cost, peak, ramp, export, violation, service, and member-decomposed credit |

tolerance, electrical violations, battery throughput, V2G, solar self-consumption, import, export, net exchange, peaks, load factor, runtime, and participant outcomes. A reward revision is accepted when it fixes a diagnosed learning signal under held-out evaluation; a larger return alone is not evidence of a better controller.

### 5.3.4 Replay, Demonstrations, and Rare Events

PPO and the deterministic off-policy methods require different records. A PPO rollout stores the exact pre-squash sample, old log-probability, value estimate, executed action, reward, and termination/truncation state. A policy-eligibility flag prevents teacher, warm-start, or phase-out decisions from entering the clipped policy-ratio objective as if the current actor had sampled them. Short rollouts are flushed at episode or topology boundaries and bootstrap only when the successor state remains valid.

The off-policy buffer began as a joint list of transitions and became a compact set of preallocated arrays. It stores per-agent current and successor states, final action, reward, and done state. Residual and imitation experiments add separate current base action, successor base action, and cloning target. The separation matters: the critic must evaluate the action that affected the environment, a residual target must reconstruct the base policy at the successor state, and a stronger teacher may supply a cloning target that was never executed.

Uniform replay remains the reference. The multi-agent event-aware buffer mixes uniform and weighted samples using

$$p_k = \varepsilon + \lambda_r g(r_k) + \lambda_a h(a_k^{\text{beh}}) + \lambda_e e_k, \qquad \Pr(k) = \frac{1-\eta}{M} + \eta \frac{p_k^\alpha}{\sum_{j=1}^{M} p_j^\alpha}, \tag{5.10}$$

where $g$ can select absolute, negative, or positive reward magnitude, $h$ measures registered behaviour-action activity, $e_k$ is an external observation-event boost, $M$ is the active buffer size, and $\eta$ is the weighted fraction of a batch. Priorities are capped when configured. The buffer can restrict $h$ to EV dimensions and form separate positive-action strata for each charger. A further stratum retains steps in which several chargers act concurrently; a maximum over the joint action would otherwise make one active charger indistinguishable from two.

Observation-derived boosts cover registered EV departure/service states and combined EV–PV–price–peak events. Multi-step returns propagate a sparse consequence across the preceding transitions, with max, mean, or sum aggregation of their event boosts. These mechanisms do not replace the uniform ablation. Repeatedly sampling penalties can distort ordinary operation, so rare-event coverage, critic stability, service, and routine-period cost are read together.

Fixed-vector replay rejects a change in state or action dimension. The structured MATD3 buffer takes a different approach: every transition carries a complete layout signature containing token families, entity types, instance identities, feature names, action names, and widths. Batches contain one signature only. A topology boundary flushes the pending multi-step queue as truncated under the old signature; learning resumes when the new bucket contains enough samples. Historical buckets remain available, while a building-count change clears the controller state because no safe remapping has been declared.

### 5.3.5 Optimisation, Checkpoints, and Export Continuity

Several safeguards were added after long runs made small numerical failures expensive. They include running reward or return normalisation, target clipping, selectable mean-squared or Huber critic loss, gradient clipping, delayed actor updates, target-policy smoothing, soft target updates, automatic entropy-temperature tuning in MASAC, and optional mixed-precision training. Actor diagnostics separate policy, behaviour-cloning, residual, saturation, storage, and V2G terms. Critic diagnostics retain target scale, absolute temporal-difference error, twin-critic disagreement, gradient norm, and update time. A configuration may disable a mechanism for an ablation; it cannot change it silently between candidates.

Full checkpoints contain the model and target networks, optimisers, training clocks, replay or rollout state, reward normalisers, exploration state, behaviour-cloning progress, and Python, NumPy, PyTorch, and CUDA random-number states where applicable. Inference checkpoints omit training-only state. Restore validates algorithm, building order, observation and action dimensions, bounds, and, for structured policies, the complete layout signature before mutating a live controller.

This transactional rule prevents a partially restored process from continuing with an actor, optimiser, and replay buffer from different runs.

Deterministic actors can be exported per building to ONNX together with ordered observations, encoders, action bounds, names, topology, and provenance. Residual composition, local safety, and price-conditioning adapters remain external to several neural graphs. The exporter refuses to call such an artefact independently deployable unless the required runtime dependency is declared. This final check connects algorithm engineering to the deployment lifecycle: the action observed in a simulator evaluation must be reproducible from the exported policy set and its manifest.

## 5.4 Level I: Transparent and Constraint-Aware Controllers

Transparent controllers answer two questions that learning alone cannot settle. They show whether the scenario can be controlled without violating its basic obligations, and they reveal how much improvement follows from information and domain logic before training begins. They also provide deterministic trajectories against which observation encoding, action scaling, settlement, and KPI export can be checked.

### 5.4.1 Business-as-Usual and the Rule-Based Ladder

The baseline ladder maps repository implementations to stable scientific roles. Class names may evolve as compatibility code is removed; the information and decision contract of each level remains fixed.

Table 5.6: Transparent controller ladder used in the experiments.

| Level | Repository policy | Decision logic | Scientific role |
|---|---|---|---|
| BAU without stationary storage | `NormalNoBatteryPolicy` | Immediate EV charging towards service, earliest feasible deferrable start, and no stationary-battery action | Native demand and service denominator |
| BAU with storage | `NormalPolicy` | The same service behaviour with simple PV self-consumption by the stationary battery | Isolates the value and side effects of installed storage |
| Basic informed control | `RBCBasicPolicy` | Current/near-term price, EV energy gap, time to departure, deferrable urgency, and storage limits | Minimum credible intelligent baseline |
| Strong local control | `RBCSmartLocalPolicy` | Local PV and load, causal forecasts, headroom, price, service slack, conservative V2G, and asset limits; community access disabled | Strict local reference and learning teacher |
| Strong community control | `RBCCommunityPolicy` | Local rules augmented with community import, export, surplus, headroom, and settlement context | Non-learning community comparator |

BAU is not designed to be deliberately poor. An EV charges when it is connected, a mandatory deferrable service starts at the earliest feasible time, and the user receives the requested service. The no-battery variant provides the clean denominator; the storage variant shows how much ordinary self-consumption changes without price or community optimisation. These cases also prevent a learned controller from receiving credit for simply activating an asset that the reference left unused.

RBC Basic introduces the smallest amount of anticipatory reasoning. Let $E_{c,t}^{\mathrm{req}}$ be the energy still required by an EV session, $H_{c,t}$ the remaining connected steps, and $\bar{P}_c^{\mathrm{ch}}$ its maximum charging power. A simple urgency measure is

$$\rho_{c,t}^{\mathrm{EV}} = \frac{E_{c,t}^{\mathrm{req}}}{H_{c,t}\,\bar{P}_c^{\mathrm{ch}}\,\eta_c\,\Delta t}. \tag{5.11}$$

Values close to one leave little freedom to defer charging. The rule charges at the minimum rate required to remain feasible and uses cheap periods when slack exists. Deferrables follow the same logic through remaining duration and latest start. Storage responds to declared cheap and expensive price regions while respecting its SoC range.

RBC Smart Local adds solar, demand, headroom, and causal forecast context. It uses available PV before importing for flexible demand, reserves enough EV energy for the departure target, and only permits V2G when the remaining connection window can recover the discharged energy. Deferrable services start when their slack becomes small or when local surplus is available. The stationary battery supports local demand and service obligations before it pursues price arbitrage. The strict-local implementation neutralises community access even when a raw simulator observation contains those columns. This is essential because an RBC consumes named observations directly and could otherwise see information that was removed only from the neural encoder.

RBC Community introduces aggregate context. Its local decisions react to community surplus, import pressure, and the declared settlement rule. This controller tests a common source of confounding: a learned multi-agent policy can outperform a local RBC because it receives a community signal, not because learning discovered a better use of the same information. The community-aware rule makes that information advantage visible.

The ladder is evaluated incrementally. BAU without storage is compared with BAU with storage; Basic with BAU; Smart Local with Basic; and Community with the matched local and community references. This order attributes changes to storage availability, deadline and price awareness, richer local information, and community information. It also supplies fallbacks of different complexity. The strongest rule is not assumed to dominate every KPI: conservative service and headroom logic can trade cost or solar use for a larger operating margin.

### 5.4.2 MILP Reference Problems

Rule-based control embeds domain knowledge in conditionals. MILP exposes the same decision problem through variables, constraints, and an objective. It is useful as a benchmark because a

schedule can be audited step by step and its optimality statement can be restricted to a precise mathematical model.

For a storage asset, non-negative charge and discharge variables $p_t^+$ and $p_t^-$ are linked to Equation (5.4). Binary variables prevent simultaneous charge and discharge where the formulation requires it. An EV session $s$ attached to charger $c$ has zero power outside $[t_s^{\mathrm{arr}}, t_s^{\mathrm{dep}})$ and must satisfy

$$E^{\mathrm{EV}}_{s,t_s^{\mathrm{dep}}} \geq E_s^{\mathrm{EV,min}}. \tag{5.12}$$

A deferrable service has one binary start variable inside its admissible window. Its power trajectory is the convolution of that start with the complete cycle profile. Total and phase-specific exchange constraints apply to the combined inflexible demand, PV, storage, EV, and deferrable schedule.

The local retail objective is

$$\min \sum_t \lambda_t \sum_{i \in \mathcal{N}} G_{i,t} \Delta t, \qquad G_{i,t} \geq P_{i,t}^{\mathrm{grid}}, \quad G_{i,t} \geq 0, \tag{5.13}$$

where export has no retail credit under the declared counterfactual. The community model replaces the independent imports by

$$\min \sum_t \lambda_t G_t \Delta t, \qquad G_t \geq \sum_i P_{i,t}^{\mathrm{grid}}, \quad G_t \geq 0. \tag{5.14}$$

The symmetric internal transfer price cancels in the aggregate community objective used by the current dataset contract. Explicit member settlement variables are still needed when individual bills, unequal weights, fairness, or asymmetric tariffs enter the objective.

Four scopes prevent the expression "the MILP" from hiding different decision problems.

Table 5.7: MILP reference scopes and permitted interpretation.

| Scope | Controlled decisions | Permitted interpretation |
|---|---|---|
| Local fixed-service | Stationary battery of each building; EV and deferrable trajectories fixed to the matched service controller | Conditional local opportunity for storage scheduling |
| Community fixed-service | Stationary batteries scheduled jointly; EV and deferrables fixed | Conditional community opportunity under the fixed service trajectories |
| Local total-energy | Stationary battery, EV/V2G, and deferrable starts inside one building | Optimum of the supplied local linear/MILP model, followed by a simulator-feasible upper schedule after replay |
| Community total-energy | All represented flexible assets and the community exchange objective | Optimum of the supplied community model, followed by a simulator-feasible upper schedule after replay |

Service is handled lexicographically. The solver first maximises physically attainable mandatory service, then enforces the registered gates, and only then minimises cost. This avoids a schedule that lowers its objective by deliberately missing a departure or omitting a deferrable cycle. Each

solve records its status, primal objective, lower bound, relative gap, horizon, forecast assumptions, and wall time.

A model solution is replayed through the simulator before it is used as controller evidence. Replay checks nonlinear efficiencies, initial conditions, session boundaries, action conventions, phase assignment, and settlement. A small model–replay gap supports the fidelity of that bounded formulation. It does not prove global optimality for every mechanism implemented by the simulator. The resulting language remains precise: "optimal for the supplied model and feasible on replay" is stronger and more useful than calling every perfect-foresight schedule optimal.

The implemented reference family currently uses perfect information and therefore estimates opportunity; it is not an operational controller. The repository fixes a separate rolling-horizon protocol for the operational variant: seven days of causal lookahead, one day committed after each solve, exact carry-over of storage and EV state and of deferrable cycles already in progress, and an audited fallback when a solve fails. The seven-day horizon covers the longest EV session and deferrable window in the current annual suite while avoiding the size of one monolithic annual MILP. This rolling schedule can enter the controller comparison only after its complete simulator replay and solve-failure audit. The perfect-information schedules may also provide training demonstrations, but only on intervals permitted by the data split.

### 5.4.3 Evidence Produced at Level I

Level I produces more than baseline cost. The RBC ladder tests whether every asset and service can be traversed for a complete year, whether richer information changes the intended actions, and whether requested actions remain inside their bounds. MILP identifies the gap left by the strongest rule and distinguishes a weak controller from a physically narrow opportunity. Together, the two families supply three reference points: ordinary operation, a credible low-cost heuristic, and a model-based schedule under explicit assumptions.

Their limitations are different. RBC quality depends on anticipated cases and manual priority rules. MILP quality depends on model fidelity, forecast quality, horizon length, and solve time. Both normally assume that the assets and decision variables are known before execution. These boundaries motivate the adaptive level without removing the value of either reference.

## 5.5 Level II: Adaptive Building Control

The building-level experiments isolate the value of learning from the value of coordination. Each member owns one learner and receives only the observations admitted by the strict local surface. The environment still contains the whole community so that shared physical conditions are reproduced, but one building's actor does not read another building's private state. This formulation also matches a plausible deployment boundary: a local controller can operate when the community service is temporarily unreachable.

### 5.5.1 Strict Local Learning Contract

For member $i$, the stochastic game in Equation (5.5) reduces to a local MDP with observation $o_{i,t}$, action $a_{i,t}$, and local reward $r_{i,t}^{\mathrm{loc}}$. A configuration containing $N$ PPO or TD3 learners is built as an ensemble of $N$ independent actor–critic pairs. Replay, value estimation, normalisation, checkpoints, and exported models remain separate. The global building index is preserved in each artefact so that a local policy cannot later be assigned to the wrong observation or action contract.

Local observations cover asset state, inflexible demand, PV, service deadlines, current price, and causal forecast features authorised by the experiment. The reward uses the member's physical retail cost or another explicitly local objective. Community settlement and private member aggregates are excluded. RBC Smart Local and the individual MILP share this boundary.

Three policy modes are separated throughout the experiments:

**Autonomous.** The actor proposes the complete local action. Safety logic only enforces the declared physical envelope.

**Residual.** A deterministic local controller supplies a feasible base action and the actor learns a bounded correction.

**Service-anchored.** EV and deferrable obligations remain with a verified service controller while the actor optimises a restricted asset set, usually stationary storage. The runtime dependency is stated explicitly.

The autonomous mode tests the strongest learning claim but also presents the largest search space. Residual control tests whether learning can improve a strong operating point without rediscovering service logic. Service anchoring isolates economic adaptation where the complete action problem is not yet learned reliably. These modes cannot be pooled under one algorithm name.

### 5.5.2 Proximal Policy Optimisation

PPO represents the on-policy branch of the comparison (Schulman et al. 2017). A stochastic actor with parameters $\theta$ produces a bounded continuous action. The implementation samples a Gaussian latent action, applies the hyperbolic tangent transformation, and accounts for that transformation in the log-probability. A value network estimates the expected return from the local observation.

The clipped policy objective is

$$\mathscr{L}^{\mathrm{clip}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right], \tag{5.15}$$

where $r_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\theta_{\mathrm{old}}}(a_t \mid o_t)$ and $\hat{A}_t$ is an advantage estimate. Clipping limits the influence of a large policy update. The complete loss includes a value term and an entropy term, with gradient clipping and a KL diagnostic used to detect an excessive update.

PPO is useful here for three reasons. Its stochastic actor explores bounded continuous decisions without an external deterministic noise process. Its on-policy updates provide a clean separation between actions sampled from the current policy and actions supplied by a teacher. The same implementation can also act as a community coordinator over a lower-frequency action space.

The cost is sample use. Once a rollout has contributed to the PPO update, it cannot be replayed indefinitely as if it came from the current policy. Teacher-controlled transitions are therefore excluded from the policy-ratio objective. They can train the value function or a separate behaviour-cloning loss under a declared rule, but their log-probability is not presented as an on-policy sample. Topology or layout changes also require the current rollout to be closed before the policy interface is rebuilt.

### 5.5.3 Twin Delayed Deep Deterministic Policy Gradient

TD3 represents the off-policy deterministic branch (Fujimoto et al. 2018). In the strict local implementation it is the one-agent reduction of the MATD3 code path: the joint state and action contain one member only, while the twin-critic, delayed-update, and replay mechanisms remain unchanged.

Two critics $Q_{\phi_1}$ and $Q_{\phi_2}$ estimate the action value. The target is

$$y_t = r_t + \gamma(1-d_t)\min_{j\in\{1,2\}} Q_{\bar{\phi}_j}\left(o_{t+1}, \mathrm{clip}\left(\mu_{\bar{\theta}}(o_{t+1}) + \varepsilon_t\right)\right), \tag{5.16}$$

where $d_t$ marks termination and $\varepsilon_t$ is clipped target-policy noise. Taking the minimum reduces optimistic value estimates. The actor is updated less frequently than the critics, and target networks move through soft updates. Replay allows several gradient steps from previously collected transitions, giving TD3 greater sample reuse than PPO.

That reuse creates its own risks. A replay buffer can contain actions generated by an earlier policy, a teacher, or a different exploration regime. If behaviour actions are mixed without labels, the critic and any auxiliary imitation loss learn from an ambiguous distribution. The implementation therefore retains the executed behaviour action and its source. Warm-start, residual, and teacher configurations fail early when the declared teacher is unavailable.

TD3 provides a useful bridge to the multi-agent level. A local TD3 result shows what deterministic twin-critic learning can achieve without non-stationarity from other learning agents. A later MATD3 comparison can then attribute changes to joint training and coordination instead of to the underlying TD3 corrections alone.

### 5.5.4 Residual Control, Service Protection, and Local Safety

The earliest autonomous policies showed why cost cannot be the only gate. Frequent EV charging or V2G can reduce one short-window objective while missing departure requirements or creating electrical pressure. A local action adapter was introduced to preserve the meaning of service and physical headroom during learning.

In residual form, the applied proposal is

$$a_{i,t}^{\text{prop}} = a_{i,t}^{\text{base}} + g_i\,\delta_{\theta_i}(o_{i,t}), \qquad a_{i,t}^{\text{req}} = \Pi_{\mathscr{A}_{i,t}^{\text{loc}}}\left(a_{i,t}^{\text{prop}}\right), \tag{5.17}$$

where $g_i$ bounds the authority of the learned residual and $\Pi$ applies the declared local envelope. $g_i = 0$ reproduces the base policy and provides a neutral control. Per-building gains are admissible only when calibrated on development data and then frozen.

The local envelope protects action bounds, SoC, charger availability, deadline-feasible EV service, atomic deferrable starts, and observed electrical headroom. EV protection is intentionally deadline-feasible rather than an instruction to charge at the average required rate from the moment of arrival. The controller retains flexibility until postponing another step would make the service impossible. If headroom is absent from the observation contract, the adapter does not invent a zero limit. Missing evidence and an actual zero limit have different meanings.

Projection is recorded, not hidden. The scorecard reports both the unsafe energy requested before projection and the residual violation after the simulator applies the action. A policy that relies continuously on projection has not learned the constrained problem, even if the physical trace is safe. This distinction links Level II directly to safety and deployability: an adapter used during evaluation must also exist, with the same semantics, in the exported inference path.

The local learning comparison therefore includes autonomous PPO/TD3, residual variants, service-anchored variants, and their neutral controls. The decisive questions are whether the learned component improves beyond its base, whether the gain survives a disjoint interval, how broadly it is distributed across buildings, and how much external protection remains necessary.

### 5.5.5 Entity-Structured PPO

The fixed-vector PPO actor is tied to the number and order of features and actions present during training. AgentTransformerPPO was implemented to test a narrower structural question: can one building policy retain the identities of known asset types while the number of their active instances changes? The entity adapter constructs three token families for each building and processes them with a Transformer encoder (Vaswani et al. 2017):

$$[\mathscr{T}_{i,t}^{\text{SRO}}, \mathscr{T}_{i,t}^{\text{NFC}}, \mathscr{T}_{i,t}^{\text{CA}}] \xrightarrow{\text{Transformer}} [\{h_{i,c,t}^{\text{CA}}\}_c, \bar{h}_{i,t}], \qquad a_{i,c,t} = f_{\text{act}}(h_{i,c,t}^{\text{CA}}), \quad V_i(o_{i,t}) = f_V(\bar{h}_{i,t}), \tag{5.18}$$

where SRO denotes shared read-only context, NFC is one derived non-flexible-consumption token, and CA denotes a controllable asset. One projection is learned per declared entity type rather than per asset instance. The actor head is applied to each CA embedding in the simulator action order, while the value head uses the mean-pooled building representation. A second charger of a known type can therefore reuse the charger projection and actor head without introducing a permanently reserved vector slot.

Each building still owns an independent tokenizer, Transformer, actor, critic, optimiser, value normaliser, and rollout buffer. This is structured local PPO, not a centralised multi-agent critic. On-policy integrity remains strict: the rollout keeps the exact pre-tanh sample and is closed before an old layout is replaced. Compatible asset-instance changes preserve type projections, neural weights, optimiser state, normalisation state, and counters. A new entity type, changed feature width, altered action meaning, or changed building population causes rejection or a full rebuild rather than silent adaptation.

The implementation includes deterministic RBC demonstrations, actor-only pretraining, an auxiliary post-PPO cloning step, local action projection, transactional topology rollback, and per-building ONNX export. Demonstrations are grouped by the layout signature on which they were collected. The exported graph remains specific to the current topology, and runtime safety is external unless separately reproduced. AgentTransformerPPO is therefore an implemented representation baseline for variable counts of known assets. It does not solve changing action semantics, health-dependent capability loss, or population-wide credit assignment.

## 5.6 Level III: Decentralised Multi-Agent Control

Independent local policies cannot observe the effect of simultaneous decisions on a shared community objective. A collection of otherwise successful battery controllers can charge during the same low-price interval and create a new peak. Training each actor against a changing population of other learners also makes the transition distribution non-stationary from the actor's local perspective. Level III introduces centralised training with decentralised execution to address these effects.

### 5.6.1 Centralised Training and Decentralised Execution

At execution, agent $i$ uses only its authorised observation:

$$a_{i,t} = \mu_{\theta_i}(o_{i,t}). \tag{5.19}$$

During training, a central critic can condition on $s_t$ or on the concatenated observations and actions of the active agents. This removes part of the apparent non-stationarity from value estimation because the critic sees the decisions that contributed to the next state. Private observations remain a training privilege and are not required by the deployed actor.

This structure is well matched to a community in which each building retains a local controller. It does not make communication free. Collecting joint training data, building a central critic, synchronising policies, and distributing compatible checkpoints all contribute to RQ2 and RQ4. The critic input also grows with the number and dimension of agents when simple concatenation is used.

### 5.6.2 MADDPG and the Original EnergAIze Formulation

MADDPG extends deterministic policy gradients to a multi-agent setting (Lowe et al. 2017). Agent $i$ owns a local actor $\mu_{\theta_i}$ and a central critic

$$Q_i\left(s_t, a_{1,t}, \ldots, a_{N,t}\right). \tag{5.20}$$

The critic is trained with a Bellman target formed from target actors and a target critic. The actor gradient follows

$$\nabla_{\theta_i} J_i \approx \mathbb{E}\left[\nabla_{a_i} Q_i(s, a_1, \ldots, a_N)\big|_{a_i=\mu_{\theta_i}(o_i)} \nabla_{\theta_i} \mu_{\theta_i}(o_i)\right]. \tag{5.21}$$

Each critic can use an individual reward, a community reward, or a declared combination. The original EnergAIze design used one agent per dwelling or controllable node, continuous charging actions, local execution, a replay buffer, soft target updates, and early exploration guided by an RBC. This design answered the practical question that motivated the first phase of the research: local policies could be trained with knowledge of community interactions and later executed without a central action server.

The historical hourly 17-member case is retained to reproduce that result. It is not silently compared with the new 15-minute annual suite. Resolution, assets, prices, services, simulator behaviour, and baselines changed. The historical experiment establishes provenance; the canonical Core-30 campaign tests the method under the current contract.

MADDPG leaves several measurable risks. Deterministic critics can overestimate actions, learning is sensitive to reward and exploration scale, and the joint critic grows with the complete observation and action layout. A fixed actor head also binds a policy to the assets present during training. These issues motivate a controlled MATD3 comparison while preserving MADDPG as an important reference.

### 5.6.3 MATD3

MATD3 applies the principal TD3 corrections to each central multi-agent critic (Ackermann et al. 2019). Agent $i$ owns two critics, $Q_{i,1}$ and $Q_{i,2}$. Its target uses the lower estimate:

$$y_{i,t} = r_{i,t} + \gamma(1-d_t) \min_{j\in\{1,2\}} Q_{i,j}^{\text{target}}\left(s_{t+1}, \tilde{a}_{1,t+1}, \ldots, \tilde{a}_{N,t+1}\right), \tag{5.22}$$

where each $\tilde{a}_{k,t+1}$ includes bounded target smoothing. Actor and target updates occur less frequently than critic updates. The replay, actor layout, observations, actions, reward, and training budget are held constant against MADDPG wherever possible. This isolates critic structure, target construction, policy smoothing, and update delay.

The comparison is read across more than return. Critic disagreement and overestimation proxies test the mechanism directly. Seed variance and catastrophic training runs reveal stability. Cost, service, peak, ramping, import/export, and violations show whether a change in value estimation

improves the control task. Training time, memory, checkpoint size, and inference time identify the resource price of the second critic.

MATD3 is treated as an algorithmic stage, not as a relabelled final solution. The standard algorithm is prior work. Thesis-specific contributions require a named modification, a controlled ablation, and evidence under the REC problem. Residual control, behaviour cloning, event-aware replay, local safety, or hierarchical composition qualify as separate mechanisms only when their effect is isolated.

### 5.6.4 Alternative MARL Comparators

MADDPG and MATD3 belong to one deterministic off-policy family. A broader comparison checks whether their behaviour follows from that family. The current repository contains the alternatives in Table 5.8.

Table 5.8: MARL comparator families and the hypothesis tested by each one.

| Method | Training structure | Question addressed | Principal cost |
|---|---|---|---|
| IPPO | Independent PPO actor–value pair for each member | Can stable on-policy local updates compete without a central critic? | No direct central credit assignment |
| MAPPO | Decentralised PPO actors with a central value function (Yu, Velu, et al. 2022) | Does central value information help without deterministic action-value learning? | On-policy sample demand and central value growth |
| HAPPO | Sequential PPO-style agent updates (Kuba et al. 2022) | Does controlling the order and trust region of multi-agent updates improve stability for heterogeneous agents? | Sequential update cost and order dependence |
| MASAC | Stochastic decentralised actors with central twin critics, based on entropy-regularised SAC (Haarnoja et al. 2018) | Does entropy-driven exploration improve continuous community control? | Entropy tuning and stochastic training variance |

**IPPO.** IPPO uses the same squashed-Gaussian actor, value network, GAE calculation, clipped objective, entropy term, value loss, minibatches, and KL diagnostic as the local PPO branch. Each member estimates value from its own observation and reward. It is the cleanest test of whether several independent on-policy learners are sufficient; any community improvement must arise through the environment and reward rather than a central training input.

**MAPPO and HAPPO.** MAPPO keeps decentralised actors but evaluates advantage with a central value input. It can use joint information during training without learning an action-value function over the full joint action. HAPPO uses the same central-value principle and updates agents sequentially in a fixed or random registered order. The sequential update makes the policy change of one agent explicit before the next is optimised. This is relevant for heterogeneous members, although it increases update cost and makes the order an experimental variable.

**MASAC.** MASAC replaces deterministic actors and external action noise with stochastic Gaussian actors. Each agent has two central critics and an entropy temperature $\alpha_i$. Its actor minimises

$$\mathscr{L}_{\pi_i}^{\text{MASAC}} = \mathbb{E}\left[\alpha_i \log \pi_i(a_i \mid o_i) - \min_{j\in\{1,2\}} Q_{i,j}(s, a_1, \ldots, a_N)\right], \tag{5.23}$$

and the temperature can be adjusted automatically towards a target entropy based on the local action dimension. The comparison tests whether continued entropy-driven exploration improves coverage of continuous REC actions after the initial random phase. Checkpointing includes both critic pairs, the stochastic actors, entropy parameters and optimisers, replay, and inherited training state.

These methods enter after the reference and data contracts are stable. They are screening comparators, not automatically equal thesis contributions. A candidate advances only after it passes the same EV, deferrable, electrical, SoC, and outage gates as MADDPG and MATD3. The matrix prevents extensive training of an algorithm that has not yet shown feasible short-window behaviour.

### 5.6.5 Entity-Structured MATD3

AgentTransformerMATD3 combines the entity-token actor introduced in Section 5.5.5 with off-policy centralised training. Each building owns an independent deterministic actor stack and one pair of independent central critics. A critic tokenises every building, injects the corresponding action into each controllable-asset embedding, pools the action-conditioned representation of that building, and aggregates buildings with a permutation-invariant Deep Sets operation before producing $Q_i$. The critic therefore accepts different numbers of known asset instances and avoids making building order itself a learned spatial coordinate.

The learning rule retains MATD3's lower twin-critic target, clipped target noise, delayed actor updates, and soft target networks. Only actor $i$ receives gradients when its policy is updated; the other actors' actions are detached. Optional residual control, replay-based BC, a separate RBC demonstration reservoir, local price conditioning, local action projection, multi-step returns, reward normalisation, and selectable critic loss are implemented as named switches. These switches are not folded into the definition of Transformer MATD3: each produces a separate configuration and ablation.

Dynamic asset-instance changes are handled transactionally. The controller validates the complete old and new layout, flushes pending multi-step transitions under the old signature, reuses compatible type projections and heads, and rolls back the whole operation if attachment fails. Replay samples only the current complete signature. A changed building count performs a full rebuild; mixed-signature batches and cross-layout checkpoint restore are not supported. ONNX exports are per building and per topology, with residual, safety, and price adapters declared as external runtime dependencies.

This method is the strongest structured baseline produced before the next chapter. It shows how far variable token counts, shared type projections, signature-aware replay, and a set-based central

critic can go while the catalogue of types and their feature meanings remain fixed. It also makes the residual problem precise: asset count can change within the known schema, but a failed sensor, a new action type, a changed dependency, or a building joining the population still lacks one shared runtime control contract.

### 5.6.6 Evidence Produced at Level III

Level III tests three claims separately. The first is decentralised execution: each actor must reproduce its action from local inputs without the training critic. The second is coordination: community outcomes must improve beyond strict local learning and a strong community RBC under matched information. The third is learning quality: the improvement must survive independent seeds and justify its computational cost.

The principal comparison uses Core-30-Nominal. Core-30-Safety tests simultaneous actions under shared headroom, while Core-30-Health evaluates degraded inputs and commands that do not alter the actor layout. Fixed-vector methods are not credited with dynamic-topology capability. A join, departure, or new charger changes the dimensions of their actor or critic and therefore defines an explicit applicability failure unless the method declares and validates a separate representation strategy.

## 5.7 Prior Knowledge and Data-Efficient Learning

Reinforcement learning is expensive when every useful behaviour has to be rediscovered through interaction. The controller studies already contain several sources of knowledge: transparent rules, optimisation schedules, previously trained policies, and archived trajectories. The question is how to reuse that knowledge without hiding a permanent dependency or preventing the policy from improving beyond its source.

### 5.7.1 Rule-Guided Exploration and Residual Policies

The original EnergAIze exploration strategy mixed a plausible RBC action with a learned deterministic action and decaying noise. A generic form is

$$a_t = (1-\alpha_t)a_t^{\mathrm{RBC}} + \alpha_t \mu_\theta(o_t) + \varepsilon_t, \qquad \alpha_t \in [0,1]. \tag{5.24}$$

Early transitions remain close to a service-aware trajectory; policy authority increases as training progresses. This avoids spending most of the initial replay buffer on disconnected EV actions, impossible deferrable starts, or saturation at action bounds. The schedule of $\alpha_t$, the noise process, and the treatment of mixed actions in replay are part of the method.

Residual control, introduced in Equation (5.17), retains the base throughout operation instead of phasing it out. It is stronger operationally because zero residual has an exact meaning and the controller returns to a known behaviour when the learned correction is unavailable. It is also a

narrower learning claim. A residual policy has not learned the complete service problem when the base still supplies EV and deferrable decisions.

The two mechanisms are evaluated separately. Guided exploration is judged by early feasibility, useful replay coverage, and transitions required to reach a fixed performance threshold. Residual control is judged by improvement over exact neutral parity, required projection, and robustness when the residual is disabled. Neither is compared with an autonomous policy without labelling the difference in runtime authority.

### 5.7.2 Behaviour Cloning and Demonstration Use

Behaviour cloning trains an actor on observation–action pairs $(o, a_E)$ produced by an expert. For continuous actions, the basic loss is

$$\mathscr{L}_{\mathrm{BC}}(\theta) = \frac{1}{|\mathscr{D}_E|} \sum_{(o,a_E)\in\mathscr{D}_E} w(o,a_E)\,\|\pi_\theta(o) - a_E\|_2^2, \tag{5.25}$$

where $w$ can give sufficient representation to rare positive EV, V2G, or deferrable actions. Uniform action loss can otherwise be dominated by the many intervals in which an asset is idle. Registered sources include the executed replay action, a warm-start policy, a separate teacher policy, deterministic RBC Smart demonstrations, and compatible replay-validated oracle schedules. The expert's information boundary must be no richer than the policy's declared training contract unless the experiment is explicitly studying privileged supervision.

One target is not suitable for every action. The loss can weight EV, stationary-storage, deferrable, and other dimensions separately; positive EV and deferrable targets can receive additional sample weight; and long sequences of zero actions can be down-weighted without being discarded. In residual training, the cloning target is clipped to the actor's available residual authority and the unreachable fraction is logged. This avoids asking the residual actor to reproduce a teacher action that its configured gain cannot express.

PPO and off-policy actors use demonstrations differently. PPO keeps expert transitions outside the on-policy rollout and applies actor-only pretraining or an auxiliary loss after the PPO epochs. TD3, MADDPG, MATD3, and MASAC can read labelled cloning targets from replay and can run a bounded actor-only pretraining phase before normal policy updates. The structured implementations also support a separate per-layout demonstration reservoir. Critics, target networks, replay, and reward normalisers are excluded from a behaviour-cloning optimiser step.

Dataset aggregation would address the states visited by an imperfect student by querying the expert during those rollouts (Ross et al. 2011). No standalone DAgger learner is registered in the current repository. The experiments reported here use fixed demonstrations, replay-labelled targets, scheduled teacher phase-out, or auxiliary BC; they are not labelled as DAgger results.

The main comparison reports initial policy quality, environment transitions to a fixed control threshold, improvement beyond the teacher, and failure after the teacher is removed. A policy

whose exported artefact still calls the teacher is classified as a composed controller. It cannot be presented as an autonomous learned policy.

### 5.7.3 Replay as a Reusable Knowledge Source

The replay mechanisms in Section 5.3.4 support more than repeated critic updates. Behaviour actions preserve a deterministic base, cloning actions preserve an independent teacher, event strata keep rare service decisions visible, and layout signatures state where an example remains meaningful. A replay archive can therefore support actor pretraining and controlled transfer between runs that share the same observation and action contract.

The repository also contains a conventional single-agent TD-error prioritised buffer based on Prioritised Experience Replay (Schaul et al. 2016). The multi-agent MADDPG/MATD3 path deliberately rejects that buffer because it does not preserve the joint-transition contract. Its canonical weighted buffer uses the reward, registered observation events, and behaviour-action strata described in Equation (5.10). The two mechanisms are reported under their exact class and sampling rule rather than the generic label "prioritised replay".

### 5.7.4 Offline Pretraining and Policy Distillation

Offline RL studies whether a controller can be trained from a fixed archive without exploratory interaction (Levine et al. 2020). This is attractive when historical operation exists and unsafe trial-and-error is unacceptable. It also creates a strict support problem: a value function can assign unrealistic value to actions that are rare or absent from the dataset.

The implemented offline stage is actor-only behaviour-cloning pretraining. It samples a bounded replay or demonstration archive, updates only the actor-side representation and policy, optionally synchronises the target actor, and records the completed update count in the checkpoint. RBC and controlled-policy trajectories retain entity, action, reward, scenario, and seed provenance. Evaluation uses simulator replay on a disjoint interval and coverage diagnostics over observations, actions, assets, and services.

Conservative Q-Learning and Implicit Q-Learning provide relevant fully offline alternatives (Kostrikov et al. 2022; Kumar et al. 2020). Neither is registered as an executable controller in the current Algorithms repository, so neither appears as a Chapter 5 result row. Their discussion defines the support problem that a future fixed-dataset extension would have to address; it is not evidence that those algorithms were run.

An offline policy can initialise later simulator training, provide a policy when online interaction is restricted, or serve as a teacher for a smaller model. It is not labelled robust merely because it trained without online errors. Robustness depends on whether the evaluation remains inside the support of the dataset and whether faults or configurations absent from the archive are handled safely.

Policy distillation transfers the actions or distributions of a larger or composed teacher to a student (Rusu et al. 2015). In the implemented controller line, this appears as supervised actor training from RBC, oracle, replay, or coordinator targets rather than as a separate registry entry called "Distillation". The relevant result is a performance–resource curve: model size, inference latency, memory, energy use, and retained control quality. A student must also preserve the observation and action semantics of its deployment target. Distillation can compress a policy; it does not repair an incompatible interface.

These knowledge-reuse mechanisms answer part of RQ2 by reducing interactions or runtime cost, part of RQ3 by avoiding unsafe early exploration, and part of RQ4 by producing smaller or more reproducible artefacts. They leave a common boundary. The teacher, dataset, replay record, and student are normally tied to a declared observation and action contract.

## 5.8 Level IV: Hierarchical Community Coordination

Central control gives one policy direct authority over every asset, but its action space and information requirements grow with the community. Independent local control preserves autonomy and scale, yet it can create coincident behaviour. Hierarchical coordination places a smaller community decision above the local controllers. The coordinator shapes the conditions under which local decisions are made; it does not replace asset-level service logic.

### 5.8.1 Decision Decomposition

The implemented hierarchy contains a community coordinator and one local leaf per building. At environment step $t$, the coordinator observes a declared community summary $z_t$ and emits a signal $m_t$. Leaf $i$ receives its original local observation and the authorised signal:

$$m_t = \pi_\psi^{\mathrm{CC}}(z_t), \qquad a_{i,t} = \pi_i^{\mathrm{leaf}}(o_{i,t}; m_{i,t}). \tag{5.26}$$

The local policy continues to enforce EV deadlines, deferrable continuity, storage limits, and building headroom. The coordinator acts every $K$ simulator steps and holds its signal between decisions. With 15-minute data, $K = 4$ gives an hourly community action while local leaves still act every 15 minutes. The interval is an experimental variable: a shorter interval increases authority and data volume but can create oscillatory signals; a longer interval is easier to operate but can miss short flexibility windows.

The signal is an effective price multiplier. $m = 1$ leaves the local price path and controller behaviour unchanged. Values below one encourage selected flexible demand; values above one can discourage import or encourage discharge where the local controller has such authority. The implementation permits the broad range $[0.5, 1.5]$. The annual campaigns use a narrower registered interval, commonly $[0.5, 1.3]$, and confirmatory experiments restrict it further when the causal probes identify regions with no useful response.

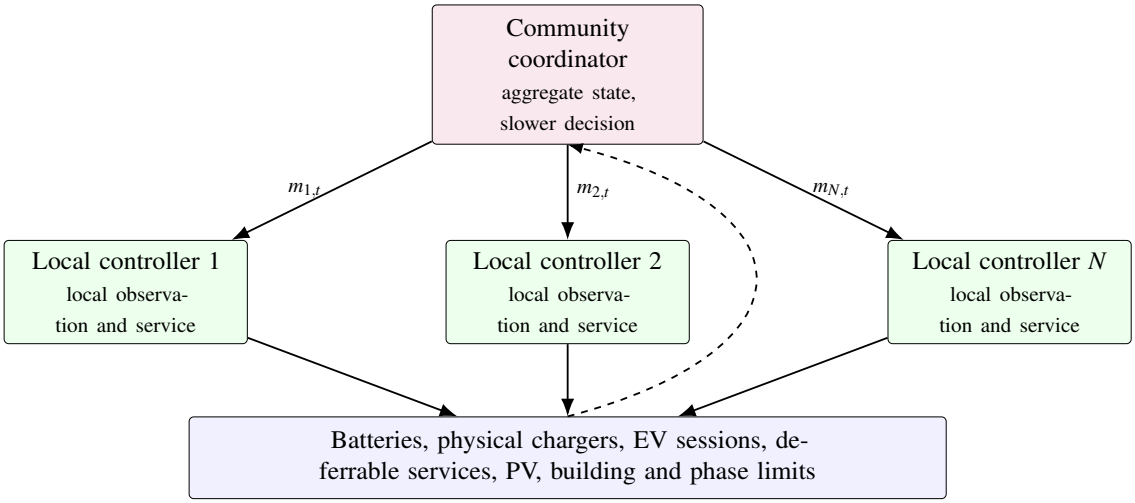



Figure 5.2: Hierarchical community coordination. The coordinator changes a compact signal; local controllers retain asset-level decisions and service protection. The dashed return path carries community aggregates and outcomes.

### 5.8.2 Global and Member-Specific Coordination

Two coordinator levels expose a clear scalability–fairness trade-off.

**Global coordination.** A Level-1 coordinator emits one multiplier for the community. Its input contains time, price, aggregate import/export, PV, flexible capacity, headroom, and recent history. Input and output sizes need not grow with the number of members. The same signal cannot distinguish a flexible office from a household close to an EV deadline; distribution remains the responsibility of the local leaves.

**Member-specific coordination.** A Level-2 coordinator emits one multiplier per building and can receive compact member summaries. It can target flexibility and distribute benefits more deliberately. Its output grows with $N$, membership order becomes part of the contract, and a joining member changes the policy interface. A set-based encoder can reduce order dependence in the input, but fixed per-member outputs still retain a population-size assumption.

The hierarchy is evaluated first over deterministic RBC leaves and then over frozen learned leaves. RBC leaves make the signal response easy to inspect. Learned leaves test composition with a stronger adaptive controller. Jointly training coordinator and leaves is a separate experiment because it changes both policies and complicates credit assignment. A poor joint result does not show that hierarchy itself is ineffective; it can indicate that the local competence was overwritten during community learning.

### 5.8.3 Causal Signal Path and Neutral Parity

A price signal has value only if the leaf reacts to it through a mechanism that was designed or trained for that signal. Applying a multiplier directly to a neural observation can move a frozen actor outside its training distribution. Applying it only to a reward has no effect during inference. Replacing the leaf's base controller at the same time as the coordinator is introduced prevents attribution.

The hierarchy therefore follows four causal requirements:

1. a neutral multiplier reproduces the standalone leaf, including exported trajectories and KPIs;
2. a non-neutral fixed probe changes the intended local action channel before a learned coordinator is trained;
3. the leaf remains frozen and deterministic during the coordinator comparison unless joint learning is the named experimental variable; and
4. the training reward uses the same cost and settlement semantics as the final scorecard, with service and violation terms retained as gates.

The signal-aware local base receives the multiplier while the frozen neural residual retains its original observations. This preserves the distribution on which the actor was trained and gives the coordinator a causal actuator. Fixed multiplier and time-schedule probes determine whether discounts, surcharges, current price, or forecast price actually affect executed actions. A region with no response is not explored repeatedly by PPO.

The PPO coordinator uses aggregate observations and a bounded Gaussian action. Its reward can contain community settled cost, excess import, ramping, export, electrical violations, and EV service. These components are normalised from a matched neutral run. Cost remains visible as the primary outcome; peak, ramping, throughput, emissions, and fairness are reported separately. Reward-weight changes define ablations rather than hidden tuning.

Teacher actions used to pretrain the coordinator are stored outside the on-policy rollout. The final teacher action cannot cross into the first PPO batch with a fabricated log-probability. Deterministic evaluation occupies its own episode and performs no update. These details matter because a coordinator acts relatively few times per year; one contaminated boundary can materially alter an early PPO update.

### 5.8.4 Hierarchical Comparison

The experimental matrix varies one axis at a time: global or member-specific output, RBC or learned leaf, neutral/fixed/learned signal, 15-minute or hourly coordinator interval, aggregate or member-aware input, and cost-first or grid-aware reward. The baseline is always the same leaf with a neutral signal. Fixed causal schedules are diagnostic references and are labelled in-sample when their trigger was derived from the same annual trajectory.

The principal evidence asks whether the coordinator lowers matched cost without losing EV or deferrable service, whether it reduces or merely shifts physical exchange, and how benefits are distributed. A community gain concentrated in a few members is reported even when the aggregate result is positive. Signal variance, action saturation, tracking response, leaf interventions, and inference latency explain how the coordinator reached its outcome.

Hierarchy establishes a useful decomposition when a compact community action improves or protects a community outcome while leaves preserve local competence. It retains two structural assumptions. The coordinator and leaves must share a tested interpretation of the signal, and member-specific outputs normally assume a fixed set and ordering of recipients. These assumptions become part of the open limitations rather than being hidden inside the pipeline.

## 5.9 Cross-Method Synthesis and Open Limitations

The complete campaign is read as a sequence of solved subproblems. A controller advances only after establishing a capability under a stated contract; the largest aggregate reward is insufficient evidence. Table 5.9 fixes that interpretation before the quantitative results are inserted.

The final results block draws from one machine-readable ledger. Each row identifies the dataset manifest, simulator and Algorithms commits, controller configuration, information surface, seeds, training and evaluation intervals, hardware, checkpoint, reference run, and evidence level. The main chapter retains the comparisons that decide a scientific question. Seed tables, learning diagnostics, unsuccessful recipes, solver schedules, and member-level exports remain in Appendix A.

### 5.9.1 Contribution to the Research Questions

RQ1 receives partial but important evidence from the controller progression. Strict local PPO/TD3 and IPPO establish individual decision boundaries. MADDPG, MATD3, MAPPO, HAPPO, and MASAC introduce different forms of coupled training, while community rewards and hierarchy expose the trade-off between participant and collective outcomes. The structured PPO and MATD3 variants admit variable counts of known asset instances. New semantics, health-dependent capabilities, and changing populations remain outside the complete capability established here.

RQ2 is addressed through explicit scaling dimensions. The campaign reports simulator transitions, wall time, update count, CPU/GPU resources, memory, critic and actor size, checkpoint size, communication assumptions, and inference latency. Independent local actors scale differently from concatenated central critics; global coordinators scale differently from member-specific outputs. Premium-100 provides a held-back admission and resource test for methods whose interfaces are compatible. No result is described as scalable without naming the dimension that increased and the resource or control quality that was maintained.

RQ3 is addressed through service-first MILP objectives, deadline-aware RBCs, local action envelopes, requested-versus-applied violation traces, matched Safety and Health scenarios, and failure-aware evaluation. These mechanisms prevent several known unsafe actions and reveal

Table 5.9: Scientific role and remaining boundary of each controller stage.

| Problem | Method stage | Capability tested | Boundary retained |
|---|---|---|---|
| Ordinary service and asset value | BAU variants | Service-respecting denominator and isolated storage effect | No deliberate adaptation or coordination |
| Plausible low-cost dispatch | RBC ladder | Transparent priorities, causal information use, deterministic fallback, and expert actions | Manually maintained rules and known action semantics |
| Auditable constrained scheduling | Local and community MILP | Explicit objective, service constraints, model bounds, and replayable reference schedules | Forecast/model fidelity, solve cost, and predefined variables |
| Adaptive local control | PPO and TD3 | Learning under strict local information, continuous actions, and exported per-building policies | Fixed building interface and no learned community coordination |
| Independent on-policy control | IPPO | Same PPO implementation and budget across several independent members | No central credit assignment |
| Decentralised coordination | MADDPG | Centralised training with local actor execution | Single critic estimates, fixed joint layout, and training sensitivity |
| More stable deterministic MARL | MATD3 | Twin-critic and delayed-update comparison under the same REC contract | Additional compute and the same fixed actor/critic structure |
| Alternative MARL updates | MAPPO, HAPPO, and MASAC | Central value learning, sequential policy updates, and entropy-regularised central-critic learning | On-policy sample cost, update-order effects, or entropy tuning |
| Known-type structural change | AgentTransformerPPO and AgentTransformerMATD3 | Variable counts of known asset instances, semantic tokens, and signature-aware training | Fixed type catalogue and feature meanings; population changes rebuild learned state |
| Use of prior knowledge | Guided, residual, imitation, replay, actor pretraining, and distillation mechanisms | Faster or safer learning and reuse of rules, models, trajectories, and policies | Dependence on teacher/data support and interface compatibility |
| Local–community decomposition | Hierarchical coordination | Compact community signal above service-aware local control | Tested signal semantics and fixed recipient/interface assumptions |

degradation under selected faults. They do not amount to universal formal safety certification. A controller can still depend on an observation that later becomes stale or on a mask constructed for the original asset layout.

RQ4 receives the algorithm-side contract needed by a later lifecycle: deterministic inference, versioned configuration, encoded observation metadata, one artefact per actor, checkpoint lineage, ONNX export, and explicit runtime dependencies. A service-anchored policy is marked as dependent on its teacher or safety adapter. These records make deployment assessment possible; they do not by themselves govern promotion, distributed rollout, monitoring, update, or recovery.

### 5.9.2 Open Limitations

The progression leaves six connected limitations.

**Partly fixed control interfaces.** The flat neural controllers expect the observation and action dimensions present during training. The structured PPO and MATD3 variants can add or remove instances of a known asset type inside existing buildings, provided feature widths and action semantics remain compatible. A new type, changed dependency, changed building population, or member-specific coordinator output can still invalidate learned state. Padding can accommodate a predefined maximum catalogue, but it does not express a genuinely new action meaning.

**Constraint knowledge outside the policy.** RBCs, MILP, safety adapters, and simulator projection carry physical knowledge that a neural policy may not have learned. This is acceptable when the composition is explicit and deployable. It becomes fragile when a bound, phase relation, or service rule changes while the adapter or policy remains stale. Frequent projection can also hide poor learned behaviour.

**Teacher and dataset support.** Imitation and offline learning inherit the coverage and assumptions of their data. An expert generated with perfect future information can overstate operational performance. A dataset without faults, new assets, or rare services cannot support strong claims in those regions. Removing the teacher after training can expose a dependence that was hidden by mixed actions.

**Growth of central training structures.** Concatenated critics grow with the number and heterogeneity of agents. More buildings increase memory, batch cost, and the amount of joint experience required. Parameter sharing reduces actor count only when roles and interfaces are compatible; it does not remove critic or communication growth automatically.

**Causal authority and fairness in hierarchy.** A compact community signal may have a weak or uneven effect on local controllers. A global multiplier can favour members with large batteries and leave inflexible members with no opportunity to benefit. Per-member outputs improve targeting at the cost of a fixed population interface and more difficult credit assignment. Aggregate savings therefore require member-level interpretation.

**Limits of the evidence domain.** The annual suite is a controlled hybrid representation of Portuguese REC conditions, not a sample of every community. One year, one settlement contract, bounded electrical-service models, and selected failure modes cannot establish universal controller behaviour. Full-year in-sample replay, held-out temporal evaluation, scale tests, and field evidence answer different questions and remain labelled accordingly.

These limitations share a practical theme: the controller line now contains specialised methods for local adaptation, multi-agent credit, stochastic exploration, structured entities, prior knowledge, and hierarchy. Their assumptions about entities, information, actions, constraints, and runtime dependencies still do not form one reconfigurable control contract. The next chapter begins with that unresolved control problem.

## 5.10 Chapter Summary

The controller research developed through four levels of complexity. BAU, rule-based control, and MILP established transparent behaviour, explicit constraints, and strong references. PPO and TD3 isolated adaptive control at building level. IPPO, MAPPO, HAPPO, MADDPG, MATD3, and MASAC exposed independent, central-value, deterministic central-critic, sequential, and entropy-regularised multi-agent alternatives. AgentTransformerPPO and AgentTransformerMATD3 added variable-cardinality representations for known asset types. Prior knowledge was reused through guided and residual actions, demonstrations, labelled replay, actor pretraining, and distillation. Hierarchical control separated a slower community signal from local asset and service decisions.

The common protocol fixes information surfaces, dataset roles, training budgets, service and safety gates, statistical reading, and computational evidence before controller results are compared. The shared engineering contract records how observations are encoded, how requested and applied actions differ, why reward profiles changed, how rare transitions and demonstrations enter training, and which runtime dependencies remain outside an exported network. Earlier methods retain roles as baselines, teachers, fallbacks, or bounded solutions. The remaining limitations concern partially fixed interfaces, externally maintained constraint knowledge, dataset support, central-training growth, hierarchical authority, fairness, and the scope of the available evidence.

# Chapter 6

# TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems

Chapter 5 followed the controller research from transparent rules and optimisation to single-agent RL, conventional MARL, entity-structured policies, knowledge reuse, and hierarchical coordination. That progression solved real parts of the energy-control problem, but its learning interfaces remained partly fixed. A controller could learn to schedule the assets it had been given; changing the number of agents, the instances owned by one agent, the meaning of an observation, or the set of safe actions could still invalidate the model or the software around it.

The distinction matters in an operational community. Adding a second charger does not merely append another number to the state. It introduces a physical module, an independently occupied connection, a new EV session, another action group, and new competition for the same total and per-phase electrical limits. A stale EV state of charge does not have the same consequence as a missing price forecast. The first can make V2G unsafe; the second can leave local operation available under poorer economic information. When a member joins or leaves, the population changes without erasing the temporal history of the members that remain. Fixed vectors, masks, and manually rebuilt heads can represent bounded versions of these cases, but they do not by themselves preserve what each value and action means.

TI-MARL was developed to make that changing interface an explicit part of the control problem. The implemented method begins with one technology-neutral typed contract for each registered member. A deterministic compiler binds that contract to the entities, runtime values, topology events, and health evidence supplied by the simulator. A parameter-shared local actor then processes the variable set of observations and active action groups, while an analytic projector applies the local electrical and service constraints derived from the same interface. TI-PPO uses a local typed critic; TI-MAPPO replaces it with a permutation-equivariant central set critic during training. Both execute through decentralised actors.

The running example is a building with two EV chargers, a stationary battery, and a deferrable service. One charger may be V2G-capable while the other is charge-only. Their EV sessions have different arrival states, targets, and departure times, yet both chargers share the building connection and can lie on different phases. This example exposes the four requirements used throughout the thesis. Dynamic instances and stable semantic types address heterogeneous control; shared parameters and variable-cardinality encoders address scalability; health closure and local projection address safety and robustness; versioned interfaces, compatibility signatures, checkpoints, and execution traces establish the algorithm-side basis for deployability.

This chapter explains the method as it exists in the Algorithms repository rather than as a list of possible extensions. It identifies the contribution precisely, develops the compiler–actor–projector–learner path, records the engineering decisions needed to train it, and fixes the experimental protocol. Quantitative control results remain provisional until the confirmation campaign is frozen. Verified software and runtime evidence is reported separately from those pending comparisons.

## 6.1 Scientific Positioning and Problem Formulation

### 6.1.1 Closest Approaches and the Residual Problem

Several neighbouring research lines already solve parts of the problem. Parameterised-action methods combine a discrete action type with continuous parameters (Li, Tang, et al. 2022; Xiong et al. 2018). Variable-action policies can process action candidates whose cardinality, order, or semantic content changes (Jain et al. 2020; Sinii et al. 2024). Modular and cross-embodiment policies reuse one controller across different sensors, actuators, and morphologies (Doshi et al. 2025; Huang et al. 2020; Luketina et al. 2025). Heterogeneous and task-open MARL address different action semantics, changing tasks, and dynamic populations (Anil et al. 2025; Yu, Lin, et al. 2024). Sensor-dropout training improves robustness to missing modalities (Liu et al. 2017), while joint-action shields prevent unsafe combinations of actions (Elsayed-Aly et al. 2021).

Variable action sets, parameterised actions, heterogeneous agents, changing populations, graph encoders, actor–critic optimisation, and safety monitors are established classes of mechanism. TI-MARL contributes the relationship among them. The same runtime contract should identify the active entities, bind observations to their sources, instantiate the action groups and ports that currently exist, propagate health and dependency changes into port validity and bounds, preserve local information boundaries, and remain meaningful to the policy, critic, safety logic, deployment validator, and explanation trace.

This position is especially important for assets that enter and leave. A padded universal vector can reserve ten charger slots and mask the unused positions. Such a representation is often sufficient when the maximum topology and all action semantics are known in advance. It becomes less natural when a newly admitted module brings a coherent observation–action contract, when several instances of the same type must be distinguished, or when a failed observation changes the safe

domain of only the dependent actions. TI-MARL tests whether an explicit entity-bound typed interface provides a useful inductive and operational structure in those cases.

The method builds on the progression established in Chapter 5. Table 6.1 identifies the problem solved by each family and the assumption that TI-MARL relaxes.

Table 6.1: TI-MARL as a cumulative extension of the controller progression in Chapter 5.

| **Stage** | **Problem addressed** | **Assumption that remained** | **TI-MARL extension** |
|---|---|---|---|
| RBC and fallback control | Feasible, transparent, low-cost operation | Rules and device interfaces are manually fixed | Health-aware interface compilation and typed fallback scope |
| MILP and optimisation | Explicit objectives and constraints | Variables and system dimensions are defined before solving | Runtime entity/action instances and typed local constraints |
| PPO and TD3 | Adaptive control without a complete online optimiser | One agent and one fixed observation–action signature | TI-RL as the one-agent typed-interface reduction |
| IPPO, MAPPO, HAPPO, MADDPG, MATD3, and MASAC | Independent and centralised-training multi-agent control | Fixed agent/action layout and fixed-vector central training inputs | Local typed actors with either local critics or a central set critic over active agents |
| Entity-structured PPO and MATD3 | Variable counts of known asset instances and signature-aware learning | Known type catalogue, fixed feature semantics, and building-count rebuild | Runtime typed observations, action ports, dependencies, and health-aware compilation |
| Behaviour cloning and actor pretraining | Reuse of expert and historical data | Demonstrations are tied to a particular interface layout | Schema- and port-referenced trajectories and replay |
| Hierarchical coordination | Separation of local and community objectives | Participants, roles, and signals are predefined | Community signals as authorised typed observations under dynamic membership |

The scientific claim concerns decision problems in which the interface itself is a time-varying structured object. The novelty is not PPO, attention, parameter sharing, health rules, or action projection in isolation. It lies in their executable connection: a versioned per-agent contract is compiled into entity-bound observations and action groups; the shared actor consumes that compiled object; validity, dynamic bounds, health closure, and local feasibility use the same semantics; and

stable identities carry transitions, checkpoints, compatibility decisions, and traces across topology change. The experiments in Sections 6.8 and 6.9 test whether this connection offers value beyond masked, entity-structured, and conventional dynamic-population baselines.

### 6.1.2 Typed-Interface Decision Processes

The one-agent reduction isolates interface reconfiguration from cross-agent learning. A TI-MDP is defined by

$$\mathscr{M}^{\mathrm{TI}} = \left\langle \mathscr{S}, \Sigma, \mathscr{H}, \mathscr{C}, \Gamma^{\mathrm{loc}}, P, P_{\Sigma}, R, \gamma \right\rangle, \tag{6.1}$$

where $\mathscr{S}$ is the physical and operational state space, $\Sigma_t$ is the current versioned capability schema, $\mathscr{H}_t$ is the runtime health state, $\mathscr{C}$ is the Typed Interface Compiler, and $\Gamma_t^{\mathrm{loc}}$ is the local typed relation and constraint graph. $P_{\Sigma}$ represents interface events such as module addition, module removal, sensor degradation, or recovery. Learning in this one-agent model is denoted TI-RL. It supports controlled evaluation of multiple chargers, variable action groups, health-aware contraction, and local bundle feasibility without introducing multi-agent credit assignment.

A TIMG adds an active agent population, local information boundaries, coupled outcomes, and shared resources:

$$\mathscr{G}^{\mathrm{TI}} = \left\langle \mathscr{S}, \mathscr{N}, \Sigma, \mathscr{H}, \mathscr{C}, \Gamma, P, P_{\Sigma}, \{R_i\}_{i \in \mathscr{N}}, \gamma \right\rangle. \tag{6.2}$$

At time $t$, $\mathscr{N}_t \subseteq \mathscr{N}$ denotes the active population. Each agent $i$ has a schema $\Sigma_{i,t}$ and health state $\mathscr{H}_{i,t}$. The physical state is augmented with the active population and interface state:

$$\tilde{s}_t = \left(s_t, \mathscr{N}_t, \{\Sigma_{i,t}, \mathscr{H}_{i,t}\}_{i \in \mathscr{N}_t}, \Gamma_t\right). \tag{6.3}$$

The compiler produces the policy-facing interface of agent $i$:

$$\mathscr{I}_{i,t} = \mathscr{C}\left(\Sigma_{i,t}, \mathscr{H}_{i,t}, \mathrm{view}_i(\tilde{s}_t)\right) = \left(\mathscr{V}_{i,t}, \mathscr{E}_{i,t}, \mathscr{G}^a_{i,t}, \mathscr{P}_{i,t}, \mathscr{B}_{i,t}\right), \tag{6.4}$$

where $\mathscr{V}_{i,t}$ and $\mathscr{E}_{i,t}$ are the typed interface nodes and relations, $\mathscr{G}^a_{i,t}$ is the variable set of active action-group instances, $\mathscr{P}_{i,t}$ is the set of valid action-port instances, and $\mathscr{B}_{i,t}$ contains the current parameter domains and local hard constraints. The view function enforces the information boundary: local actors receive their local interface and explicitly authorised aggregate context, not the private observations of other agents.

The policy produces one decision for each active action-group instance:

$$a_{i,t} = \{a_{i,g,t}\}_{g \in \mathscr{G}^a_{i,t}}, \qquad a_{i,g,t} = (p_{i,g,t}, \eta_{i,g,t}), \tag{6.5}$$

where $p_{i,g,t}$ is a selected valid port and $\eta_{i,g,t}$ is its parameter vector. The local action is a variable structured bundle whose contents follow the active interface. A constraint-aware construction $\Phi_i$ maps raw decoded group decisions into a locally feasible bundle:

$$\bar{a}_{i,t} = \Phi_i\left(a_{i,t}, \Gamma_{i,t}^{\mathrm{loc}}\right), \qquad \bar{a}_{i,t} \in \mathscr{A}_{i,t}^{\mathrm{loc}}. \tag{6.6}$$

The multi-agent joint action is

$$\bar{\mathbf{a}}_t = (\bar{a}_{i,t})_{i \in \mathscr{N}_t}. \tag{6.7}$$

In the implemented v1 path, $\bar{\mathbf{a}}_t$ is encoded into simulator commands and the simulator remains authoritative for the applied action and residual physical violation. The analytic projector acts within each member and therefore enforces declared local total and per-phase headroom; it does not redistribute power between members. A future shared corrector could be expressed as $\mathscr{S}_\Gamma(\bar{\mathbf{a}}_t, \Gamma_t)$, but no result in this chapter relies on one. The physical transition is

$$s_{t+1} \sim P\left(\cdot \mid s_t, \bar{\mathbf{a}}_t\right). \tag{6.8}$$

Local and community outcomes are coupled through this joint transition. A representative reward decomposition is

$$r_{i,t} = r_{i,t}^{\mathrm{loc}} + \alpha_i r_t^{\mathrm{com}} - \beta_i c_{i,t}^{\mathrm{loc}} - \eta_i c_t^{\mathrm{shared}}, \tag{6.9}$$

where local terms capture cost, service, comfort, or degradation; the community term captures peak, ramping, renewable utilisation, or a shared demand-response objective; and the constraint terms distinguish local and shared violations or interventions. When $|\mathscr{N}_t| = 1$ and $\Gamma_t$ contains only local relations, the model reduces to TI-RL.

The formal objects are domain-independent. In energy, a module can be a charger or battery, an entity can be an EV session, a community observation can be transformer loading, and a port can be CHARGE-EV$(p)$. In robotics, the same structure can represent a sensor module, a manipulated object, workspace occupancy, and a parameterised grasp action. The thesis evaluates the method in energy systems while preserving this broader interpretation.

The reduction also clarifies the experimental sequence. TI-PPO tests whether the typed local interface and its reconfiguration can be learned without central training. TI-MAPPO keeps the same actor and execution path but uses the active population during value estimation. Differences between the two can therefore be attributed to centralised training rather than to a different deployed controller.

## 6.2 Implemented TI-MARL Architecture

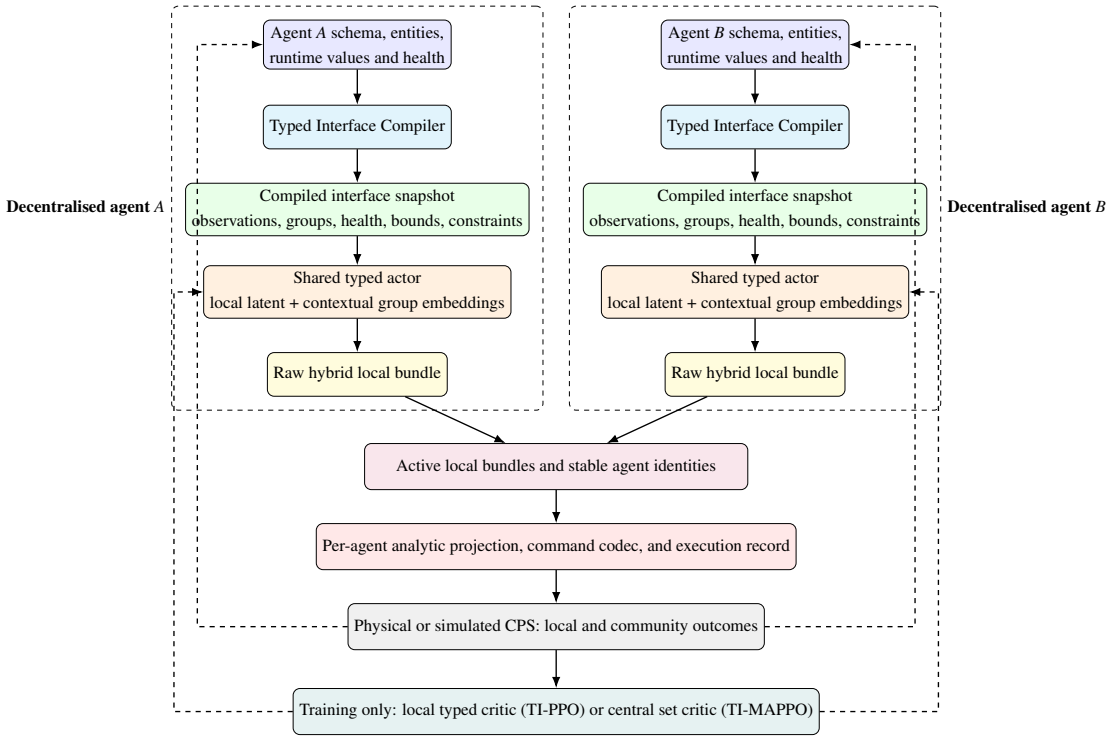



Figure 6.1: Implemented TI-MARL architecture. Each active member is compiled into an entity-bound snapshot and evaluated by the same typed actor. The raw hybrid bundle is projected against that member's service and electrical constraints before command encoding. TI-MAPPO uses the active set of member summaries only during training.

Figure 6.1 separates the runtime path from the training path. Each actor evaluation receives one compiled local interface and authorised aggregate signals. It does not receive the raw private state of another member. TI-MAPPO's critic pools the active agent set during training; TI-PPO uses only the local interface. The actor, compiler, projector, and command codec are otherwise the same.

The method contains five principal components.

The architecture is deliberately asymmetric between training and operation. The actor remains local in both phases. Centralisation is introduced only in TI-MAPPO's value function and the experiment infrastructure, where it can improve credit assignment without creating a runtime dependency on unrestricted global state. Community observations remain typed observation parts with explicit scope, provenance, time, and health.

Table 6.2: Main TI-MARL components and their interpretation in the running energy example.

| Component | Generic role | Energy example |
|---|---|---|
| Typed Interface Compiler | Binds deployment-neutral member contracts to current entities, observations, events, health, groups, bounds, and constraints | Adds a known charger group when it becomes active; scopes failure to the affected group; contracts V2G when its evidence is stale |
| Hierarchical typed encoder | Aggregates observations by channel and sensor before forming a local latent state | Keeps two charger sessions distinct through semantic and structural identities without learning an embedding for a concrete charger identifier |
| Shared hybrid actor | Produces one categorical mode and one bounded continuous fraction for each active group | Controls two chargers, one battery, and one deferrable service with shared group-type heads rather than an asset-specific output head |
| Analytic local projector | Applies validity, dynamic bounds, service floors, and joint total/per-phase headroom | Preserves an urgent EV or mandatory deferrable start while scaling competing charging decisions to the available connection capacity |
| Local or central set critic | Supplies TI-PPO local values or TI-MAPPO values conditioned on the active member set | Retains the same decentralised actor while comparing independent and community-aware value estimation |

## 6.3 Entity-Bound Runtime Typed Interfaces

### 6.3.1 Schemas, Entities, and Dynamic Module Instances

A capability schema describes the potential interface of an agent. It identifies the agent role, module types, observation templates, action-group templates, constraints, compatibility requirements, and fallback semantics. Runtime state identifies which module instances and controlled entities are active now. Separating the two prevents fast-changing values such as SoC or connection state from being treated as schema revisions.

A building can contain several instances of the same module type. In the running example, agent $A$ contains two charger instances, one stationary battery, one deferrable-service module, and one main meter:

$$\mathscr{M}_{A,t} = \{m_1^{\text{meter}}, m_1^{\text{charger}}, m_2^{\text{charger}}, m_1^{\text{battery}}, m_1^{\text{def}}\}. \tag{6.10}$$

Each charger can be related to a currently connected EV session. The observation

```
semantic type:  EV_SOC
entity:  ev_session_17
```

```
source module:  charger_1
```

cannot be confused with the SoC of the vehicle connected to `charger_2`. The corresponding action group is similarly bound:

```
group type:  EV_CHARGER_MODE
owner module:  charger_1
target entity:  ev_session_17
```

The compiler represents these relationships as typed edges such as `owns`, `connected_to`, `observes`, `controls`, and `requires`. Entity binding is encoded in the interface semantics and does not depend on position in a flat vector.

When `charger_3` is installed, a module-addition event creates a new module node, its observation parts, one EV-charger action-group instance, valid port instances, and the relevant constraint edges. The neural network is not extended with a new permanent head. The same group and port decoder used for the existing chargers is applied to the new instance. Removing the charger removes that subgraph. A semantically unknown module is admitted only when a compatible parent type and decoder are available; otherwise its learned control path remains inactive and fallback semantics apply.

The implementation deliberately separates four identifiers that flat action layouts often conflate. A member owns the local controller boundary; a charger identifies installed infrastructure; an EV identifies a vehicle; and a session records the temporary occupancy of one charger by one EV. A domestic charger can retain the same vehicle over many sessions, whereas one office charger can serve several vehicles in one day. The action group belongs to the physical charger, while service observations belong to the currently bound session. Session replacement therefore rebinds runtime values without rebuilding an unchanged charger structure.

### 6.3.2 Typed Observation Parts and Community Context

An observation part is a typed object with an identity and declared semantics. A representative definition is

$$o_k = (\tau_k, \mathrm{id}_k, \mathrm{entity}_k, \mathrm{source}_k, x_k, u_k, q_k, h_k, t_k, s_k), \tag{6.11}$$

where $\tau_k$ is the semantic type, $x_k$ the value, $u_k$ the unit, $q_k$ quality or uncertainty metadata, $h_k$ the health state, $t_k$ the timestamp, and $s_k$ the scope. The scope can be self, local, neighbourhood, community, or global. The source can be a physical sensor, derived signal, forecast, external platform, or coordinator message.

Community information is represented through the same object model. A building may receive numerical aggregates such as community net demand, transformer loading, remaining headroom, photovoltaic surplus, a price multiplier, or a demand-response target. These values do not reveal the raw private state of another building. Their provenance and health remain explicit:

```
semantic type: COMMUNITY_TRANSFORMER_LOADING
scope: community; source: platform_service
unit: percent; health: HEALTHY
```

TI-MARL does not require flexibility to be represented through an explicit negotiation object. In the energy instantiation, an EV connected from 08:00 until 18:00 with a current SoC, target SoC, maximum charging power, and departure requirement already exposes the information from which a policy can learn temporal flexibility. Useful derived quantities such as time to departure, remaining required energy, or required average charging power can be supplied as typed observation parts when they improve learning or interpretability. Their use is an experimental design choice and introduces no separate offer phase or community allocator.

Table 6.3 lists representative observation parts for the running example.

Table 6.3: Representative typed observation parts for a building with two chargers. Values are illustrative.

| **Semantic type** | **Entity/source** | **Value** | **Scope** | **Decision meaning** |
|---|---|---|---|---|
| House net power | main meter | 2.1 kW | self | Current aggregate import used to evaluate local electrical limits |
| EV SoC | session 17 / charger 1 | 0.35 | local | Energy state of the EV controlled through charger 1 |
| EV target SoC | session 17 | 0.80 | local | Minimum service target at departure |
| Departure time | session 17 | 10 h | local | Temporal slack available to the charging policy |
| EV SoC | session 28 / charger 2 | 0.62 | local | Distinct SoC bound to charger 2 and its connected session |
| Battery SoC | battery 1 | 0.70 | local | Available storage state |
| Community headroom | platform service | 5.5 kW | community | Aggregate remaining shared capacity made visible to all authorised agents |
| Community peak state | platform service | high | community | Community context used by local actors and the reward |

### 6.3.3 Dynamic Action Groups and Typed Parameterised Ports

The output interface is organised around action-group instances. Each controllable module creates one or more groups whose ports are mutually exclusive within the group but can be combined with ports from other groups. For example,

$$\mathscr{P}_{\text{charger}_1,t} = \{\text{IDLE-EV}, \text{CHARGE-EV}(p), \text{DISCHARGE-EV}(p)\}, \tag{6.12}$$

$$\mathscr{P}_{\text{battery}_1,t} = \{\text{IDLE-BATTERY}, \text{CHARGE-BATTERY}(p), \text{DISCHARGE-BATTERY}(p)\}, \tag{6.13}$$

$$\mathscr{P}_{\text{def}_1,t} = \{\text{IDLE}, \text{START}\}. \tag{6.14}$$

An action port is a typed callable capability:

$$p = \left(\tau_p, \mathrm{id}_p, \mathrm{group}_p, \mathrm{owner}_p, \Theta_p, U_p, B_{p,t}, V_{p,t}, D_p, K_p\right), \tag{6.15}$$

where $\Theta_p$ is the parameter schema, $U_p$ the units, $B_{p,t}$ the current parameter domain, $V_{p,t}$ the validity state, $D_p$ the observation and capability dependencies, and $K_p$ the resource effects used by local constraints, critics, and optional safety checks.

A new charger creates another instance of the charger action group. It does not create another set of network parameters. The implemented actor emits a categorical mode and a Beta-distributed fraction in $[0,1]$ for every active group. The fraction is relative to the currently compiled physical upper bound and is converted to a signed simulator command only once by the codec. This convention avoids applying a health-dependent or topology-dependent bound twice. A deferrable start is indivisible: after START is selected, its typed cycle profile determines the subsequent demand rather than a continuous power fraction.

Table 6.4: Representative action ports for two charger instances and one stationary battery.

| **Port** | **Owner/target** | **Parameter** | **Current validity and bounds** | **Dependencies and local effects** |
|---|---|---|---|---|
| Charge EV | charger 1 / session 17 | power [kW] | valid, $[0, 7.2]$ | connected session; increases house import |
| Discharge EV | charger 1 / session 17 | power [kW] | valid, $[0, 4.0]$ | V2G, trustworthy SoC, reserve; reduces house import |
| Charge EV | charger 2 / session 28 | power [kW] | valid, $[0, 11.0]$ | distinct connection and EV state; increases house import |
| Discharge EV | charger 2 / session 28 | power [kW] | absent | charger 2 has no V2G capability |
| Discharge battery | battery 1 | power [kW] | valid, $[0, 3.0]$ | battery SoC and reserve; reduces house import |

For example, suppose `charger_1` is rated at 7.4 kW but the current connection and EV state reduce its compiled charging bound to 4.0 kW. A sampled fraction of 0.75 requests 3.0 kW. The network output remains in the same numerical domain when the available bound later changes; the interface gives the output its current physical meaning. The trace stores the selected mode, fraction, compiled bound, requested command, projected command, and applied simulator value separately.

### 6.3.4 Health-Aware Causal Compilation

Health is part of the interface semantics and is also available to the neural network. The implemented states are HEALTHY, DEGRADED, STALE, MISSING, FAILED, and UNKNOWN. Fault evidence

remains a separate record. A simulator event such as `dropout` or `stuck` is not mapped to one universal health label; the compiler considers event duration, freshness, criticality, declared availability, channel semantics, and recovery hysteresis.

A charger failure disables only the associated charger group and its ports. Other modules controlled by the member remain active. A stale EV SoC can preserve charging while blocking V2G because the discharge reserve can no longer be established. Failure of the main house meter has a wider consequence: the complete member is isolated when aggregate import or phase headroom cannot be verified. Loss of a community signal removes that coordination input but leaves the local controller available. During a grid outage, charging, V2G, and a new deferrable start are blocked; safe stationary-storage discharge may remain available when its local dependencies still hold.

The compiler computes the transitive effect of a health event through a dependency closure:

$$\mathscr{H}^{\star}_{i,t} = \mathrm{Closure}\left(\mathscr{H}_{i,t}, D_{i,t}\right), \tag{6.16}$$

which identifies affected observations, ports, groups, bounds, local constraints, and operational modes. For the EV SoC example, the closure can be summarised as

$$\textsc{healthy} \Rightarrow \textsc{discharge-ev} \in [0, P^{\mathrm{v2g}}_{\max,t}], \tag{6.17}$$

$$\textsc{stale} \Rightarrow \textsc{discharge-ev} \notin \mathscr{P}^{\mathrm{valid}}_{i,t}, \tag{6.18}$$

$$\textsc{failed} \Rightarrow \textsc{discharge-ev} \notin \mathscr{P}^{\mathrm{valid}}_{i,t}. \tag{6.19}$$

This behaviour differs from replacing a missing value with zero and asking the policy to infer the safety consequence. The dependency rule changes the decision interface itself. Recovery is explicit: a group or port returns only after its declared fresh-data and hysteresis conditions hold. The closure log records the subject, state, rule, consequence, and duration used in that decision.

The compiler rebuilds structural plans only when topology changes. Values, health evidence, session identity, validity, and bounds are refreshed at every step without treating routine runtime churn as a new neural architecture. This separation is important in the 15-second stress case, where value and session updates occur far more frequently than asset installation or removal.

## 6.4 Local Typed-Interface Policies

### 6.4.1 Local Information Boundaries and Parameter Sharing

Each agent executes a logically local TIP. A building actor receives its compiled interface, optional memory, and authorised aggregate community observations. It does not receive the raw loads, EV states, or private service requirements of other buildings. This boundary is identical during evaluation and deployment; privileged information used by a central critic during training is not silently passed to the actor.

The current implementation shares one actor across all compatible members:

$$\pi_i = \pi_\theta \left( \mathscr{I}_{i,t}, m_{i,t} \right), \tag{6.20}$$

Parameter sharing does not imply shared observations. House $A$ and House $B$ execute the same weights on different local snapshots, and member role remains an encoded feature. The compiler rejects unknown semantic or action-group types rather than silently allocating a random representation. This keeps zero-shot admission limited to new instances and compositions of known contracts.

This structure supports population changes without a permanent actor slot per agent. A new building instantiates the appropriate local policy family after schema compatibility has been checked. Removing a building removes its actor instance from the active population; the shared parameter set and the actors of the remaining agents are unchanged.

### 6.4.2 Hierarchical Typed Encoding and Local Latent State

The compiled local interface is encoded as variable-cardinality typed sets. The hierarchy follows the simulator's measurement structure: observation parts are grouped into channels, channels into sensors, and sensors into the local member representation. Structural relations remain available when an action group asks for its relevant observations. This is narrower than a general graph neural network and matches the implementation used in the experiments.

Each observation token combines its normalised value with quality and validity indicators, health, semantic type, sensor type, channel type, unit, scope, declared use, a deterministic semantic fingerprint, and temporal-horizon features. The first mapping is

$$z_k^o = E_\theta^o \left( x_k, q_k, h_k, e_{\tau_k}, e_{\text{sensor}}, e_{\text{channel}}, e_{u_k}, e_{s_k}, e_{\text{use}}, f_k^{\text{id}}, f_k^{\text{time}} \right). \tag{6.21}$$

Permutation-equivariant relational layers aggregate observation tokens within their channels and sensors, followed by learned pooling over the local sensor set:

$$z_{i,t} = F_\theta \left( \left\{ \left\{ \left\{ z_k^o \right\}_{k \in c} \right\}_{c \in s} \right\}_{s \in i} \right). \tag{6.22}$$

The common latent $z_{i,t}$ summarises the complete local decision context. Concrete asset identifiers are not learned as permanent embeddings. Instance distinction is carried by the compiled structural binding and deterministic observation identity, while semantic families determine reusable parameters. Reordering observations, chargers, or members therefore does not change the intended meaning.

Each active action group starts from its type embedding and the local latent. In the action-conditioned variant, the group also queries observation tokens related to its owner, target entity, and dependencies. The two chargers can use the same mode and fraction heads while attending

to different sessions. Attention across the active group set then exposes local competition before decoding. Adding a charger adds one group evaluation; it does not change the actor or critic parameter count.

During replay and behaviour-cloning updates, immutable type indices and set membership are prepared once and reused across optimisation epochs. Runtime values and health are never cached as static input. Packed evaluation removes repeated per-agent device transfers while preserving the same set semantics. These changes affect throughput, not the mathematical information surface.

### 6.4.3 Dynamic Grouped Action Decoding

The actor does not contain a permanent neural head for each asset instance. A group-type head is applied to every active group of that type. For group $g$ and candidate mode $m$,

$$\ell_{i,g,m,t} = f_\theta^{\text{select}}\left(z_{i,t}, z_{i,g,t}, e_{\tau_g}\right), \tag{6.23}$$

and the categorical mode is selected after invalid modes have been masked:

$$\Pr\left(m_{i,g,t} = m\right) = \text{Softmax}_{m \in \mathscr{M}_{i,g,t}^{\text{valid}}}\left(\ell_{i,g,m,t}\right). \tag{6.24}$$

A second head generates positive Beta parameters. Sampling $u_{i,g,t} \sim \text{Beta}(\alpha_{i,g,t}, \beta_{i,g,t})$ gives the bounded fraction

$$\eta_{i,g,t} = u_{i,g,t}, \qquad (\alpha_{i,g,t}, \beta_{i,g,t}) = d_{\tau_g}(z_{i,g,t}). \tag{6.25}$$

The log probability used by PPO is the sum of the categorical and Beta terms. At training time the two parts are sampled. Deterministic replay can use categorical argmax or an `expected_signed` rule that combines charge and discharge probabilities, the Beta mean, a configurable gain, a deadband, and a non-idle logit margin. These controls were added because an apparently small stochastic preference can become an implausibly persistent deterministic charge or discharge command after export.

All groups are decoded as one bundle from the same local context. A representative raw bundle is

$$a_{A,t}^{\text{raw}} = \left\{ \begin{array}{l} \texttt{charger_1} : \text{CHARGE-EV}(4.0\ \text{kW}), \\ \texttt{charger_2} : \text{CHARGE-EV}(3.5\ \text{kW}), \\ \texttt{battery_1} : \text{DISCHARGE-BATTERY}(1.0\ \text{kW}), \\ \texttt{deferrable_1} : \text{START} \end{array} \right\}. \tag{6.26}$$

The bundle-level representation allows the actor to learn priorities among the two EV sessions, battery use, a pending service, and community conditions. It does not by itself guarantee that the combined power remains within local electrical limits or that an urgent service remains feasible. Those requirements are handled by the final local construction.

### 6.4.4 Constraint-Aware Local Bundle Construction

Local hard constraints should not depend exclusively on a negative reward. An actor can learn to reduce violations, but safety-critical electrical and service limits require explicit semantics. TI-MARL keeps the learned preference in the raw bundle and applies a constraint-aware parameterisation to obtain the final local bundle.

Let $a_{i,t}^{\mathrm{raw}}$ denote the decoded bundle and $\mathscr{F}_{i,t}^{\mathrm{loc}}$ the feasible set induced by the local typed graph. The construction is

$$\bar{a}_{i,t} = \Phi_i\left(a_{i,t}^{\mathrm{raw}}, \mathscr{F}_{i,t}^{\mathrm{loc}}\right), \qquad \bar{a}_{i,t} \in \mathscr{F}_{i,t}^{\mathrm{loc}}. \tag{6.27}$$

The v1 mapping is an analytic joint projector. It first rejects a removed or invalid group, applies a forced typed fallback when one exists, clips the learned fraction to $[0,1]$, and respects the minimum operating power declared by the selected port. It then reserves mandatory deferrable starts and the EV charging power needed to remain within the configured service schedule. Remaining charge and discharge requests are jointly scaled against total and phase-specific import or export headroom. A command that falls below its typed operating floor after scaling returns to IDLE.

For the running example, one local constraint is

$$P_t^{\mathrm{ch1}} + P_t^{\mathrm{ch2}} + P_t^{\mathrm{def}} - P_t^{\mathrm{bess,dis}} \le P_{A,t}^{\max}. \tag{6.28}$$

This total limit is evaluated together with phase incidence. If both chargers lie on L1, unused headroom on L3 cannot make their combination feasible. A three-phase service contributes to all three phase rows according to its declared incidence. The projector can retain a configurable headroom reserve and a completion margin for deferrables. It does not compare one member with another or optimise the community objective.

Two diagnostics distinguish a useful feasibility mechanism from one that hides poor policy learning:

$$I_t^{\mathrm{loc}} = \mathbb{1}\left[a_{i,t}^{\mathrm{raw}} \notin \mathscr{F}_{i,t}^{\mathrm{loc}}\right], \tag{6.29}$$

$$D_t^{\mathrm{loc}} = \left\|\bar{a}_{i,t} - a_{i,t}^{\mathrm{raw}}\right\|. \tag{6.30}$$

Frequent or large adjustments indicate reliance on the projector and weak learning of coherent bundles. Requested, projected, and applied actions remain separate in the transition record, so a physically safe trajectory cannot conceal an infeasible raw policy. The optional shield-aware update removes an intervened group from the PPO policy loss and can distil the final feasible decision into that group; unrelated groups remain eligible for learning.

## 6.5 Multi-Agent Learning and Community Coordination

### 6.5.1 Joint Training and Decentralised Execution

TI-MARL does not require a community controller agent. Each building acts independently from its own compiled interface and authorised community observations. The method becomes multi-agent because the local bundles form one joint action, the physical transition depends on that joint action, local actions compete for shared resources, and rewards include local and community consequences.

At one training step,

$$\bar{\mathbf{a}}_t = (\pi_1(\mathscr{I}_{1,t}), \ldots, \pi_n(\mathscr{I}_{n,t})) \tag{6.31}$$

is applied to the shared environment. The transition produces the next local interfaces, local rewards, and common community outcomes. A low community headroom followed by simultaneous charging from two buildings can increase peak and ramping penalties or create a residual simulator violation. Over repeated interaction, the agents learn how local state and authorised aggregate signals relate to the behaviour of the other agents.

Joint training means that actor updates are derived from the same joint transition. It does not mean that actors share private runtime inputs or that one global actor replaces them. Stable member identities govern the rollout when population changes. A joining member has no predecessor in Generalised Advantage Estimation; a departing member receives a terminal boundary; members that remain can bootstrap across the topology event. The rollout is not cleared and the networks are not resized.

### 6.5.2 Local and Central Set Critics

A conventional central critic often concatenates all agent observations. That representation is tied to a fixed order and population. TI-PPO avoids centralisation and estimates $V_i$ from the local typed snapshot. TI-MAPPO retains the same actor but estimates values from the variable active set.

The implemented central critic encodes each member with the same typed snapshot encoder, mean-pools the active local latents into a community summary, and evaluates each stable member identity from its local and community representations:

$$V_{\phi,i,t} = g_\phi \left( z_{i,t}, \frac{1}{|\mathscr{N}_t|} \sum_{j \in \mathscr{N}_t} z_{j,t} \right). \tag{6.32}$$

The mapping is permutation equivariant: reordering members reorders their values but does not alter their association. Cardinality can change without reconstructing a joint input layer. Mean pooling is intentionally modest. It captures community context and supports the current single-REC experiments, but it does not distinguish several feeder neighbourhoods with identical aggregate summaries. A typed resource-graph critic is a reasonable extension for that case and is not claimed as part of v1.

### 6.5.3 TI-PPO and TI-MAPPO

TI-MARL defines the interface, actor structure, local bundle semantics, and population handling. The current learner supports two controlled PPO variants.

**TI-PPO.** The shared actor and a parameter-shared local critic are updated from local member trajectories. This is the typed one-agent learner applied independently across the active population. It isolates the effect of the interface and actor from centralised value estimation.

**TI-MAPPO.** The actor is paired with the central set critic in Equation (6.32). The critic may use the active joint training context; the actor retains its local information boundary. A checkpoint pins the backbone and critic kind, so a TI-PPO state cannot be loaded into TI-MAPPO by accident.

For a transition generated by old parameters $\theta_{\text{old}}$, the hybrid group probability ratio is

$$r_{i,g,t}(\theta) = \exp\left[\log \pi_\theta(m_{i,g,t}, \eta_{i,g,t} \mid \mathscr{I}_{i,t}) - \log \pi_{\theta_{\text{old}}}(m_{i,g,t}, \eta_{i,g,t} \mid \mathscr{I}_{i,t})\right]. \tag{6.33}$$

With joint-agent credit, the clipped PPO objective uses the sum of group log probabilities for that member. With typed-group credit, the ratio and advantage remain attached to group $g$:

$$L^{\text{group}}(\theta) = \mathbb{E}_{i,g,t}\left[\min\left(r_{i,g,t}(\theta)\hat{A}_{i,g,t}, \text{clip}(r_{i,g,t}(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,g,t}\right)\right]. \tag{6.34}$$

The group critic supplies a matching baseline. EV service and charger-constraint consequences are routed to the responsible charger family; stationary-storage consequences reach the storage group; deferrable completion reaches its start group. Economic settlement and genuinely shared grid effects remain common. This avoids teaching the second charger that it caused a service failure belonging to the first, while preserving community coupling.

TI-MATD3 was considered because Chapter 5 found value in deterministic twin-critic methods. It would require a principled relaxation of the categorical mode and a variable-population twin critic. That implementation is not part of the present contribution and is retained as future comparative work rather than a method evaluated here.

### 6.5.4 Community Observations, Rewards, and Learned Coordination

Community coordination is learned from numerical aggregate observations and coupled outcomes. Typical community-scoped parts include transformer loading, remaining headroom, net community demand, photovoltaic surplus, current peak state, price multipliers, and demand-response targets. They are ordinary typed observations with explicit source, timestamp, health, and scope.

The policy receives no symbolic flexibility offer. An EV's current SoC, target SoC, departure time, charging limits, connection state, and local price or community context are sufficient for the actor to learn whether charging can be delayed or reduced. Derived features such as remaining required energy and time to departure can be included when supported by the experimental protocol. They remain observation parts and introduce no separate negotiation mechanism.

The local and community reward terms expose the trade-off to the learning algorithm. A building can reduce its own cost while contributing to a community peak, or preserve community headroom while missing an EV service target. Reporting keeps participant outcomes, community outcomes, service, constraint interventions, and distributional effects separate. A single scalar return is used for optimisation where required, with the underlying metrics still visible.

### 6.5.5 Training Stability and Time Resolution

Moving the controller to 15-minute data increased the effective learning horizon and exposed numerical failures that were less visible in the earlier hourly experiments. TI-MARL therefore treats the physical duration of a transition as part of the learner contract. When $\gamma_{\text{ref}}$ and $\lambda_{\text{ref}}$ are declared for reference duration $\Delta t_{\text{ref}}$, the factors used at runtime are

$$\gamma_{\Delta t} = \gamma_{\text{ref}}^{\Delta t/\Delta t_{\text{ref}}}, \qquad \lambda_{\Delta t} = \lambda_{\text{ref}}^{\Delta t/\Delta t_{\text{ref}}}. \tag{6.35}$$

The implementation normalises critic targets, uses a Huber value loss by default, clips policy ratios and gradient norms, and records approximate KL divergence, clipping fraction, explained variance, and finite-gradient checks. Advantages can be normalised globally or per stable member identity. Entropy coefficients can differ by action-group family; a charger and a deferrable start do not need the same exploration pressure. A policy anchor can constrain movement from a selected actor and can be reset deliberately when a checkpoint is resumed.

The rollout is on-policy and retains the raw typed decision, projected decision, intervention scope, stable agent and group identities, reward decomposition, termination semantics, current and next snapshot hashes, and simulator execution feedback. Teacher actions and auxiliary-planning examples never enter that PPO rollout. This boundary matters: an update remains attributable to experience generated by the current policy even when demonstrations are used elsewhere in training.

### 6.5.6 Reusing Prior Knowledge without Extending Runtime Authority

The work in Chapter 5 showed that a capable rule-based controller can make early training markedly less erratic. TI-MARL reuses `RBCSmart` through a typed behaviour-cloning warm start. The teacher acts on the physical simulator interface; its requested commands are decoded back into the active typed groups and stored with the exact interface snapshot on which they were valid. Pretraining updates the actor only. The critic is not trained from teacher returns, and the complete demonstration snapshots are discarded after pretraining.

Idle actions dominate many annual trajectories. A plain average cross-entropy loss can therefore learn an impressive cloning accuracy while missing the charge, discharge, V2G, or start events that make the teacher useful. The implementation supports bounded inverse-frequency weighting and a hierarchical mode mean. It can run a final unweighted calibration phase so that balancing rare actions does not permanently distort the deployed prior.

Two optional causal auxiliaries were added after the warm-start experiments exposed sparse service decisions. The EV planner labels typed states using only current service variables and declared forecasts: urgent charging, price-aware delay, authorised V2G with departure reserve, or idle. The storage planner identifies PV capture, low-price charging, high-price discharge, and idle from the same deployment-visible surface. Each auxiliary predicts both mode and fraction, trains the actor only, and keeps a bounded reservoir stratified by decision reason. Rare urgent or arbitrage examples therefore survive without contaminating the PPO rollout.

These planners are training devices, not runtime controllers. They neither issue commands nor override the analytic projector. Pure TI-MAPPO, behaviour-cloning warm start, EV auxiliary, storage auxiliary, and their combinations remain separate ablations. This separation makes the improvement over Chapter 5 testable: prior knowledge is attached to a typed and causal state, while the learned actor still owns execution.

## 6.6 Typed Constraints, Safety, and Runtime Robustness

### 6.6.1 Typed Constraint Relations

The TCG records the physical relations required to interpret a typed action. Its objects represent members, modules, controlled entities, action groups, resources, and constraints. A charger port carries its contribution to building import and the phases it occupies. Health and topology events can remove that contribution or change its domain.

Within a member, these relations supply the analytic projector and the trace. At community level they define what the aggregate observations, reward components, and residual violation KPIs refer to. The current set critic does not perform message passing over the TCG. This distinction prevents the data contract from being mistaken for an implemented graph-learning architecture.

For two buildings sharing one transformer, a hard constraint can be written as

$$\sum_{i \in \mathcal{N}_t} K_i^{\text{import}}(\bar{a}_{i,t}) \leq P^{\max}_{\text{transformer},t}. \tag{6.36}$$

The actors receive aggregate indications such as community loading or headroom and learn coordinated behaviour through the reward and, for TI-MAPPO, the set critic. The TCG gives the KPI and assurance layers an unambiguous definition of the resource being shared.

### 6.6.2 Boundary of Shared-Constraint Enforcement

Decentralised policies and a central training critic do not provide a hard guarantee on a constraint coupled across members. The implemented projector can guarantee only the local feasible set encoded for one member. Shared limits are represented in the simulator, exposed through authorised aggregate observations, penalised in the reward where declared, and measured from applied trajectories. A residual shared violation therefore remains a failed safety outcome, even when every local bundle was feasible.

An external minimum-correction monitor is a deployment option, not part of the evaluated v1 algorithm. Adding it would require an allocation rule for competing members and service obligations; that rule would itself affect fairness and performance. The thesis does not hide this unresolved choice behind a generic “shield”. The final experiments report what the learned joint action achieved before any such future cross-member correction.

### 6.6.3 Runtime Assurance and Degraded Modes

Runtime assurance handles conditions that cannot be delegated to policy quality. These include incompatible schema versions, unknown action types, compiler failure, missing safety-critical sensors, inference timeout, and actuator failure. Responses are scoped according to the dependency relations.

The compiler’s concrete responses are group enablement, forced mode and fraction, dynamic bound contraction, and member isolation. A failed charger disables one group and leaves the others active. A stale SoC blocks the dependent discharge port. A failed main meter can isolate the complete member because its local limit can no longer be verified. A missing community signal retains local operation. Recovery requires the declared health and consistency conditions; one unverified sample is insufficient.

The runtime-assurance decision and the learned action remain separate records. Scientific evaluation and real-world audits can assess the policy without attributing fallback actions to it.

## 6.7 Compatibility and Typed Explainability

### 6.7.1 Interface Snapshots and Admission

Every decision is associated with an immutable interface snapshot containing active entities, observation parts, action groups, ports, validity, bounds, dependencies, health, and schema versions. The snapshot allows an experiment transition to be reconstructed even when the runtime topology subsequently changes.

Before a checkpoint is restored, its compatibility signature is checked against the current contract, schema, type registry, health rules, learning backbone, critic, actor context, and credit-assignment mode. A new instance of a known charger type can be admitted without changing network architecture. A semantically unknown type is rejected. Compiler migration requires explicit opt-in and matching contract hashes; tensor compatibility alone is insufficient.

This contract links the algorithmic method to Deployability. It prevents a model from being resumed or moved merely because tensor dimensions happen to match. The lifecycle mechanisms in Chapter 7 build on the same distinction.

### 6.7.2 Checkpoint, Export, and Actor-Only Handoff

The `ti_marl_checkpoint_v1` format stores the actor, critic, optional typed-group critic, optimisers, rollout, random-number states, compiler state, behaviour-cloning state, resolved configuration, software versions, and compatibility signature. Checkpoint replacement is atomic. Restoring to a different compatible member composition is allowed because stable semantic types, rather than the active population, determine network shape.

Two exports have different purposes. The research artefact retains actor and critic state for evaluation and is marked `deployable: false`. A separate `ti_marl_deployment_bundle_v1` omits the critic and optimisation state and carries the actor, resolved typed interfaces, compiler and health rules, projector configuration, normalisation declaration, versions, and compatibility signature. This actor-only bundle defines the intended handoff to the inference lifecycle, but the current generic serving path does not yet certify it as a production TI-MARL deployment. Chapter 7 preserves that boundary.

### 6.7.3 Execution-Level Decision Traces

Typed interfaces support explanations grounded in the actual decision path. A trace can identify

- the active entities, modules, and schema versions;
- the local and community observations available to the actor;
- observations that were degraded, stale, missing, or failed;
- the action groups and ports that existed at that step;
- invalid ports and the dependency or health rule that invalidated them;
- the port and parameter selected for each active group;
- any local feasibility adjustment and the active local constraint;
- simulator execution feedback, including requested and applied values; and
- the final action executed by each asset.

For example, a trace can state that `charger_1` selected CHARGE-EV while DISCHARGE-EV was unavailable because the bound EV SoC source was stale. It can separately record that `charger_2` was scaled to preserve the L1 headroom and whether the simulator applied the resulting request. These statements follow compiler and constraint semantics; they are not inferred from attention weights.

Transitions are written to buffered gzip-compressed JSONL chunks. Each transition references the current and next content-addressed snapshots. Complete snapshots are deduplicated and emitted

periodically or when topology and health events require them. This keeps the audit trail useful on long high-frequency runs without serialising the complete interface graph at every step.

### 6.7.4 Formal and Operational Properties

The method supports several properties whose validity can be checked independently of control performance.

**Entity-binding integrity.** Every observation and action instance identifies its source, owner, and target entity. The compiler rejects ambiguous or orphaned bindings.

**Interface soundness.** A compiled valid port has an active owner module, satisfied hard dependencies, and a recognised parameter schema. Health-dependent contraction is deterministic under a fixed snapshot.

**Local bundle feasibility.** The final local bundle belongs to the local feasible set when the construction layer terminates normally and the encoded constraints are correct.

**Information-boundary preservation.** The deployed actor consumes only local interface objects and authorised aggregate observations. Privileged critic information is excluded from the actor package.

**Safe admission.** Unknown or incompatible schema elements do not become executable action ports merely because a numerical embedding can be produced.

**Trace consistency.** The explanation record references the same interface snapshot, selected ports, constraint rules, and final action used by execution.

These properties do not prove global optimality, universal robustness, or the correctness of every physical model. They define testable invariants that separate interface and runtime correctness from empirical policy quality.

## 6.8 Experimental Methodology and Baselines

### 6.8.1 Comparison Design and Reference Methods

TI-MARL uses the information boundaries, reward lineage, scorecard, and annual datasets established in Chapters 4 and 5. A stable-interface comparison is required before any reconfiguration advantage is claimed. The subsequent comparisons alter one declared dimension at a time: local interface, population, health, physical pressure, or combined operation.

The executable reference set is intentionally smaller than the Chapter 5 inventory. BAU and `RBCSmart` anchor cost, service, and feasibility. The selected fixed-layout PPO/MAPPO

or MATD3 configuration represents conventional learning where its interface is compatible. `AgentTransformerPPO` and `AgentTransformerMATD3` test variable counts of known entity types. TI-PPO and TI-MAPPO isolate local typed learning from centralised value estimation. The MILP oracle remains a bound on bounded scheduling windows, not a direct dynamic-interface baseline.

Table 6.5: Structural baselines required to isolate the TI-MARL contribution.

| **Baseline** | **Question tested** |
|---|---|
| BAU and `RBCSmart` | Does learning improve cost and community outcomes without losing the service and safety behaviour of an inspectable controller? |
| Selected fixed-layout RL/MARL | Is nominal performance attributable to the learning recipe rather than the typed interface? |
| AgentTransformer variants | Does compilation add value beyond variable counts of already known entity types and a structured encoder? |
| TI-PPO / TI-MAPPO | What changes when the actor and runtime path remain fixed but the critic gains the active community set? |
| No projector / reward-only constraints | How much feasibility comes from learned behaviour and how much from explicit local construction? |
| Local context / action-conditioned context | Do structural relations between a group and its observations improve control of repeated asset instances? |
| Joint-agent / typed-group credit | Does routing service and constraint consequences to responsible groups improve learning without removing shared economic credit? |
| Pure PPO / BC / causal auxiliaries | Which gains arise from typed representation and which require prior knowledge? |

### 6.8.2 Reconfiguration Scenarios, Ablations, and Metrics

The scenarios are organised from controlled changes to integrated operation.

**Development and stable control.** Core-15-Stripped provides short debugging and ablation runs with a static, lightly constrained composition. Core-30-Nominal supplies the main stable-interface comparison. The same physical members and 15-minute year are retained when the experimental dimension changes.

**Safety, health, and topology.** Core-30-Safety increases total and per-phase pressure. Core-30-Health introduces sensor, forecast, communication, actuator, and asset evidence. Core-30-Dynamic changes members and assets, including chargers independently of their host member. Core-30-Combined overlays these dimensions and demand-response events after they have been interpreted separately.

**Integrated confirmation.** Premium-100-Clean retains the heterogeneous 70-residential, 20-small-service, and 10-larger-member catalogue, dynamic topology, several chargers per equipped

service building, limits, services, and settlement without injected faults. Premium-100-AllIn adds coincident failure events. The AllIn run is reserved for final confirmation; it is not used to select hyperparameters.

**Held-out semantics.** Held-out tests use new instance counts or compositions of known types. An unknown semantic or action-group type is expected to fail admission. Calling that rejection "zero-shot generalisation" would weaken the claim rather than strengthen it.

Ablations alter one implemented mechanism: semantic metadata, entity binding, health closure, dynamic bounds, action-conditioned group context, analytic projection, community observations, community reward, central set critic, typed-group credit, behaviour cloning, EV planning, storage planning, deterministic decoding, or compatibility validation. The evaluation reports five evidence groups:

1. control and service performance;
2. reconfiguration and held-out-composition behaviour;
3. Safety and Robustness, including local adjustments and assurance interventions;
4. Scalability and runtime cost across agents, observations, groups, ports, and topology events; and
5. Deployability and explainability, including admission, compatibility, and trace completeness.

### 6.8.3 Temporal Split, Seeds, and Selection Gates

Each annual dataset contains 35,040 action steps. Four 1,344-step development windows cover different seasons: 4032–5375, 12792–14135, 21552–22895, and 30312–31655. Four disjoint windows, 6720–8063, 15480–16823, 24240–25583, and 33000–34343, are retained for confirmation. The remaining 24,288 steps are available for training and tuning. A complete replay on the same year after training is reported as a transductive annual benchmark, not as held-out evidence.

Neural development uses seeds 123, 456, and 789. The confirmation set uses 1009, 2027, 3037, 4051, and 5059, paired with simulator seeds 2601–2604 where the scenario requires them. Checkpoint choice sees development results only. The selected policy must satisfy all of the following before cost is considered:

- total applied electrical-violation energy no greater than 0.5 kWh;
- at least 99% of feasible EV departures meeting minimum acceptable service;
- no decrease in feasible EV target-tolerance rate or deferrable service relative to the paired reference;
- no more than 5% deterioration in daily peak ratio or ramping ratio;

- no more than two percentage points loss in solar self-consumption.

Among policies that pass, promotion requires at least 0.1% cost improvement. Daily peak, ramping, and solar self-consumption resolve ties in that order. The final scorecard also reports all-time peak, load factor, battery throughput, V2G export, community import/export/net exchange, members that improve, training time, and deterministic inference latency. Failing a hard gate cannot be offset by a lower scalar cost.

## 6.9 Results and Comparative Analysis

The evidence is divided into two levels. Software and runtime tests establish that the claimed interface, topology, feasibility, persistence, and trace mechanisms execute as described. Control superiority requires the frozen multi-seed campaign in Section 6.8.3. Passing the first level does not substitute for the second.

### 6.9.1 Implemented and Verified Behaviour

The focused v1 regression suite was rerun against Algorithms commit `58fb6370`. All 115 tests passed. The suite covers compiler contracts, policy runtime, checkpoint and trace contracts, the public typed-interface path, one dynamic real-simulator vertical slice, and the 15-second topology stress case. Table 6.6 states what this evidence supports.

This evidence establishes implementation correctness under the exercised cases. It does not show that TI-MAPPO is cheaper than the best Chapter 5 controller, that it generalises to every compatible composition, or that the 500 ms test ceiling is a suitable deployment latency. Those are empirical questions.

### 6.9.2 Multiple Assets, Reconfiguration, and Held-Out Compositions

This analysis measures whether a policy trained over typed entity and group instances remains competent when the number of chargers changes, EV sessions move between chargers, modules are added or removed, buildings join or leave, and known module types appear in held-out combinations. Results report both control quality and interface correctness: entity-binding errors, invalid actions, admission decisions, retraining requirements, and recovery after topology changes.

The key comparison is against the fixed-layout selected controller and the implemented Agent-Transformer variant. A positive result requires more than accepting a different tensor length. It must preserve the relation between the new entity, its observations, action group, health, bounds, and affected constraints. Unknown types must be rejected, not assigned an arbitrary embedding.

Table 6.6: Verified TI-MARL implementation evidence in the audited repository state.

| Capability | Observed test evidence | Claim supported |
|---|---|---|
| Typed compilation and actor semantics | Determinism, order invariance, semantic distinction, unknown-type rejection, dynamic bounds, and multi-instance groups pass | The runtime interface has stable meaning beyond tensor position |
| Local feasibility and service | Total/per-phase projection, EV service floor, V2G reserve, typed minimum power, and deferrable must-start cases pass | Final local bundles satisfy the encoded local contract while retaining raw/final evidence |
| TI-PPO and TI-MAPPO | Both local and central-set critic paths train in the real-simulator vertical slice | The two declared backbones share one dynamic actor/runtime path |
| Dynamic population | Join/leave GAE rules, checkpoint restoration across compatible compositions, and constant parameter count pass | Known members and assets can change without resizing the networks or discarding survivor history |
| Health and trace closure | Sensor, communication, actuator, and asset events propagate to health, validity, closure logs, commands, and execution records | Degradation scope and requested/applied action lineage are machine-auditable |
| High-frequency topology | Six shifted asset events produce seven structural versions in a 15-second fixture; compiler p95 is required to remain below 500 ms | Runtime values and sessions do not trigger unnecessary structural rebuilds |

### 6.9.3 Health-Aware Robustness and Local Safety

Health experiments compare typed dependency-based recompilation with value replacement and mask-only handling where those baselines are executable. They measure service preservation, affected scope, detection duration, group disablement, bound contraction, unnecessary shutdown, fallback use, violation energy, and recovery hysteresis.

Local safety results separate policy quality from the constraint-aware bundle construction. The analysis reports raw-bundle feasibility, final-bundle feasibility, adjustment frequency and magnitude, and the effect of removing local constraint semantics. Main-meter failure, charger failure, and stale SoC provide distinct failure scopes and should not collapse into one binary robustness score.

### 6.9.4 Multi-Agent Coordination and Shared Constraints

Community coordination is evaluated through aggregate observations, local and community reward components, and critic structure. The comparison includes TI-PPO, TI-MAPPO, the selected Chapter 5 MARL baseline, and the no-community-observation/reward ablations. It reports cost, participant benefit coverage, community peak and ramping, import/export, solar use, demand-response delivery, and residual shared-constraint pressure.

A credible TI-MARL result should show that central training or community information changes the applied joint trajectory, not merely the critic loss. Since v1 contains no cross-member corrector, residual shared violations are attributed directly to the learned joint behaviour and simulator enforcement.

### 6.9.5 Scalability and Runtime Cost

Scale experiments vary the number of members, observations, chargers and EV sessions, action groups, ports, and topology events. Actor latency, compiler latency, local-projector latency, critic latency, memory, training throughput, checkpoint size, and trace volume are measured alongside control performance.

The results identify which dimension increased and which remained fixed. Shared heads keep model parameters constant when another known charger is added, but token processing and inference cost still grow. The set critic avoids rebuilding a fixed joint vector, while its pooling and encoder cost still grows with the active population.

### 6.9.6 Ablations, Explainability, and Boundary Conditions

The ablations determine whether typed semantics, entity binding, health closure, action-conditioned group decoding, local feasibility, the set critic, group credit, prior knowledge, and compatibility checks materially affect the claimed capability. Trace evidence is audited for completeness and fidelity: every record must match the snapshot, selected modes and fractions, active constraints, adjustments, commands, and simulator execution feedback.

Boundary results are reported explicitly. If the AgentTransformer baseline matches TI-MARL under the tested reconfiguration, the typed interface remains useful for software contracts and traces but the learning claim narrows. If the method requires retraining for every new combination of known modules, the compositional claim is weakened. If the projector intervenes frequently, safety is being supplied by the explicit layer rather than learned bundle coherence. If cost improves by missing service or shifting harm to a minority of members, the policy fails the protocol.

## 6.10 Discussion, Generality, and Contribution to the Research Questions

TI-MARL extends the methods of Chapter 5 by changing the object over which the controller operates. The actor no longer assumes one fixed list of assets and action coordinates; it acts over an entity-bound interface compiled from the modules, sessions, health evidence, and bounds that exist now. TI-MAPPO's critic no longer requires a permanent concatenated population; it evaluates the active member set. The same semantics support local constraints, checkpoint compatibility, admission, and decision traces.

RQ1 is addressed by stable member, charger, EV, and session identities; variable observation and action-group sets; known-type admission; topology-aware transitions; and a shared actor that preserves local information boundaries. The member can gain or lose a charger without inventing a new output coordinate, while the policy remains responsible for the active local bundle.

RQ2 is addressed conditionally. Shared type heads and set encoders keep the parameter count independent of the number of known instances. Packed evaluation and cached structural preparation reduce repeated work. The central critic accepts a variable population. None of these makes computation constant: tokens, groups, compiler work, trace volume, and sample requirements still grow and are measured explicitly.

RQ3 is addressed by causal health derivation, dependency closure, forced typed modes, dynamic bounds, EV and deferrable service floors, total and per-phase projection, and requested–projected–applied evidence. The guarantee is local and conditional on the encoded physical model. Shared community safety remains an empirical outcome because v1 has no cross-member hard corrector.

RQ4 receives the algorithm-side contract: resolved interfaces, content-addressed snapshots, compatibility hashes, topology-aware checkpoints, actor-only handoff, and execution traces. These records make governance possible; they do not implement promotion, distributed rollout, monitoring, or rollback, which are developed in Chapter 7.

Typed explainability is a cross-cutting outcome. Mode identities, entity bindings, health rules, bounds, local adjustments, and simulator execution feedback are part of the executable contract. They can be reported directly, without translating anonymous output indices after the decision. This does not explain the internal causal reasoning of the network; it provides a faithful account of what the policy could observe and invoke, what the compiler excluded, and how the final command was obtained.

The formal model can represent modular robots, industrial systems, edge services, and other CPSs with changing capabilities and local/shared resources. The evidence in this thesis remains energy-specific. Generality applies to the abstraction and implementation pattern. Performance in another domain requires new experiments, constraints, and safety evidence.

The principal limitations are concrete. Known semantic and action-group families remain necessary. Mean-pooled community context cannot represent every feeder or phase topology. Aggregate signals can be delayed or insufficient. The projector is only as correct as its electrical

and service contract, and repeated intervention can hide weak policy learning unless raw actions are reported. Auxiliary planners encode useful causal heuristics but can bias the policy towards their coverage. The current actor-only handoff is not yet a certified production serving path. Most importantly, final control and generalisation claims still depend on the multi-seed confirmation campaign.

## 6.11 Chapter Summary

TI-MARL treats the current observation–action interface of each agent as a versioned, entity-bound runtime object. The Typed Interface Compiler derives observation parts, dynamic action-group instances, parameterised ports, health-dependent validity, bounds, dependencies, and local constraint relations from the active modules and entities. A local policy encodes the variable interface into one shared local representation and jointly decodes an action bundle across the controllable groups that currently exist. New instances reuse the same type-conditioned decoders instead of requiring permanent asset-specific heads.

The method is multi-agent through the joint physical transition, coupled local and community outcomes, shared resources, dynamic population, and TI-MAPPO's central set critic. Execution remains decentralised: each actor receives its local interface and authorised aggregate community numbers. The analytic projector applies validity, service, and local electrical constraints. Health events change the interface at the appropriate scope, from one port to the complete member.

This structure links the controller progression to the four requirements of the thesis. Dynamic heterogeneity becomes an interface and transition problem; scalability is tied to parameter sharing and measured variable-cardinality cost; safety and robustness acquire explicit health and feasibility semantics; deployability begins with compatibility, persistence, and trace contracts. The following chapter moves from that algorithm-side boundary to the lifecycle required to reproduce, validate, package, deploy, monitor, update, and recover policies and policy sets.

# Chapter 7

# EnergAIze Policy Lifecycle: Experiment Orchestration and Distributed Inference

The preceding chapters establish how controllers are represented, trained, and evaluated. None of those steps alone produces a dependable control service. An experimental result becomes usable only when its origin can be reconstructed, its policy artefacts remain associated with the observations and actions they expect, and an approved runtime can be installed, inspected, changed, and recovered on the intended infrastructure.

This problem is particularly visible in MARL. The deployable object can contain several policies or one shared policy associated with several agent roles. Loading the model files successfully does not establish that their preprocessing, interfaces, safety dependencies, communication assumptions, and versions remain mutually compatible. A partial update can succeed technically and still create a policy combination that was never evaluated. Governance must cover the policy set and its execution contract, including the checkpoint bytes.

EnergAIze addresses the problem through a connected implementation spanning experiment configuration, the Job Orchestrator, a common Worker Agent, heterogeneous local and remote compute, shared evidence storage, policy export, a deployment catalogue, the energAIze inference service, and an operator-facing web interface. The same pipeline has served Docker workers on a server and personal computer, Slurm and Singularity execution on Deucalion, remote GPU execution through Union INESC TEC, and an NVIDIA Jetson worker with a target-specific runtime image. The output of training is packaged in a bundle that the inference service can load without recreating the model's observation ordering, encoders, action names, or bounds manually.

CityLearn v3 remains part of this lifecycle. Its environment, dataset, scenario, event, interface, reward, constraint, baseline, and evaluation versions belong to the experiment record. The simulator supports controlled regression before a controller is selected and replay after field observations expose a new condition. It does not deploy policies itself; it supplies the reproducible world against which policy behaviour and compatibility are assessed.

## 7.1 Deployment Problem and Lifecycle Contract

Generic MLOps already provides experiment tracking, artefact lineage, registries, packaging, serving, deployment, and monitoring (Kreuzberger et al. 2023; Sculley et al. 2015; Zaharia et al. 2018). RLOps adds the environment, interaction data, policy evaluation, and safety context of reinforcement learning (Li, Thomas, et al. 2022). Distributed RL frameworks provide workers, learners, replay, and multi-policy mappings (Hoffman et al. 2020; Hu et al. 2023; Liang et al. 2018; Zhou et al. 2023). EnergAIze uses these foundations but must preserve additional energy-control and MARL semantics across independently evolving repositories and heterogeneous execution targets.

**Experiment contract.** A reproducible EnergAIze job binds the source revision and container image to a resolved configuration, simulator and dataset versions, scenario and event definitions, observation and action interfaces, reward and constraint settings, algorithm parameters, random seeds, training and evaluation intervals, and requested compute profile. The resulting record includes logs, progress, KPIs, checkpoints, exported policies, runtime information, and a terminal status. A correction produces a derived run and leaves the provenance of the completed experiment unchanged.

**Policy bundle and policy set.** Serialisation stores model weights. A deployable bundle also preserves the topology, ordered observations, encoders, named actions, bounds, artefact paths, algorithm format, and optional production aliases required to reconstruct inference. For a multi-agent controller, the manifest identifies every per-agent artefact or the mapping of a shared policy to its agents. This structure provides the minimum packaging unit for a policy set even where promotion and compatibility decisions remain operator-controlled.

**Deployment and operation.** A deployment record connects a bundle to an inference target and records which manifest the target reports as active. Runtime health, structured logs, request identifiers, provider responses, and control KPIs then associate observed behaviour with that deployment. A previous bundle can remain available for an operator-controlled reversal. Coordinated activation across several independent targets requires stronger readiness and compatibility semantics than the current per-target switch. It remains a lifecycle boundary and carries no implicit guarantee.

Figure 7.1 summarises the provenance chain. The entities are connected by immutable identities and stored artefacts, while evaluation and approval decisions determine whether a completed job is eligible for deployment.

## 7.2 End-to-End EnergAIze Lifecycle Architecture

The implemented architecture contains two connected control paths. The experiment path validates a request, assigns it to a compatible worker, executes Algorithms against a versioned simulator and

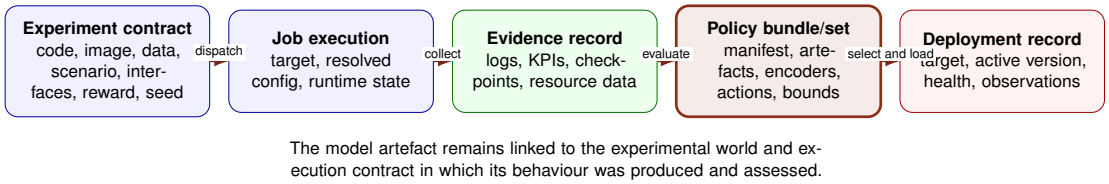



Figure 7.1: Lifecycle entities and provenance relations connecting an experiment specification to the policy bundle and its observed deployment.

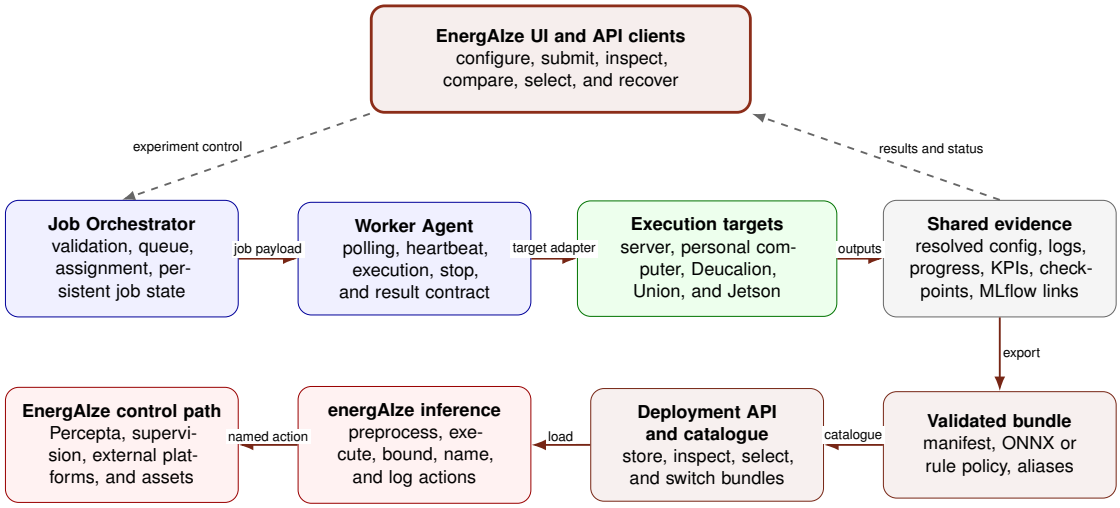



Figure 7.2: Implemented EnergAIze lifecycle architecture. Experiment orchestration and heterogeneous execution produce traceable evidence and a policy bundle; deployment services catalogue and load the bundle into an observable inference target connected to the wider EnergAIze control path.

dataset, and returns evidence and policy artefacts. The deployment path catalogues a self-contained bundle, loads it into a selected inference service, and exposes health and runtime logs. Shared storage, explicit manifests, and the web interface connect both paths without making the UI, worker, or inference service responsible for all lifecycle decisions.

The services have distinct responsibilities. The Job Orchestrator owns training and evaluation jobs, worker state, queueing, and result collection. The deployment backend owns the bundle catalogue and communication with configured inference targets. The inference service owns the active policy pipeline and request-time execution. The UI provides a human-facing control and observation surface over these services, without acting as scheduler or policy runtime.

Figure 7.2 presents the concrete architecture. Technology-specific adapters remain behind the Worker Agent and inference boundaries, allowing the same scientific configuration and bundle contract to cross environments with different execution mechanisms.

The architecture contributes to experimental scalability by allowing independent workers to execute jobs concurrently, but worker count is not evidence of controller scalability. It contributes to deployability by preserving common experiment and inference contracts across target-specific adapters. Safety and robustness remain present as validation requirements, action bounds, external

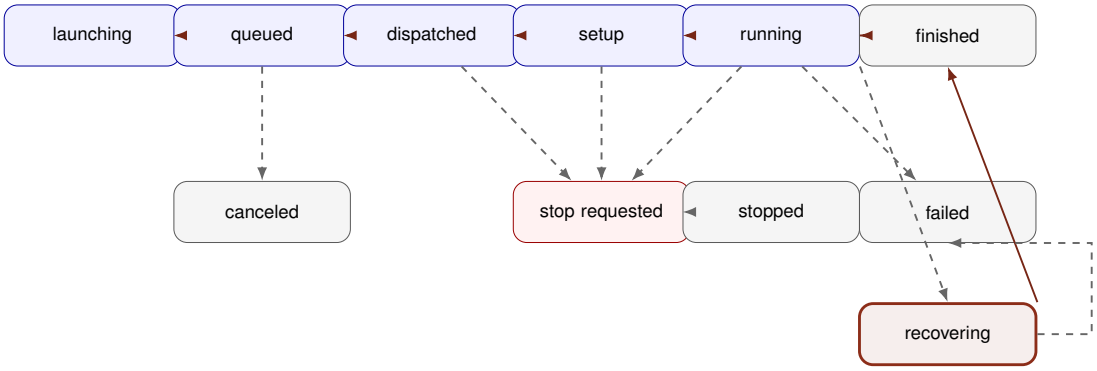



Figure 7.3: Implemented job states. Recovery after successful remote Union computation is represented separately from rerunning the experiment, while cancellation, cooperative stop, and failure remain explicit terminal outcomes.

supervision, health information, explicit failure states, and recovery behaviour. These connections allow systems evidence to be reported without conflating it with algorithmic performance.

## 7.3 Experiment Orchestration on Heterogeneous Compute

### 7.3.1 Job Orchestrator and Persistent Job State

A job begins with a user or API client selecting an experiment configuration, dataset, execution image, and either a specific host or a CPU/GPU worker profile. The Job Orchestrator validates the request, writes a persistent job record, resolves container-visible paths, and places a compact scheduling record in the queue. The dispatched payload contains the image, command, environment, volumes, and resolved configuration path required by the worker; workers do not reinterpret the scientific configuration independently.

The shared storage root is the source of truth for configuration snapshots, job metadata, state, logs, progress, results, and installed artefacts. A completed job directory contains the resolved configuration that was actually executed together with the user's original request. Personal folders and archive metadata organise jobs without moving or rewriting their evidence. The UI can reorganise this metadata while workers and recovery mechanisms retain stable evidence paths.

The implemented job state machine is shown in Figure 7.3. Setup is explicit for remote environments whose container image, input archive, or scheduler request must be prepared before Algorithms starts. Union adds a non-terminal recovering state after successful remote computation because result installation can fail or be delayed independently of training.

### 7.3.2 Common Worker Agent Contract

Every worker advertises an identity, execution mode, capacity, software version, heartbeat time, and relevant resource capabilities. It requests a compatible job, executes the complete payload supplied by the orchestrator, streams or synchronises logs, publishes progress and state, and installs

final results under the shared job identity. The shared protocol allows the orchestrator and workers to evolve independently and supports rolling upgrades through capability negotiation.

Attempt fencing prevents a late status from an old execution from changing a requeued job. A capable worker receives an opaque token for one dispatch attempt and returns it with each status update. Requeueing invalidates that token; a stale worker that receives a rejection terminates its superseded Docker container, Slurm job, or Union run. Cooperative stop uses the opposite direction of the same contract: the orchestrator records the stop request, while the worker terminates the execution mechanism it owns and reports the final state.

The common contract hides execution mechanics, not their resource consequences. Container startup, SIF preparation, scheduler wait, input transfer, remote authentication, result download, and edge image compatibility are recorded and evaluated by target class. A run executed on Deucalion or Union must remain scientifically traceable to the same resolved experiment contract while retaining its distinct infrastructure evidence.

### 7.3.3 Execution Target Profiles

Table 7.1 summarises the target adapters exercised by the implementation. The table emphasises why one generic remote-worker description would be misleading.

Table 7.1: Heterogeneous EnergAIze worker targets and target-specific execution requirements.

| Target | Execution mechanism | Distinct requirements | Lifecycle role |
|---|---|---|---|
| Institutional server | Docker worker with shared storage | Persistent service, Docker image, NFS-visible job data, CPU/GPU capability advertisement | Baseline always-available orchestration and container execution |
| Personal computer | Docker worker started on demand | VPN and NFS preparation, optional local GPU, controlled shutdown without abandoning an active job | Adds opportunistic compute while preserving the central job record |
| Deucalion | SSH, Slurm, and Singularity/Apptainer adapter | Explicit target selection, partition and wall-time validation, OCI-hosted SIF cache, remote dataset staging, scheduler and SSH monitoring | HPC execution under a batch scheduler and project storage model |
| Union INESC TEC | Union task with runner and Algorithms sidecar | Remote GPU allocation, object-store transfer, device-flow or API-key authentication, durable remote-run and result-recovery state | Concurrent remote GPU training and recoverable artefact return |
| Jetson Xavier | Docker worker with NVIDIA L4T image | ARM64 and JetPack-compatible image, NVIDIA runtime, limited local storage, VPN/NFS availability | Edge-class execution and target-specific resource compatibility |

Local Docker workers execute the payload directly against the mounted shared directory. The personal-computer helper coordinates VPN access, NFS mounting, worker startup, graceful

completion, and unmounting. This enables the workstation to contribute a GPU without making its intermittent availability invisible to the orchestrator.

The Deucalion adapter translates the same job into a remote workspace, Slurm request, and Singularity/Apptainer invocation. Versioned SIF images are obtained from an OCI repository and cached by tag. Configuration and selected datasets are staged under a job-specific path, logs are synchronised during execution, and final results return to the shared EnergAIze directory. Partition wall-time constraints are validated before submission, preventing an avoidable scheduler rejection.

The Union adapter packages the resolved input and launches a remote task whose runner coordinates an Algorithms sidecar. Temporary object storage carries inputs and results without requiring every Union container to mount the EnergAIze NFS. A successful remote computation enters recovering until its archive is downloaded, checked, installed, and acknowledged. Persisted worker state makes these steps idempotent across bridge restarts and avoids repeating expensive training when only result transfer failed.

The Jetson path exposes a different form of heterogeneity. The job image must match ARM64, the installed JetPack/L4T software, CUDA, and the NVIDIA container runtime. This target shows that deployment and experimental scalability depend on image and hardware compatibility in addition to CPU count.

## 7.4 Experiment Evidence and Operator Supervision

The EnergAIze UI is the operator-facing surface of the lifecycle. It exposes experiment configurations and datasets, queued and completed jobs, worker health and capacity, execution progress, logs, results, and KPI comparisons. It also links jobs to their MLflow records where tracking is enabled. These views provide one place to inspect the experiment without transferring scheduling or execution authority into the browser.

Job ownership, personal folders, and archive operations separate scientific organisation from physical storage. Moving a job in the UI changes its organisational metadata and leaves the evidence directory in place. Active jobs cannot be deleted; they must first reach a terminal state through completion, cancellation, or cooperative stop. This protects the relationship between an in-flight worker and the job record it is updating.

The UI also reflects target-specific states that a generic job list would hide. Deucalion partition limits inform the submission controls. Union authentication requests expose only the browser verification URL and code, while credentials remain in the worker's persistent authentication store. A result-recovery action can resume an interrupted Union artefact transfer when remote computation has already succeeded; it cannot silently resubmit the training run.

Logs, progress files, result payloads, checkpoints, simulator exports, system metrics, and MLflow are complementary forms of evidence. The UI makes them accessible, but visual availability is not validation. A final scientific result still requires a frozen configuration, comparable baseline, seed protocol, complete output, and an archived analysis that states its evidence level.

## 7.5 Policy Packaging and Distributed Inference

### 7.5.1 Validated Bundle Contract

Algorithms writes a self-contained inference bundle after training or baseline evaluation. Neural controllers export one or more ONNX policy artefacts; deterministic controllers export rule-based JSON policies. The accompanying `artifact_manifest.json` records the manifest version, experiment and run identity, topology, per-agent observation and action dimensions, ordered observation names, encoders, action names, action bounds, reward metadata, algorithm format, and relative artefact paths. An optional alias map reconciles production feature names with the canonical training interface.

The experiment runner includes the export path. Before finalising the job output, the training side validates the manifest structure, supported artefact format, relative paths, presence of all files, per-agent indices, dimensions, and ONNX compatibility. This establishes a concrete hand-off between the Algorithms and inference repositories without a separate manual conversion.

The bundle preserves more than model portability. Observation ordering and categorical or periodic encoders are reconstructed by the serving process. Per-agent action names and bounds keep heterogeneous controller outputs associated with their physical meanings. Where electrical topology metadata is available, charger and site-limit configuration can accompany the artefact. Full physical safety nevertheless remains outside the neural graph unless the controller explicitly embeds it; action bounds in a manifest do not replace the Fail-Safe Supervisor or device protection.

### 7.5.2 energAIze Inference Runtime

The energAIze inference service exposes a stable HTTP interface for model-backed and rule-based decisions. Loading a bundle first parses its manifest, resolves optional aliases, discovers the available agent artefacts, rebuilds each preprocessing pipeline, and creates the corresponding ONNX Runtime or deterministic policy executor. A default agent can be selected while the complete set of loaded agent indices remains visible through the service state.

Each inference request is flattened into canonical feature keys, remapped through declared aliases, transformed by the training-compatible encoders, and passed to the selected policy. Outputs are mapped back to named actions, clamped to the declared bounds, and rounded according to the runtime contract. Request identifiers and structured logs preserve the execution trace. The health endpoint reports whether a bundle is configured, the active manifest, loaded agents, execution providers, alias file, and GPU availability.

This runtime boundary allows a controller to change without rewriting the API or Percepta integration. It also allows a rule-based controller and an ONNX policy to use the same deployment path, which is useful for baselines, staged authority, and operator-controlled fallback. The current general serving path supports ONNX and rule-based bundles. A TI-MARL controller whose dynamic typed-interface compiler remains part of execution requires that compiler and its state to

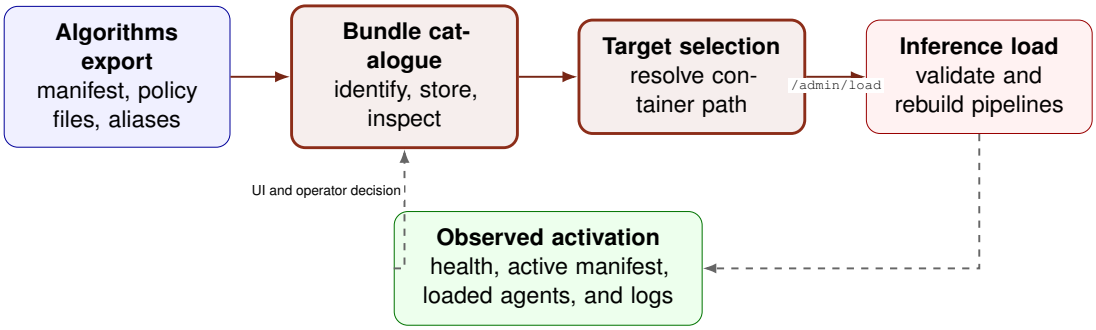



Figure 7.4: Implemented bundle management and inference activation sequence. Bundle selection is explicit and observable for each configured target; coordinated fleet-wide activation remains a separate lifecycle requirement.

be packaged and served with the policy; a research PyTorch bundle is not automatically equivalent to the standard inference contract.

### 7.5.3 Inference Targets and Bundle Management

The deployment backend maintains a content-identified bundle catalogue in shared storage. Uploading a bundle directory preserves its internal files, derives a bundle identity, and records its manifest and artefact location. Operators can inspect the bundle contents through the UI before selecting it. The configured deployment contains distinct inference services for HQ, São Mamede, and RH01, each exposing its own health and active-manifest state.

Selecting a bundle for a target causes the backend to resolve the path visible inside that inference container and call its administrative load endpoint. The inference service constructs the new pipelines and returns the load result; the backend then probes health and reports the active manifest to the UI. A bundle reported as active by a reachable target cannot be deleted from the catalogue. Previous bundles can remain available and be selected again if an operator decides to reverse a change.

Figure 7.4 shows this implemented sequence. The procedure is per target. It supports explicit version selection and reversal, but it is not yet an atomic policy-set rollout across several disconnected targets.

## 7.6 Monitoring, Failure Handling, and Recovery

Lifecycle monitoring separates several questions. Infrastructure monitoring determines whether workers, schedulers, containers, storage, and inference services are reachable. Experiment monitoring determines whether a job is advancing and producing expected outputs. Data monitoring determines whether observations are complete, current, and plausible. Control monitoring evaluates cost, service, peak, ramping, energy exchange, equality, safety interventions, and other domain

KPIs. Coordination monitoring determines whether the active policy set remains compatible and behaves coherently.

The implemented orchestration path provides heartbeats, worker versions, active-job counts, capability and resource information, explicit status transitions, live logs, progress, terminal results, and stale-worker policies. Dispatched jobs can be requeued when an attempt never starts. Running jobs without durable remote state fail explicitly and are never assumed complete. Attempt fencing prevents the original execution from later overwriting the result of a replacement attempt.

Remote execution introduces additional recovery semantics. A temporary Deucalion SSH interruption is tolerated for a configured interval before failure. Union persists remote run identity, upload, submission, result installation, cleanup, and terminal acknowledgement as separate steps. A bridge restart resumes the unfinished step. Successful remote compute is not rerun solely because result download, authentication, signed URLs, or local storage temporarily failed.

Inference monitoring exposes service readiness, active bundle identity, loaded agents, execution providers, GPU availability, and structured request logs. The deployment UI provides live and historical log inspection and can select an earlier bundle. This is an operator-controlled rollback mechanism. Automatic KPI-triggered rollback, atomic multi-target activation, compatibility barriers across independent inference services, and complete coordination-aware recovery remain to be demonstrated before they can be described as implemented guarantees.

The Fail-Safe Supervisor and external device protections remain the authority for physical action safety. Runtime output clamping, an available rule-based bundle, or a healthy inference process are useful mechanisms, but none alone proves that the complete physical control path is safe or robust. Operational evidence must report policy proposals, supervisor interventions, provider acceptance, executed actions, and recovery behaviour separately.

## 7.7 Systems Evaluation, Availability, and Contribution to RQ4

**Experiment reconstruction.** A completed job is reconstructed from its job identity in a clean environment. The audit compares source and image, resolved configuration, simulator and dataset, scenario, interfaces, reward, seed, runtime target, and resulting metrics. Reproduction tolerances distinguish deterministic environment outputs from stochastic learning outcomes.

**Cross-target execution.** Representative jobs are executed through server Docker, the personal-computer worker, Deucalion, Union, and Jetson where algorithm and resource requirements permit. The comparison reports queue and scheduler wait, setup time, orchestration overhead, execution time, resource use, artefact-transfer time, and output completeness. Scientific KPI differences are interpreted only after configuration, precision, image, and seed differences have been excluded.

**Failure and recovery.** Tests cover worker disappearance before and after dispatch, stale attempts, cooperative stop, Deucalion connection loss, Union bridge restart, interrupted result transfer,

expired signed URLs, insufficient local recovery space, and inference-target unavailability. The expected outcome records whether work is requeued, failed, stopped, retained for recovery, or completed without duplicate compute.

**Bundle and inference parity.** The exported bundle is loaded in a clean inference service and evaluated against saved observations from the training-side controller. The comparison checks feature ordering, encoders, per-agent selection, named actions, numerical tolerance, bounds, and rule-based behaviour. Target measurements include load time, inference latency, memory, CPU/GPU provider, request rate, and log overhead.

**Deployment control.** Catalogue upload, duplicate handling, file inspection, target selection, health verification, active-bundle deletion protection, and re-selection of a previous bundle are exercised for each configured inference target. A separate experiment is required before claiming coordinated policy-set rollout or automatic rollback across targets.

**Implementation and availability.** The lifecycle implementation is distributed across the Simulator, Algorithms, Job Orchestrator, Worker Agent, backend API, energAIze inference, EnergAIze UI, and infrastructure repositories. Each repository has a distinct responsibility and test suite. The final reproducibility record must identify release, commit, image, licence, contribution ownership, and archived experiment evidence. Repository size and commit counts can contextualise engineering effort but are not scientific results.

**Contribution to RQ4 and current boundary.** The implemented system establishes an end-to-end contract connecting reproducible experiments, heterogeneous execution, evidence return, policy export, bundle cataloguing, target-specific inference, health observation, explicit version switching, and several failure-recovery mechanisms. It demonstrates that heterogeneous compute does not require a separate scientific experiment format for each target and that training and inference can share observation, encoding, action, and topology metadata.

The broader policy-set lifecycle remains only partially automated. Bundle validation is structural and target loading is explicit; final controller promotion still requires scientific judgement. Coordinated readiness barriers, automatic compatibility decisions across independently deployed policy sets, KPI-triggered rollback, and dynamic TI-MARL serving require additional implementation and evidence. The current answer to RQ4 combines the concrete lifecycle achieved here with the MARL-specific semantics that future automation must preserve.

## 7.8 Chapter Summary

The EnergAIze lifecycle connects controller development to distributed execution through explicit, inspectable contracts. The Job Orchestrator preserves job identity and state; the Worker Agent

adapts a common payload to server, personal-computer, Deucalion, Union, and Jetson execution; shared storage and MLflow retain evidence; Algorithms exports self-contained policy bundles; the deployment backend catalogues and selects those bundles; energAIze inference reconstructs and executes the policy pipeline; and the UI exposes the resulting jobs, workers, artefacts, targets, health, and recovery actions to the operator.

This infrastructure makes deployability testable through the execution path, evidence, and target behaviour associated with a checkpoint. It also exposes its present limits: physical safety remains supervised outside the model, worker recovery is more mature than automatic policy-set recovery, and per-target bundle switching is not atomic distributed rollout. The next chapter examines how the architecture, controllers, lifecycle services, external platforms, and physical assets were combined at the evidence levels achieved in OPEVA and DemFlex.

# Chapter 8

# EnergAIze in Operation: Real-World Deployment and Validation

The preceding chapters developed the representation, simulation, control, and lifecycle mechanisms required to move learned energy control beyond an isolated experiment. This chapter examines that movement through two distinct demonstrators. The OPEVA demonstrator connects Renewable Energy Community operation, residential and corporate charging installations, external platforms, vehicle information, simulation, and distributed control. The DemFlex demonstrator provides a second and independently attributable setting. Their sites, assets, control authority, evaluation periods, and results are kept separate even where both instantiate the EnergAIze architecture.

Real-world evidence spans several levels. Historical measurements can drive a simulation; a target device can execute a policy without actuating an asset; a controller can operate in shadow beside the active system; and a bounded field test can apply physical commands. None of these results establishes sustained closed-loop operation by itself. Each demonstrator is reported with its physical configuration, integration path, controller authority, duration, interventions, and evidence level.

## 8.1 Evidence Levels and Comparative Reading

The validation uses the following hierarchy:

1. **real-data simulation**: historical operational data drive the simulator;
2. **target-hardware inference**: the controller runs on the intended class of device;
3. **hardware- or software-in-the-loop**: the policy interacts with physical or emulated interfaces in a closed loop;
4. **shadow operation**: decisions are produced and compared but not applied;
5. **limited physical or field control**: actions affect assets during a bounded pilot; and

6. **sustained operational control**: repeated closed-loop use forms part of normal site operation.

The evidence level is part of each claim. Real-data simulation supports long controller comparisons over identical periods, without showing whether a device accepted the requested action. Integration tests establish protocol and interface compatibility, while long-term energy benefit requires a different study. Physical control exposes latency, rejection, fallback, and user constraints; a short pilot still provides no seasonal evidence. The two demonstrators are assessed against the same questions: which assets were represented, which information was available, which decisions were produced, which commands were applied, which protection remained external, and what outcome was measured.

Figure 8.1 maps each thesis demonstrator to the exact evidence level and action authority it supports.

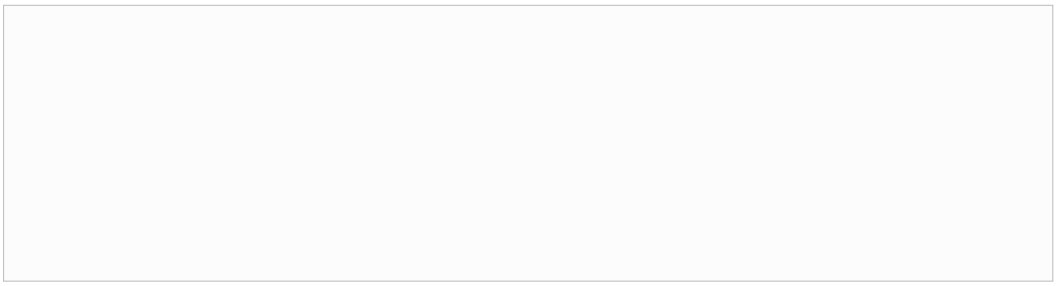


Figure 8.1: Validation evidence levels and the claims supported by each level.

Controller selection follows the promotion logic in Chapter 7. A method is included in a demonstrator only when its control performance, stability, interface compatibility, runtime resources, and protection mechanisms are adequate for the intended evidence level. A transparent rule-based controller remains available as a reference or fallback. The EnergAIze MADDPG controller preserves continuity with the OPEVA work; later MATD3, hierarchical, or TI-MARL policies require their own target-scoped validation before receiving physical authority.

## 8.2 OPEVA Demonstrator

### 8.2.1 Control Problem and Physical Configuration

OPEVA Demo 9 addresses the coordination problem created when EV charging is added to buildings that already have non-controllable demand, photovoltaic generation, storage, and site-power limits. Charging immediately at connection can satisfy the driver while increasing a building or community peak. Delaying or modulating the session can reduce cost and align demand with local photovoltaic production, but only within the vehicle's connection window and required departure energy. Corporate installations add a further allocation problem because several simultaneous charging requests can exceed the power available to the charging switchboard or site (Salvan and OPEVA Consortium 2026).

The demonstrator connects three complementary contexts. The Cleanwatts Living Lab represents a heterogeneous community of real residential participants, including consumers without controllable assets, prosumers with photovoltaic production, and flexsumers with batteries or domestic EV charging. The i-charging S. Mamede factory represents an industrial site with one 50 kW DC charger, two AC charging units with two outlets each, photovoltaic generation, meters, and a local controller. The i-charging headquarters represents a larger office installation with one 50 kW DC charger, eighteen 22 kW AC chargers, photovoltaic generation, building loads, meters, and a common charging switchboard. Simultaneous charging can request more than 400 kW; site-level power allocation is a physical requirement with direct consequences for the optimisation problem (Salvan and OPEVA Consortium 2026).

Across the three sites, the reported installation comprises four domestic 7.4 kV A AC chargers, two corporate 7.4 kV A AC units with two outputs, eighteen corporate 22 kV A AC chargers, two 50 kW DC chargers, two local charging controllers, and three AC meters. These devices are distributed across installations with different demand, generation, storage, communication, and control capabilities. The resulting heterogeneity is central to the demonstrator: the controller cannot assume that every participant owns the same assets or exposes the same telemetry.

The later real-data study associated with the demonstrator represents 17 installations. The first 15 form a heterogeneous residential and commercial REC; the remaining two represent an office and an industrial facility with photovoltaic generation and multiple EV chargers. The dataset contains 18 months of operational measurements aligned to 15-minute intervals. This simulation configuration and the physical Demo 9 installation are related research objects, but they are not interchangeable: the dataset supports long-horizon comparison, while the physical sites provide interface, device, and bounded-actuation evidence.

### 8.2.2 EnergAIze Integration Path

The demonstrator instantiates EnergAIze as a path from heterogeneous physical interfaces to a controlled action. Cleanwatts provides the community-facing platform for residential measurements, photovoltaic and consumption forecasts, batteries, energy prices, and selected actuation. My.i-charging monitors charging sessions and connects the residential and corporate chargers to the wider system. At the corporate sites, omni-e local controllers acquire charger and meter data and enforce local power-management functions. OCPP connects chargers with the charging platform, Modbus TCP/IP connects selected meters and chargers to local controllers, OCPI exchanges charging information and profiles between platform boundaries, and REST or RabbitMQ interfaces connect the data and algorithm services (Salvan and OPEVA Consortium 2026).

Percepta and the associated Data Provider middleware isolate the controller from those provider-specific details. Receivers acquire measurements from each platform; translators normalise identifiers, units, timestamps, and message formats; per-installation managers align the required decision interval, identify asset relationships, and handle data gaps; and predictors invoke the policy associated with the installation. The reverse path sends the validated result to the provider responsible

for the corresponding battery or charger. Training and inference use the same physical meanings while following different data paths: historical records are assembled into reproducible datasets for simulation and learning, whereas runtime messages are collected and aggregated periodically for the active controller (Fonseca, Sousa, et al. 2025; Salvan and OPEVA Consortium 2026).

Vehicle SoC illustrates why the integration layer is part of the control problem. Many chargers report session status and energy but do not expose the vehicle's current SoC. PulseCharge obtains available vehicle telemetry through manufacturer-facing services and combines it with user-declared or predicted flexibility. Recurrent user schedules provide a low-interaction baseline, predictions use historical routines when available, and explicit user input has priority when a journey departs from the routine. Consumption, photovoltaic-production, and flexibility prediction services provide additional derived observations and can temporarily compensate for unavailable direct measurements. Missing or low-confidence information remains distinguishable from a healthy observation because it changes which charging or V2G decisions can be justified.

The Flexibility Scheduling Algorithm receives the resulting observations and produces building-level actions for controllable assets. CityLearn and EVLearn reproduce the relevant building, EV, charger, battery, and community conditions so that rule-based, optimisation, single-agent, and multi-agent controllers can be trained or compared without trial and error on the physical installations (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Fonseca, Ferreira, Cabral, Severino, and Praça 2024). At runtime, the policy can execute centrally as one model instance per installation in a cloud environment or locally beside an installation. In both arrangements, provider and device protections retain the final authority to accept, limit, reject, or override a command.

Figure 8.2 presents the three OPEVA sites, the assets available at each, and the data and command path through the partner platforms and EnergAIze components.

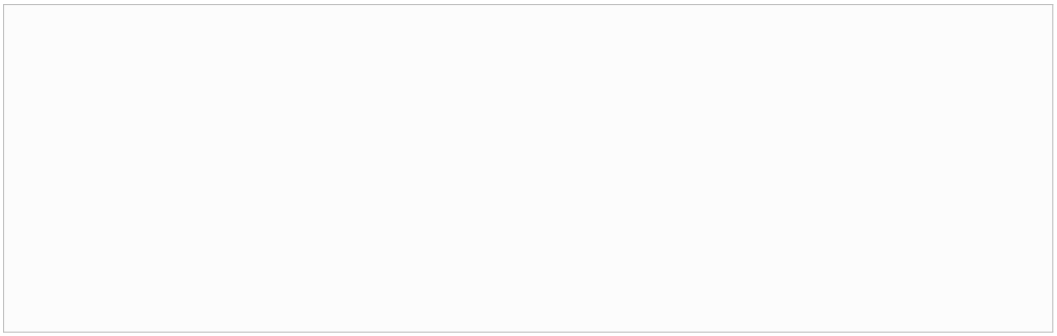


Figure 8.2: OPEVA Demo 9 sites, assets, platforms, and EnergAIze integration path.

### 8.2.3 Scenarios and Validation Strategy

Four use cases organise the demonstrator. Their value lies in the progression from a common residential charging problem to V2G, in-transit charging, and constrained fleet charging. Their implementation levels must remain explicit.

Table 8.1: OPEVA Demo 9 scenarios and the evidence boundary reported in deliverable D6.3.

| Scenario | Control question | Reported implementation | Boundary |
|---|---|---|---|
| 1 | Schedule residential charging within the connection window and departure requirement | Simulation and real setting | Physical action authority and duration require final log confirmation |
| 2 | Use a connected EV for local storage and V2G | Simulation and partial real integration | Bidirectional charger operation remained simulated |
| 3 | Select and reserve an in-route charging option using trip, tariff, range, and community information | Simulation and partial integration | Public charging and community/charger communication were not available for complete physical execution |
| 4 | Prioritise several corporate charging requests under building loads, photovoltaic production, user needs, and site-power limits | Simulation and partial real integration | Testing was bounded to avoid disturbing office and industrial operation |

The real-data simulation study compares unmanaged operation, passive REC energy sharing, rule-based scheduling, and EnergAIze coordination under common data and pricing assumptions. The 18-month records are aligned to 15-minute intervals, with accumulated and instantaneous quantities treated differently during resampling. Short gaps, longer missing periods, timestamp inconsistencies, physically invalid values, and post-gap accumulation spikes are handled according to declared data-type rules. This processing enables a common experiment but does not erase the fact that some controller inputs were interpolated, repeated, estimated, or unavailable in the original system.

The final evidence must keep this campaign separate from the earlier nine-building benchmark reported in D6.3. That benchmark combined demonstrator measurements with CityLearn data and simulated EV flexibility. It supported the initial controller and simulator evaluation, whereas the later 17-installation campaign addresses a different population, time resolution, data horizon, and performance-and-equity question. Numerical results from the two campaigns are not pooled.

### 8.2.4 Results and Demonstrated Boundary

The four-building operational study provides preliminary evidence that the complete data and coordination path can work with real participants and imperfect measurements. It reports approximately 9% lower daily peak demand and 5% lower energy cost for the evaluated coordinated operation (Fonseca, Sousa, et al. 2025). These values are interpreted with the reported site, baseline, duration, weather, participant behaviour, and degree of direct control; they are not extrapolated to all 17 installations or to sustained autonomous operation.

The later real-data simulation campaign provides the long-horizon comparison. Relative to the unmanaged reference, EnergAIze reports a 10.2% reduction in grid energy demand, a 13.6% reduction in energy cost, a 14.2% improvement in renewable self-consumption, a 20.7% reduction in peak demand, and a 30.1% reduction in ramping under the stated scenario and pricing assumptions. Participant-level results remain heterogeneous, which is scientifically important:

aggregate improvement does not establish that participants with different assets and flexibility obtain equal benefits.

The systems evidence is similarly layered. The Cleanwatts and my.i-charging integration exchanged charging-session data; OCPI charging profiles provided a path for control; local controllers operated at the corporate sites; the middleware produced periodic real-data messages for inference; and selected Cleanwatts-device actuation tests had begun at the reporting point (Salvan and OPEVA Consortium 2026). The same report described containerised model and middleware deployment as an early-stage activity. OPEVA established substantial evidence from real platforms, real data, local controllers, and bounded actuation. V2G, in-route charging, fleet-wide policy deployment, and sustained closed-loop operation retained scenario-specific limitations.

Figure 8.3 aligns one audited OPEVA period with observation quality, policy state, requested and applied charging actions, site demand, limits, and interventions.

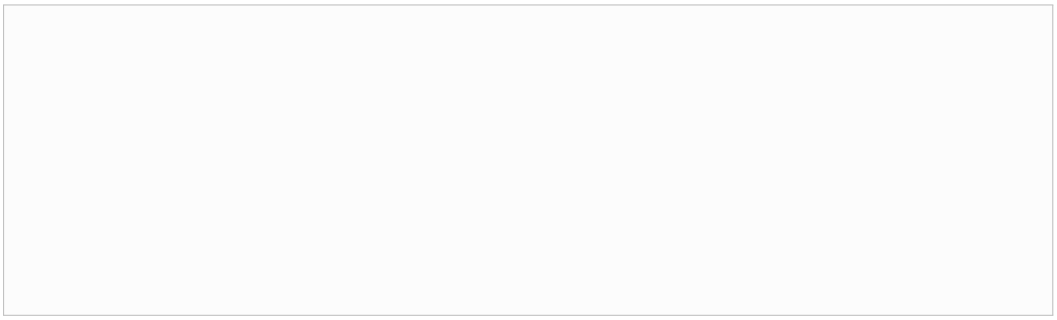


Figure 8.3: Timeline of one audited OPEVA control or integration period.

## 8.3 DemFlex Demonstrator

DemFlex constitutes the second demonstrator context of this chapter. It is kept separate from OPEVA because its physical setting, available assets, data lineage, control authority, experimental periods, and deployment history define a different body of evidence. The same EnergAIze abstractions can be reused, but compatibility of an interface or component does not permit a result, asset count, or evidence level to be transferred between demonstrators.

## 8.4 Cross-Demonstrator Lessons and Simulation-to-Operation Differences

The demonstrators expose differences that are easily hidden by a nominal simulator. Real timestamps and APIs are not perfectly aligned. SoC can be unavailable or estimated. User-declared departure times can be absent or incorrect. Physical platforms can reject, quantise, delay, or override commands. Assets can be unavailable for reasons not represented in the training scenario. Edge devices can restart and communication can be interrupted.

These differences are classified as data, interface, dynamics, execution, constraint, or lifecycle shifts. The appropriate response depends on the source of the mismatch. A unit or timestamp error requires a data-contract correction; a provider-specific action requires an adapter; a known observation failure requires an explicit degraded mode; a persistent distribution shift can justify retraining; and an incompatible capability must be rejected when numerical imputation would erase its semantics. The learning algorithm is changed only when the evidence identifies it as the relevant failure source.

Fail-safe behaviour is evaluated through proposed and executed actions separately. Missing or stale critical data can contract the current interface, activate conservative bounds, invoke a rule-based fallback, hold a previously safe value, or suspend learned control. The number, duration, cause, and effect of these interventions are part of the result. A stable physical trajectory produced almost entirely by external correction is evidence for a useful assurance layer, but weak evidence for the policy itself.

Figure 8.4 turns the observed differences between simulation assumptions and demonstrator conditions into explicit findings, mitigations, and remaining limits.

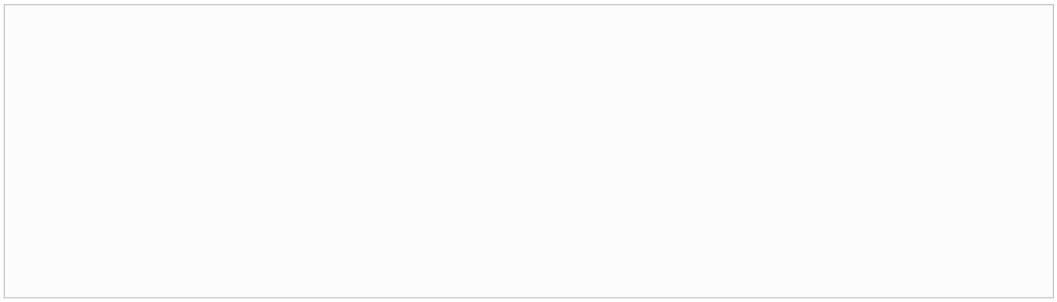


Figure 8.4: Observed differences between simulation assumptions and demonstrator conditions.

The evidence remains bounded by the number of sites, deployment duration, seasonal coverage, partner access, manual steps, and degree of direct actuation. Weather, occupancy, prices, and participant behaviour can confound energy outcomes. Where randomised field periods are unavailable, matched baselines, replay, and counterfactual simulation can support interpretation without extending the causal claim beyond the study design.

Figure 8.5 reports the final demonstrator evidence without collapsing simulated, integration, and physical outcomes into one score.

## 8.5 Contribution to the Research Questions and Chapter Summary

The two demonstrators contribute to all four research questions by testing interactions that the preceding chapters isolate experimentally. For RQ1, they identify the heterogeneous interfaces, participant and asset configurations, and availability changes that occurred outside a fixed benchmark. For RQ2, they establish the installation, device, data, and runtime scales actually reached, while larger controlled population claims remain in simulation. For RQ3, they expose data-quality

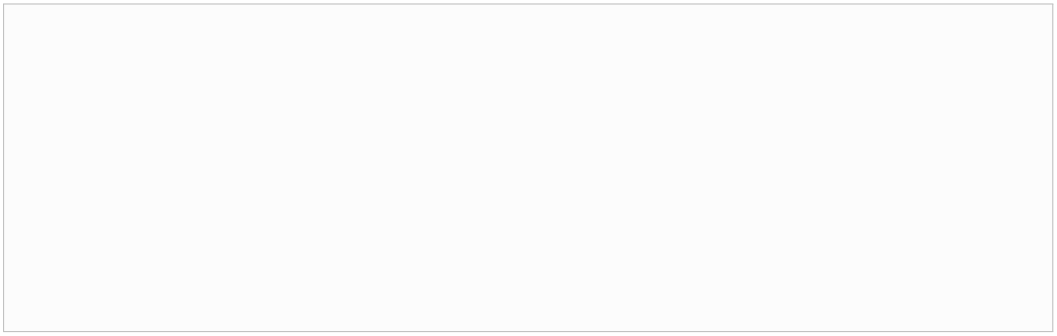


Figure 8.5: Verified OPEVA and DemFlex results by evidence level.

failures, missing vehicle state, physical limits, platform interventions, and degraded operation. For RQ4, they test the path from historical data and training to packaging, external-platform integration, edge or cloud inference, monitoring, updates, and recovery.

**Chapter synthesis.** OPEVA demonstrates how EnergAIze joins heterogeneous community and charging assets, external platforms, data normalisation, vehicle and flexibility information, simulation, learned control, and bounded actuation. Its strongest current evidence combines long-horizon real-data simulation, real-platform integration, local charging control, and preliminary field outcomes, while several scenarios remain partially simulated or bounded by platform access and site-operation constraints. DemFlex is treated as a separate demonstrator so that its contribution can be stated from its own configuration and evidence.

Together, the demonstrators show what changes when the architectural and algorithmic contributions meet physical assets, provider interfaces, imperfect data, runtime resources, and human authority. The next chapter synthesises that evidence with the controlled results from Chapters 4–7 and answers the research questions explicitly.

# Chapter 9

# General Discussion and Answers to the Research Questions

This thesis started from a practical difficulty. A controller that performs well in a simulator is still several steps away from controlling a distributed physical system. Its observations must retain the same meaning outside the experiment, its actions must remain valid for the assets that are currently available, physical and service constraints must be enforced, and the complete decision path must survive software, hardware, and organisational change. These conditions become harder when participants, chargers, vehicles, sessions, measurements, and computing targets do not remain fixed.

The work addressed that difficulty through one cumulative argument. EnergAIze established the system boundaries and common contracts. The simulator made heterogeneous assets, reconfiguration, failures, and physical limits reproducible. The controller progression identified what rule-based, optimisation-based, single-agent, and multi-agent methods solve under increasingly demanding conditions. TI-MARL then changed the policy-facing object from a fixed vector into a compiled, entity-bound interface. The lifecycle infrastructure connected experiments to heterogeneous compute, policy export, target-specific inference, monitoring, and recovery. OPEVA and DemFlex provide the real-world contexts in which the assumptions and limits of those contributions are examined.

The discussion does not select one algorithm as a universal winner. It asks what the combined evidence establishes, which mechanism produced each result, and how far that result can be carried beyond the evaluated setting. This distinction matters in a thesis that contains software verification, controlled simulation, real-data simulation, target-runtime evidence, integration with external platforms, and bounded physical operation. These evidence levels answer different questions.

## 9.1 What the Combined Evidence Establishes

Five bodies of evidence support the thesis argument.

The simulator establishes that the conditions motivating the research can be represented together and audited. The annual dataset suite contains heterogeneous residential and service members, several asset combinations, multiple chargers per member, EV sessions, deferrable loads, local settlement, total and per-phase limits, health events, and changes in membership and asset availability. The simulator work is important because a policy cannot be credited for robustness or reconfiguration when those conditions are absent from the environment. Its tests and audits support claims about representation, execution, and reproducibility; they do not by themselves support a claim about controller quality.

The controller progression establishes that complexity must earn its place. Rule-based controllers provide transparent feasibility and strong references. Optimisation exposes the objective, constraints, and value of information. Single-agent RL tests adaptation without the coordination burden. MADDPG and MATD3 introduce decentralised continuous multi-agent control and address part of the instability observed during learning. Behaviour cloning, guided exploration, replay strategies, offline learning, distillation, and hierarchy tackle data use, training effort, reuse, and the separation of local and community decisions. Their contribution is both methodological and diagnostic: each stage identifies a problem that can be solved without assuming that every later mechanism is already necessary.

TI-MARL establishes a different form of controller contract. The audited implementation compiles a versioned member schema and current runtime evidence into typed observations, dynamic action-group instances, health-dependent validity, bounds, dependencies, and local constraints. One shared actor processes these compiled local interfaces and produces a hybrid action bundle for the groups that exist at that step. TI-PPO uses a local typed critic; TI-MAPPO retains the same actor and runtime path while estimating value from the active member set. The analytic projector applies local service and electrical constraints before command encoding. The focused regression suite provides implementation evidence for these semantics, dynamic known-type composition, topology-aware trajectories, checkpoint restoration, health closure, and requested–projected–applied traces. It does not replace the multi-seed control comparison.

The lifecycle work establishes that experiment execution and policy serving can share a governed hand-off without forcing every computing target to adopt the same infrastructure. The Job Orchestrator retains persistent job identity and state. Worker adapters execute the same scientific payload through local Docker, the personal workstation, Deucalion, Union, and Jetson-compatible paths. Algorithms exports validated ONNX or rule-based bundles for the general inference path. The deployment backend catalogues bundles and activates a selected version on a target-specific inference service, while the UI exposes state, logs, health, and operator-controlled reversal. The evidence supports heterogeneous execution and per-target bundle management. It does not yet support atomic TI-MARL policy-set rollout, automatic KPI-triggered rollback, or production certification of the typed actor bundle.

The demonstrators establish why the separation between these layers is necessary. OPEVA combines residential and corporate charging, photovoltaic generation, storage, site limits, external platforms, local controllers, vehicle information, prediction, simulation, and learned control. The reported four-building study found approximately 9% lower daily peak and 5% lower energy cost in its evaluated setting (Fonseca, Sousa, et al. 2025). The later 17-installation real-data simulation reported reductions of 10.2% in grid energy demand, 13.6% in energy cost, 20.7% in peak demand, and 30.1% in ramping, together with a 14.2% improvement in renewable self-consumption. Those results retain the baseline, data-processing, action-authority, and evidence-level boundaries stated in Chapter 8. They are not evidence of sustained autonomous control across every OPEVA site. DemFlex is analysed as a separate body of evidence and is not populated with OPEVA assumptions.

Table 9.1 condenses the role of each evidence layer. The claim boundary is as important as the result because it prevents a unit test, a long simulation, or a successful API exchange from being presented as physical controller validation.

Table 9.1: Role of the principal evidence layers in the thesis argument.

| Evidence layer | What it establishes | What it does not establish alone |
|---|---|---|
| Simulator tests and audits | Correct representation and execution of the declared scenario, assets, events, constraints, and interfaces | Controller superiority or field benefit |
| Controlled algorithm experiments | Relative control quality, learning behaviour, ablation effects, and sensitivity under matched scenarios | Transfer to an unseen site or reliable physical actuation |
| TI-MARL regression and stress tests | Executability and traceability of typed compilation, variable groups, local projection, topology change, and persistence | Better cost or coordination than the strongest baseline |
| Lifecycle and runtime tests | Reconstructable jobs, heterogeneous execution, bundle validation, inference loading, observation, and recovery behaviour | Automatic coordinated rollout or physical safety |
| Real-data and demonstrator evidence | Behaviour under real measurements, platforms, devices, users, and bounded action authority | Universal, seasonal, or cross-domain effectiveness |

## 9.2 Answer to RQ1: Dynamic and Heterogeneous Multi-Agent Control

RQ1 asks how MARL control can accommodate heterogeneous and changing populations, capabilities, and availability while balancing local and collective objectives. The answer developed in this thesis is to make the current control interface part of the decision process instead of hiding it in a permanent vector layout.

The formulation separates the member, the physical asset, the EV, and the charging session. These identities have different lifetimes and cannot safely be collapsed into one index. A member can remain active while a charger becomes unavailable. One charger can serve several EV sessions

over time. A service building can expose several independently controlled chargers. The typed interface records these relations and associates every observation, action group, bound, health rule, and constraint with the entity to which it applies.

At runtime, the Typed Interface Compiler constructs one deterministic local snapshot from the versioned schema and current system evidence. The shared actor encodes the observations that exist and decodes a mode and bounded fraction for each active action group. A newly active instance of a known charger type reuses the same semantic encoder and group head; the network does not gain a new output coordinate. Stable member identities preserve survivor trajectories when members join or leave, while EV-session rebinding changes the relevant entity relation without forcing an unnecessary structural rebuild.

The multi-agent component is retained through the joint physical transition, shared resources, local and community reward terms, authorised aggregate observations, and, in TI-MAPPO, a central set critic used only during training. Execution remains decentralised. A member does not need the raw private state of every other participant. This boundary makes the information assumption visible and allows TI-PPO and TI-MAPPO to be compared without changing the deployed actor.

Local and collective objectives cannot be reduced to aggregate cost. The evaluation therefore includes EV service, deferrable completion, individual cost outcomes, the number of members that improve, peak, ramping, renewable self-consumption, import, export, and residual constraint pressure. A community improvement that depends on a small group receiving worse service fails the intended interpretation even when total cost decreases.

The answer is bounded to known semantic and action-group families. TI-MARL can admit new instances and tested combinations of those families; it does not infer the meaning or safety requirements of an arbitrary unknown device. An unsupported type is rejected or routed to an explicit fallback. This is a controlled form of plug-and-control, not unrestricted zero-shot control.

## 9.3 Answer to RQ2: Multi-Dimensional Scalability of Learning and Coordination

RQ2 asks under which conditions learning and coordination can maintain useful control as population, heterogeneity, reconfiguration, and environment complexity increase under constrained resources. The results support a conditional answer. Scalability is an operating envelope with several axes, not a property that can be assigned from agent count alone.

The simulator contributes data formats, annual scenarios, memory corrections, structured interfaces, and repeatable event injection. The experiment infrastructure separates scientific scale from campaign throughput: several workers can reduce wall-clock time without changing the complexity of the controller. The deployment targets add another dimension because a policy that trains successfully on a remote GPU may still fail the memory, latency, image, or architecture requirements of an edge device.

The controller mechanisms address complementary bottlenecks. Decentralised execution limits runtime dependence on a central joint action. Parameter sharing prevents the number of actor parameters from growing with every known asset instance. The hierarchical typed encoder aggregates observations by channel and sensor before constructing group context. Dynamic group heads avoid a preallocated universal output vector. TI-MAPPO's set critic accepts a variable active population without rebuilding a fixed concatenated input. Packed evaluation and cached structural preparation reduce repeated runtime work.

None of these mechanisms makes cost constant. More observations and action groups still require more encoding and decoding work. A larger active population increases critic and simulator work. Topology events add compilation and trace cost. Training sample requirements can grow even when parameter count does not. The appropriate evidence reports control quality together with compiler, projector, actor, and critic latency; training throughput; memory; trace volume; communication; and checkpoint size.

The answer to RQ2 is therefore expressed as the largest tested combination that satisfies the declared control and resource gates, together with the bottleneck encountered next. Core-15 supports rapid development, Core-30 isolates nominal, safety, health, and dynamic conditions, and Premium-100 combines scale, heterogeneous members, several chargers, reconfiguration, faults, and physical pressure. Results obtained in one profile do not imply the same envelope under another hardware target, communication pattern, or control interval.

## 9.4 Answer to RQ3: Safe and Robust Operation

RQ3 asks how physical and service constraints, uncertainty, faults, reconfiguration, and the difference between training and use can be incorporated and assessed. The evidence supports a layered answer in which policy learning, deterministic feasibility, physical protection, and monitoring retain separate responsibilities.

The simulator represents total and per-phase limits, asset bounds, charging and departure requirements, deferrable deadlines, outages, data problems, communication failures, device unavailability, and changes in topology. This establishes the conditions under which safety and robustness can be evaluated. It also makes the disturbance type, duration, severity, and affected scope part of the experiment record.

TI-MARL uses the same typed semantics in observation construction, action validity, dynamic bounds, health closure, and local feasibility. The actor proposes a raw hybrid bundle. The analytic local projector then enforces validity, EV service floors, V2G reserve, mandatory deferrable starts, local total headroom, and local per-phase headroom. The command and trace retain the raw, projected, and applied values. Projector intervention can therefore be measured instead of being hidden inside the environment.

Health handling is scoped through declared dependencies. Stale vehicle information can contract the affected EV actions without disabling an unrelated stationary battery. A failed charger

can remove its group while leaving the member active. Loss of a critical main meter can force a wider degraded mode. Recovery uses explicit hysteresis so that a fluctuating signal does not repeatedly enable and disable control. These rules convert several robustness cases into observable interface changes.

The guarantee remains local and conditional on the correctness of the encoded model. The current TI-MARL implementation contains no cross-member hard corrector for a shared feeder or community limit. Shared constraints are exposed, learned, and evaluated from the applied trajectory; a residual violation is a failed outcome. Device protection and the external Fail-Safe Supervisor retain authority in physical operation. A low violation count after frequent external correction is evidence that the assurance layer worked, not that the policy learned safe behaviour.

Robustness is reported through degradation and recovery, not a single nominal score. Relevant outcomes include service loss, violation energy, invalid requests, fallback duration, unnecessary shutdown, recovery time, and the difference between requested and executed actions. A method is robust within the tested disturbance envelope when it maintains the declared service and safety gates or degrades in the controlled manner specified by the experiment.

## 9.5 Answer to RQ4: Governed and Deployable MARL Lifecycle

RQ4 asks which abstractions and mechanisms allow coordinated learned control to be reproduced, governed, and deployed across heterogeneous and changing edge systems. The implemented answer begins with lineage and ends with an observed target state.

An experiment is identified by its resolved configuration, source and image revisions, simulator and dataset, scenario, interface, reward, seed, target, logs, checkpoints, results, and KPI analysis. The Job Orchestrator maintains the persistent job state. Workers translate a common payload into the execution contract of local Docker, the personal workstation, Deucalion, Union, or Jetson-class infrastructure. Target-specific authentication, scheduling, staging, images, and result recovery do not alter the scientific identity of the job.

The general inference path packages the policy with the information needed to reconstruct its input and output semantics. The manifest records topology, ordered observations, encoders, actions, bounds, algorithm format, and artefact paths. The inference service validates the bundle, rebuilds preprocessing, selects the declared agent, performs ONNX or rule-based execution, maps outputs to named bounded actions, and exposes health and active-bundle identity. The backend catalogues bundles and loads a selected bundle into a configured target. Previous versions remain available for operator-controlled reversal.

TI-MARL contributes an algorithm-side deployment contract through resolved interfaces, compiler state and rules, compatibility signatures, stable identities, actor-only bundles, and execution traces. The current generic inference service does not yet certify that typed actor bundle as a production serving path. The distinction prevents a research checkpoint from being described as deployed merely because its weights can be serialised.

The demonstrated lifecycle is strongest for reproducible experiment execution, heterogeneous worker adaptation, explicit remote recovery state, validated general bundles, per-target loading, and operator-visible reversal. Atomic activation of a coordinated policy set across disconnected targets, automatic compatibility decisions, KPI-triggered rollback, and complete TI-MARL serving remain outside the implemented guarantee. These are not peripheral deployment details: they determine whether the policy combination observed at runtime is the combination that was validated.

RQ4 is therefore answered by a concrete lifecycle and by the boundaries it exposes. Deployability is demonstrated through preserved meaning, target compatibility, observable activation, and recoverable state. Continued coordinated operation requires further evidence when several policies and intermittent targets must change together.

Figure 9.1 links each answer to the corresponding contribution, evidence level, and boundary.

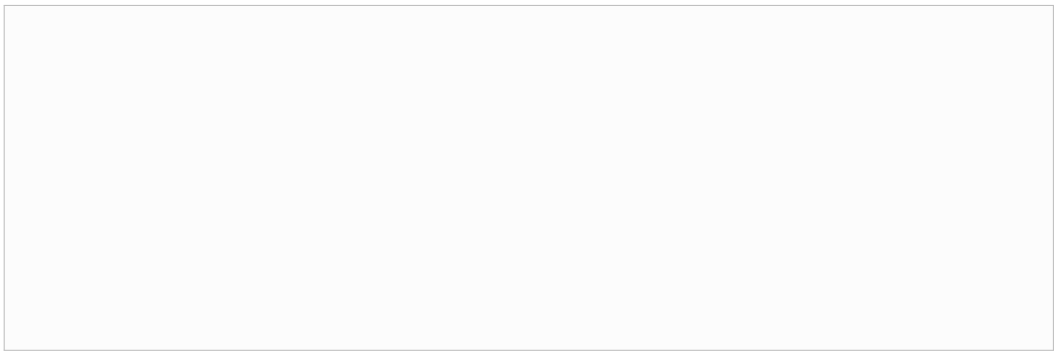


Figure 9.1: Research-question, contribution, finding, and evidence map.

## 9.6 Interactions among the Four Requirements

The four requirements affect one another, often in ways that a single reward or benchmark conceals.

**Scalability and control quality.** Parameter sharing and aggregation reduce model growth, but aggressive aggregation can remove distinctions required for local service or phase-specific limits. More expressive typed interfaces preserve those distinctions at the cost of additional tokens, compilation, and trace data. Scale must therefore be reported together with the control information retained.

**Safety and performance.** The local projector can reject a nominally profitable action because it violates service or electrical limits. That reduction in reward is not a loss of controller quality under the physical problem. Frequent projection, however, indicates that the actor has not internalised the feasible region. Raw and final bundles are needed to distinguish valid conservatism from weak learning.

**Robustness and specialisation.** A policy tuned to one community, season, or stable asset set may outperform a more transferable policy in nominal operation. Typed representations, health-aware compilation, historical data, and broader scenario coverage can improve transfer while increasing training and validation cost. The preferred balance depends on how often the site changes and how expensive retraining or a failed decision is.

**Deployability and model complexity.** A controller can pass simulation gates and still exceed a target's memory or latency budget. A model can fit each target separately while its preprocessing, compiler, communication, or update protocol remains incompatible. Resource and compatibility gates belong to promotion because they influence which controller can be used, not merely how it is installed.

**Reconfiguration and assurance drift.** An interface change can invalidate a trained action head, an observation alias, a safety bound, a fallback dependency, or a previously validated policy combination. The typed interface and lifecycle meet at this point. The controller must expose the current contract, and the operational system must decide whether the resulting composition remains admissible.

Figure 9.2 summarises these interactions without treating the four terms as independent scores.

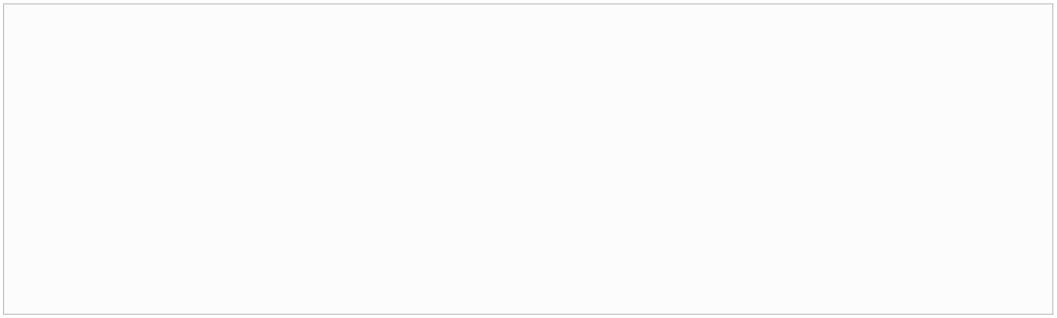


Figure 9.2: Interactions among scalability, deployability, safety, and robustness.

## 9.7 Scientific Contributions, Generality, and Responsible Use

The central scientific contribution is TI-MARL's executable connection between a versioned control contract and multi-agent learning. Its novelty does not lie in PPO, attention, parameter sharing, health rules, or projection separately. The method compiles an entity-bound interface; evaluates it through one shared typed actor; associates decisions with current groups and physical meanings; applies health, validity, bounds, and local feasibility through the same semantics; and carries stable identities and compatibility evidence across topology change. TI-PPO and TI-MAPPO isolate the effect of local and central-set value estimation while retaining that common actor and runtime path.

This principal contribution rests on six supporting outcomes: the evidence-based formulation of the integrated real-world MARL problem; the EnergAIze research architecture; the audited simulator and annual scenario suite; the progression of control and learning solutions; the experiment and inference lifecycle; and the demonstrator evidence. They should not be presented as seven unrelated novelties. Together they explain why the algorithm has the interface it has and how its claims can be tested and carried towards use.

The empirical conclusions remain energy-domain conclusions. A typed charger action does not become a robotic manipulation action by renaming it. The transferable part is the problem structure: agents can gain or lose known capabilities; observations and actions belong to identifiable entities; availability changes validity and bounds; local decisions interact through shared resources; and policies must retain their meaning across training and runtime. Application to robotics, industrial equipment, traffic control, or distributed edge services requires new schemas, physical constraints, baselines, and evidence.

Responsible community control also requires participant-level interpretation. Lower aggregate cost can coexist with worse outcomes for a minority, and average EV service can hide one missed departure. The scorecard therefore includes benefit coverage, service, constraints, and intervention alongside energy and cost. Local inference reduces some data transfer but does not remove privacy risks from logs, forecasts, identities, or model updates. Cybersecurity, certification, regulation, and long-term organisational governance constrain deployment but are not claimed as primary contributions of this thesis.

## 9.8 Limitations and Threats to Validity

**Construct validity.** Dynamic population, safety, robustness, scalability, and real-world operation have several meanings. The thesis separates session changes from member admission, local projection from shared physical safety, nominal noise from declared faults, population size from resource scale, and real-data simulation from physical control. Some ambiguity remains when source projects use broader terminology.

**Internal validity.** RL comparisons depend on implementation, reward, information access, tuning effort, checkpoint selection, seeds, and stochastic environments. Common manifests, paired windows, hard promotion gates, strong baselines, ablations, and independent seeds reduce this risk. They cannot compensate for an unequal information or tuning budget, which must remain visible in each comparison.

**External validity.** The communities, assets, tariffs, weather, participants, devices, and event distributions bound the results. Core and Premium scenarios broaden controlled coverage, while OPEVA and DemFlex expose conditions absent from nominal simulation. Neither large simulation nor one demonstrator establishes universal performance.

**Physical and causal validity.** The simulator and local projector depend on the correctness of their electrical, asset, service, and settlement models. Field outcomes can be confounded by occupancy, weather, price, maintenance, participant behaviour, platform changes, and manual intervention. Matched baselines and replay improve interpretation but do not create randomised field evidence.

**Implementation and attribution.** The work builds on CityLearn, established algorithms, open-source packages, student work, partner platforms, and collaborative European projects. Releases, repository history, contribution statements, data ownership, and demonstrator records are required to distinguish original design, adaptation, implementation, integration, data provision, and experiment execution.

## 9.9 Implications and Open Research Problems

The results point to a smaller set of open problems than the broad real-world MARL label suggests. Unknown asset families still require admission semantics, policy selection, and safety configuration. Shared hard constraints need allocation mechanisms whose effect on fairness and service can be evaluated. Assurance rules must detect when their own observations or physical models are no longer trustworthy. Continual and offline improvement must preserve promotion evidence and previously validated behaviour. Coordinated updates over intermittent networks need explicit mixed-version and local recovery semantics. Longer replicated deployments are required to study seasonal drift, maintenance, interventions, and benefit distribution.

The thesis therefore answers the four research questions at different evidence levels. RQ1 receives a concrete typed-interface formulation and verified implementation. RQ2 receives mechanisms and an explicit multi-dimensional evaluation envelope. RQ3 receives layered local feasibility, health-aware operation, disturbance protocols, and a clear boundary around shared safety. RQ4 receives an implemented experiment-to-inference lifecycle with honest limits around coordinated rollout and TI-MARL serving. The quantitative strength of each answer remains tied to the final evidence reported in its contribution chapter.

The main conclusion is that the difficult part of real-world MARL lies in preserving meaning across layers. The physical problem, simulator, policy interface, feasibility logic, experiment, package, target, and applied action cannot be evaluated as unrelated objects. EnergAIze provides the system context for those relations, and TI-MARL makes the changing local control contract an explicit part of learning. Chapter 10 condenses this argument and states what the thesis ultimately establishes.

# Chapter 10

# Conclusions and Future Research

This thesis investigated how multi-agent reinforcement learning can move from controlled experiments towards use in distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems. Renewable Energy Communities and electric-vehicle charging provided a demanding empirical setting: local assets interact through shared infrastructure; participants expose different information and flexibility; chargers, vehicles, and sessions change independently; physical and service limits cannot be ignored; and the controller must eventually run through real software, devices, and organisations.

The research did not produce one isolated algorithm followed by an application. It produced a connected path. EnergAIze defined the architecture and responsibilities. The simulator made the relevant conditions reproducible. A progression of controllers established which problems could be solved with transparent rules, explicit optimisation, single-agent learning, conventional MARL, data-efficient learning, and hierarchy. TI-MARL addressed the remaining fixed-interface assumption. Experiment orchestration and distributed inference carried controller artefacts across heterogeneous computing environments. OPEVA and DemFlex then provided distinct real-world contexts for testing the resulting claims.

## 10.1 What the Research Established

The state-of-the-art review showed that scalable MARL, dynamic teams, safe learning, physical testbeds, deployment tooling, and long-running intelligent systems already contain strong individual solutions. The residual problem was their composition. The assumptions that make one mechanism effective often remain implicit and can fail when the system interface, physical constraints, population, information quality, or runtime changes.

The first conclusion is that the experimental world is part of the method. The simulator is not a neutral source of reward. Its identities, timebase, units, assets, events, constraints, observations, actions, settlement, and data-quality rules determine what a controller can learn and what a result means. The annual dataset suite and the CityLearn v3 extensions make heterogeneous members,

multiple chargers, EV sessions, deferrable services, phase limits, health problems, and topology changes available under repeatable conditions. This foundation permits the later algorithmic claims to be tested instead of assumed.

The second conclusion is that controller complexity must be justified by the control problem. Rule-based control remains valuable for transparency, feasibility, development, and fallback. Optimisation remains valuable when the model and horizon are available and an explicit bound or operational schedule is needed. Single-agent methods isolate adaptation from coordination. Conventional MARL becomes useful when distributed information and repeated interaction matter. MATD3, guided exploration, behaviour cloning, replay strategies, offline learning, distillation, and hierarchy address specific problems of stability, data use, runtime cost, and local–community decomposition. Their value does not disappear when TI-MARL is introduced.

The third conclusion concerns the object presented to the policy. Fixed observation and action arrays are appropriate while all relevant entities and meanings are known in advance. They become fragile when one building gains a second charger, an EV session moves, a device loses a capability, or a measurement failure changes which action remains defensible. TI-MARL represents the active local interface as a versioned, entity-bound object. A deterministic compiler derives typed observations, dynamic groups, health-dependent validity, bounds, dependencies, and local constraints. A shared actor produces one coherent hybrid bundle for the groups that currently exist. TI-PPO and TI-MAPPO use the same actor and execution path, differing in whether value estimation is local or conditioned on the active population.

The fourth conclusion is that learned actions and feasible actions must remain distinguishable. The actor produces a raw bundle; the analytic projector applies local service and electrical requirements; the device or simulator returns the applied outcome. Keeping these stages in the trace reveals whether good physical behaviour came from the policy, the feasibility layer, external protection, or a combination. The current guarantee is local and model-dependent. Shared limits remain an empirical multi-agent outcome because the evaluated TI-MARL version contains no cross-member hard corrector.

The fifth conclusion is that scale has several independent costs. Parameter sharing can keep model size stable while token processing, critic work, communication, simulation, or sample demand grows. Distributed workers can shorten a campaign without making the algorithm itself more scalable. A policy can train on a GPU and still fail an edge latency or compatibility requirement. Population, heterogeneity, topology events, memory, throughput, communication, training time, inference time, and control quality must be reported together.

The sixth conclusion is that deployment starts before a model file is copied and continues after it is loaded. The experiment, source, image, simulator, dataset, interface, reward, seed, checkpoint, package, target, active version, and observed actions form one evidence chain. The implemented lifecycle reconstructs jobs across heterogeneous workers, validates general inference bundles, loads selected versions into target-specific services, exposes health and logs, and permits operator-controlled reversal. Automatic coordinated rollout, KPI-triggered rollback, and production

TI-MARL serving remain separate engineering and research steps.

Finally, the real-world work shows why these distinctions matter. OPEVA connects real participants, corporate charging, local controllers, external platforms, vehicle and flexibility information, historical data, simulation, and bounded actuation. Its reported studies show improvements in cost, peak, grid demand, ramping, and renewable use within their declared settings. They also expose missing SoC, asynchronous data, platform boundaries, site limits, partial scenario implementation, and heterogeneous participant outcomes. DemFlex supplies a second demonstrator whose claims must be derived from its own site, controller, data, authority, and duration.

## 10.2 Answers to the Research Questions

**RQ1: dynamic and heterogeneous multi-agent control.** Changing participants and assets are handled by separating stable agent identity from module, entity, and session identity and by compiling the active control interface at runtime. Known observation and action semantics are reused across new instances, while health and validity determine which groups can act. A shared local actor preserves decentralised execution; aggregate context, coupled outcomes, and the TI-MAPPO set critic support community coordination. Unknown semantic families require explicit admission or fallback instead of an arbitrary embedding.

**RQ2: multi-dimensional scalability.** Scalability is achieved conditionally through structured simulation, parameter sharing, hierarchical typed encoding, dynamic group heads, a variable-population set critic, cached preparation, distributed experiment workers, and target-specific inference profiles. These mechanisms remove several fixed-size assumptions but do not make computation independent of population or interface size. The answer is the measured envelope that satisfies both control and resource gates, together with the bottleneck that follows it.

**RQ3: safe and robust operation.** Safety and robustness are incorporated through explicit scenario constraints and faults, health-aware interface compilation, dynamic validity and bounds, local analytic projection, service floors, degraded modes, fallback, device protection, and requested–projected–applied traces. Disturbances are evaluated by type, severity, degradation, intervention, and recovery. The thesis supports conditional local feasibility and empirical shared-constraint assessment, not universal formal certification.

**RQ4: governed and deployable lifecycle.** Reproducible job identity, heterogeneous worker adapters, persistent remote state, validated policy bundles, target profiles, inference health, active-version records, logs, and operator-controlled reversal connect experiments to serving. TI-MARL adds resolved interface, compiler, compatibility, persistence, and trace contracts to that handoff. Coordinated atomic rollout and automated policy-set recovery remain beyond the current implementation evidence.

## 10.3 Principal Contributions

TI-MARL is the principal algorithmic contribution. Its novelty lies in the executable connection between a versioned, entity-bound control contract and multi-agent learning. The same typed semantics organise what the actor observes and invokes, how health changes validity, how local feasibility is applied, how topology-aware state is restored, and how decisions are traced. This contribution is narrower and more defensible than claiming novelty for PPO, attention, parameter sharing, or action projection separately.

Six supporting contributions make that method meaningful and testable:

1. an evidence-based formulation of the integrated problem separating simulated performance from real-world readiness;
2. the EnergAIze architecture, which assigns responsibilities and contracts across data, simulation, control, experimentation, deployment, and operation;
3. the CityLearn v3 simulation and annual scenario foundation for heterogeneous, event-driven, constrained, and reconfigurable REC and EV experiments;
4. the progressive body of rule-based, optimisation, RL, MARL, data-efficient, and hierarchical solutions and the conditions under which each remains useful;
5. the governed experiment and inference lifecycle spanning heterogeneous compute, reproducible artefacts, general bundle export, target activation, observation, and recovery; and
6. real-world-oriented evidence from OPEVA and DemFlex, kept separate by demonstrator and evidence level.

The software produced during the research includes the simulator, datasets and validation tools, controller implementations, TI-MARL compiler and runtime, orchestration services, worker adapters, inference service, backend and UI support, experiment configurations, and demonstrator integrations. Software volume is evidence of engineering effort, not scientific validity. Tests, comparisons, ablations, systems measurements, releases, and demonstrator records provide the corresponding evidence.

Figure 10.1 summarises how the contributions combine without presenting them as independent inventions.

## 10.4 Assessment of the Central Proposition

The central proposition stated that operationalising MARL in distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems requires the integrated co-design of system representations and simulation environments, scalable learning and coordination, safe and robust control processes, and lifecycle-aware distributed deployment infrastructure.

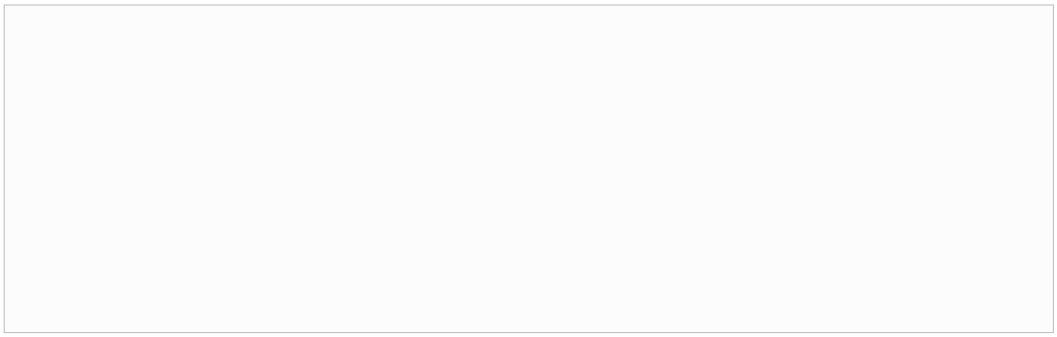


Figure 10.1: Contribution stack and cumulative thesis argument.

The body of work supports that proposition within the evaluated energy-domain scope. No individual layer carries the complete claim. A learning result depends on the world and interface against which it was produced. A feasible action depends on the current entities, measurements, bounds, and physical model. A checkpoint remains meaningful only while its preprocessing, compatibility, and execution contracts are preserved. A healthy process does not prove acceptable control, and a successful physical command does not prove seasonal benefit. The integrated design is necessary because a change at one layer can invalidate evidence obtained at another.

The proposition is supported most strongly as a systems and method argument, with different empirical depths across its components. The simulator, progressive controllers, TI-MARL implementation, lifecycle, and OPEVA evidence establish concrete parts of the path. Final multi-seed comparisons, complete cross-target systems measurements, production TI-MARL serving, DemFlex evidence, and long-duration field operation determine the remaining quantitative reach. This boundary narrows the conclusion without weakening its central lesson.

## 10.5 Limits of the Conclusions

The empirical evidence is concentrated in Renewable Energy Communities, buildings, batteries, photovoltaic systems, EVs, and charging infrastructure. Mechanism-level transfer to another CPS is plausible where agents have changing known capabilities, entity-bound observations and actions, local and shared constraints, and heterogeneous runtime targets. Performance and safety do not transfer without new domain models and evidence.

The evaluated scenarios cover a wider range of topology, faults, physical pressure, and community size than the original benchmarks, but they remain designed scenarios. Field evidence is bounded by sites, duration, authority, participants, weather, platform access, and manual intervention. The local projector depends on the correctness of the encoded constraints. The central set critic uses a deliberately modest community summary. Unknown asset families, formal shared safety, adversarial security, certification, and fully autonomous fleet-wide updates remain outside the demonstrated contribution.

## 10.6 Future Research Directions

**Admission of new asset families.** A stronger plug-and-control process should identify a new asset, verify its schema and physical model, select a compatible policy or fallback, test local and shared constraints, and monitor its effect after admission. This requires governance for schema evolution as well as more asset types.

**Shared physical assurance.** Local projection should be complemented by allocation methods for shared feeder, transformer, or community constraints. The allocation rule must expose its consequences for service and fairness; a central corrector is not neutral merely because it restores feasibility.

**Larger and richer TI-MARL structures.** Future work should compare TI-PPO and TI-MAPPO with graph and attention-based critics, alternative learning backbones, larger populations, multiple communities, and richer electrical topology. These experiments must separate gains from the typed interface from gains produced by the optimiser or critic.

**Assurance under model and data error.** The compiler and projector should represent confidence in measurements and physical relations, detect inconsistent assumptions, and report when a guarantee no longer applies. Formal barrier, reachability, or shield methods are relevant where suitable models exist, but must also adapt to interface change.

**Continual and distributed learning.** Historical, offline, federated, and continual learning can reduce direct exploration and support adaptation across communities. Promotion gates, replayable evidence, privacy constraints, and protection against forgetting must remain part of the update process.

**Coordinated deployment over intermittent networks.** Mixed versions, partial activation, disconnection, local rollback, and recovery need explicit policy-set semantics. The next lifecycle step is to determine which combinations remain admissible while independent targets change at different times.

**Long-duration and replicated real-world studies.** Seasonal and multi-site studies should report energy and cost together with service, participant benefit, data incidents, projection and fallback interventions, rejected commands, resource use, policy changes, and operator actions. Such records are needed to distinguish an effective policy from an effective surrounding assurance system.

**Fairness, explanation, security, and governance.** Participant-level benefit, explanation of binding and projection decisions, human override, privacy, cybersecurity, certification, and regulatory accountability deserve dedicated research beyond their role as constraints in this thesis.

## 10.7 Closing Remarks

The work began with learning-based energy coordination and expanded as real systems exposed assumptions that a nominal benchmark could leave invisible. Feasible control led to explicit optimisation. Adaptation led to decentralised coordination. Training instability and data scarcity led to stronger learning and reuse mechanisms. Local and community objectives led to hierarchy. Changes in assets, sessions, health, and action meaning led to TI-MARL. Reproducibility and physical use led to orchestration, packaging, inference, monitoring, and recovery.

The final lesson is simple but demanding. A credible intelligent controller must expose what it controls, what it can currently observe and invoke, how an action becomes feasible, which experiment produced the policy, which runtime is executing it, what was actually applied, and what evidence supports its continued use. EnergAIze supplies the surrounding system, TI-MARL supplies a policy formulation for changing typed interfaces, and the simulator, controller progression, lifecycle, and demonstrators supply the evidence needed to test both. Together, they move the discussion from a successful learning curve towards an auditable path for intelligent control in changing physical systems.

# References


Ackermann, J., V. Gabler, T. Osa, and M. Sugiyama (2019). *Reducing Overestimation Bias in Multi-Agent Domains Using Double Centralized Critics*. DOI: 10.48550/arXiv.1910.01465. arXiv: 1910.01465 [cs.LG].

Albrecht, S. V., F. Christianos, and L. Schäfer (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. Cambridge, MA: MIT Press. URL: https://www.marl-book.com.

Almilaify, Y., K. Nweye, and Z. Nagy (2023). "SCALEX: Scalability Exploration of Multi-Agent Reinforcement Learning Agents in Grid-Interactive Efficient Buildings". In: *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 261–264. DOI: 10.1145/3600100.3623749.

Anil, G., P. Doshi, D. A. Redder, A. Eck, and L.-K. Soh (2025). "MOHITO: Multi-Agent Reinforcement Learning using Hypergraphs for Task-Open Systems". In: *Proceedings of the Forty-first Conference on Uncertainty in Artificial Intelligence*. Vol. 286. Proceedings of Machine Learning Research. PMLR, pp. 149–171. URL: https://proceedings.mlr.press/v286/anil25a.html.

Biagioni, D., X. Zhang, D. Wald, D. Vaidhynathan, R. Chintala, J. King, and A. S. Zamzam (2022). "PowerGridworld: A Framework for Multi-Agent Reinforcement Learning in Power Systems". In: *Proceedings of the Thirteenth ACM International Conference on Future Energy Systems*. Association for Computing Machinery, pp. 565–570. DOI: 10.1145/3538637.3539616.

Blad, C., S. Bøgh, C. Kallesøe, and P. Raftery (2023). "A Laboratory Test of an Offline-Trained Multi-Agent Reinforcement Learning Algorithm for Heating Systems". In: *Applied Energy* 337, p. 120807. DOI: 10.1016/j.apenergy.2023.120807.

Blum, D., J. Arroyo, S. Huang, J. Drgoňa, F. Jorissen, H. T. Walnum, Y. Chen, K. Benne, D. Vrabie, M. Wetter, and L. Helsen (2021). "Building Optimization Testing Framework (BOPTEST) for Simulation-Based Benchmarking of Control Strategies in Buildings". In: *Journal of Building Performance Simulation* 14.5, pp. 586–610. DOI: 10.1080/19401493.2021.1986574.

Campoy-Nieves, A., A. Manjavacas, J. Jiménez-Raboso, M. Molina-Solana, and J. Gómez-Romero (2025). "Sinergym: A Virtual Testbed for Building Energy Optimization with Reinforcement Learning". In: *Energy and Buildings* 327, p. 115075. DOI: 10.1016/j.enbuild.2024.115075.

Candela, E., L. Parada, L. Marques, T.-A. Georgescu, Y. Demiris, and P. Angeloudis (2022). "Transferring Multi-Agent Reinforcement Learning Policies for Autonomous Driving Using Sim-to-Real". In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8814–8820. DOI: 10.1109/IROS47612.2022.9981319.

Cao, D. et al. (2020). "Reinforcement Learning and Its Applications in Modern Power and Energy Systems: A Review". In: *Journal of Modern Power Systems and Clean Energy* 8.6, pp. 1029–1042. DOI: 10.35833/MPCE.2020.000552.

Charbonnier, F., T. Morstyn, and M. D. McCulloch (2022). "Scalable Multi-Agent Reinforcement Learning for Distributed Control of Residential Energy Flexibility". In: *Applied Energy* 314, p. 118825. DOI: 10.1016/j.apenergy.2022.118825.

Chen, D., K. Chen, Z. Li, T. Chu, R. Yao, F. Qiu, and K. Lin (2022). "PowerNet: Multi-Agent Deep Reinforcement Learning for Scalable Powergrid Control". In: *IEEE Transactions on Power Systems* 37.2. DOI: 10.1109/TPWRS.2021.3100898.

Chen, P., S. Liu, X. Wang, and I. Kamwa (2023). "Physics-Shielded Multi-Agent Deep Reinforcement Learning for Safe Active Voltage Control with Photovoltaic/Battery Energy Storage Systems". In: *IEEE Transactions on Smart Grid* 14.4, pp. 2656–2667. DOI: 10.1109/TSG.2022.3228636.

Chen, X., G. Qu, Y. Tang, S. H. Low, and N. Li (2022). "Reinforcement Learning for Selective Key Applications in Power Systems: Recent Advances and Future Challenges". In: *IEEE Transactions on Smart Grid* 13.4, pp. 2935–2958. DOI: 10.1109/TSG.2022.3154718.

Chu, T., J. Wang, L. Codecà, and Z. Li (2020). "Multi-Agent Deep Reinforcement Learning for Large-Scale Traffic Signal Control". In: *IEEE Transactions on Intelligent Transportation Systems* 21.3. DOI: 10.1109/TITS.2019.2901791.

Chu, Y., Z. Wei, X. Fang, S. Chen, and Y. Zhou (2022). "A Multiagent Federated Reinforcement Learning Approach for Plug-In Electric Vehicle Fleet Charging Coordination in a Residential Community". In: *IEEE Access* 10, pp. 98535–98548. DOI: 10.1109/ACCESS.2022.3206020.

Claus, C. and C. Boutilier (1998). "The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems". In: *Proceedings of the Fifteenth National Conference on Artificial Intelligence*. AAAI Press, pp. 746–752.

Direção-Geral de Energia e Geologia (2023). *Consumo por Município e Tipo de Consumidor*. Official Portuguese electricity-consumption statistics. URL: https://www.dgeg.gov.pt/pt/estatistica/energia/eletricidade/consumo-por-municipio-e-tipo-de-consumidor/ (visited on 08/21/2026).

Doshi, R., H. R. Walke, O. Mees, S. Dasari, and S. Levine (2025). "Scaling Cross-Embodied Learning: One Policy for Manipulation, Navigation, Locomotion and Aviation". In: *Proceedings of the 8th Conference on Robot Learning*. Vol. 270. Proceedings of Machine Learning Research. PMLR, pp. 496–512. URL: https://proceedings.mlr.press/v270/doshi25a.html.

Dulac-Arnold, G., N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester (2021). "Challenges of Real-World Reinforcement Learning: Definitions, Benchmarks and Analysis". In: *Machine Learning* 110.9, pp. 2419–2468. DOI: 10.1007/s10994-021-05961-4.

E-REDES (2018). *Manual de Ligações à Rede Elétrica de Serviço Público*. Official connection manual. URL: https://www.e-redes.pt/sites/eredes/files/2019-02/EDPDistribui%C3%A7%C3%A3o_Manual_Ligacoes_Rede_2018_9%C2%AA_edi%C3%A7%C3%A3o.pdf (visited on 08/21/2026).

Eichelbeck, M., H. Markgraf, and M. Althoff (2026). "CommonPower: A Framework for Safe Data-Driven Smart Grid Control". In: *IEEE Transactions on Smart Grid* 17.1, pp. 71–82. DOI: 10.1109/TSG.2025.3616402.

Elsayed-Aly, I., S. Bharadwaj, C. Amato, R. Ehlers, U. Topcu, and L. Feng (2021). "Safe Multi-Agent Reinforcement Learning via Shielding". In: *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, pp. 483–491. arXiv: 2101.11196. URL: https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p483.pdf.

Entidade Reguladora dos Serviços Energéticos (2026a). *Manual do Utilizador: Simulador de Potência Contratada*. Official guidance on Portuguese low-voltage contracted-power levels. URL: https://www.erse.pt/media/02ibhrwu/manualutilizador_pt.pdf (visited on 08/21/2026).

— (2026b). *Mobilidade Elétrica: Como Funciona*. URL: https://www.erse.pt/consumidores-de-energia/mobilidade-electrica/como-funciona/ (visited on 08/21/2026).

European Parliament and Council of the European Union (2018). *Directive (EU) 2018/2001 of the European Parliament and of the Council of 11 December 2018 on the Promotion of the Use of Energy from Renewable Sources*. L 328, pp. 82–209. Official Journal of the European Union. URL: https://eur-lex.europa.eu/eli/dir/2018/2001/oj.

Faia, R., B. Ribeiro, C. Goncalves, L. Gomes, and Z. Vale (2023). "Multi-Agent Based Energy Community Cost Optimization Considering High Electric Vehicles Penetration". In: *Sustainable Energy Technologies and Assessments* 59, p. 103402. DOI: 10.1016/j.seta.2023.103402.

Fan, T.-H., X. Y. Lee, and Y. Wang (2022). "PowerGym: A Reinforcement Learning Environment for Volt-Var Control in Power Distribution Systems". In: *Proceedings of the 4th Annual Learning for Dynamics and Control Conference*. Vol. 168. Proceedings of Machine Learning Research. PMLR, pp. 21–33.

Foerster, J. N., G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson (2018). "Counterfactual Multi-Agent Policy Gradients". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 2974–2982. DOI: 10.1609/aaai.v32i1.11794.

Fonseca, T., L. L. Ferreira, B. Cabral, R. Severino, K. Nweye, D. Ghose, and Z. Nagy (2025). "EVLearn: Extending the CityLearn Framework with Electric Vehicle Simulation". In: *Energy Informatics* 8.1, p. 16. DOI: 10.1186/s42162-024-00445-w.

Fonseca, T., L. L. Ferreira, B. Cabral, R. Severino, and I. Praça (2024). "EnergAIze: Multi Agent Deep Deterministic Policy Gradient for Vehicle to Grid Energy Management". In: *arXiv preprint arXiv:2404.02361*. DOI: 10.48550/arXiv.2404.02361. arXiv: 2404.02361.

Fonseca, T., L. L. Ferreira, J. Landeck, L. Klein, P. Sousa, and F. Ahmed (2022). "Flexible Loads Scheduling Algorithms for Renewable Energy Communities". In: *Energies* 15.23, p. 8875. DOI: 10.3390/en15238875.

Fonseca, T., C. Sousa, R. Venâncio, P. Pires, R. Severino, P. Rodrigues, P. Paiva, and L. L. Ferreira (2025). "Control of Renewable Energy Communities using AI and Real-World Data". In: *arXiv preprint arXiv:2505.17321*. DOI: 10.48550/arXiv.2505.17321. arXiv: 2505.17321.

Fujimoto, S., H. van Hoof, and D. Meger (2018). "Addressing Function Approximation Error in Actor-Critic Methods". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1587–1596. URL: https://proceedings.mlr.press/v80/fujimoto18a.html.

Gonçalves, R., D. Gomes, and M. Antunes (2025). "Intelligent Management of Renewable Energy Communities: An MLaaS Framework with RL-Based Decision Making". In: *Energies* 18.13, p. 3477. DOI: 10.3390/en18133477.

Gronauer, S. and K. Diepold (2022). "Multi-Agent Deep Reinforcement Learning: A Survey". In: *Artificial Intelligence Review* 55.2, pp. 895–943. DOI: 10.1007/s10462-021-09996-w.

Gryga, L. and B. Rossi (2021). "Co-Simulation of Smart Grids: Dynamically Changing Topologies in Failure Scenarios". In: *Proceedings of the 6th International Conference on Complexity, Future Information Systems and Risk*. SciTePress, pp. 63–69. DOI: 10.5220/0010448700630069.

Gu, S., J. G. Kuba, Y. Chen, Y. Du, L. Yang, A. Knoll, and Y. Yang (2023). "Safe Multi-Agent Reinforcement Learning for Multi-Robot Control". In: *Artificial Intelligence* 319, p. 103905. DOI: 10.1016/j.artint.2023.103905.

Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine (2018). "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1861–1870. URL: https://proceedings.mlr.press/v80/haarnoja18b.html.

Hamzah, M. et al. (2023). "Distributed Control of Cyber Physical System on Various Domains: A Critical Review". In: *Systems* 11.4, p. 208. DOI: 10.3390/systems11040208.

Hardy, T. D., B. Palmintier, P. L. Top, D. Krishnamurthy, and J. C. Fuller (2024). "HELICS: A Co-Simulation Framework for Scalable Multi-Domain Modeling and Analysis". In: *IEEE Access* 12, pp. 24325–24347. DOI: 10.1109/ACCESS.2024.3363615.

Hermans, B., S. S. W. Walker, J. H. A. Ludlage, and L. Özkan (2024). "Model Predictive Control of Vehicle Charging Stations in Grid-Connected Microgrids: An Implementation Study". In: *Applied Energy* 368, p. 123210. DOI: 10.1016/j.apenergy.2024.123210.

Hernandez-Leal, P., B. Kartal, and M. E. Taylor (2019). "A Survey and Critique of Multiagent Deep Reinforcement Learning". In: *Autonomous Agents and Multi-Agent Systems* 33.6, pp. 750–797. DOI: 10.1007/s10458-019-09421-1.

Hoffman, M. W., B. Shahriari, J. Aslanides, G. Barth-Maron, F. Behbahani, T. Norman, A. Abdolmaleki, A. Cassirer, F. Yang, K. Baumli, S. Henderson, A. Novikov, S. Gómez Colmenarejo, S. Cabi, Ç. Gülçehre, T. Le Paine, A. Cowie, Z. Wang, B. Piot, and N. de Freitas (2020). "Acme: A Research Framework for Distributed Reinforcement Learning". In: *arXiv preprint arXiv:2006.00979*. arXiv: 2006.00979 [cs.LG].

Hu, S., Y. Zhong, M. Gao, W. Wang, H. Dong, X. Liang, Z. Li, X. Chang, and Y. Yang (2023). "MARLlib: A Scalable and Efficient Multi-Agent Reinforcement Learning Library". In: *Journal of Machine Learning Research* 24.315, pp. 1–23. URL: https://www.jmlr.org/papers/v24/23-0378.html.

Huang, W., I. Mordatch, and D. Pathak (2020). "One Policy to Control Them All: Shared Modular Policies for Agent-Agnostic Control". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 4455–4464. URL: https://proceedings.mlr.press/v119/huang20d.html.

Hussain, S., C. Lai, and U. Eicker (2023). "Flexibility: Literature Review on Concepts, Modeling, and Provision Method in Smart Grid". In: *Sustainable Energy, Grids and Networks* 35, p. 101113. DOI: 10.1016/j.segan.2023.101113.

Intelligent Environments Laboratory (2026). *CityLearn: A Gymnasium Environment for Demand Response*. URL: https://github.com/intelligent-environments-lab/CityLearn (visited on 08/20/2026).

Jain, A., A. Szot, and J. J. Lim (2020). "Generalization to New Actions in Reinforcement Learning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 4661–4672. URL: https://proceedings.mlr.press/v119/jain20b.html.

Kaelbling, L. P., M. L. Littman, and A. R. Cassandra (1998). "Planning and Acting in Partially Observable Stochastic Domains". In: *Artificial Intelligence* 101.1–2, pp. 99–134. DOI: 10.1016/S0004-3702(98)00023-X.

Kostrikov, I., A. Nair, and S. Levine (2022). "Offline Reinforcement Learning with Implicit Q-Learning". In: *International Conference on Learning Representations*. arXiv: 2110.06169. URL: https://openreview.net/forum?id=68n2s9ZJWF8.

Kreuzberger, D., N. Kühl, and S. Hirschl (2023). "Machine Learning Operations (MLOps): Overview, Definition, and Architecture". In: *IEEE Access* 11, pp. 31866–31879. DOI: 10.1109/ACCESS.2023.3262138.

Kuba, J. G., R. Chen, M. Wen, Y. Wen, F. Sun, J. Wang, and Y. Yang (2022). "Trust Region Policy Optimisation in Multi-Agent Reinforcement Learning". In: *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=EcGGFkNTxdJ.

Kumar, A., A. Zhou, G. Tucker, and S. Levine (2020). "Conservative Q-Learning for Offline Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 33. DOI: 10.48550/arXiv.2006.04779. arXiv: 2006.04779.

Lee, E. A. (2008). "Cyber Physical Systems: Design Challenges". In: *2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, pp. 363–369. DOI: 10.1109/ISORC.2008.25.

Lee, Z. J., S. Sharma, D. Johansson, and S. H. Low (2021). "ACN-Sim: An Open-Source Simulator for Data-Driven Electric Vehicle Charging Research". In: *IEEE Transactions on Smart Grid* 12.6, pp. 5113–5123. DOI: 10.1109/TSG.2021.3103156.

Levine, S., A. Kumar, G. Tucker, and J. Fu (2020). *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*. DOI: 10.48550/arXiv.2005.01643. arXiv: 2005.01643 [cs.LG].

Li, B., H. Tang, Y. Zheng, J. Hao, P. Li, Z. Wang, Z. Meng, and L. Wang (2022). "HyAR: Addressing Discrete-Continuous Action Reinforcement Learning via Hybrid Action Representation". In: *International Conference on Learning Representations*. arXiv: 2109.05490. URL: https://openreview.net/forum?id=wQkaGq7Vz6q.

Li, P., J. D. Thomas, X. Wang, A. Khalil, A. K. Ahmad, R. Inacio, S. Kapoor, A. Parekh, A. Doufexi, A. Shojaeifard, and R. J. Piechocki (2022). "RLOps: Development Life-Cycle of Reinforcement Learning Aided Open RAN". In: *IEEE Access* 10, pp. 113808–113826. DOI: 10.1109/ACCESS.2022.3217511.

Liang, E., R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan, and I. Stoica (2018). "RLlib: Abstractions for Distributed Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 3053–3062. URL: https://proceedings.mlr.press/v80/liang18b.html.

Littman, M. L. (1994). "Markov Games as a Framework for Multi-Agent Reinforcement Learning". In: *Proceedings of the Eleventh International Conference on Machine Learning*. Morgan Kaufmann, pp. 157–163. DOI: 10.1016/B978-1-55860-335-6.50027-1.

Liu, G. H., A. Siravuru, S. Prabhakar, M. Veloso, and G. Kantor (2017). "Learning End-to-End Multimodal Sensor Policies for Autonomous Navigation". In: *Proceedings of the 1st Conference on Robot Learning*. Vol. 78. Proceedings of Machine Learning Research. PMLR, pp. 249–261. URL: https://proceedings.mlr.press/v78/liu17a.html.

Lowe, R., Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch (2017). "Multi-Agent Actor-Critic for Mixed Cooperative–Competitive Environments". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 6379–6390.

Luketina, J., J. Lanchantin, S. Sukhbaatar, and A. Szlam (2025). "Compositional Interfaces for Compositional Generalization". In: *Proceedings of the 3rd Conference on Lifelong Learning*

*Agents*. Vol. 274. Proceedings of Machine Learning Research. PMLR, pp. 692–709. URL: https://proceedings.mlr.press/v274/luketina25a.html.

Nagy, Z., J. R. Vázquez-Canteli, S. Dey, and G. Henze (2021). "The CityLearn Challenge 2021". In: *Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. Association for Computing Machinery, pp. 218–219. DOI: 10.1145/3486611.3492226.

Nweye, K., K. Kaspar, G. Buscemi, T. Fonseca, G. Pinto, D. Ghose, S. Duddukuru, P. Pratapa, H. Li, J. Mohammadi, L. L. Ferreira, T. Hong, M. Ouf, A. Capozzoli, and Z. Nagy (2025). "CityLearn v2: Energy-Flexible, Resilient, Occupant-Centric, and Carbon-Aware Management of Grid-Interactive Communities". In: *Journal of Building Performance Simulation* 18.1, pp. 17–38. DOI: 10.1080/19401493.2024.2418813.

Nweye, K., S. Sankaranarayanan, and Z. Nagy (2023). "MERLIN: Multi-Agent Offline and Transfer Learning for Occupant-Centric Operation of Grid-Interactive Communities". In: *Applied Energy* 346, p. 121323. DOI: 10.1016/j.apenergy.2023.121323.

Oliehoek, F. A. and C. Amato (2016). *A Concise Introduction to Decentralized POMDPs*. Cham: Springer. DOI: 10.1007/978-3-319-28929-8.

OMIE (2023). *Day-Ahead Market Hourly Price in Portugal: Official File Archive*. URL: https://www.omie.es/en/file-access-list?dir=+Day-ahead+market+hourly+price+in+Portugal&parents%5B0%5D=%2F&parents%5B1%5D=Day-ahead+Market&parents%5B2%5D=1.+Prices&realdir=marginalpdbcpt (visited on 08/21/2026).

— (Oct. 1, 2025). *The Day-Ahead Market Begins Trading in 15-Minute Periods*. Official market notice. URL: https://www.omie.es/sites/default/files/2025-10/2501001_go_live_mtu15_md_en_vf.pdf (visited on 08/21/2026).

Orfanoudakis, S., C. Diaz-Londono, Y. E. Yılmaz, P. Palensky, and P. P. Vergara (2025). "EV2Gym: A Flexible V2G Simulator for EV Smart Charging Research and Benchmarking". In: *IEEE Transactions on Intelligent Transportation Systems* 26.2, pp. 2410–2421. DOI: 10.1109/TITS.2024.3510945.

Orfanoudakis, S., N. K. Panda, P. Palensky, and P. P. Vergara (2025). "GNN-DT: Graph Neural Network Enhanced Decision Transformer for Efficient Optimization in Dynamic Environments". In: *arXiv preprint arXiv:2502.01778*. DOI: 10.48550/arXiv.2502.01778. arXiv: 2502.01778.

Patrício, J. T., T. Lopes, A. Rocha, J. Martins, and R. A. Lopes (2026). "Development and Real-World Deployment of a Multi-Agent System to Support the Operation of Renewable Energy Communities". In: *Energy Reports* 15, p. 108967. DOI: 10.1016/j.egyr.2025.108967.

Pigott, A., C. Crozier, K. Baker, and Z. Nagy (2022). "GridLearn: Multiagent Reinforcement Learning for Grid-Aware Building Energy Management". In: *Electric Power Systems Research* 213, p. 108521. DOI: 10.1016/j.epsr.2022.108521.

Ponse, K., J. F. Kleuker, T. M. Moerland, and A. Plaat (2025). "Chargax: A JAX-Accelerated EV Charging Simulator". In: *Reinforcement Learning Journal* 6, pp. 364–383.

Pordata (2023). *Consumo de Energia Elétrica por Consumidor: Total e por Tipo de Consumo*. Series based on DGEG data; 2023 values provisional. URL: https://www.pordata.pt/portugal/consumo+de+energia+eletrica+por+consumidor+total+e+por+tipo+de+consumo-1231 (visited on 08/21/2026).

Puech, A., T. Rigaut, W. Templier, and M. Tournoud (2024). "Controlling Large Electric Vehicle Charging Stations via User Behavior Modeling and Stochastic Programming". In: *arXiv preprint arXiv:2402.13224*. DOI: 10.48550/arXiv.2402.13224. arXiv: 2402.13224.

Rashid, T., M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson (2018). "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research, pp. 4295–4304. URL: https://proceedings.mlr.press/v80/rashid18a.html.

Reis, I. F. G., I. Gonçalves, M. A. R. Lopes, and C. H. Antunes (2021). "Business Models for Energy Communities: A Review of Key Issues and Trends". In: *Renewable and Sustainable Energy Reviews* 144, p. 111013. DOI: 10.1016/j.rser.2021.111013.

Ross, S., G. J. Gordon, and J. A. Bagnell (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Ed. by G. Gordon, D. Dunson, and M. Dudík. Vol. 15. Proceedings of Machine Learning Research. PMLR, pp. 627–635. URL: https://proceedings.mlr.press/v15/ross11a.html.

Rusu, A. A., S. G. Colmenarejo, C. Gulcehre, G. Desjardins, J. Kirkpatrick, R. Pascanu, V. Mnih, K. Kavukcuoglu, and R. Hadsell (2015). *Policy Distillation*. DOI: 10.48550/arXiv.1511.06295. arXiv: 1511.06295 [cs.LG].

Sadeghianpourhamami, N., J. Deleu, and C. Develder (2020). "Definition and Evaluation of Model-Free Coordination of Electrical Vehicle Charging With Reinforcement Learning". In: *IEEE Transactions on Smart Grid* 11.1, pp. 203–214. DOI: 10.1109/TSG.2019.2920320.

Salvan, L. and OPEVA Consortium (Apr. 2026). *OPEVA D6.3: Presentation of All Final Demonstrators*. Tech. rep. D6.3. Version 1.0, approved 22 April 2026. Chips Joint Undertaking Project 101097267 — OPEVA.

Schaul, T., J. Quan, I. Antonoglou, and D. Silver (2016). "Prioritized Experience Replay". In: *International Conference on Learning Representations*. arXiv: 1511.05952. URL: https://arxiv.org/abs/1511.05952.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). *Proximal Policy Optimization Algorithms*. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs.LG].

Sculley, D., G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison (2015). "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems*. Vol. 28, pp. 2503–2511.

Sebastián, E., T. Duong, N. Atanasov, E. Montijano, and C. Sagüés (2025). "Physics-Informed Multiagent Reinforcement Learning for Distributed Multirobot Problems". In: *IEEE Transactions on Robotics* 41, pp. 4499–4517. DOI: 10.1109/TRO.2025.3582836.

Shapley, L. S. (1953). "Stochastic Games". In: *Proceedings of the National Academy of Sciences* 39.10, pp. 1095–1100. DOI: 10.1073/pnas.39.10.1095.

Shi, W., J. Cao, Q. Zhang, Y. Li, and L. Xu (2016). "Edge Computing: Vision and Challenges". In: *IEEE Internet of Things Journal* 3.5, pp. 637–646. DOI: 10.1109/JIOT.2016.2579198.

Shibata, K., T. Jimbo, and T. Matsubara (2023). "Deep Reinforcement Learning of Event-Triggered Communication and Consensus-Based Control for Distributed Cooperative Transport". In: *Robotics and Autonomous Systems* 159, p. 104307. DOI: 10.1016/j.robot.2022.104307.

Shojaeighadikolaei, A., Z. Talata, and M. Hashemi (2024). "Centralized vs. Decentralized Multi-Agent Reinforcement Learning for Enhanced Control of Electric Vehicle Charging Networks". In: *arXiv preprint arXiv:2404.12520*. DOI: 10.48550/arXiv.2404.12520. arXiv: 2404.12520.

Sinii, V., A. Nikulin, V. Kurenkov, I. Zisman, and S. Kolesnikov (2024). "In-Context Reinforcement Learning for Variable Action Spaces". In: *Proceedings of the 41st International Conference on*

*Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 45773–45793. URL: https://proceedings.mlr.press/v235/sinii24a.html.

SoftCPS Research Group (2026a). *Simulator: Research Fork of CityLearn for Renewable Energy Community Studies*. URL: https://github.com/Soft-CPS-Research-Group/Simulator (visited on 08/20/2026).

— (Aug. 16, 2026b). *SoftCPS REC Simulator*. Version 1.7.0. Python package. URL: https://pypi.org/project/softcpsrecsimulator/ (visited on 08/20/2026).

Sousa, C., T. Fonseca, L. L. Ferreira, R. Venâncio, and R. Severino (2025). "Percepta: High Performance Stream Processing at the Edge". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 1–4. DOI: 10.1109/ETFA65518.2025.11205806.

Srithapon, C. and D. Månsson (2023). "Predictive Control and Coordination for Energy Community Flexibility with Electric Vehicles, Heat Pumps and Thermal Energy Storage". In: *Applied Energy* 347, p. 121500. DOI: 10.1016/j.apenergy.2023.121500.

Su, H., K. Shi, J. Y. J. Chow, and L. Jin (2020). *Dynamic Queue-Jump Lane for Emergency Vehicles under Partially Connected Settings: A Multi-Agent Deep Reinforcement Learning Approach*. DOI: 10.48550/arXiv.2003.01025. arXiv: 2003.01025.

Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. URL: https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

Tan, M. (1993). "Multi-Agent Reinforcement Learning: Independent versus Cooperative Agents". In: *Proceedings of the Tenth International Conference on Machine Learning*. Morgan Kaufmann, pp. 330–337. DOI: 10.1016/B978-1-55860-307-3.50049-6.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., pp. 5998–6008. URL: https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need.

Vázquez-Canteli, J., J. Kämpf, G. Henze, and Z. Nagy (2019). "CityLearn v1.0: An OpenAI Gym Environment for Demand Response with Deep Reinforcement Learning". In: *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 356–357. DOI: 10.1145/3360322.3360998.

Vázquez-Canteli, J. R. and Z. Nagy (2019). "Reinforcement Learning for Demand Response: A Review of Algorithms and Modeling Techniques". In: *Applied Energy* 235, pp. 1072–1089. DOI: 10.1016/j.apenergy.2018.11.002.

Wang, J. et al. (2022). "Cooperative and Competitive Multi-Agent Systems: From Optimization to Games". In: *IEEE/CAA Journal of Automatica Sinica* 9.5, pp. 763–783. DOI: 10.1109/JAS.2022.105506.

Wang, X., N. Nimalsiri, Y. Xia, P. Wang, and S. Halgamuge (2026). "HetGPS: Scalable Graph Multi-Agent Reinforcement Learning with Physics-Anchored Adaptive Safety for EV Charging". In: *arXiv preprint arXiv:2608.00679*. Preprint, released August 2026. DOI: 10.48550/arXiv.2608.00679. arXiv: 2608.00679.

Wilk, P., N. Wang, and J. Li (2024). "Multi-Agent Reinforcement Learning for Smart Community Energy Management". In: *Energies* 17.20, p. 5211. DOI: 10.3390/en17205211.

Wu, A., K. Nweye, and Z. Nagy (2023). "Heterogeneous Multi-Agent Reinforcement Learning for Grid-Interactive Communities". In: *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 314–315. DOI: 10.1145/3600100.3626276.

Wu, H., D. Qiu, L. Zhang, and M. Sun (2024). "Adaptive Multi-Agent Reinforcement Learning for Flexible Resource Management in a Virtual Power Plant with Dynamic Participating Multi-Energy Buildings". In: *Applied Energy* 374, p. 123998. DOI: 10.1016/j.apenergy.2024.123998.

Xiong, J., Q. Wang, Z. Yang, P. Sun, L. Han, Y. Zheng, H. Fu, T. Zhang, J. Liu, and H. Liu (2018). *Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space*. DOI: 10.48550/arXiv.1810.06394. arXiv: 1810.06394 [cs.LG].

Yeh, C., V. Li, R. Datta, J. Arroyo, N. Christianson, C. Zhang, Y. Chen, M. M. Hosseini, A. Golmohammadi, Y. Shi, Y. Yue, and A. Wierman (2023). "SustainGym: Reinforcement Learning Environments for Sustainable Energy Systems". In: *Advances in Neural Information Processing Systems*. Vol. 36, pp. 59464–59476.

Yu, C., A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu (2022). "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games". In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 24611–24624. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets_and_Benchmarks.html.

Yu, C., X. Yang, J. Gao, J. Chen, Y. Li, J. Liu, Y. Xiang, R. Huang, H. Yang, Y. Wu, and Y. Wang (2023). "Asynchronous Multi-Agent Reinforcement Learning for Efficient Real-Time Multi-Robot Cooperative Exploration". In: *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pp. 1107–1115. arXiv: 2301.03398. URL: https://arxiv.org/abs/2301.03398.

Yu, X., Y. Lin, S. Wang, K. Lv, and S. Han (2024). *Improving Global Parameter-sharing in Physically Heterogeneous Multi-agent Reinforcement Learning with Unified Action Space*. DOI: 10.48550/arXiv.2408.07395. arXiv: 2408.07395 [cs.MA].

Zaharia, M., A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar (2018). "Accelerating the Machine Learning Lifecycle with MLflow". In: *IEEE Data Engineering Bulletin* 41.4, pp. 39–45.

Zhang, Q., K. Dehghanpour, Z. Wang, F. Qiu, and D. Zhao (2021). "Multi-Agent Safe Policy Learning for Power Management of Networked Microgrids". In: *IEEE Transactions on Smart Grid* 12.2, pp. 1048–1062. DOI: 10.1109/TSG.2020.3034827.

Zhang, Y., Z. Zhang, J. Ji, M. Quiñones-Grueiro, W. Barbour, D. Gloudemans, G. Zachár, C. Weston, G. Biswas, and D. B. Work (2025). "Real-World Deployment and Assessment of a Multi-Agent Reinforcement Learning-Based Variable Speed Limit Control System". In: *IEEE Intelligent Transportation Systems Magazine*. DOI: 10.1109/MITS.2025.3592040. arXiv: 2503.01017.

Zhang, Y., Z. Zhang, M. Quiñones-Grueiro, W. Barbour, C. Weston, G. Biswas, and D. B. Work (2024). "Field Deployment of Multi-Agent Reinforcement Learning Based Variable Speed Limit Controllers". In: *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*, pp. 776–783. DOI: 10.1109/ITSC58415.2024.10919506. arXiv: 2407.08021.

Zhou, M., Z. Wan, H. Wang, M. Wen, R. Wu, Y. Wen, Y. Yang, Y. Yu, J. Wang, and W. Zhang (2023). "MALib: A Parallel Framework for Population-Based Multi-Agent Reinforcement Learning". In: *Journal of Machine Learning Research* 24.150, pp. 1–12. URL: https://www.jmlr.org/papers/v24/22-0169.html.

Zhu, D., B. Yang, Y. Liu, Z. Wang, K. Ma, and X. Guan (2022). "Energy Management Based on Multi-Agent Deep Reinforcement Learning for a Multi-Energy Industrial Park". In: *Applied Energy* 311, p. 118636. DOI: 10.1016/j.apenergy.2022.118636.

Zhu, Z., R. d. S. Braz, J. Bhandari, D. Jiang, Y. Wan, Y. Efroni, L. Wang, R. Xu, H. Guo, A. Nikulkov, D. Korenkevych, U. Dogan, F. Cheng, Z. Wu, and W. Xu (2024). "Pearl: A Production-Ready Reinforcement Learning Agent". In: *Journal of Machine Learning Research* 25.273, pp. 1–30. URL: https://www.jmlr.org/papers/v25/24-0196.html.

# Appendix A

# Reproducibility and Supplementary Material

This appendix organises the material required to reproduce and audit the principal findings without interrupting the main argument.

## A.1 Claim and Evidence Inventory

A master inventory maps every retained claim to the relevant research question, method, repository and commit, scenario, dataset, experiment identifier, seeds, metrics, result table or figure, evidence level, publication, author contribution, and known limitation. This mapping defines the evidence path used throughout Chapters 4–9.

## A.2 Scenario and Environment Manifests

Reconstruction of the principal experiments requires schemas or manifests that record temporal resolution, units, buildings, assets, data sources, event streams, constraints, observation and action interfaces, reward definitions, KPI configuration, software versions, and referenced data checksums.

## A.3 Canonical Annual Dataset Contract

This section freezes the human-readable contract for the annual suite used in Chapters 4–6. The row-level record remains authoritative. Each family contains `dataset_manifest.json`, `settlement_contract.json`, variant schemas, Parquet catalogues for members, electrical services, chargers, EVs, sessions, and deferrables, event files, daily OMIE provenance, validation output, and `file_checksums.sha256`. The generator seed is 2023 and the contract version is `2023-q15-v1.9`. All families span 35,040 consecutive 900-second UTC intervals in 2023 and retain `Europe/Lisbon` local-calendar and daylight-saving attributes. Routine instants are

first constructed in Lisbon civil time and only then converted to UTC; the missing spring hour is shifted forward by one hour and the first occurrence of the repeated autumn hour is selected deterministically. A complete annual controller run appends an observation-only terminal boundary at 1 January 2024 00:00, yielding 35,041 state points and exactly 35,040 action transitions without adding a physical energy or settlement row.

### A.3.1 Calendar, price forecasts, and settlement

The current-price column is the official Portuguese OMIE 2023 day-ahead series converted from EUR/MWh to EUR/kWh. Each physical hourly value is repeated over four 15-minute steps; the 23- and 25-period daylight-saving days are preserved by the continuous UTC calendar. Forecast columns target 1, 6, and 24 hours. For a delivery interval whose day-ahead result is available by the declared conservative 13:00 local cut-off, the forecast contains that published value; before the cut-off it contains the corresponding previous-day value. The entity bundle also exposes 15-minute and 3-hour horizons. These use the shortest stored horizon capable of targeting the same delivery step, read at the corresponding earlier issue time. At an episode cold start with no such issue in the retained window, the current price is used. This rule is tested independently against the controller-visible observation and prevents the interface from reconstructing an unpublished realised price from the full annual table.

Settlement matches member surplus and demand within the same step, uses equal importer weights and a local price equal to 80% of the grid-import price, and assigns zero remuneration to residual grid export. The retained grid-only counterfactual disables local matching but preserves the same member demand, generation, actions, and OMIE price. Taxes, network tariffs, supplier margins, and regulated export compensation are outside this experimental accounting contract.

### A.3.2 Demand, PV, and stationary storage

Demand profiles combine class- and routine-specific intraday structure, seasonal variation, correlated daily multipliers, local peak-time shifts, reduced-activity days, and correlated member-level short-term variation. The raw trajectory is scaled to an annual target sampled by rejection from the bounded log-normal ranges in Table A.1; this avoids an artificial point mass at either bound. The class centres are 2,490, 16,580, and 175,900 kWh per consumer-year and are anchored to Portuguese 2023 residential, non-domestic, and industrial averages (Direção-Geral de Energia e Geologia 2023; Pordata 2023). The ranges and routine frequencies create controlled scenario heterogeneity; they are not empirical confidence intervals or inferred population proportions.

Table A.2 lists the behaviour strata. The assignment is deterministic and family-independent, so the first 15 physical members remain identical between Core-15-Stripped and Core-30. Within an archetype, member-specific amplitudes, centres, activity reductions, annual targets, and short-term variation remain independently reproducible. Occupancy varies with the routine: commuter homes

Table A.1: Static-asset generation contract by member class. PV and BESS entries apply only to members that own the corresponding asset.

| Member class | Annual demand target | PV contract | Stationary BESS contract |
|---|---|---|---|
| Residential | 1.1–6.5 MWh; centre 2.49 MWh | 1.5–8.0 kW; target annual coverage 0.58–1.32 | 5, 7.5, 9.6, 10, 13.5, or 15 kWh; 2.5–6 kW product classes |
| Small service/commercial | 7–45 MWh; centre 16.58 MWh | 8–40 kW; target annual coverage 0.42–1.02 | 20, 30, 40, 50, 60, or 75 kWh; 10–30 kW product classes |
| Larger consumption | 75–350 MWh; centre 175.9 MWh | 30–150 kW; target annual coverage 0.32–0.82 | 60, 80, 100, 120, 160, or 200 kWh; 30–80 kW product classes |

are largely unoccupied during the working day, home-working and home-day members retain daytime occupancy, and service occupancy follows its opening or process window.

Table A.2: Routine archetypes used to construct heterogeneous demand and flexibility.

| Member class | Five stratified archetypes | Coupled effects |
|---|---|---|
| Residential | Early commuter; standard commuter; late commuter; home worker; predominantly home-day | Morning/evening/lunch balance, weekday absence, weekend profile, reduced-activity days, EV arrival/departure, and appliance request time |
| Small service/commercial | Office hours; six-day retail; split-day clinic; extended hospitality; early workshop | Opening days and hours, split or extended operation, occupancy, customer/employee charging windows, and type/timing of the deferrable service |
| Larger consumption | Day process; extended process; continuous process; morning-intensive process; evening-intensive process | Shift coverage, weekend reduction, maintenance days, shared-charger visits, and batch/process flexibility windows |

PV capacity is obtained from the annual demand target, the common-profile specific yield, and the coverage range in Table A.1, subject to the stated class bounds. Orientation is sampled as east, south, or west with probabilities 0.25, 0.50, and 0.25; derating is sampled between 0.84 and 0.98 and combined with reproducible member-level shading. Generation is stored as absolute per-step energy. Stationary BESS capacity is selected from the product-like classes in the table by a stratified fraction of average daily demand, with a lower sizing floor tied to host PV. Nominal power is selected from the compatible product class and does not exceed 0.65 times energy capacity. Efficiency is sampled from 0.920–0.965, initial SoC from 0.25–0.60, depth of discharge from 0.80, 0.85, 0.90, or 0.95, standing-loss coefficient from 0.0005–0.0015, and capacity-loss coefficient from $0.5 \times 10^{-5}$–$1.5 \times 10^{-5}$. A single-phase battery follows its member's assigned phase; a battery at a three-phase member is connected across all phases. These are benchmark parameter classes, not a market-share model.

### A.3.3 Electrical connections and phase allocation

Table A.3 gives the rule used to construct member-level connection limits. The levels reproduce Portuguese connection classes at the resolution required by the simulator (E-REDES 2018; Entidade Reguladora dos Serviços Energéticos 2026a). CityLearn advances active energy, so each kVA value is mapped to the same numerical kW ceiling under an explicit unity-power-factor assumption. For a balanced three-phase connection, each phase receives one third of the total ceiling; for a

single-phase connection, the complete ceiling is assigned to one of L1–L3. Residential single-phase assignments rotate deterministically across members. The export ceiling is $\min(I_b^{\max}, P_b^{\mathrm{PV}} + P_b^{\mathrm{BESS}})$ and is divided using the same phase rule. This is a connection-headroom benchmark, not a voltage, reactive-power, protection, or feeder-power-flow model.

Table A.3: Electrical-service and charger contract. Charger powers are compatible with the charging classes described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b).

| Configuration | Contracted levels (kVA) | Installed charger power | Phase rule |
|---|---|---|---|
| Residential, ordinary single-phase | 3.45, 4.60, 5.75, 6.90, or 10.35 | none, 3.7 kW, or 7.4 kW | Member and charger assigned wholly to a rotated L1, L2, or L3 community phase |
| Residential, one three-phase charger | 13.80 or 17.25 | one 11 kW charger | Uncontrollable demand and charger divided over L1–L3 |
| Residential, two chargers | 17.25 or 20.70 | two independently controlled 7.4 kW chargers | Chargers connected to two rotated individual phases; shared total and per-phase limits |
| Small service/commercial | 20.70, 27.60, 34.50, or 41.40 | two 11 kW chargers where equipped; 5.5 kW V2G limit | Balanced three-phase connection; each charger spans all phases |
| Larger consumption | 69.0, 103.5, or 138.0 | four or six 22 kW chargers where equipped; 11 kW V2G limit | Balanced three-phase BTE-level surrogate; each charger spans all phases |

The member-level catalogue records connection type, contracted kVA, the originally sampled class, active-power assumption, native demand and export peaks, total import/export limits, L1–L3 limits, default split, and asset phase connection. The charger catalogue separately records each physical charger and its phase. This permits two chargers at one member to receive independent actions while remaining jointly constrained by one connection. A rejection check increases the connection class when the uncontrolled native trace would otherwise exceed the sampled level before any flexible action is applied. This affects only two Premium members: Member 094 is raised from 69 to 138 kVA for native peaks of 111.5233 kW import and 97.5324 kW export, and Member 100 is raised from 69 to 103.5 kVA for a 99.6939 kW import peak. The original level and upgrade flag remain in the schema; no other family member requires this correction.

Electrical reporting distinguishes the pre-projection pressure of the requested action from the post-projection residual of the applied trajectory. Both are integrated as energy and counted as events. The residual calculation is reconstructed from total and phase power histories with a $10^{-5}$ kW numerical tolerance and is the hard safety criterion. Requested pressure remains a controller diagnostic and can be non-zero while the residual is zero. Robustness reporting follows an analogous identity rule: observation, forecast, and action corruptions retain field- or port-level counts, whereas asset-unavailable duration counts each physical asset once per affected time step, irrespective of the number of internal fields that expose its status.

### A.3.4 EVs, sessions, and deferrable services

EV capacities are sampled from 40, 50, 58, 64, 75, and 82 kWh, with 0.90 depth of discharge and initial SoC between 0.30 and 0.60. Residential chargers use one persistent vehicle identity;

shared service chargers use pools of four vehicles. Residential visit probability, arrival centre, following-morning departure centre, temporal spread, and trip-energy scale depend on the host routine and differ between weekdays and weekends. Service and larger-consumption chargers use two to four routine-dependent probabilistic windows on their eligible operating days. Independent charger-level draws allow two chargers at the same member to receive different sessions. All times are jittered and rounded to the 15-minute contract; cross-midnight normalisation prevents two sessions from occupying one physical charger simultaneously.

Required session energy is sampled from class-conditional log-normal scenario distributions with a 6 kWh residential base median multiplied by a 0.66–1.12 routine factor, a 9 kWh small-service median, and a 15 kWh larger-consumption median. It is limited to 85% of $\Delta t\, P^{\max} \eta$ for residential sessions, 72% for shared service sessions, and 72% of EV capacity before arrival and target SoC are written. The catalogue also records the minimum full-power steps and residual temporal slack. Every ordinary session is individually deliverable at 95% charger efficiency. Joint service can nevertheless become infeasible after total/phase limits, concurrent sessions, topology changes, demand-response obligations, or injected failures are introduced. Table A.4 freezes the generated catalogue counts and the zero-infeasibility audit result.

Every occupied charger row additionally stores the explicit session identity and an optional current-SoC boundary reference. The reference follows the neutral constant-rate trajectory from the declared arrival SoC to the departure target and is consulted only when an episode, member, charger, or new session becomes active inside an occupied interval. It prevents an arbitrary catalogue initial SoC from being substituted for omitted history; it does not overwrite the controller-driven SoC during an uninterrupted session. This rule covers the 0, 3, 4, and 69 back-to-back session boundaries in Micro, Core-15, Core-30, and Premium, respectively, including the case in which the same residential EV begins the new session and charger occupancy remains continuous. The audit verifies that session identity agrees with the catalogue, the reference is finite and monotonic while connected, absent while disconnected, equal to arrival SoC at the session boundary, and leaves a deliverable remainder at every intermediate step. At runtime, a separate conservation check sums positive connected-session changes in stored EV energy and compares them with charger input. Any SoC gain not supported by metered input, beyond numerical tolerance, fails the controller-level audit.

Table A.4: Generated EV identities, annual session counts, and topology-observable departure obligations. A censored session is absent from the controller's service denominator because its member or physical charger does not exist at the relevant boundary.

| **Family/variant** | **EVs** | **Sessions** | **Observable** | **Censored** |
|---|---|---|---|---|
| Micro-4-Q | 2 | 554 | 554 | 0 |
| Core-15-Stripped | 7 | 1,757 | 1,757 | 0 |
| Core-30 static | 15 | 3,774 | 3,774 | 0 |
| Core-30 Dynamic/Combined | 15 | 3,774 | 3,690 | 84 |
| Premium-100 Clean/AllIn | 212 | 41,173 | 28,108 | 13,065 |

All 47,258 distinct catalogue sessions across the four families are individually feasible before shared constraints and injected events are applied. For the dynamic variants, censoring is split by cause without double counting: Core-30 has 56 member-inactive and 28 charger-inactive sessions; Premium-100 has 13,048 and 17, respectively. The annual controller audit reconstructs these counts from the topology calendar and requires the recorded departure-event denominator to match. This separates an unavailable catalogue service from a failure to serve an obligation that the controller actually faced.

Each deferrable asset alternates between two complete cycle profiles and receives a member-specific energy multiplier from 0.82 to 1.22. Residential appliance types comprise washing machine, dishwasher, washer–dryer, domestic-hot-water cycle, and a mixed dishwasher/laundry class. Service types comprise cleaning or preconditioning, cold storage or cleaning, sterilisation or laundry, hospitality dishwashing or hot water, and compressor or pumping. Larger-member types comprise batch process, process auxiliaries, maintenance or pumping, thermal preparation, and late batch process. Requests occur probabilistically on routine-eligible days. Residential occurrence probabilities range from 0.52 to 0.70 and `must_run` probabilities from 0.66 to 0.76; the corresponding ranges are 0.82–0.90 and 0.88–0.94 for small services, and 0.90–0.94 and 0.94–0.97 for larger members. Earliest start, window length, completion margin, deadline, and priority vary reproducibly by asset and day. A new request is omitted if its earliest admissible start would overlap the preceding request's completion window for the same appliance. The schedule retains cycle identity, earliest/latest start, deadline, priority, and mandatory-service flag so completion, delay, and unserved energy can be evaluated.

The complete Micro, Core-15, Core-30, and Premium catalogues contain 214, 928, 1,892, and 7,360 deferrable cycles, respectively. All are observable in the static variants. Under the declared dynamic topology, Core-30 has 1,657 fully observable cycles, 234 whose start window occurs entirely while the member or appliance is absent, and one whose full start window is visible but whose completion interval crosses an asset-removal boundary. Premium has 5,950, 1,409, and one in the same categories. The final category is policy-dependent because the cycle can be completed before removal or interrupted by it; the annual audit accepts the corresponding completed-plus-missed denominator only within the explicit one-cycle interval. No-start cases are topology-censored, not unserved requests.

### A.3.5 Topology and robustness calendars

Topology events change whether an entity exists in the active scenario; robustness events leave it in the topology but alter data, commands, or availability. Event logs and controller-facing masks preserve these semantics. Reinstallation creates a new runtime instance from the declared template and initializes a stateful asset at its boundary state, while the pre-removal instance remains in historical KPI aggregation. It does not imply unobserved state continuity through the interval in which the asset is absent. Tables A.5 and A.6 summarise the UTC topology calendar; exact time steps, target and source identifiers, and restoration templates remain in each variant schema.

An `add_asset` event constructs a new runtime instance from the immutable catalogue template. Paired removal/restoration entries have replacement or recommissioning semantics: the restored capability starts from its declared initial state, and no hidden SoC, degradation, or unfinished-cycle continuity is inferred across the absence. Temporary loss of the same equipment is modelled through a robustness-unavailability event; the entity identity and accumulated state remain present while the selected telemetry or control channel is unavailable. These semantics avoid attributing an undocumented state-reset convention to a controller result.

Table A.5: Core-30 dynamic topology calendar. Paired dates denote removal and restoration.

| **UTC date/time** | **Target** | **Change** |
|---|---|---|
| 01 Feb 06:00 | Member 016 | First entry |
| 04 Mar 12:00 / 04 Apr 18:00 | Member 006, charger 02 | Removal / restoration |
| 26 May 20:00 / 06 Jun 06:00 | Member 003, BESS | Removal / restoration |
| 16 Jun 16:00 | Member 027 | First entry |
| 18 Aug 04:00 / 28 Aug 14:00 | Member 017, deferrable | Removal / restoration |
| 08 Sep 00:00 | Member 029 | First entry |
| 28 Sep 20:00 / 09 Oct 06:00 | Member 018, PV | Removal / restoration |
| 19 Oct 16:00 | Member 030 | First entry |
| 09 Nov 12:00 | Member 019 | Permanent exit |
| 10 Dec 18:00 | Member 008 | Permanent exit |

Table A.6: Premium-100 dynamic topology calendar. Member ranges denote simultaneous first-entry cohorts.

| **UTC date/time** | **Target** | **Change** |
|---|---|---|
| 31 Jan / 02 Mar / 01 Apr | Members 061–065 / 081–085 / 091–093 | First-entry cohorts |
| 15 Apr / 25 Apr | Member 025, charger 02 | Removal / restoration |
| 31 May / 30 Jun | Members 066–070 / 086–090 | First-entry cohorts |
| 07 Jul / 17 Jul | Member 012, PV | Removal / restoration |
| 28 Jul / 02 Aug | Member 001, BESS | Removal / restoration |
| 30 Jul / 29 Aug | Members 094–096 / 097–100 | First-entry cohorts |
| 07 Aug / 18 Aug | Member 071, charger 02 | Removal / restoration |
| 08 Sep / 18 Sep | Member 073, deferrable | Removal / restoration |
| 28 Sep | Members 005, 006, 072 | Permanent exits |
| 27 Nov | Members 020–022 | Permanent exits |

Health and AllIn event files cover missing non-controllable-load observations, biased price forecasts, delayed storage actions, and temporary charger unavailability. Premium-100-AllIn also includes PV telemetry loss, charger-command dropout, a service-charger outage, a stationary-storage outage, and selected grid-outage intervals. Four non-overlapping demand-response requests cover upward and downward directions, different durations, tolerances, activation prices, and shortfall penalties; the Premium power targets are three times the Core-30 targets. These controlled scenario interventions carry no empirical incident-frequency claim.

The final structural audit verifies cadence and daylight saving, explicit civil-time localization policy, exact mapping between session timestamps and quarter-hour indices, the 365 official OMIE source records and their hashes, hourly-to-quarter-hour expansion, publication-aware price forecasts,

all member and asset counts, referential integrity, explicit session identity, session occupancy and deliverability, mid-session SoC boundary initialization, non-overlapping deferrable deadlines, Portuguese contracted-power classes, phase sums, topology coverage, clean-twin equality, and every generated-file checksum. Across the suite, 1,817 structural and cross-family checks pass. The diversity audit also measures exact-duplicate traces and schedules, pairwise and cross-archetype profile correlations, peak-clock dispersion, annual demand, PV and BESS distributions, EV arrivals, duration, energy and slack, and deferrable-window dispersion; it reports no finding. Nine week-long schema rollouts complete 6,048 control transitions with finite observations and rewards. Across the regular and compressed rollouts, 31,565 controller-visible price-feature values match their causal inputs; the compressed dynamic timelines apply all 120 topology events and independently verify the corresponding member/asset state, the stationary-battery SoC at activation, and the physical PV balance. The full simulator regression contains 457 passing tests. The scientific audit classifies the suite as ready for benchmarking within the declared scope. The remaining limitations are the absence of a population-level goodness-of-fit claim and of feeder-level electrical physics.

### A.3.6 Matched annual baseline scorecard

The final campaign contains two independently identified controller runs for each schema and binds every result to the same dataset-contract version, current family checksum, exact Simulator source digest, runtime attestation, and policy-pair fingerprint. Table A.7 gives the cost and service results. The cost column reports BAU followed by RBC Smart; the percentage column is the matched RBC change. EV values use the feasible-departure denominator, while deferrable values use the topology-valid completed-plus-missed denominator. Exact EV-target rates are omitted because both declared references stop charging within 0.04 SoC of the requirement; minimum and tolerance rates are the relevant service measures.

Table A.7: Annual matched cost and service scorecard. Paired entries are BAU/RBC Smart.

| Variant | Cost (EUR) BAU/RBC | RBC Δ (%) | EV minimum BAU/RBC (%) | EV tolerance BAU/RBC (%) | Deferrable BAU/RBC (%) |
|---|---|---|---|---|---|
| Micro-4-Q | 1229.62/1169.80 | −4.86 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Core-15-Stripped | 4078.73/3964.39 | −2.80 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Core-30-Nominal | 23497.82/23060.93 | −1.86 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Core-30-Safety | 23496.46/23080.49 | −1.77 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Core-30-Health | 23498.14/23061.00 | −1.86 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Core-30-Dynamic | 12972.56/12676.37 | −2.28 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Core-30-Combined | 12971.27/12695.37 | −2.13 | 100.00/100.00 | 100.00/100.00 | 100/100 |
| Premium-100-Clean | 113639.38/112209.55 | −1.26 | 99.67/99.73 | 99.66/99.72 | 100/100 |
| Premium-100-AllIn | 113610.37/112170.26 | −1.27 | 99.63/99.65 | 99.62/99.64 | 100/100 |

Premium Clean records 93/77 BAU/RBC minimum misses among 28,108 observable departures. These occur only at Members 091, 095, and 097, which host four, four, and six shared 22 kW chargers: the BAU distribution is 76, 13, and 4, and RBC Smart yields 62, 14, and 1. AllIn records 107/103 misses and spreads the additional cases over fault-affected service members. The catalogue audit independently found every session to be individually feasible. The annual misses measure

simultaneous service, connection, and event pressure, with impossible requests already excluded by the generator audit.

Table A.8 retains the stress-case evidence. Requested pressure is integrated before the feasibility projection; residual violation is reconstructed from the applied total and per-phase trajectories. Robustness counts are identical for the matched policies and record physical asset–time-step unavailability, not the number of internal status fields. Four demand-response events have valid baselines in every declared run.

Table A.8: Annual safety, demand-response, and robustness scorecard for the stress variants. Paired entries are BAU/RBC Smart.

| Variant | Requested pressure BAU/RBC (kWh) | Residual BAU/RBC (kWh) | DR compliance BAU/RBC | DR shortfall BAU/RBC (kWh) | Health events/ unavailable steps |
|---|---|---|---|---|---|
| Core-30-Safety | 6.281/0 | 0/0 | — | — | — |
| Core-30-Health | 0/0 | 0/0 | — | — | 4/48 |
| Core-30-Combined | 4.696/0 | 0/0 | 0.340/0.281 | 228.923/264.090 | 4/48 |
| Premium-100-Clean | 4235.047/1465.524 | 0/0 | 0.469/0.437 | 705.288/765.391 | — |
| Premium-100-AllIn | 4267.160/1494.777 | 0/0 | 0.640/0.627 | 394.175/427.110 | 8/120 |

All 18 runs have zero EV energy-accounting shortfall, zero residual electrical violation, and complete every runtime-visible deferrable cycle with zero unserved energy. These integrity results do not erase controller limitations. RBC Smart increases ramping relative to BAU in six schemas and reduces demand-response compliance in Combined and both Premium variants. The detailed machine-readable scorecard retains these outcomes as controller watches and does not relabel them as dataset defects.

The member and asset ranges in this appendix are complemented by the row-level supplementary inventory: the family manifests and Parquet catalogues enumerate every member, physical charger, EV, session, PV system, stationary battery, deferrable appliance, phase assignment, connection level, limit, routine, event target, and source checksum. This machine-readable inventory is authoritative where a 100-member row listing would otherwise duplicate the released schema without adding a scientific distinction.

## A.4 Simulator Audits and Demonstrative Results

This section retains the detailed evidence that supports, but would interrupt, the architectural and methodological account in Chapter 4. The results are bound to the archived simulator checkout and declared workloads. They establish implementation consistency and demonstrate that the corresponding mechanisms are executable and observable; they do not by themselves establish external validity or controller optimality. The dataset-specific structural audit and annual BAU/RBC Smart scorecards remain in Appendix A.3.

### A.4.1 Software and contract verification

Table A.9: Verification and validation evidence reported for the simulator manuscript checkout.

| **Check** | **Failure mode protected against** | **Reported evidence** |
|---|---|---|
| Full regression suite | General simulator behaviour and compatibility across the extended codebase | 457 passing tests; 18 warnings |
| Targeted REC requirements | Incorrect KPIs, event semantics, topology propagation, entity interfaces, and multi-community behaviour | 103 passing tests; 8 warnings |
| Physics and time-unit audit | Energy/power inconsistencies across temporal resolutions | 16 of 16 scenarios passed at 60, 300, 900, and 3600 s |
| Entity/change audit | Unstable identities, relations, masks, action rows, or configuration versions | Strict contract pass |
| Canonical annual-suite audit | Misaligned 2023 clocks or civil-time routines, incorrect OMIE expansion or publication boundary, duplicate profiles or identities, overlapping services, infeasible EV sessions, invalid Portuguese connection classes, incomplete checksums, unmatched scenario twins, or topology events without physical effect | 1,817 structural and cross-family checks pass; diversity and deterministic-generation audits pass without findings; zero individually infeasible EV sessions; nine week-long rollouts, 31,565 causal-price checks, and 120 physically verified topology-event applications pass; Core-15 is the declared physical subset of Core-30 |
| BAU/KPI sanity audit | Non-finite values, duplicate keys, or inconsistent reference deltas | 3317 rows per export; 1401 BAU-derived rows; no duplicate keys |
| Thermal/HVAC compatibility | Regressions in inherited thermal and heat-pump behaviour | 12 passing tests |

These counts characterise the evaluated evidence bundle and should be regenerated if the implementation or dataset contract changes. They complement independently calculable golden cases, which remain necessary to detect systematic balance or service errors that could be concealed by plausible annual trajectories.

### A.4.2 Selected capability demonstrations

The demonstrations below retain the observed values beside the precise interpretation that each case can support.

Table A.10: Selected application evidence and the bounded claim supported by each example.

| Example | Observed evidence | Supported interpretation |
|---|---|---|
| Historical 17-member REC/EV reference | BAU: 28.7 kEUR, 166.8 MWh import, 30.0 $tCO_2$; community RBC: 22.0 kEUR, 133.3 MWh, 22.8 $tCO_2$, approximately 91% EV within-tolerance service and 100% flexible-load service | Service, settlement, and constraint indicators prevent a cost-only reading; transparent RBCs are references, not proof of optimality. |
| Temporal resolution | The same annual trace yields peaks of 96.8, 188.2, and 281.5 kW at 1 h, 15 min, and 15 s | A declared timestep changes the observable capacity problem even when annual energy is preserved. |
| Historical 17-member settlement | 17.0 MWh shared; cost changes from 24.8 to 22.0 kEUR; all 17 members save under the configured rule | Participant and accounting outcomes are observable; equity and legal applicability remain separate questions. |
| Demand response | Shortfall changes from 101.1 to 51.6 kWh; compliance from 0.43 to 0.70; cost after settlement from 863.6 to 790.6 EUR | Requests, delivery, shortfall, and settlement can be audited without claiming profitability or controller optimality. |
| Dynamic topology | Four member/asset events propagate to active counts, identified rows, relations, actions, and logs | Runtime change is represented consistently; controller adaptation must be evaluated separately. |
| Controlled failures | Missing observations, forecast bias, command dropout, and asset unavailability appear in their respective channels and exports | Disturbances are repeatable and measurable; robustness outside the declared classes is not inferred. |
| Multiple communities | Four independent RECs retain local and combined results; 2.45 MWh of simultaneous surplus/demand is identified | Portfolio evidence is preserved without claiming physical coupling or a market-clearing mechanism. |

### A.4.3 Performance and scalability smoke benchmarks

The smoke benchmarks compare matched configurations and isolate the computational effect of interface detail, community multiplicity, storage format, and action-feedback retention.

Table A.11: Computational-cost evidence from matched simulator configurations.

| Comparison | Observed change | Interpretation |
|---|---|---|
| Flat versus basic entity input | 5.30 to 5.51 ms/step, approximately +4.1% | Stable identities, relations, masks, and action feedback add a small measured overhead in the matched case. |
| Flat versus detailed entity input | 4.88 to 10.18 ms/step; 165 flat values versus 2538 table values and 302 feature names | The richest interface more than doubles payload processing but remains quantitatively characterised. |
| One, two, and four communities | 5.02, 7.08, and 11.41 ms/step; four communities produce 5227 KPI rows, including 107 combined rows | Four communities take 2.27 times the single-community step time while retaining local and portfolio evidence. |
| 15-second CSV versus Parquet | Approximately 2.4–2.6 GB becomes 12–13 MB; initialisation falls by 34–38% | Columnar storage addresses data loading and storage; it does not materially change post-load physics. |
| Entity-feedback memory regression | Approximately +2071 MiB reduced to approximately +5 MiB over 700 targeted steps | The audit exposed and bounded a major implementation leak in the exercised configuration. |

The basic entity contract remains close to flat execution in the matched case, while the detailed bundle deliberately exchanges a much richer payload for approximately twice the step time. The four-community configuration corresponds to approximately 88 steps/s. These observations make the computational cost visible for the reported studies; they do not imply linear scaling to arbitrary population sizes, detailed feeder models, or distributed deployment targets.

## A.5 Simulator KPI Catalogue

The core controller scorecard in Chapter 4 is a deliberately small interpretation layer over a broader CityLearn v3 KPI surface. The complete machine-readable output retains the entity or aggregation level, controller condition, baseline condition, unit, and raw value. It is produced through the primary `evaluate_v2()` interface. Table A.12 records the implemented families and the principal quantities used to interpret them. Exact code-level names are versioned with the simulator documentation and exported result, which remain authoritative when the catalogue evolves.

Table A.12: Expanded CityLearn v3 KPI catalogue.

| **Family** | **Principal quantities** | **Typical scope and unit** |
|---|---|---|
| Energy and grid exchange | Net electricity consumption; total grid import and export; net grid energy; average daily and maximum import/export peaks; ramping; load factor; zero-net-energy indicators | Building, community, or portfolio; kWh, kW, or ratio |
| Cost and settlement | Grid import cost; export revenue; local import/export payment; cost after local and demand-response settlement; grid-only counterfactual; raw, BAU-difference, and valid BAU-ratio rows | Member, community, or portfolio; EUR or ratio |
| Participant benefit distribution | Members with lower cost than the matched reference; benefit-coverage fraction; minimum, maximum, median, and dispersion of member savings | Member distribution; count, fraction, or EUR |
| Emissions | Imported-energy emissions; average carbon intensity; raw and BAU-relative emissions | Building, community, or portfolio; $kgCO_2$ or $kgCO_2$/kWh |
| PV and local sharing | PV generation, self-consumption, self-sufficiency, local import/export, matched local exchange, community surplus and deficit, sharing ratios | Member or community; kWh or ratio |
| Stationary storage | Charged/discharged energy, losses, state of charge, cycling and degradation-related outputs where enabled, requested/applied power difference | Asset, building, or community; kWh, kW, ratio, or state |
| EV and charger service | Connected steps, arrivals, topology-valid departures, target/minimum/tolerance success, departure deficit, charged and V2G energy, required average power, slack, charger clipping, and EV energy-accounting shortfall | EV, charger, member, or community; count, ratio, kWh, kW, or time |
| Deferrable service | Requested, started, completed, missed, topology-censored, and interrupted cycles; served/unserved energy; mean and maximum start delay; deadline violations | Asset, member, or community; count, ratio, kWh, or time |
| Electrical feasibility | Requested total/per-phase pressure, rejected or reduced power, residual applied violation, violation duration, state-bound violations, and feasible headroom | Asset, member, phase, or community; kW, kWh, count, or time |

| Family | Principal quantities | Typical scope and unit |
|---|---|---|
| Demand response | Request and active-step counts; requested, delivered, credited, and shortfall energy; compliance; invalid baselines; payment, penalty, and net revenue; up/down delivery | Request, community, or portfolio; count, kWh, ratio, or EUR |
| Topology and health | Active member/asset counts and periods; additions/removals; altered observation and forecast values; missing data; changed/lost actions; equipment-unavailable asset–step counts | Entity, member, community, or portfolio; count, time, or mixed |
| Controller interface | Entity-table and row counts; relations; feature and action identities; masks; feasible action ranges; requested, clipped, applied, and rejected actions with reasons | Controller, entity, or action port; count or native action unit |
| Computational effort | Training wall time, inference latency, simulated steps per second, initialisation, reset and export time, peak memory, and the associated hardware, software, population, interface, and training-budget context | Controller or experiment; ms, s, h, steps/s, or MiB |
| Multiple communities | Per-community KPI tables; summed energy, cost, emissions, payments, requests, and events; declared weighted ratios; combined portfolio results | Community or portfolio; inherited unit |
| Inherited thermal, HVAC, and comfort | Heat-pump electricity, unmet thermal demand, comfort violations, occupant overrides, thermal resilience, and HVAC clipping where the selected scenario enables those inherited models | Building or district; kWh, count, time, or ratio |

Three interpretation rules apply across this catalogue. First, reward and KPI are distinct: reward is controller-facing and may combine selected objectives, whereas KPIs remain evaluative outputs. Second, additive quantities may be summed across members or communities, but ratios require a declared denominator or reporting weight. Third, requested-pressure and applied-residual quantities must not be conflated: the former describes controller intent before feasibility projection, while the latter establishes whether the executed trajectory violated the configured physical boundary. The archived time series and event logs remain necessary whenever an aggregate KPI cannot explain when or why an outcome occurred.

## A.6 Controller, TI-RL, and TI-MARL Configurations

Complete controller definitions cover passive and random operation, rule-based policies, MILP replay policies, strict-local PPO and TD3, IPPO, MAPPO, HAPPO, MADDPG, MATD3, MASAC, the entity-structured PPO and MATD3 variants, hierarchical controllers, and the typed-interface variants introduced in Chapter 6. There is no standalone SAC registry entry: the entropy-regularised implementation is MASAC. `SingleAgentRL` is a configuration placeholder and is not an executable method.

### A.6.1 Executable Controller Inventory

Table A.13: Executable Algorithms registry grouped by scientific role.

| **Role** | **Registered names** | **Configuration evidence retained** |
|---|---|---|
| Passive and diagnostic | `NormalNoBatteryPolicy`, `NormalPolicy`, `RandomPolicy` | Asset availability, service logic, seed, observation/action names, and export contract |
| Rule-based | `RBCBasicPolicy`, `RBCSmartPolicy`, `RBCSmartLocalPolicy`, `RBCCommunityPolicy`, `RuleBasedPolicy` | Information boundary, price and forecast use, EV/V2G thresholds, deferrable logic, headroom rule, and fallback scope |
| Signal-aware rules | `SignalAwareRBC`, `SignalAwareRBC SmartLocal`, `FixedPriceSignal`, `CausalPriceSignal` | Signal range, neutral value, update interval, recipient scope, and causal response probe |
| Oracle replay | `FixedServiceOracle ReplayPolicy`, `TotalHomeOracle ReplayPolicy`, `TotalOracleReplayPolicy` | Solver model, horizon, information assumption, status/gap, source interval, replay tolerance, and fallback |
| Strict local RL | `PPO`, `TD3` | One learner per member, local profile, reward, network, optimiser, rollout/replay, exploration, safety, seed, and checkpoint |
| Flat MARL | `IPPO`, `MAPPO`, `HAPPO`, `MADDPG`, `MATD3`, `MASAC` | Actor information, critic/value scope, team-reward mix, update order, entropy or target-noise settings, replay/rollout, and comparison reference |
| Entity-structured learning | `AgentTransformerPPO`, `AgentTransformerMATD3` | Tokenizer schema, layout signature, known type widths, topology version, homogeneous replay/rollout rule, runtime adapters, and per-topology export |

| Role | Registered names | Configuration evidence retained |
|---|---|---|
| Hierarchical control | `BuildingAgent`, `CommunityCoordinator`, `CCLevel1`, `CCLevel2` | Leaf identity and checkpoint, frozen state, coordinator inputs/outputs, signal interval, price adapter, reward allocation, and neutral-parity run |
| Typed interface | `TIMARL` | Interface registry, compiler and health-rule hashes, hierarchical typed encoder, grouped hybrid actor, analytic local projector, TI-PPO/TI-MAPPO critic, compatibility signature, and trace versions |

### A.6.2 Observation, Action, and Training Record

Every neural run archives the raw simulator interface, selected information surface, encoding profile, ordered raw and encoded observation names, ordered action names, physical bounds, building identities, and entity/topology metadata. Fixed-vector campaigns identify one of `minmax_space`, `maddpg_v1`, `maddpg_v2_compact`, `maddpg_v3_operational`, or `maddpg_v3_realtime`. Structured campaigns additionally archive the tokenizer configuration, SRO/NFC/CA segments, type feature widths, excluded features, action-to-CA order, and full layout signature.

The action record distinguishes actor output, exploration action, deterministic base, residual correction, composed action, controller projection, simulator request, and applied action whenever the selected path contains those stages. For off-policy learning, current and successor base actions and the independent cloning target are preserved separately. For PPO, the exact pre-tanh action, old log-probability, value, policy-eligibility flag, and termination/truncation state remain attached to the rollout sample.

The training manifest records the actor and critic classes and layers, learning rates, discount, target-update coefficient, rollout or replay capacity, batch and minibatch sizes, update cadence, PPO epochs and clipping, GAE, entropy settings, critic loss, target clipping, gradient clipping, target smoothing, actor delay, reward/return normalisation, multi-step return, exploration schedule, semantic noise scales, behaviour-cloning sources and weights, residual authority, local-safety settings, AMP/device use, seed, checkpoint mode, and export dependencies. A field absent from a method is recorded as not applicable rather than assigned a silent default in the result table.

### A.6.3 TI-MARL Reproducibility Contract

The public input to `TIMARL` is one technology-neutral `typed_agent_interface_v1` YAML definition for each registered member, together with an explicit simulator-binding registry. The

simulator owns runtime entities, topology events, values, fault evidence, and execution feedback. The compiler owns semantic binding, health derivation, dependency closure, mode validity, dynamic bounds, feasibility inputs, and fallback semantics. Generated defaults are expanded into the resolved interface archived with the run.

Table A.14: TI-MARL v1 configuration and evidence fields retained for reproduction.

| **Area** | **Required record** |
|---|---|
| Interface | Contract version; member definitions; simulator bindings; semantic, sensor, channel, unit, scope, use, and action-group registries; schema and health-rule hashes |
| Actor | Latent width; attention heads; relational layers; local or action-conditioned group context; categorical modes; Beta-fraction heads; deterministic strategy, gain, deadband, and non-idle margins by group type |
| Critic | TI-PPO/local or TI-MAPPO/set pairing; optional typed-group critic; value-target normalisation; Huber or MSE value loss |
| PPO | Learning rate; reference $\gamma$ and GAE $\lambda$; discount timebase; clip; epochs; rollout length; entropy by group family; advantage-normalisation scope; KL target; gradient limit |
| Credit and projection | Joint-agent or typed-group credit; intervention-aware masking; intervention distillation; EV service strategy and margins; total/per-phase headroom reserve; deferrable completion margin |
| Prior knowledge | Behaviour-cloning teacher, demonstration episodes, balancing method, calibration; EV and storage planning coefficients, label parameters, per-reason reservoir capacities, and samples |
| Population | Stable member order; joining/departing transition treatment; active/registered members; topology version; structural recompilation count |
| Persistence | `ti_marl_checkpoint_v1`; learning architecture; actor, critic, group critic, optimisers, rollout, RNG, compiler, resolved config, versions, hashes, and migration decision |
| Execution evidence | Raw and projected typed bundles; interventions; encoded commands; `entity_action_execution_v1`; current/next snapshot hashes; trace chunk and deduplication settings |
| Handoff | Research artefact deployment flag; actor-only bundle format; resolved interfaces; compiler and health rules; projector; normalisation; compatibility signature; omitted critic and optimiser state |

The audited implementation state is Algorithms commit `58fb6370` with simulator package

version 1.8.0. Its focused TI-MARL suite contains 115 passing tests across compiler, policy runtime, runtime contracts, public typed interfaces, a real-simulator dynamic vertical slice, and a 15-second topology stress case. This count identifies the audited state; later releases must report their own test inventory rather than inheriting it.

### A.6.4 Reward Profile Ledger

The profile numbers below are provenance identifiers rather than an ordered claim of improvement. A later profile records a diagnosed change in the control signal; it becomes preferable only when the matched evaluation supports that decision.

Table A.15: Implemented Chapter 5 reward-profile lineage.

| Identifier | Registered profile | Change retained for reproduction |
|---|---|---|
| Base | Cost Hard Constraint | Configurable local/community cost, electrical and outage penalties, EV service and V2G, storage bounds/throughput, deferrables, import, peak, ramp, export, emissions, settlement, and timestep scaling |
| V2 | Cost Service Guard | Protects EV service from cost savings created by V2G below the target |
| V3 | Cost Service Cost Balanced | Rebalances service pressure and cost and introduces a light storage-throughput term |
| Local V1 | Local Cost Service | Removes community settlement and aggregate penalties to enforce a strict-local reward boundary |
| Local V2 | Local Scorecard Guard | Raises electrical, EV, storage, and deferrable failure costs around the frozen scorecard gates |
| Local V3 | Local Economic Safety | Removes dense recoverable urgency when deadline-feasible projection already protects service |
| Individual V3 | Individual Scorecard Aligned | Uses each member's settlement row while actors and critics remain independent; labelled as a market/broadcast reward rather than a strict-local ablation |
| V4 | Community Band | Adds community settlement and EV over-service tolerance to the service-first objective |

| Identifier | Registered profile | Change retained for reproduction |
|---|---|---|
| V41 | Community Storage Band | Strengthens storage discipline and EV target-band pressure |
| V42 | Community Service Band | Raises service pressure after cost-improving under-service and over-service were observed |
| V43 | Community Battery Value | Restores physical storage fallback bounds, uses observed limits, scales by timestep, and lowers the cycling penalty |
| V44 | Community Smooth Service | Uses quadratic dense EV terms so recoverable deficits do not dominate long high-frequency episodes |
| V45 | Community Feasible Service | Computes schedule pressure from effective charging power under building/phase headroom and penalises unrecoverable schedule deficit |
| V46 | Community Feasible Precision | Caps only training penalties for infeasible shortfall and increases over-service precision; raw diagnostics remain uncapped |
| V47 | Community Feasible Precision, tighter band | Retains V46 feasibility and tightens the upper service band |
| V49 | Community Storage Value | Reduces throughput suppression and raises settlement/peak value so useful storage operation remains learnable |
| V50 | Community Deadline Value | Extends bounded deadline shaping while retaining the effective-power feasibility calculation |
| V51 | Community Precision Value | Rebalances minimum service, over-service, and community value after excessive charging |
| V52 | Community Peak Deadline | Adds stronger explicit peak/export pressure beside deadline service and settlement |
| V53 | Community Residual Constraint | Defines a community-shape objective for bounded residual learning over an RBC teacher |
| V54 | Community Dense EV Residual | Makes sparse EV consequences visible during connected periods for conservative residual control |

| Identifier | Registered profile | Change retained for reproduction |
|---|---|---|
| V55 | Community Storage Residual | Removes uncontrollable service terms only when EV and deferrable actions are copied exactly from the audited teacher |
| CC | Community Coordinator reward | Price-arbitrage-centred coordinator objective retained for historical reproduction |
| CC Level 1 | Aggregate coordinator reward | Normalised cost, peak excess, ramp, export, violation, and selectable settlement aggregation |
| CC Level 2 | Member-aware coordinator reward | Adds EV harm, member-decomposed settlement and constraint credit, team mixing, neutral counterfactual, and running normalisation |
| CC Phase 1 | Market-making phase reward | Retains the intermediate coordinator formulation used during the hierarchy's historical development |

Each final run stores the complete reward class and parameter mapping. The abbreviated identifiers above are insufficient to reproduce a run on their own. Reward component exports, raw and normalised totals, settlement parameters, seconds per timestep, phase/charger map, and any running state are retained with the experiment evidence.

## A.7 Hyperparameters, Seeds, and Statistical Procedures

The supplementary record contains hyperparameter tables, search ranges, selected values, random seeds, training budgets, checkpoint-selection rules, evaluation windows, statistical tests, confidence intervals, and the treatment of failed runs.

## A.8 Supplementary and Negative Results

Supplementary evidence includes additional learning curves, participant-level results, scale curves, disturbance-severity curves, ablations, failure cases, methods that did not improve the selected objective, and configurations rejected before deployment. Negative results are retained where they explain a design choice in Chapters 5 or 6.

## A.9 Lifecycle and Deployment Records

Lifecycle records include experiment-manifest examples, policy-set manifests, compatibility rules, validation reports, package specifications, target profiles, deployment sequences, mixed-version tests, rollback records, monitoring schemas, soak-test summaries, and failure-recovery traces.

## A.10 Software, Data, and Archival Record

The archival record lists repositories, releases, licences, archived commits or DOIs, container images, external dependencies, datasets and access conditions, partner-restricted components, hardware specifications, and commands used to reproduce the principal figures and tables. Restricted artefacts are accompanied by a stated limitation and the closest reproducible substitute.